ridez.io

Complete Technical Documentation & Knowledge Base

Generated from `docs/project-technical-documentation/` · commit 6eb54b7 · 2026-11-02

Contents

1. ridez.io — Technical Documentation Knowledge Base
2. 00 — Project overview
3. 01 — Technology stack
4. 02 — Folder and file structure (file-by-file)
5. 03 — System architecture
6. 04 — Frontend documentation
7. 05 — Backend documentation
8. 06 — API documentation
9. 07 — Database documentation
10. 08 — Authentication, authorization and security
11. 09 — Feature-by-feature explanation
12. 10 — Complete data flow
13. 11 — Third-party integrations and external services
14. 12 — State management and frontend flow
15. 13 — Background processes and real-time functionality
16. 14 — Error handling and edge cases
17. 15 — Application execution and deployment
18. 16 — Dependencies and package explanation
19. 17 — Technical decisions and project design
20. 18 — Problems, limitations and future improvements
21. 19 — Manager explanation guide
22. 20 — Technical interview questions and answers
23. 21 — Quick revision sheet
24. Architecture diagrams
25. Entity-relationship diagram
26. Frontend diagrams
27. Real-time and background processing diagrams
28. Admin sequence diagrams
29. Authentication sequence diagrams
30. Completion, payment and rating sequence diagrams
31. Ride sequence diagrams
32. State machines

ridez.io — Technical Documentation Knowledge Base

This folder is a complete, developer-level knowledge base for the **ridez.io** taxi-booking application in this repository. Everything here was written by reading the actual source code (commit `6eb54b7`, branch `fresh-main`, 2026-11-02). Where behaviour is inferred rather than read directly from code, it is marked **(inferred)**. Where something could not be verified, it is marked **(not verified)**.

> **Scope of verification.** All file paths, function names, endpoints, table names and fields were checked against the source. The automated test suite was **not executed** while writing these docs, because the checkout has no Python virtualenv or `node_modules`, and installing dependencies was out of scope. Test *names and intent* are documented from reading the test files.

How to read this

If you want to…	Start with
Explain the product in 5 minutes	19 — Manager explanation guide
Revise the night before a meeting	21 — Quick revision sheet
Understand the big picture	00 — Overview → 03 — Architecture
Find what a file does	02 — Folder and file structure
Look up an endpoint	06 — API documentation
Look up a table	07 — Database documentation
Trace a feature end to end	09 — Features and 10 — Data flow
Prepare for hard questions	20 — Interview questions and 18 — Problems

Documents

#	Document	Contents
00	Project overview	What the app does, users and roles, features, workflow, high-level diagram
01	Technology stack	Every technology and why it is used
02	Folder and file structure	File-by-file explanation of the whole repository
03	System architecture	Layers, request lifecycle, concurrency model, error architecture
04	Frontend documentation	React app, routing, every component and hook
05	Backend documentation	Flask app factory, every module and function
06	API documentation	All 39 HTTP route handlers (43 logical operations) + Socket.IO contract
07	Database documentation	All 19 tables, ERD, constraints, migrations
08	Authentication and security	JWT + refresh cookies, RBAC, CSRF, rate limits, gaps
09	Feature-by-feature explanation	19 features traced end to end
10	Complete data flow	Step-by-step lifecycle of data for each major flow
11	Third-party integrations	Google Routes, Maps JS, Places, CDN, Render
12	State management	Session store, workspace polling, forms, routing

#	Document	Contents
13	Background and real-time processes	Matching worker, outbox, Socket.IO, GPS loop
14	Error handling	How each failure class is handled, gaps
15	Deployment and execution	Local run, Docker, Render, CI, backups
16	Dependencies	Every package, version, purpose and usage
17	Technical decisions	Design patterns, trade-offs, documented vs inferred decisions
18	Problems and improvements	Bugs, risks, tech debt, recommendations
19	Manager explanation guide	Speaking scripts
20	Technical interview questions	70+ questions with project-specific answers
21	Quick revision sheet	Paths, functions, endpoints, tables, env vars

Diagrams

All diagrams are Mermaid, so they render on GitHub and in most Markdown viewers and remain editable.

Diagram file	Contents
diagrams/architecture.md	High-level system, backend module dependencies, deployment topology
diagrams/erd.md	Entity-relationship diagram of all tables
diagrams/state-machines.md	Ride, driver, payment, mock payment, session state machines
diagrams/sequences-auth.md	Registration, login, refresh, logout, request authentication
diagrams/sequences-rides.md	Location, quote, booking, matching, acceptance, trip, cancellation, trip map
diagrams/sequences-payments-ratings.md	Completion, cash, online/mock QR, ratings
diagrams/sequences-admin.md	Driver review, vehicle, fare and revenue settings, audit
diagrams/realtime.md	Matching worker loop, outbox dispatch, client refresh
diagrams/frontend.md	Route tree and component tree

Relationship to the older docs in `docs/`

The repository already contains shorter, topic-specific notes in `docs/` (for example `docs/SESSIONS.md`, `docs/RENDER_DEPLOYMENT.md`, `docs/PRODUCT_BLUEPRINT.md`). Those were left unchanged. This knowledge base supersedes them as a single reference, and points out where they describe **planned** behaviour that the code does not implement (for example, the blueprint mentions Redis and SQL Server, but the running system uses neither). See 17 — Technical decisions.

Terminology used consistently

Term	Meaning in this codebase
Rider	Passenger account (`users.role = 'rider'`)
Driver	Driver account (`users.role = 'driver'`); must be approved by an admin
Admin	Operator account (`users.role = 'admin'`); cannot self-register
Quote	A 5-minute, immutable fare estimate (`fare_quotes` row)
Ride	A booked trip (`rides` row), identified by a UUID string
Invitation	A pending ride offer to one driver (`ride_invitations` row)
Round	One 30-second matching window at a given radius (`ride_match_rounds` row)

Term	Meaning in this codebase
Active claim	A row in `active_rides` that marks a user as busy with one ride
Outbox event	A row in `outbox_events` that the worker turns into a Socket.IO `workspace.changed` push
Workspace	The JSON snapshot returned by `GET /api/v1/workspace`, and the React screen built from it
Tick	One iteration of the 1-second background loop (`app/realtime.py::tick`)
Allocation / settlement	The driver/company split recorded in `revenue_ledger`; it is accounting only, not a money transfer

00 — Project overview

What the application does

ridez.io is a web-based taxi booking platform (think a small, self-hosted ride-hailing service) with three role-specific workspaces in one React single-page app:

- **Riders** search pickup/destination addresses, receive an itemized, locked fare quote, book a ride, follow the matched driver live on a map, share a per-ride OTP at pickup, pay (cash, or a simulated "online" QR payment), and rate the driver.
- **Drivers** register and wait for admin approval, go "available" while sharing browser GPS, receive ride invitations within an expanding radius, accept or reject them, progress the trip (head to pickup → arrive → start with OTP → complete), confirm cash payments, see earnings, and rate the rider.
- **Admins** approve, suspend and reinstate drivers, record vehicle details, configure versioned INR fare rules and surge, configure effective-dated driver/company revenue splits, monitor live metrics, and browse an audit history.

The backend is a single Flask process that serves the REST API under `/api/v1`, a Socket.IO server for push notifications, a 1-second background loop that drives matching deadlines, and (in production) the compiled React app itself.

What problem it solves

Matching a passenger with a nearby, available, approved driver, agreeing on a fair price up front, and carrying the trip safely to completion with an auditable money split. Specifically, the code addresses:

Problem	How the code addresses it
Price disputes	Fare is computed server-side and stored as an **immutable quote** (`fare_quotes`), including the policy version and surge reason. The ride uses the quote total; later rate changes cannot reprice it.
Finding a driver	**Expanding-radius matching**: 500 m, +500 m every 30 s, up to `MATCH_MAX_RADIUS_M` (default 5 km), only for drivers with GPS fresher than 90 s (`app/matching.py`).
Double booking	Database uniqueness on `active_rides.user_id` and **conditional UPDATEs** ensure one driver per ride and one ride per person even under concurrency.
Wrong passenger / wrong car	**Per-ride OTP** the driver must enter to start the trip; vehicle details are **snapshotted** onto the ride at acceptance.
Driver earnings transparency	Revenue split recorded once per completed ride in `revenue_ledger`; earnings count as "received" only once payment is confirmed.
Operational control	Admin approval, suspension with visible reasons, audit trail of every admin change.

Primary objectives (as reflected in code and `docs/PRODUCT_BLUEPRINT.md`)

1. A complete, working ride lifecycle from quote to settlement.
2. Correctness under concurrency (competing acceptances, cancellations racing completions, etc.).
3. Honest labelling: simulated routes, test payments and allocations are always labelled as such in the UI and API (e.g. `test_only: true`, "Allocation recorded · not a payment").
4. Privacy: riders never see revenue splits; drivers never see the rider's OTP; admins never see precise driver coordinates; people never see comments written about them.

Users and roles

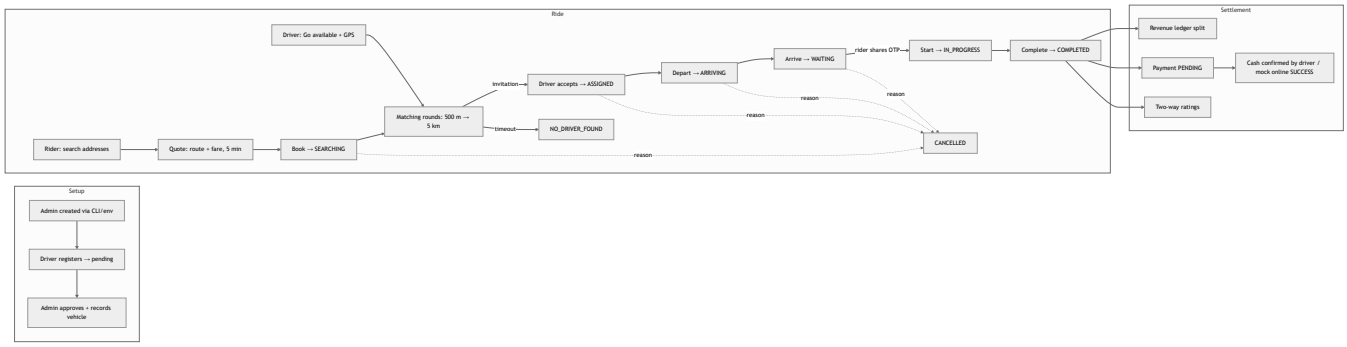
Role	How the account is created	Initial status	What they can do
`rider`	Public registration (`POST /api/v1/auth/register`)	`active`	Quote, book, cancel, pay, rate, view history
`driver`	Public registration with role `driver`	`pending`	Sign in and view workspace; after approval: go available, accept rides, progress trips, confirm cash, rate riders, view earnings
`admin`	**Only** CLI `flask create-admin` or `flask bootstrap-admin` (env vars). Public registration rejects `admin`.	`active`	All admin pages and endpoints; can view any ride

Account `status` values (`users.status`): `active`, `pending`, `suspended` (drivers only, enforced by CHECK constraint), `disabled`. **Note:** no code path or endpoint sets `disabled`; it can only be set directly in the database. Disabled users cannot log in, refresh, or use tokens.

Main features

Feature	Status
Mobile + password registration/login, permanent account codes	Implemented
Persistent sessions (15-min JWT + rotating 30-day HttpOnly refresh cookie), logout, logout all devices	Implemented
Driver approval / suspension / reinstatement with audit	Implemented
Admin-recorded vehicle details, snapshotted on each ride	Implemented
Driver GPS availability with freshness, replay and plausibility checks	Implemented (browser tab must stay open)
Google Places address autocomplete + map click selection	Implemented (needs browser key)
Route + fare quotes (Google Routes or deterministic simulator)	Implemented
Versioned fare policy with zone/traffic/demand surge	Implemented
Expanding-radius matching with persistent rounds	Implemented
Accept/reject, trip lifecycle, per-ride OTP	Implemented
Pre-trip cancellation with reasons	Implemented
Revenue allocation with effective-dated splits	Implemented
Payment method choice, driver cash confirmation	Implemented
"Online" payment	**Simulated only** — mock QR page, no bank/UPI integration
Two-way star ratings with tags and admin-only comments	Implemented
Live Google trip map and ETA card	Implemented for Google-mode rides
Real-time updates (Socket.IO + outbox) with polling fallback	Implemented
Admin metrics dashboard and audit history	Implemented
Profile editing, account recovery, password reset	**Not implemented** (UI says "later milestone")
SMS OTP, native background tracking, real payments, refunds, incident handling	**Not implemented**
Multi-instance scaling (Redis, shared rate limits)	**Not implemented**; system must run as one process

Overall application workflow



Technology stack at a glance

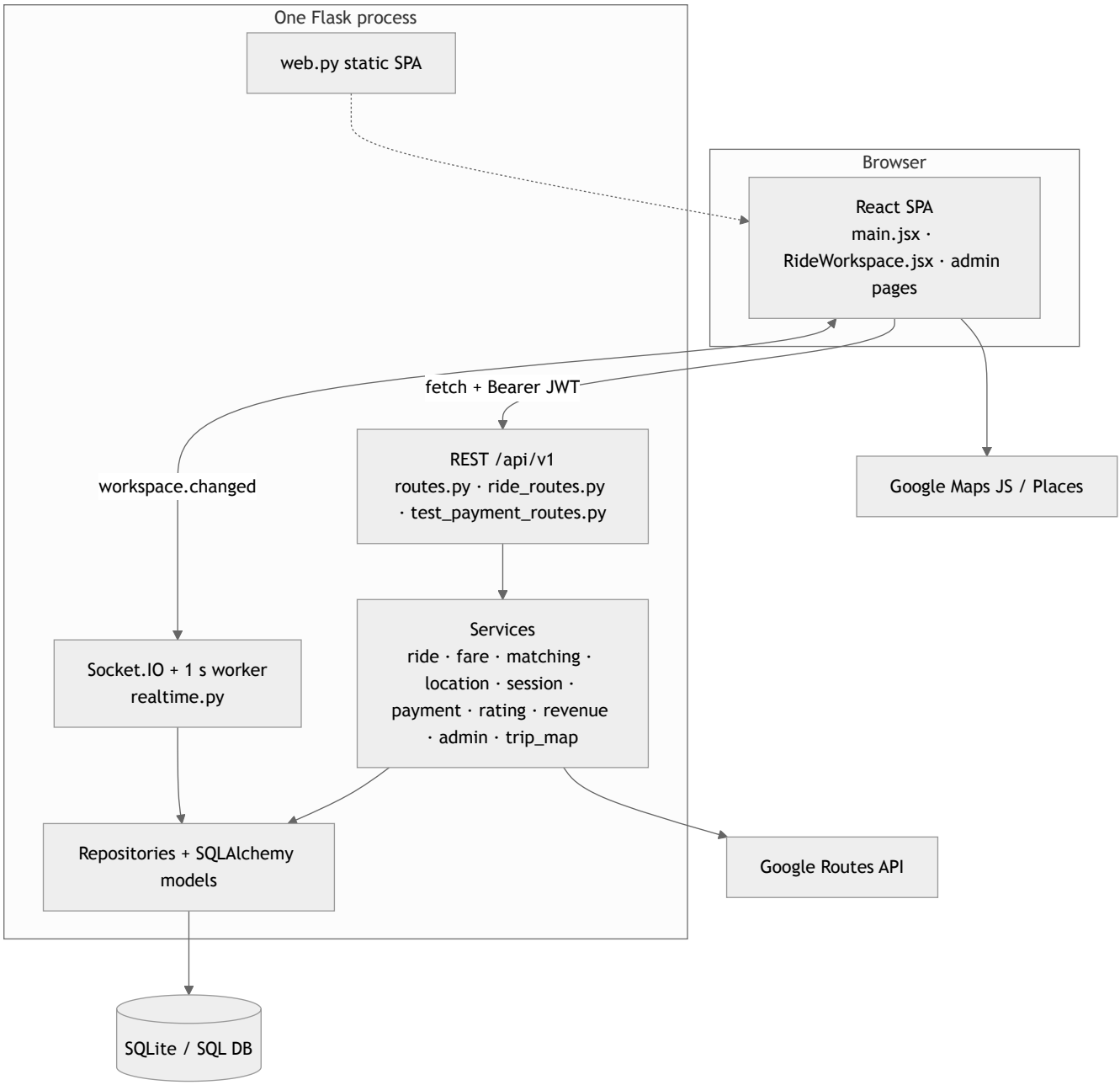
Layer	Technology
Frontend	React 19, React Router 7, Vite 6, socket.io-client 4, lucide-react icons, qrcode
Backend	Python 3.12, Flask 3.1, Flask-SQLAlchemy 3.1 / SQLAlchemy 2.1, Flask-Migrate (Alembic), Flask-SocketIO 5 (threading mode), PyJWT, phonenumbers, requests
Database	SQLite in development and in the Render deployment; SQL Server (pyodbc) targeted but **not validated**
External	Google Routes API (server), Google Maps JavaScript API + Places API (New) (browser)
Runtime	Gunicorn (1 worker, gthread, 100 threads) in Docker on Render with a persistent disk
CI	GitHub Actions: pytest, Vite build, Docker build, container smoke test

Full rationale: 01 — Technology stack.

How the major systems work together

1. The browser loads the SPA (from Vite in development, or from Flask's `web.py` in production).
2. The SPA restores the session by calling `POST /api/v1/auth/refresh` with the HttpOnly cookie, getting a 15-minute JWT that lives only in memory.
3. Every screen is driven by one authoritative JSON snapshot, `GET /api/v1/workspace`, assembled by `ride_service.snapshot()` according to role.
4. User actions call specific endpoints (quote, book, accept, …). Each handler validates, performs conditional writes, inserts **outbox events** for every affected user, and commits.
5. The background loop (every second) advances matching deadlines and turns outbox rows into Socket.IO `workspace.changed` messages. Clients receiving one simply re-fetch `/workspace`. If the socket is down, clients poll every 2 seconds.
6. Fare quotes and live trip maps call Google Routes from the server; address search and map tiles call Google from the browser.

High-level architecture diagram



More diagrams: diagrams/architecture.md.

01 — Technology stack

Versions below are taken from `frontend/package.json` (ranges) and `backend/requirements.lock` (exact pins). Rationale is labelled **documented** when stated in the repo (code comments, README, `docs/`) and **inferred** otherwise.

Frontend

Technology	Version	Role in this app	Why it is used
React	`^19.0.0`	All UI components (`frontend/src/*.jsx`)	Documented: blueprint mandates "React with JavaScript". Function components + hooks only; no class components.
react-dom	`^19.0.0`	`createRoot` in `main.jsx`	Required renderer for React 19
React Router	`^7.0.0` (`react-router-dom`)	`BrowserRouter`, nested `Routes`, `Navigate`, `NavLink`, `useParams` in `main.jsx` and `TestPayment.jsx`	Client-side routing for `/rider/*`, `/driver/*`, `/admin/*`, `/test-payment/:token`
Vite	`^6.0.0`	Dev server with proxy (`vite.config.js`), production build to `frontend/dist`	Inferred: fast dev server; built-in `import.meta.env` for `VITE_*` browser variables; simple proxy for `/api` and `/socket.io` WebSockets
@vitejs/plugin-react	`^4.3.0`	JSX transform and fast refresh	Standard Vite React integration
socket.io-client	`^4.8.1`	`io()` in `RideWorkspace.jsx::useWorkspace`	Must match the Flask-SocketIO (python-socketio 5 / Engine.IO 4) protocol
lucide-react	`^0.468.0`	Icons across all pages	Inferred: lightweight, tree-shakeable SVG icons
qrcode	`^1.5.4`	`QRCode.toDataURL` in `TestPayment.jsx`	Generates the mock payment QR image in the browser
Plain CSS	—	`frontend/src/styles.css`	No CSS framework

No state-management library (Redux, Zustand, React Query) is used. State is held in React hooks plus a small module-level store in `authSession.js`. See 12 — State management.

No TypeScript, no ESLint config, no frontend unit-test runner. There is one Playwright browser script (`frontend/tests/qr-session.browser.cjs`) that is not wired into CI.

Backend

Technology	Version	Role	Why
Python	3.12 (`.python-version` = `3.12.12`)	Runtime	Documented in README: 3.14 is unsupported by the pinned `pyodbc` wheel
Flask	3.1.0	HTTP framework, app factory `create_app()` in `app/__init__.py`, one Blueprint `api`	Documented in blueprint: "Flask REST API"

Technology	Version	Role	Why
Werkzeug	3.1.9	Password hashing (`generate_password_hash` / `check_password_hash`), `ProxyFix`, routing	Comes with Flask
Flask-SQLAlchemy	3.1.1	`db = SQLAlchemy()` session and model base	ORM integration
SQLAlchemy	2.1.6	2.0-style `select()`, `update()`, `delete()` statements in repositories/services	Needed for conditional `UPDATE … WHERE` used for concurrency control
Flask-Migrate / Alembic	4.1.0 / 1.20.1	`flask db upgrade`; 13 migrations in `backend/migrations/versions`	Versioned schema changes; `/ready` checks DB is at the migration head
Flask-SocketIO / python-socketio / python-engineio	5.5.1 / 5.17.0 / 4.15.1	`socketio` server in `app/realtime.py`, `async_mode='threading'`	Push `workspace.changed` notifications; threading mode avoids eventlet/gevent
simple-websocket / wsproto	1.1.0 / 1.3.2	WebSocket transport for threading mode	Transitive dependency of Flask-SocketIO
PyJWT	2.10.1	HS256 access tokens in `session_service.py`	Stateless bearer token, but paired with a DB session check
itsdangerous	2.2.0	`URLSafeTimedSerializer` for mock QR tokens (`test_payment_routes.py`)	Signed, time-limited capability URLs; ships with Flask
phonenumbers	8.13.55	`services.normalize()` → E.164	Ensures one account per real number regardless of formatting
requests	2.32.5	Google Routes call in `route_provider.compute`	Simple HTTP client with timeouts
python-dotenv	1.2.1	Loads `backend/.env` in `create_app` (never overrides real env vars)	Local configuration
tzdata	2025.2	IANA zone data for `zoneinfo` in `fare_service` (surge schedules)	Needed on systems (Windows, slim containers) without OS tz database
pyodbc	5.2.0	SQL Server driver	Documented target DB is SQL Server. **Not used** in the default SQLite deployment
gunicorn	26.2.0	Production WSGI server (`gunicorn.conf.py`)	One `gthread` worker with 100 threads
pytest	8.3.4	Backend tests in `backend/tests`	Note: it is also installed in the production image because it is in `requirements.lock`

Database

Environment	Database	Source
Local development	SQLite file (`DATABASE_URL` default `sqlite:///ridez.db`)	`app/__init__.py`
Tests	In-memory SQLite (`sqlite://`) or temp files for concurrency tests	`backend/tests/test_auth.py::app` fixture
Render (Blueprint)	SQLite on persistent disk `sqlite:////var/data/ridez.db`	`render.yaml`

Environment	Database	Source
Intended production	SQL Server via `mssql+pyodbc`	README, `.env.example`. **No validation exists**, and the Docker image does not include Microsoft ODBC Driver 18

External services

Service	Used from	Purpose
Google Routes API (`routes.googleapis.com/directions/v2:computeRoutes`)	Backend `route_provider.py`	Road distance/duration/polyline for quotes and live trip maps
Google Maps JavaScript API (`maps.googleapis.com/maps/api/js`, script libraries `marker`, `geometry`; `places` is loaded on demand with `importLibrary`)	Browser `googleMaps.js`	Map rendering, polyline decoding, Places autocomplete
Google Places API (New)	Browser `AddressSearch.jsx`	`AutocompleteSuggestion.fetchAutocompleteSuggestions`, `place.fetchFields`
jsDelivr CDN	`/api/v1/docs` HTML	Loads Swagger UI assets
Render	Hosting	Docker web service, TLS, persistent disk
GitHub Actions	CI	`.github/workflows/checks.yml`

Details: 11 — Third-party integrations.

Tooling and infrastructure

Tool	File	Purpose
Docker (multi-stage)	`Dockerfile`	Stage 1 `node:22-bookworm-slim` builds the SPA; stage 2 `python:3.12-slim-bookworm` runs Gunicorn as user `ridez` (uid 10001)
Render Blueprint	`render.yaml`	One Docker web service, Starter plan, 1 GB disk at `/var/data`
Native Render scripts	`deploy/render-build.sh`, `deploy/render-start.sh`	Alternative to Docker: Python runtime build/start
GitHub Actions	`.github/workflows/checks.yml`	pytest → npm build → docker build → `deploy/smoke.py`
Smoke test	`deploy/smoke.py`	Starts the real image, checks migrations, login, cookies, booking, cancel, restart persistence, logout
Backup	`deploy/backup.py`	Online SQLite backup with integrity check

How the pieces fit (why this stack)

- **Single origin.** Flask serves the SPA (`web.py`), REST API and Socket.IO from one origin. This is why there is no CORS configuration and why the refresh cookie can be `SameSite=Lax` with `Path=/api/v1/auth`. In development Vite's proxy recreates the same-origin setup.
- **Threading instead of async.** `Flask-SocketIO(async_mode='threading')` + Gunicorn `gthread` keeps everything in ordinary blocking Python, compatible with SQLAlchemy sessions and the `requests` library,

with no monkey-patching. The README states this was deliberately chosen to avoid installing Redis for local Windows development.

- **SQLAlchemy Core-style statements** (`update(...).where(...)`) are central: almost every race-sensitive operation is a conditional UPDATE whose `rowcount` decides who won.

02 — Folder and file structure (file-by-file)

This document explains **every tracked file** in the repository (120 files). For each source file it lists purpose, the important functions with inputs/outputs, dependencies, callers, the tables/APIs touched, and what breaks if it is changed or removed. Deeper walkthroughs of complex logic live in 04 — Frontend and 05 — Backend.

Repository tree

```
ridez.io/
├── .github/workflows/checks.yml     CI pipeline
├── .dockerignore  .gitignore  .node-version  .python-version
├── Dockerfile                       Multi-stage image (SPA build + Python runtime)
├── render.yaml                      Render Blueprint (1 Docker web service + disk)
├── requirements.txt                 Root shim → backend/requirements.lock
├── README.md  TEST_PAYMENTS.md      Human docs
├── backend/
│   ├── .env.example                 Backend env template (no secrets)
│   ├── requirements.txt             Direct dependencies
│   ├── requirements.lock            Full pinned dependency set (used everywhere)
│   ├── run.py                       Dev entry point (Socket.IO dev server + worker)
│   ├── wsgi.py                      Production WSGI entry (Gunicorn)
│   ├── gunicorn.conf.py             Gunicorn config + worker start hook
│   ├── app/                         Flask application package (23 modules)
│   ├── migrations/                  Alembic environment + 13 versions
│   └── tests/                       18 pytest modules
├── frontend/
│   ├── index.html  vite.config.js  package.json  package-lock.json  .env.example
│   ├── src/                         24 JS/JSX/CSS modules
│   └── tests/qr-session.browser.cjs Manual Playwright check
├── deploy/                          start.sh, render-build.sh, render-start.sh, smoke.py,
backup.py
└── docs/                            Topic notes + this knowledge base
```

Untracked/generated directories you may see locally: `backend/instance/` (auto-generated dev secrets `jwt.key`, `codes.key` and the dev SQLite DB — git-ignored), `frontend/node_modules/`, `frontend/dist/`, `backend/frontend_dist/` (copied SPA build for native Render), `backend/.venv*`. All are ignored by `.gitignore`.

Backend: `backend/app/`

The package follows a **routes** → **services** → **repositories** → **models** layering (documented in `docs/PRODUCT_BLUEPRINT.md`: "Keep controllers thin; services own business rules and repositories own persistence"). In practice some services also run queries directly.

`backend/app/__init__.py` — application factory

Purpose. Creates and configures the Flask app. It is the single place where configuration, secrets validation, extensions, blueprints, real-time and CLI commands are wired together.

Why it exists. The factory pattern (`create_app(config)`) lets tests create isolated apps with in-memory databases and lets `run.py` and `wsgi.py` share configuration.

Symbol	Signature / type	Responsibility
`db`	`SQLAlchemy()`	Global Flask-SQLAlchemy handle imported everywhere as `from . import db`
`create_app`	`(config: dict | None = None) -> Flask`	See logic below
CLI `init-db`	`flask --app run init-db`	`db.create_all()`; refuses outside development
CLI `check-routing`	`[--probe]`	Prints route provider and whether the Google server key is set / matches `backend/.env` (never prints the key). `--probe` makes one real `compute()` call
CLI `create-admin`	prompts `--mobile --name --password` (hidden, confirmed)	`services.register(..., role='admin', allow_admin=True)`
CLI `bootstrap-admin`	no args	`bootstrap.bootstrap_admin()` from `BOOTSTRAP_ADMIN_*` env vars

`create_app` **logic, step by step**

1. `load_dotenv(backend/.env, override=False)` — real environment variables always win.
2. `development = APP_ENV == 'development'` (default `development`).
3. In development, if `JWT_SECRET` or `USER_CODE_SECRET` are missing, generate 32-byte hex secrets once into `instance/jwt.key` and `instance/codes.key` (opened with `'x'` mode and `0o600` permissions) and reuse them on restart — so sessions and account codes survive restarts.
4. Set `SQLALCHEMY_DATABASE_URI` (`DATABASE_URL`, default `sqlite:///ridez.db` only in development), `JWT_SECRET`, `USER_CODE_SECRET`.
5. Merge the `config` argument (tests use this).
6. `setdefault` feature flags from env: `TEST_PAYMENTS_ENABLED` and `SIMULATOR_ENABLED` (default on in development, off otherwise), `ROUTE_PROVIDER` (`simulator` in development, `google` otherwise; anything else raises), `GOOGLE_MAPS_SERVER_KEY`, `AUTH_COOKIE_SECURE` (= not development), `AUTH_TRUSTED_ORIGINS` (comma list), `REFRESH_SESSION_DAYS=30`, `LOCATION_STALE_SECONDS=90`, `MATCH_ROUND_SECONDS=30`, `MATCH_MAX_RADIUS_M` (default 5000; must be a multiple of 500 in 500–10000).
7. **Fail fast**: raise `RuntimeError` naming any missing `DATABASE_URL` / `JWT_SECRET` / `USER_CODE_SECRET`. Outside development also require both secrets ≥ 32 chars, distinct, and a Google server key when `ROUTE_PROVIDER=google`.
8. `TRUST_PROXY_HEADERS=1` wraps the WSGI app in `ProxyFix(x_for=1, x_proto=1)` (trust exactly one proxy — Render).
9. `SQLALCHEMY_ENGINE_OPTIONS.pool_pre_ping=True`.
10. `db.init_app`, `Migrate(app, db)`, import `routes`, `ride_routes`, `test_payment_routes` (the latter two **register routes on the same** `api` **blueprint as an import side-effect**), register blueprint at `/api/v1`, `init_realtime(app)`, `register_frontend(app)`.
11. App-level JSON handlers for 404 (`{"error":"Resource not found"}`) and 405.

Used by: `run.py`, `wsgi.py`, every test fixture, `migrations/env.py` (through Flask-Migrate). **If removed/broken:** nothing starts. Changing defaults here changes behaviour globally (e.g. lowering `LOCATION_STALE_SECONDS` makes drivers drop out of matching sooner).

`backend/app/models.py` — database schema (19 SQLAlchemy models)

Purpose. Declares every table, column, primary/foreign key and CHECK constraint, plus `public()` serializers that decide what JSON each entity exposes.

Model (table)	Key methods	Notes
`User` (`users`)	`public() -> dict` adds `vehicle` for drivers	CHECKs `valid_user_role`, `valid_user_status` (suspended only for drivers)
`DriverPresence` (`driver_presence`)	—	`available`, `resume_available`
`FareQuote` (`fare_quotes`)	`public()` → strings for money, `currency:'INR'`, `traffic_available`, `expires_at` + `Z`	Immutable fare snapshot
`FarePolicy` (`fare_policies`)	—	JSON `settings`, manual `version` PK
`Ride` (`rides`)	—	CHECK `valid_ride_state`; vehicle snapshot columns
`ActiveRide` (`active_rides`)	—	PK `user_id` = "one active ride per person"
`Invitation` (`ride_invitations`)	—	PK `(ride_id, driver_id)` = never invite a driver twice for the same ride
`DriverLocation` (`driver_locations`)	—	Latest fix only; `updated_at` indexed
`MatchRound` (`ride_match_rounds`)	—	PK `(ride_id, round_no)`
`OutboxEvent` (`outbox_events`)	—	`delivered_at` indexed
`RevenueLedger` (`revenue_ledger`)	`public()` adds `status:'ALLOCATED'`	PK `ride_id` = allocate once
`RevenueShareSetting` (`revenue_share_settings`)	`public()`	CHECK `valid_revenue_split` (sum = 100)
`AdminAudit` (`admin_audit`)	—	`details` JSON for vehicle edits
`AuthSession` (`auth_sessions`)	—	Revocable server session
`RefreshToken` (`refresh_tokens`)	—	PK = SHA-256 hex of the token
`DriverVehicle` (`driver_vehicles`)	`public()`	Optimistic `version`
`MockPayment` (`mock_payments`)	`public()` adds `test_only: True`	CHECK status
`RidePayment` (`ride_payments`)	`public()`	3 CHECKs incl. "CONFIRMED needs method and confirmed_at"
`RideRating` (`ride_ratings`)	`public(include_comment=True)`	PK `(ride_id, rater_role)`; CHECK stars 1–5

Used by: every service and repository. **If modified:** must be accompanied by an Alembic migration; `/api/v1/ready` returns 503 until the DB is at the new head. Full field reference: 07 — Database.

`backend/app/routes.py` — the `api` Blueprint, auth decorator, core endpoints

Purpose. Creates the `api` Blueprint and the cross-cutting HTTP machinery: JSON error responses, `no-store` caching, JSON body parsing, authentication/authorization decorator, the login/register rate limiter, and auth/health/doc endpoints.

Symbol	Inputs → output	Responsibility
`api`	`Blueprint('api')`	Mounted at `/api/v1`
`no_store(response)`	after_request	Adds `Cache-Control: no-store` to every API response
`api_error(error: ApiError)`	errorhandler	Returns `{"error": message}` with `error.status`; on 401 from `/auth/refresh` or `/auth/logout-all` also **clears the refresh cookie**
`body() -> dict`	request JSON	Raises `ApiError('A JSON object is required')` (400) unless the body is a JSON object
`authenticated(role=None)`	decorator factory	Requires `Authorization: Bearer …`, calls `session_service.authenticate`, sets `g.user`, `g.auth_claims`; 403 if `role` given and differs
`limit_auth()`	before_request	For POST `/auth/login` and `/auth/register`: sliding window of 10 attempts per 60 s per `request.remote_addr`, stored in `app.extensions['auth_attempts']` (in-memory) → 429
`health()`	`GET /health`	`status`, `route_provider`, `matching_worker` (tick within 10 s), `simulator`, `storage` (`sqlite-development` or `sql-server` label)
`registration()`	`POST /auth/register`	`register(body())` → 201 `{user}`
`sign_in()`	`POST /auth/login`	`check_origin(require_header=False)` → `login()` → JSON + Set-Cookie
`me()`	`GET /me`	`{user}`
`driver()`	`GET /driver/overview` (driver)	Summary derived from `snapshot()`
`admin()`	`GET /admin/overview` (admin)	Full admin `snapshot()`
`specification()`	`GET /openapi.json`	Returns `spec.SPEC`
`docs()`	`GET /docs`	Swagger UI HTML loading assets from jsDelivr
`refresh_session()` / `logout_session()` / `logout_all_sessions()`	`POST /auth/refresh`, `/auth/logout`, `/auth/logout-all`	Delegate to `session_service`
`readiness()`	`GET /ready`	200 only if `alembic_version` equals the migration head **and** the worker ticked in the last 10 s; else 503

Imports: `services` (ApiError, login, register); lazily `session_service`, `ride_service`, `spec`, Alembic. **Used by:** `ride_routes.py` and `test_payment_routes.py` import `api`, `authenticated`, `body` from here. **If removed:** every endpoint disappears; the auth decorator is the only authorization gate.

`backend/app/ride_routes.py` — ride, driver and admin endpoints

Purpose. Thin HTTP adapters for rides, drivers, payments, ratings and admin settings. Each handler authenticates, parses the body, calls one service function and wraps the result in JSON.

Handler	Route	Role	Calls
`workspace`	`GET /workspace`	any	`ride_service.snapshot(g.user)`
`quote`	`POST /fares/quotes`	rider	**inline limiter** 10/min per user id (`extensions['quote_attempts']`), then `ride_service.quote` → 201
`book`	`POST /rides`	rider	`ride_service.book` → 201
`details`	`GET /rides/<ride_id>`	any participant/admin	`expire()`, `accessible()`, `public()`
`availability`	`POST /driver/availability`	driver	`ride_service.availability`
`location`	`PUT /driver/location`	driver	`location_service.update` + `repo.commit()`
`accept` / `reject`	`POST /rides/<id>/accept`, `/reject`	driver	`ride_service.accept` / `reject`
`payment_method`	`POST /rides/<id>/payment-method`	rider	`payment_service.choose`
`confirm_cash`	`POST /rides/<id>/confirm-cash`	driver	`payment_service.confirm_cash`
`rate`	`POST /rides/<id>/rating`	rider or driver (403 for admin)	`rating_service.submit`
`progress`	`POST /rides/<id>/<action>`	any (checked in service)	`ride_service.move`; body required for `start`, optional for `cancel`, ignored otherwise
`approve`	`POST /admin/drivers/<int:id>/approve`	admin	`ride_service.approve` → `admin_service.change_driver_status`
`fare_settings` / `update_fare_settings`	`GET` / `PUT /admin/fare-settings`	admin	`fare_service.settings` / `save`
`revenue_share` / `update_revenue_share`	`GET` / `PUT /admin/revenue-share`	admin	`revenue_service.settings` / `save`
`admin_audit` / `admin_audit_detail`	`GET /admin/audit`, `/admin/audit/<category>/<identifier>`	admin	`admin_service.audit` / `audit_detail`
`admin_drivers`	`GET /admin/drivers`	admin	`admin_service.driver_list(request.args)`
`suspend_driver` / `reinstate_driver`	`POST /admin/drivers/<int:id>/suspend`, `/reinstate`	admin	`admin_service.change_driver_status`
`trip_map`	`GET /rides/<id>/map`	rider/driver	`trip_map_service.trip_map`
`update_driver_vehicle`	`PUT /admin/drivers/<int:id>/vehicle`	admin	`admin_service.save_driver_vehicle`

The catch-all `/rides/<ride_id>/<action>` handles `depart`, `arrive`, `start`, `complete`, `cancel`. Werkzeug prefers static segments, so explicit routes like `/accept`, `/map`, `/test-payment` take precedence. **(inferred from Werkzeug routing rules; the tests exercise these routes.)**

Used by: imported for side effects in `create_app`. **If removed:** all ride/admin functionality returns 404.

`backend/app/test_payment_routes.py` — mock QR payment endpoints

Purpose. Lets testers simulate an online payment by scanning a QR code. Never moves money.

Symbol	Inputs → output	Responsibility
`enabled()`	—	404 `Test payments are disabled` unless `TEST_PAYMENTS_ENABLED`
`signer()`	→ `URLSafeTimedSerializer(JWT_SECRET, salt='mock-qr-payment-v1')`	Signs ride IDs
`payment_for(token: str) -> MockPayment`	token	`loads(token, max_age=900)`; 410 if expired/invalid; 404 if no record
`create_test_payment(ride_id)`	`POST /rides/<id>/test-payment` (rider)	Owner + COMPLETED + not already paid by cash; insert `MockPayment` PENDING (race-safe); returns `{payment, path:'/test-payment/<token>', expires_in:900}`
`view_test_payment(token)`	`GET /test-payments/<token>` (no auth)	`{payment}`
`simulate_test_payment(token)`	`POST /test-payments/<token>` (no auth) `{status}`	Conditional update that never downgrades SUCCESS; on SUCCESS calls `payment_service.confirm(ride,'ONLINE')`; notifies rider+driver

Tables: `mock_payments`, `ride_payments`, `rides`, `fare_quotes`, `outbox_events`.

`backend/app/spec.py` — OpenAPI 3.0.3 document

Purpose. Builds the dict served at `/api/v1/openapi.json` (and rendered by `/api/v1/docs`). Defines `ERROR`, `USER` schemas, helper `operation(summary, fields=None, protected=False, status='200') -> dict`, and `SPEC`, which is **mutated incrementally** at import time as features were added (the version ends at `0.8.0`).

Coverage gaps (verified): the spec does not document `/rides/{id}/test-payment`, `/test-payments/{token}`, or response bodies (only `"Success"`). Treat this document set, not the OpenAPI file, as the complete reference.

`backend/app/web.py` — serving the SPA

`register_frontend(app)` adds `GET /` and `GET /<path:path>` handlers that serve files from `FRONTEND_DIST` (`backend/frontend_dist` or `/app/frontend_dist` in Docker).

- Returns JSON 404 `Frontend build is not installed` if the folder does not exist (normal in local dev, where Vite serves the SPA).
- Rejects paths starting with `api` / `socket.io` or containing dot-segments (`.env`, `..`) → JSON 404.
- Unknown files fall back to `index.html` only for SPA routes: `login`, `register`, `rider/*`, `driver/*`, `admin/*`, `test-payment/*`. Everything else → 404.
- `assets/*` → `Cache-Control: public, max-age=31536000, immutable`; other files `no-cache`.
- An app-wide `after_request` adds security headers: `X-Content-Type-Options: nosniff`, `Referrer-Policy` (`no-referrer` on `/test-payment/` so the capability token does not leak via Referer, otherwise `strict-origin-when-cross-origin`), `Permissions-Policy: geolocation=(self)`, `X-Frame-Options: DENY`, and HSTS when the request is HTTPS.

`backend/app/bootstrap.py` — non-interactive admin creation

`bootstrap_admin() -> str`. Reads `BOOTSTRAP_ADMIN_MOBILE`, `BOOTSTRAP_ADMIN_PASSWORD` (required) and `BOOTSTRAP_ADMIN_NAME` (default `Admin`). If the mobile already belongs to a rider/driver → error (never promotes). If an inactive admin exists → error (never reactivates). If an active admin exists → no-op message. Otherwise `register(..., role='admin', allow_admin=True)`. Called by CLI `bootstrap-admin`, run from `deploy/start.sh` / `render-start.sh` when `BOOTSTRAP_ADMIN=1`.

`backend/app/services.py` — auth service, error type, trip OTP

Despite the generic name, this module contains **account and OTP logic** plus the shared exception.

Function	Inputs → output	Responsibility
`ApiError(message: str, status: int = 400)`	exception	The only error type the API layer translates to JSON
`normalize(value) -> str`	raw mobile	`phonenumbers.parse(value, None)` (country code required), validity check, E.164 output; length ≤ 40
`register(data: dict, allow_admin=False) -> User`	name, mobile, password, role	Validation, duplicate check (409), permanent code generation, password hash, insert (IntegrityError → 409)
`login(data: dict) -> (dict, str)`	mobile, password	Normalized generic failure `Mobile or password is incorrect` (401); constant-time-ish via dummy hash; rejects `disabled`; delegates to `session_service.issue`
`trip_otp(ride) -> str`	Ride	`HMAC-SHA256(USER_CODE_SECRET, "trip-otp:v2:{ride.id}") mod 1 000 000`, zero-padded to 6 digits
`verify_trip_otp(ride, driver_id, value) -> None`	Ride, int, str	5 failed attempts per driver per 60 s (in-memory, `extensions['trip_otp_attempts']`) → 429; validates 6 ASCII digits; `hmac.compare_digest`; wrong code → 400

Used by: `routes.py`, `bootstrap.py`, `__init__.py` CLI, `ride_service.py`, every service (for `ApiError`).

`backend/app/repositories.py` — user repository

Four tiny functions on `users`: `by_mobile(mobile) -> User|None`, `by_id(user_id) -> User|None`, `code_exists(code) -> bool`, `save(user) -> User` (add + commit). Used by `services.register/login` and `bootstrap`. (`by_id` is currently unused — verified by search.)

`backend/app/session_service.py` — sessions, tokens, CSRF/origin checks

Purpose. Everything about server-side sessions: issuing JWTs, rotating refresh cookies, verifying bearer tokens, revoking sessions and stopping a driver's work on logout.

Function	Inputs → output	Responsibility
`COOKIE='ridez_refresh'`, `COOKIE_PATH='/api/v1/auth'`	constants	Cookie is only sent to auth endpoints
`now()`	→ naive UTC datetime	

Function	Inputs → output	Responsibility
`digest(value) -> str`	token	SHA-256 hex; only hashes are stored
`check_origin(require_header=True)`	request	403 unless `X-Session-Request: 1` (when required); `Origin`, if present, must equal the host URL or one of `AUTH_TRUSTED_ORIGINS`; `Sec-Fetch-Site: cross-site` → 403
`cookie(response, token=None) -> response`		Sets or deletes the refresh cookie (HttpOnly, SameSite=Lax, Secure per config, max-age 30 days)
`payload(user, session) -> dict`		JWT `{sub, sid, iat, exp}` HS256, `exp = min(now+15 min, session.expires_at)`; returns `access_token, expires_in, expires_at, user`
`new_refresh(session) -> str`		`secrets.token_urlsafe(48)`; stores hash in `refresh_tokens`
`issue(user) -> (dict, str)`		Creates `auth_sessions` row (30 days) + first refresh token, commits
`valid_session(session_id, user_id=None) -> AuthSession | None`		Exists, not revoked, not expired, belongs to user
`authenticate(token) -> (User, claims)`	JWT	Decode with required claims; user exists, not disabled; session valid → else 401 `Please sign in again`
`stop_requests(user_id)`		For drivers: no-op lock UPDATE, `available=False, resume_available=False`, PENDING invitations → `CLOSED`, notify
`revoke(session)`		`revoked_at = now`, `stop_requests`
`refresh() -> (dict, str)`	cookie	Look up hash; invalid → revoke + 401; **conditional** `used_at` update → replay detection revokes session; else issue new refresh token
`logout(user=None, all_devices=False)`		All devices: revoke all user's sessions. Else find session via cookie, falling back to the Bearer token's `sid`. Then `disconnect_invalid()`

Tables: `auth_sessions`, `refresh_tokens`, `users`, `driver_presence`, `ride_invitations`, `outbox_events`.

`backend/app/ride_repository.py` — ride persistence helpers

Purpose. Persistence primitives for rides, invitations, presence, outbox and locks. Most return `bool` from `rowcount == 1` so callers know whether a conditional write won.

Function	Inputs → output	Responsibility
`get(model, key)` / `add(record)` / `commit()` / `rollback()`		Session shortcuts
`notify(*user_ids)`	ints	Inserts one `OutboxEvent` per distinct id **plus one with** `user_id=None` (admins room)
`notify_ride(ride)`	Ride	Notifies rider, every invited driver and the assigned driver
`all_rides(user) -> list[Ride]`		Rider: own; driver: assigned; admin: all. Newest 100
`eligible_drivers(at) -> list[DriverPresence]`	datetime	Available, status active, `fresh` location with `recorded_at` and `updated_at` within `LOCATION_STALE_SECONDS`, and **not** in `active_rides`
`transition(ride_id, old, new, **values) -> bool`		`UPDATE rides SET status=new … WHERE id AND status=old`

Function	Inputs → output	Responsibility
`assign(ride_id, driver_id, now, vehicle=None) -> bool`		Single conditional UPDATE to ASSIGNED that re-checks: ride SEARCHING & unexpired, PENDING unexpired invitation for this driver, driver available+active+fresh; copies vehicle snapshot
`release(ride_id)`		`DELETE FROM active_rides WHERE ride_id`
`close_invitations(ride_id, status='EXPIRED')`		PENDING → status
`expire_round(ride, at) -> bool`		Claims an overdue round by bumping `round_no` only if `round_no` and `expires_at` are unchanged (idempotent across worker/requests)
`round_for(ride)`		`MatchRound` for current round
`expired_searches(now)`		SEARCHING rides with `expires_at <= now`
`invitations(driver_id)`		Rides with a PENDING invitation for the driver and still SEARCHING
`ledger(driver_id)`		Driver's `RevenueLedger` rows
`drivers()`		Newest 100 drivers
`ride_for_quote(quote_id)`		Idempotency lookup
`set_available(driver_id, available, resume_available=None) -> DriverPresence`		Upsert presence
`lock_active_driver(driver_id, require_available=False) -> bool`		**No-op UPDATE** `status_revision = status_revision` on an active driver row — acquires a row/DB write lock so availability, acceptance and suspension serialize; returns False if the driver is not active (or not available when required)
`lock_driver(driver_id) -> bool`		Same lock regardless of status

`backend/app/ride_service.py` — ride business logic

Purpose. The core domain service: quotes, bookings, availability, acceptance, rejection, trip transitions, cancellation and the workspace snapshot.

Function	Inputs → output	Responsibility
`now()` / `money(value)`		UTC now; Decimal rounded half-up to 0.01
`simulator()`		503 if provider is `simulator` and `SIMULATOR_ENABLED` is off
`text(data, key, limit=160) -> str`		Required trimmed string of 1..limit chars
`quote(user, data) -> dict`	rider, body	Validate → `expire()` → `route_provider.compute` → `fare_service.calculate` → insert `FareQuote` (5 min) → `public()` + `polyline`
`expire()`		`matching.advance()` then `db.session.expire_all()` — every request catches up overdue deadlines itself
`accessible(user, ride_id) -> Ride`		Participant or admin, else 404 (not 403, to avoid revealing existence)
`public(ride, user, invitation=False) -> dict`		Role-aware ride JSON: quote, driver (name, code, vehicle snapshot, rating), rider (name, rating), test/real payment, cancellation, my_rating (participants), ratings (admin), settlement (driver/admin only, not in invitations)

Function	Inputs → output	Responsibility
`book(user, data) -> dict`	`{quote_id}`	Idempotent booking + first matching round
`availability(user, data) -> dict`	`{available, latitude…}`	Go online/offline; stores next-trip preference while busy
`accept(user, ride_id) -> dict`		Guarded conditional assignment
`reject(user, ride_id) -> dict`		PENDING → REJECTED (idempotent)
`cancellation(ride, user) -> dict`		`{by, by_you, reason, cancelled_at}`
`move(user, ride_id, action, data=None) -> dict`		`cancel` and driver transitions `depart/arrive/start/complete`; completion side-effects
`snapshot(user) -> dict`		The `/workspace` payload (role-specific)
`approve(user, driver_id, data)`		Delegates to `admin_service.change_driver_status(..., 'approve', ...)`

Walkthroughs: 05 — Backend.

`backend/app/matching.py` — expanding-radius matching

Function	Responsibility
`invite(ride, started_at)`	Records a `MatchRound`; if the round is still open and the quote has pickup coordinates, invites every eligible driver within `ride.radius_m` (haversine) who was **not invited before** for this ride, after `lock_active_driver(require_available=True)` succeeds
`advance()`	Up to `MATCH_MAX_RADIUS_M//500 + 1` catch-up passes: claim each overdue round, expire its invitations, then either end in `NO_DRIVER_FOUND` (max radius reached) or expand radius by 500 m with a new deadline = old deadline + 30 s and invite again. Finally marks stale driver locations `fresh=False` and notifies those drivers
`close_round(ride, outcome)`	Sets the current round's `outcome` (`ASSIGNED`, `CANCELLED`)

Called from `ride_service.expire/book/accept/move` and `realtime.tick`.

`backend/app/location_service.py` — GPS validation

Function	Inputs → output	Responsibility
`utcnow()`		Naive UTC now (used across modules)
`coordinate(value, name, bound) -> float`	number	Finite, not bool, `abs ≤ bound`
`distance_m(lat1, lng1, lat2, lng2) -> float`		Haversine metres (Earth radius 6 371 000 m)

Function	Inputs → output	Responsibility
`is_fresh(location, at=None) -> bool`		`fresh` flag and both `recorded_at` and `updated_at` newer than 90 s
`public(location) -> dict`		`{fresh, recorded_at, accuracy_m}` — **no coordinates**
`update(user, data, initializing=False) -> dict`		Status/permission rules, coordinate/accuracy/time validation, ordering, implausible-movement check, replay no-op, upsert, notify

`backend/app/route_provider.py` — routing adapter

Function	Responsibility
`denial_reason(response) -> str | None`	Extracts an allow-listed Google error reason (e.g. `API_KEY_INVALID`, `BILLING_DISABLED`) without leaking other vendor text
`seconds(value) -> int`	Parses Google duration strings like `"1234.5s"` → ceil seconds; must be 1..604 800
`compute(pickup, destination, labels) -> dict`	Returns `{provider, distance_m, duration_seconds, baseline_seconds, polyline}`. Simulator: deterministic from SHA-256 of labels. Google: `POST computeRoutes`, timeout (3 s connect, 10 s read), no redirects; maps 429/401/403/other/timeout/connection errors to specific `ApiError`s; validates every field

Used by `ride_service.quote`, `trip_map_service`, CLI `check-routing`.

`backend/app/fare_service.py` — fare policy and calculation

Symbol	Responsibility
`DEFAULT_POLICY`	₹40 base, ₹12/km, ₹2/min, ₹40 minimum, `Asia/Kolkata`, all surge off, cap 2.00×, traffic threshold 1.25 / multiplier 1.25, demand threshold 2.00 / multiplier 1.25 / min 5 requests, no zones
`current() -> (version, policy)`	Latest `FarePolicy` or `(0, DEFAULT_POLICY copy)`
`decimal_field(data, key, low='0', high='100000') -> str`	Strict 2-dp decimal validation
`validate(data) -> dict`	Validates a full policy incl. ≤ 20 zones (id, name, lat/lng, radius 100–50 000 m, multiplier 1–5, distinct weekdays 0–6, HH:MM start/end, enabled)
`settings() -> dict`	`{version, policy, history[≤20]}`
`save(user, data) -> dict`	Optimistic version check, insert `version+1`, notify admins
`scheduled(zone, local) -> bool`	Day/time window incl. overnight windows and all-day (`start == end`)
`demand_signal(pickup, at) -> (requests, ratio)`	Open SEARCHING requests within 1 km ÷ max(1, fresh eligible drivers within 1 km)
`calculate(route, pickup, at=None) -> dict`	Itemized fare; highest eligible multiplier capped at `max_multiplier`; reason text; `rate_snapshot` JSON; 422 if total > 99 999 999.99

`backend/app/revenue_service.py` — revenue split

Function	Responsibility
`active(at=None) -> dict`	Split in force at `at` (latest `effective_from ≤ at`, ties → highest version) or default `{version:0, 85.00/15.00}`

Function	Responsibility
`revision() -> int`	Highest version (0 if none)
`settings() -> dict`	`{version, active, upcoming[], history[≤100], server_time}`
`percentage(value) -> Decimal`	0–100, ≤ 2 dp
`save(user, data) -> dict`	Version check, sum = 100, reason, optional future `effective_from` (tz-aware, ≤ 366 days), records previous split
`allocate(ride, quoted, driver_id, completed_at) -> RevenueLedger`	`driver_amount = round_half_up(total × pct / 100)`, `company_amount = total − driver_amount`

`backend/app/payment_service.py` — per-ride payment

Function	Responsibility
`record_completion(ride, quoted, at)`	Insert `RidePayment` PENDING, amount = quote total
`notify(ride)`	Outbox events for rider + driver
`completed_ride(ride_id) -> (Ride, RidePayment)`	409 unless ride COMPLETED with a payment row
`choose(user, ride_id, data) -> dict`	Rider sets `CASH` / `ONLINE` while PENDING; ONLINE needs test payments enabled; idempotent
`confirm(ride_id, method, by=None) -> int`	Conditional `PENDING → CONFIRMED`; returns rowcount (first wins)
`confirm_cash(user, ride_id) -> dict`	Assigned driver confirms cash; 409 if already paid online
`confirmed_by_other_method(ride_id) -> bool`	Confirmed with a method other than ONLINE

`backend/app/rating_service.py` — ratings

`TAGS` (rider→driver tags and driver→rider tags), `summary(user_id) -> {average: '4.5'|None, count}`, `given(ride_id, role)`, `for_admin(ride_id) -> list` (with comments), `submit(user, ride_id, data) -> dict` (validation, final once submitted, idempotent same-stars retry).

`backend/app/admin_service.py` — admin operations

Function	Responsibility
`metrics() -> dict`	Ride counts by status, fresh vs stale available drivers, pending/suspended drivers, all-time allocation sums
`events()`	`UNION ALL` subquery normalizing `admin_audit`, `fare_policies`, `revenue_share_settings` into `{id, category, action, actor_id, created_at, reason, target}`
`audit(params) -> dict`	Paginated (20/page, page ≤ 100 000), category filter, escaped `ILIKE` search over reason/admin name/action
`audit_detail(category, identifier) -> dict`	Human-readable before/after rows (status, vehicle, revenue split, fare policy diff)
`DRIVER_TRANSITIONS`	`approve: pending→active`, `suspend: active→suspended`, `reinstate: suspended→active`
`change_driver_status(actor, driver_id, action, data) -> dict`	Versioned, idempotent status change + audit + `stop_requests`
`driver_list(params) -> dict`	Paginated filterable driver list with location freshness, active ride, rating

Function	Responsibility
`save_driver_vehicle(actor, driver_id, data) -> dict`	Versioned vehicle upsert + audit

`backend/app/realtime.py` — Socket.IO and the background worker

Symbol	Responsibility
`socketio`	`SocketIO(async_mode='threading')`
`init_realtime(app)`	Binds Socket.IO; registers `connect` (authenticate `auth.token`, join `user:{id}` and `admins` for admins; reject otherwise) and `disconnect` handlers; tracks connections in `extensions['ride_connections']`
`disconnect_invalid()`	Drops sockets whose JWT expired, user is disabled/missing, or session is revoked
`dispatch()`	Emit up to 200 undelivered outbox events as `workspace.changed`, mark delivered, prune delivered events older than 7 days
`tick(app)`	`advance()` + `dispatch()` in an app context; records `matching_last_tick`
`start_worker(app)`	Starts the 1-second loop once per process

`backend/app/trip_map_service.py` — live trip map

`ACTIVE` and `APPROACH` status sets; `phase_for(ride)`; `source_for(phase, driver)`; `cached_route(key, compute)` (thread-safe in-memory cache with `concurrent.futures.Future` to coalesce concurrent Google calls, 60 s TTL for success / 10 s for errors, ≤ 256 entries, 15 s wait timeout); `position(ride)` (fresh driver coordinates or None); `trip_map(user, ride_id) -> dict`.

Backend entry points and configuration

File	Purpose	Details
`backend/run.py`	**Development** entry	`create_app()`, `start_worker(app)`, `socketio.run(host='0.0.0.0', port=$PORT or 5000, use_reloader=False, allow_unsafe_werkzeug=True)`. Also the `--app run` target for Flask CLI commands locally
`backend/wsgi.py`	**Production** WSGI entry	`app = create_app()`; worker startup is left to Gunicorn's hook
`backend/gunicorn.conf.py`	Gunicorn settings	`bind 0.0.0.0:$PORT (10000)`, `workers = 1` (comment: matching loop and socket dispatcher require exactly one process), `worker_class = 'gthread'`, `threads = 100`, `preload_app = False`, `timeout 60`, `graceful_timeout 30`, `keepalive 5`, `accesslog = None` (don't log URLs with Socket.IO session IDs), `post_worker_init` → `start_worker(worker.wsgi)`
`backend/.env.example`	Template	Variable names and comments only; copy to `backend/.env`
`backend/requirements.txt`	Direct deps	12 packages

File	Purpose	Details
`backend/requirements.lock`	Full pin set (35 packages)	Used by Docker, CI, README and root `requirements.txt`

Backend migrations: `backend/migrations/`

File	Purpose
`env.py`	Standard Flask-Migrate Alembic environment; uses the app's engine and metadata; suppresses empty autogenerate revisions
`alembic.ini`	Logging configuration only
`script.py.mako`	Template for new revisions
`README`	One line: "Single-database configuration for Flask."
`versions/*.py`	13 linear revisions, head `c51e8a2f90d4`. Full table in 07 — Database

Backend tests: `backend/tests/`

All tests use pytest with the `app` / `client` fixtures from `test_auth.py` (in-memory SQLite, simulator mode, fixed test secrets) and the `actors` fixture from `test_rides.py` (two riders, two drivers, one admin, all logged in). Concurrency tests create file-backed SQLite databases under `tmp_path` and race real threads with `threading.Barrier`. Google calls are mocked with `unittest.mock.patch('app.route_provider.requests.post')`.

Module	What it verifies
`test_auth.py`	Registration, E.164 duplicate detection, validation, generic login errors, disabled accounts, role authorization, admin provisioning only via CLI, invalid/expired tokens, auth rate limit, JSON validation, docs endpoints
`test_sessions.py`	Refresh rotation and hashing, replay revocation, logout vs logout-all, CSRF/origin checks, cross-site login rejection, expired access refresh, driver logout stopping matching but keeping the active ride, Secure cookie in production, concurrent refresh with one winner, persistence across app instances, Bearer fallback logout
`test_rides.py`	Full ride + exactly-once settlement, quote validation/ownership/expiry/idempotent booking, invitation expiry/reject/cancel, permissions, simulator disabled, concurrent acceptance has one winner
`test_matching.py`	Location validation, replay/implausible movement, stale exclusion, radius expansion invites only new drivers, worker recovery of overdue rounds, socket auth/scoping/reconnect, socket expiry disconnect, outbox retry, recovery from a new app instance
`test_cancellation.py`	Pre-trip cancellation for each phase/actor, reasons, mid-trip rejection, notice expiry, race outcomes, expired search not relabelled
`test_driver_auto_availability.py`	Staying available after completion, offline preference, fresh-GPS requirement, ineligible conditions, retries, offline vs completion race
`test_driver_review.py`	Suspend/reinstate, trip preservation, version/reason validation, audit immutability, search/pagination with literal wildcards, concurrent admins, suspension vs availability/acceptance races
`test_vehicle_details.py`	Vehicle save/snapshot, history immutability, validation, admin-only audit, missing vehicle explicit, edit vs acceptance consistency

Module	What it verifies
`test_google_fares.py`	Google request shape, no fallback on errors, connection failures, missing key, malformed responses, policy validation/conflicts, surge overlap/cap/minimum, timezone/overnight zones, traffic and demand surge, quote rate limit, denial reason handling, `check-routing` CLI
`test_trip_map.py`	Route restore for both roles without repricing, phase switches, stale fallback, participant-only access, errors, no driver coordinates after completion, cache expiry and thread coalescing, approach routing
`test_revenue.py`	Defaults/permissions, invalid splits, effective-dated snapshots, boundaries, rounding extremes, same-timestamp ordering, concurrent admins
`test_ride_payments.py`	Pending payment at completion, cash flow, online success blocking cash and vice-versa, permissions, unavailable before completion
`test_mock_payments.py`	Mock QR idempotent success, financial isolation, access/expiry/disable, notifications
`test_ratings.py`	Two-way averages, comment privacy, finality/validation, participant-only, rider rating shown on invitations
`test_trip_otp.py`	Per-ride private code, required at start, previous ride code rejected, attempt limit, leading zero
`test_admin_privacy.py`	Riders never receive the split, admin metrics, audit access/filters/details/pagination
`test_bootstrap.py`	Bootstrap creates admin once, never logs credentials, never promotes/reactivates, names missing settings
`test_deployment.py`	SPA deep links and API boundaries, readiness, production config validation, proxy/secure cookie/origin/headers

---

Frontend: `frontend/`

Configuration files

File	Purpose
`package.json`	Name `ridez-web`, `type: module`. Scripts: `dev` = `vite --host 0.0.0.0`, `build` = `vite build`, `preview` = `vite preview --host 0.0.0.0`
`package-lock.json`	Exact npm dependency tree (used by `npm ci` in Docker/CI)
`vite.config.js`	React plugin; dev proxy `/api` → `API_PROXY_TARGET` (default `http://127.0.0.1:5000`, `changeOrigin:false` so the Host/Origin match for session origin checks) and `/socket.io` with `ws:true`
`index.html`	Root `<div id="root">`, loads `/src/main.jsx`, theme colour, title
`.env.example`	`VITE_GOOGLE_MAPS_BROWSER_KEY`, `VITE_GOOGLE_MAP_ID` (copy to `.env.local`). Warns never to put the server key in a `VITE_` variable

`frontend/src/main.jsx` — app shell, routing, auth context

Component / symbol	Responsibility
`Auth` context + `AuthProvider`	Holds `session`, `ready`, `restoreError`, `retryRestore`, `logout`, `signingOut`, `logoutError`. On mount subscribes to `authSession` and calls `refreshSession()` to restore from the cookie. Schedules a renewal 60 s before `expires_at` (min 1 s), retrying every 15 s on failure
`SessionLoading`, `Brand`, `Tag`, `PageHeading`	Presentational helpers

Component / symbol	Responsibility
`Landing`, `CityMap`	Public marketing page with an illustrative SVG map (explicitly not live)
`AuthPage({register})`	Login/register form. Registration: `api('/auth/register')` and shows the permanent code. Login: `signIn(fields)` then `navigate('/{role}')`. Role preselected from `?role=driver`
`Protected({role})`	Waits for restore; shows restore error with retry; redirects to `/login` if no session, or to own workspace on role mismatch
`navItems`	Sidebar items per role
`Workspace({role})`	Sidebar + nested routes (see diagrams/frontend.md)
`Account({user})`	Profile fields + "Sign out all devices"
`ComingSoon`	Fallback for unknown sub-routes
`App`	Top-level routes

If removed: the app does not render. **Calls:** `/auth/register`, plus everything via `authSession`.

`frontend/src/api.js` — HTTP client

`api(path, {token, ...fetchOptions}) -> Promise<object>`:

1. `resolveToken(token)` maps a possibly-stale token captured by a component to the **current** session token (throws if the token never belonged to this session — prevents using another user's token after a switch).
2. `fetch('/api/v1' + path, {credentials:'same-origin', headers:{'Content-Type':'application/json', Authorization:'Bearer …'}})`.
3. On 401 (and not aborted): use the already-renewed session if another request refreshed it, otherwise `refreshSession()`, then retry once.
4. Parse JSON (error if unreadable); throw `Error(data.error)` if `!response.ok`; return data.

Used by every component that talks to the backend except the auth calls in `authSession.js`.

`frontend/src/authSession.js` — session store

Module-level store: `session`, `pendingRefresh`, `generation`, `signingOut`, `listeners`, `knownTokens`, and a `BroadcastChannel('ridez-auth')`.

Export	Responsibility
`currentSession()`	Current `{access_token, expires_in, expires_at, user}` or null
`subscribeSession(listener) -> unsubscribe`	Used by `AuthProvider`
`resolveToken(token)`	See `api.js`
`refreshSession() -> Promise`	Single-flight (`pendingRefresh`), cross-tab serialized with `navigator.locks.request('ridez-refresh-cookie')`, ignores results from an older `generation`, publishes null on 401
`signIn(fields)`	`POST /auth/login` under the lock; bumps generation; publishes
`signOut(allDevices=false)`	For all devices: refresh first (to get a valid Bearer), then `POST /auth/logout-all`; otherwise `POST /auth/logout`; publishes null; broadcasts `signed-out` to other tabs

Internal `request(path, body, token)` always sends `X-Session-Request: 1` (required by the backend CSRF check).

`frontend/src/RideWorkspace.jsx` — the main role workspace

The largest component; renders rider, driver and admin dashboards from one `/workspace` snapshot.

Symbol	Responsibility
`currency`, `labels`, `actions`	INR formatter; status labels; next driver action per status
`useWorkspace(token)`	Opens an authenticated Socket.IO connection (`io({auth:{token}, transports:['polling','websocket']})`); on `workspace.changed` dedupes `event_id` (last 100) and debounces refresh by 80 ms; polls `/workspace` every 15 s when connected or 2 s when not; exposes `data, error, refresh, connection, updateAvailability`
`Countdown({deadline})`	Seconds remaining, ticking every 500 ms
`QuoteDetails({quote})`	Itemized fare rows (base, distance, time, minimum adjustment, surge) and provider description
`RideCard(...)`	One ride: status, cancellation notice, assigned details or quote, search progress, OTP input (driver at WAITING), next driver action button (with confirm before complete), cancel control, settlement (driver/admin), payment status, rating form (within 7 days), admin ratings, test payment card
`ActiveRidePanel(...)`	`useTripRoute` shared between `TripStatsCard` and `TripRouteMap` (Google rides only)
`RATING_WINDOW_MS`	7 days (frontend-only rule)
`paymentDismissed` / `rememberPaymentDismissed`	`sessionStorage` key `ridez.payment-dismissed.{id}`
`usePaymentPrompt(role, data)`	For riders: opens the payment modal when the active ride disappears and turned into a COMPLETED/PENDING-payment ride, or on first load if the latest ride completed < 30 min ago
`RideWorkspace({role, user, token, view})`	Composes everything; `perform(path, body)` is the single POST helper (busy lock, error display, availability patch, refresh); `changeDriverAvailability(available)` gets a GPS fix first when going online

`frontend/src/BookingPanel.jsx`

Rider booking form. State: `form` (labels and coordinate strings), `quote`, `error`, `locating`. `change` clears coordinates when an address label is edited in Google mode; `select(prefix, point)` sets coordinates (from AddressSearch, map clicks or "Use my location"); `submit` validates required coordinates (both in Google mode, pickup only in simulator mode) and calls `perform('/fares/quotes', payload)`; "Confirm ride" calls `perform('/rides', {quote_id})`. Renders `GoogleRouteMap` (picker mode with the quote polyline) in Google mode.

`frontend/src/AddressSearch.jsx`

Accessible combobox for Google Places (New) autocomplete. Debounces 300 ms, needs ≥ 3 chars, uses an `AutocompleteSessionToken` per search session (billing best practice), biases results to 50 km around the pickup, shows ≤ 5 suggestions, supports Arrow/Enter/Escape keys, and on choice calls

`placePrediction.toPlace().fetchFields(['formattedAddress','location'])`. Uses sequence counters to drop stale async results. Shows "Powered by Google" attribution.

`frontend/src/googleMaps.js`

`loadMaps() -> Promise<google.maps>` loads the Maps JS script once (`v=quarterly`, `libraries=marker,geometry`, `loading=async`, callback `__ridezGoogleReady`), with a 15 s timeout, `gm_authFailure` hook that dispatches `ridez.maps.error`, and clear errors when `VITE_GOOGLE_MAPS_BROWSER_KEY` is missing.

`frontend/src/GoogleRouteMap.jsx`

Renders a Google map (default centre Mumbai 19.076, 72.8777). Props `pickup`, `destination`, `polyline`, `driverPosition`, `onSelect`. With `onSelect` it is a **picker** (toggle pickup/destination, click to set). Draws `P`/`D` `AdvancedMarkerElement`s, decodes the encoded polyline with `geometry.encoding.decodePath`, fits bounds, and moves a 🚕 driver marker. Retry button re-initializes on error.

`frontend/src/TripRouteMap.jsx`

`tripPhase(ride)` (`approach`/`in_progress`/`overview`), `useTripRoute(ride, token)` (polls `/rides/{id}/map` at `refresh_after_seconds`, 15 s after errors; keyed by ride id + phase), and `TripRouteMap` (headings per phase/role, refresh button, error text, summary of driver location time).

`frontend/src/TripStatsCard.jsx`

`googleNavigationUrl(place, lat, lng)` builds a `https://www.google.com/maps/dir/?api=1&destination=…&travelmode=driving&dir_action=navigate` link. `liveRoute()` turns map data into minutes/km/arrival time only when the route originates from the driver and targets the expected point. `TripStatsCard` shows phase-specific metrics (approach, waiting, in-progress) and a "Navigate in Google Maps" button for drivers.

`frontend/src/useDriverLocation.js`

`getPosition()` — requires a secure context; tries high accuracy (15 s), then low accuracy (20 s) unless permission was denied; maps error codes 1/2/3 to friendly messages; returns `{latitude, longitude, accuracy_m, recorded_at}`. `useDriverLocation({token, enabled, available, onTrip})` — sends `PUT /driver/location` immediately then every 30 s (10 s on a trip); watches the Permissions API and, if permission is denied/revoked, posts `available:false` to stop new requests while keeping any active trip.

Smaller components

File	Responsibility	Backend calls
`AssignedRideDetails.jsx`	Prominent card after acceptance: other person + rating, car plate/model/colour (or explicit "not provided"), fare, rider OTP before start, pickup/destination	—
`CancelRideControl.jsx`	Two-step cancel with role-specific preset reasons, "Other" requires text, max 200 chars typed; builds `"Reason — notes"`	`POST /rides/{id}/cancel` via `perform`
`CancellationNotice.jsx`	Who cancelled (you / driver / rider), reason, time, "no payment or refund" note	—

File	Responsibility	Backend calls
`DriverManagement.jsx`	Admin driver list with status filter, search, pagination, approve/suspend/reinstate review form with reason, vehicle editor launch	`GET /admin/drivers`, `POST /admin/drivers/{id}/{action}`
`VehicleEditor.jsx`	Car number/model/colour + reason form	`PUT /admin/drivers/{id}/vehicle`
`FareSettings.jsx`	Edit all policy fields, surge zones (add/remove, weekdays, times), reason; shows history	`GET` / `PUT /admin/fare-settings`
`RevenueSettings.jsx`	Driver/company percentages (must total 100), optional scheduled date (browser time → UTC ISO), reason; upcoming and history	`GET` / `PUT /admin/revenue-share`
`AdminAudit.jsx`	Category/search filters, pagination, expandable before/after detail	`GET /admin/audit`, `GET /admin/audit/{category}/{id}`
`OtpInput.jsx`	Six single-digit inputs with paste, backspace, arrow/home/end navigation; reports the joined string	—
`PaymentStatus.jsx`	`paymentLabel`, `PaymentStatusLine`, `DriverCollectCard` ("Cash received"), `usePaymentConfirmedFlash` (detects PENDING→CONFIRMED transitions), `PaymentConfirmedFlash`	`POST /rides/{id}/confirm-cash` via `perform`
`Rating.jsx`	`RatingTags` context (tags come from the server), `RatingBadge`, `StarsShown`, `RatingForm` (stars, sentiment-filtered tags, comment)	`POST /rides/{id}/rating`
`TestPayment.jsx`	`useTestQr` (create QR), `TestPaymentCard` (in ride cards), `TestPaymentPage` (public `/test-payment/:token` page), `PaymentSuccess`, `PaymentPrompt` (rider modal: choose cash/online, QR, failure retry, success + rating)	`POST /rides/{id}/test-payment`, `GET/POST /test-payments/{token}`, `POST /rides/{id}/payment-method`
`styles.css`	All styling (single global stylesheet)	—

`frontend/tests/qr-session.browser.cjs`

Manual Playwright script (not in CI) that mocks the API, signs in, renders `RideWorkspace` in history view, and asserts that generating a test QR with an expired access token triggers exactly one refresh and a retry with the new token. It hard-codes default paths (`/opt/codex/...playwright`, `/usr/bin/chromium`) overridable by `PLAYWRIGHT_MODULE` / `CHROMIUM_PATH`, and requires a running Vite server at `RIDEZ_TEST_URL`.

Root and deployment files

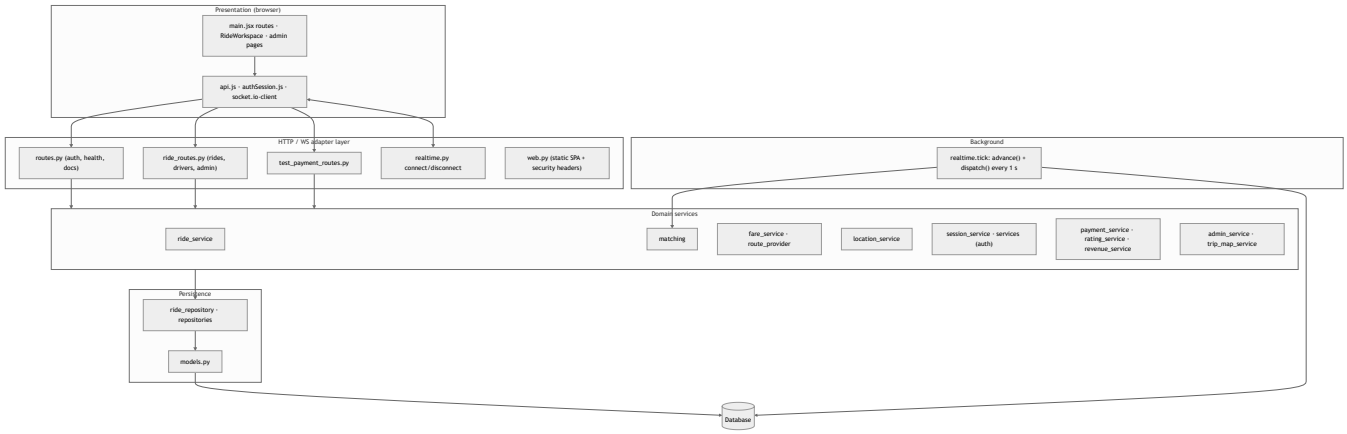
File	Purpose
`Dockerfile`	Stage `frontend`: `npm ci` (optional proxy CA secret), build with `VITE_GOOGLE_MAPS_BROWSER_KEY` / `VITE_GOOGLE_MAP_ID` build args. Stage `runtime`: Python 3.12 slim, `libodbc2`, `ca-certificates`, `gosu`, user `ridez` (10001), install `requirements.lock`, copy `app`, `migrations`, `wsgi.py`, `gunicorn.conf.py`, `start.sh`, `backup.py`, SPA into `frontend_dist`; `EXPOSE 10000`; `HEALTHCHECK` calls `/api/v1/ready`; `ENTRYPOINT /app/start.sh`, `CMD gunicorn --config gunicorn.conf.py wsgi:app`
`.dockerignore`	Excludes `.git`, `node_modules`, venvs, caches, `.env*`, `instance`, `*.db`, `*.key`, `*.pem`, `dist` — keeps secrets and local DBs out of images
`.gitignore`	Same idea for Git; keeps `.env.example`

File	Purpose
`.node-version` / `.python-version`	`22` / `3.12.12`
`render.yaml`	Render Blueprint (see 15 — Deployment)
`requirements.txt` (root)	`-r backend/requirements.lock` for Render Python services that build from the repo root
`deploy/start.sh`	Docker entrypoint: as root, `chown /var/data` then re-exec as `ridez` via `gosu`; when the command is `gunicorn`, run `flask db upgrade`, optional `bootstrap-admin`, unset `BOOTSTRAP_ADMIN*`, then `exec`
`deploy/render-build.sh`	Native Render build: pip install, check npm exists, `npm ci --include=dev`, `npm run build`, copy `frontend/dist` → `backend/frontend_dist`
`deploy/render-start.sh`	Native Render start: verify SPA build, `db upgrade`, optional bootstrap, unset bootstrap vars, `exec gunicorn`
`deploy/smoke.py`	Container end-to-end smoke test (used in CI)
`deploy/backup.py`	`python backup.py <new-file>`: SQLite online backup via `sqlite3.backup`, refuses overwrite, `PRAGMA integrity_check`
`.github/workflows/checks.yml`	CI on push to `main` and PRs
`README.md`	Setup, run, behaviour notes
`TEST_PAYMENTS.md`	Mock QR testing guide
`docs/*.md`	Older topic notes: product blueprint, screen flows, sessions, cancellations, driver availability/management, vehicle details, admin monitoring, address search & revenue, Google Maps setup, Render deployment

03 — System architecture

1. Architectural style in one paragraph

ridez.io is a **modular monolith**: one Python process hosts the REST API, the WebSocket server, the background matching worker and (in production) the static SPA. Inside the process, code is layered as **HTTP adapters** → **domain services** → **repositories** → **SQLAlchemy models**. The database is the single source of truth for every piece of business state, including matching deadlines and outgoing notifications, so the process can restart at any time and catch up. Correctness under concurrency is achieved with **conditional UPDATE statements** and **primary-key uniqueness**, not with in-memory locks. The browser treats every push message as a hint and re-reads one authoritative snapshot (`GET /workspace`).



2. Frontend architecture

- **Single bundle SPA** built by Vite; entry `frontend/src/main.jsx`.
- **Routing**: React Router with role-prefixed trees `/rider/*`, `/driver/*`, `/admin/*` guarded by `Protected`, plus public `/`, `/login`, `/register`, `/test-payment/:token`.
- **Session**: a module-level store (`authSession.js`) holds the access token **in memory only**. The refresh credential is an HttpOnly cookie the JS cannot read. `AuthProvider` exposes it via React context.
- **Data**: one hook, `useWorkspace(token)`, owns the role snapshot. All mutations go through `perform()` (POST) or direct `api()` calls in admin pages, then trigger a refresh.
- **Real-time**: one Socket.IO connection per mounted workspace, authenticated with the access token.
- **Maps**: Google Maps JS loaded lazily by `googleMaps.js` only when a map or address search mounts.

Details: 04 — Frontend, 12 — State management.

3. Backend architecture

Layer	Modules	Rules followed
Entry	`run.py` (dev), `wsgi.py` + `gunicorn.conf.py` (prod)	Exactly one process
App factory	`app/__init__.py`	Config + validation + wiring

Layer	Modules	Rules followed
Adapters	`routes.py`, `ride_routes.py`, `test_payment_routes.py`, `web.py`, `realtime.py` handlers	Parse request, enforce authentication/role, call one service, `jsonify`
Services	`ride_service`, `matching`, `fare_service`, `route_provider`, `location_service`, `session_service`, `services`, `payment_service`, `rating_service`, `revenue_service`, `admin_service`, `trip_map_service`	Validation, business rules, transactions (commit/rollback), outbox notifications
Repositories	`ride_repository`, `repositories`	Query helpers and conditional writes
Models	`models.py`	Schema + `public()` serializers

Transaction ownership: **services commit**. Repositories never commit except `repositories.save`. Adapters commit only in one place (`ride_routes.location` calls `repo.commit()` after `location_service.update`).

Details: 05 — Backend.

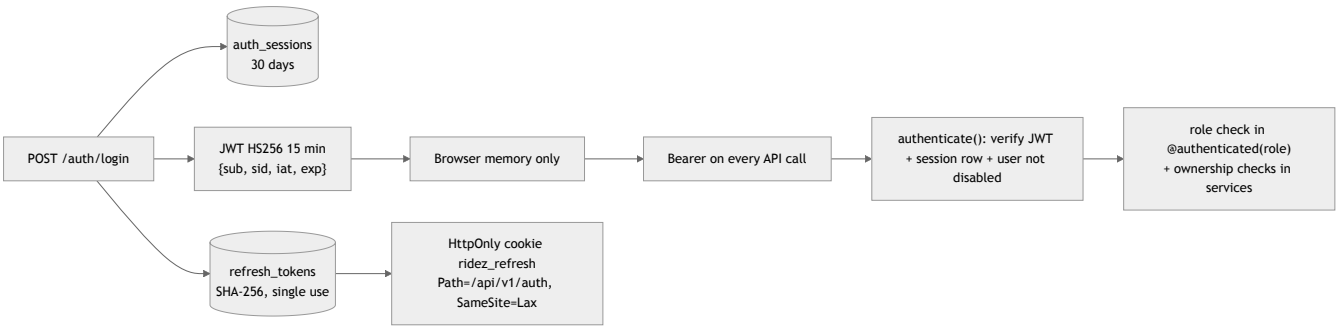
4. Database architecture

- 19 tables (see 07 — Database), all created by Alembic.
- Natural "one-row" tables keyed by user (`driver_presence`, `driver_locations`, `driver_vehicles`, `active_rides`) and by ride (`revenue_ledger`, `ride_payments`, `mock_payments`).
- Append-only/versioned history tables: `fare_policies`, `revenue_share_settings`, `admin_audit`, `ride_match_rounds`.
- Timestamps are stored as **naive UTC** (`datetime.now(timezone.utc).replace(tzinfo=None)`) and serialized with a trailing `Z`. Exception: `users.created_at` is declared timezone-aware.
- Money is `Numeric(10,2)` and handled as Python `Decimal` with `ROUND_HALF_UP`; serialized as strings (`"352.08"`) to avoid float rounding in JSON.

5. API architecture

- REST over JSON under `/api/v1`, one Flask Blueprint (`api`).
- Authentication by `Authorization: Bearer <JWT>` for all business endpoints; cookie-based only for `/auth/refresh`, `/auth/logout`, `/auth/logout-all`.
- Uniform error envelope: `{"error": "<human message>"}` with an HTTP status (400, 401, 403, 404, 409, 410, 422, 429, 502, 503, 504).
- Idempotency: booking keyed by `quote_id`; accept, transitions, cancel, reject, payment choice, cash confirmation, rating (same stars), driver review and mock payment all return the current state on exact retries.
- Optimistic concurrency with explicit `version` fields: fare policy, revenue share, driver status (`status_revision`), vehicle.
- Caching: every API response is `Cache-Control: no-store`.
- Documentation: `/api/v1/openapi.json` and `/api/v1/docs` (partial; see 06 — API).

6. Authentication and authorization architecture



Two layers of authorization:

1. **Role gate** in `routes.authenticated(role)` (e.g. only riders may quote/book).
2. **Ownership/state checks** inside services (`ride_service.accessible` , `payment_service.choose` checks `ride.rider_id` , `move` checks assigned driver, etc.). Not-owned resources return **404** to avoid revealing their existence.

Details: 08 — Authentication and security.

7. External service integrations

Integration	Direction	Trigger
Google Routes `computeRoutes`	Server → Google	Every quote in Google mode; trip map refresh (cached 60 s per ride/phase/source/target)
Google Maps JS + Places (New)	Browser → Google	Booking map, address search, trip map
Browser Geolocation	Browser	Driver availability/location loop; rider "Use my location"

8. Communication between modules (who calls whom)

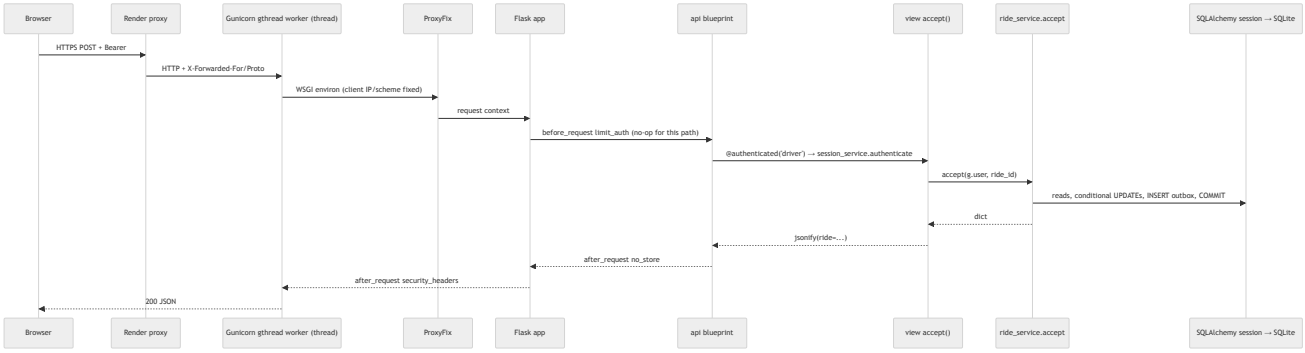
Caller	Callee	Why
Every request through `ride_service.expire()`	`matching.advance()`	Catch up overdue rounds before reading state
`ride_service.quote`	`route_provider.compute` , `fare_service.calculate`	Route then price
`ride_service.book`	`matching.invite`	First round invitations
`ride_service.move('complete')`	`revenue_service.allocate` , `payment_service.record_completion` , `location_service.is_fresh`	Settlement + auto-availability
`ride_service.public`	`rating_service.summary/given/for_admin`	Ratings on ride JSON
`ride_service.snapshot`	`admin_service.metrics` , `location_service.public`	Admin and driver data
`admin_service.change_driver_status`	`session_service.stop_requests`	Suspension stops new work
`session_service.logout`	`realtime.disconnect_invalid`	Kick revoked sockets
`trip_map_service.trip_map`	`ride_service.accessible` , `route_provider.compute`	Participant check + route
`test_payment_routes.simulate_test_payment`	`payment_service.confirm`	Online confirmation

Caller	Callee	Why
All mutating services	`ride_repository.notify / notify_ride`	Outbox events
`realtime.tick`	`matching.advance`, `dispatch`	Background progress

Note the cyclic import between `ride_service` and `admin_service` / `trip_map_service`: it is avoided at import time by importing inside functions (e.g. `from .admin_service import metrics` inside `snapshot`).

9. Request–response lifecycle

Example: `POST /api/v1/rides/{id}/accept` in production.



Errors: an `ApiError` raised anywhere in the call stack is caught by `routes.api_error` and turned into `{"error": …}` with its status. Any other exception becomes Flask's default 500 (HTML in Flask 3 unless a handler exists — **no generic 500 JSON handler exists**; see 14 — Error handling). The SQLAlchemy session is scoped to the app context and removed at teardown by Flask-SQLAlchemy, which rolls back uncommitted work.

10. Error-handling architecture

Layer	Mechanism
Services	Raise `ApiError(message, status)`; call `repo.rollback()` before raising after a failed conditional write
Blueprint	`@api.errorhandler(ApiError)` → JSON; clears refresh cookie on auth 401s
App	JSON 404 and 405 handlers
Worker	`tick` rolls back and re-raises; `loop` logs a generic message (no exception payload, to avoid leaking coordinates) and continues next second
Google	Every failure mode mapped to a specific, non-leaking message (`route_provider.compute`); trip map caches errors for 10 s
Frontend	`api()` throws `Error(data.error)`; components store it in local `error` state and render `role="alert"` blocks; workspace pauses actions while `error` is set

11. Background tasks and scheduled operations

There is no cron, Celery, or Redis. One daemon thread per process (`realtime.start_worker`) runs `tick()` every second:

1. `matching.advance()` — expire rounds, expand radius, end searches, mark stale locations.
2. `dispatch()` — disconnect invalid sockets, emit outbox events, prune old events.

Because `advance()` is also called at the start of most requests (`ride_service.expire()`), deadlines are honoured even if the worker is stalled. Both paths use the same idempotent conditional "claim" (`expire_round`) so they cannot double-process a round. Details: 13 — Background and real-time.

12. Concurrency model (critical to understand)

Mechanism	Where	Guarantees
Conditional UPDATE + `rowcount`	`transition`, `assign`, `expire_round`, `payment_service.confirm`, refresh token `used_at`, driver status change, mock payment update, payment method change	Exactly one concurrent writer wins; others get 409 or idempotent result
Primary key uniqueness	`active_rides.user_id`, `rides.quote_id`, `ride_invitations (ride_id, driver_id)`, `revenue_ledger.ride_id`, `ride_payments.ride_id`, `mock_payments.ride_id`, `ride_ratings (ride_id, rater_role)`, `fare_policies.version`, `revenue_share_settings.version`	Duplicate inserts raise `IntegrityError` → handled as 409 or idempotent return
No-op "lock" UPDATE	`lock_active_driver`, `lock_driver`, vehicle save	Serializes availability/acceptance/suspension/completion on the same driver row. In SQLite this acquires the database write lock; in SQL Server it would take a row lock **(inferred; SQL Server not validated)**
`db.session.expire_all()` / `refresh()`	After conditional writes and in `expire()`	Re-read authoritative state instead of stale identity-map objects
Single process	`gunicorn.conf.py workers=1`	In-memory rate limiters, socket connection map and trip-map cache are correct only because there is one process

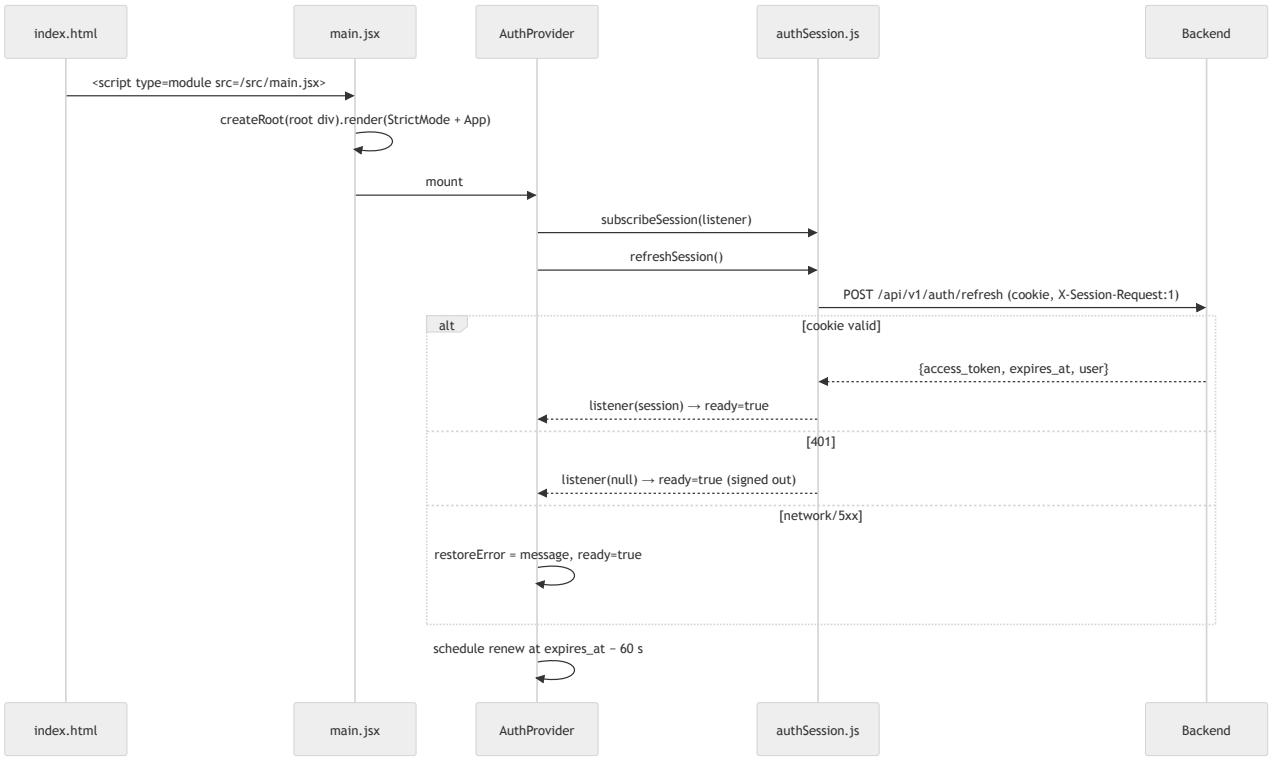
13. Configuration architecture

All configuration is environment variables read in `create_app`; there are no config classes or YAML files. Development defaults are safe for local work and production requirements are enforced at boot. Full list: 15 — Deployment and 21 — Quick revision.

04 — Frontend documentation

The frontend lives in `frontend/src/`. It is plain JavaScript + JSX (no TypeScript), React 19 function components, React Router 7, and one global stylesheet. File-level summaries are in 02; this document explains **how it works**.

1. Boot sequence



Until `ready` is true, `AuthPage` and `Protected` render `SessionLoading` ("Restoring your session…"), so the user never briefly sees the login page on reload.

2. Routing (`main.jsx::App` and `Workspace`)

Top-level routes:

Path	Element	Auth
`/`	`Landing`	public
`/test-payment/:token`	`TestPaymentPage`	public (capability token)
`/login`	`AuthPage`	redirects to `/{role}` if already signed in
`/register`	`AuthPage register`	same
`/rider/*`, `/driver/*`, `/admin/*`	`Protected role=…` → `Workspace`	session + matching role
`*`	"Wrong turn?" page	public

`Protected`:

```
if(!ready) return <SessionLoading/>;
if(restoreError && !session) return /* error + Retry + Sign in again */;
if(!session) return <Navigate to="/login" replace/>;
if(session.user.role !== role) return <Navigate to={`/${session.user.role}`} replace/>;
return <Workspace role={role}/>;
```

`Workspace` renders the sidebar from `navItems[role]` and nested routes. The nested route list is computed: `['history','earnings','drivers','rides']` filtered per role (admin: drivers, rides; driver: history, earnings; rider: history), each rendering `RideWorkspace` with a `view` prop. Admin additionally gets `settings` → `FareSettings`, `revenue` → `RevenueSettings`, `audit` → `AdminAudit`. All roles get `account` → `Account`. Unknown sub-paths render `ComingSoon`.

The production server (`backend/app/web.py`) mirrors these prefixes so a hard reload of `/rider/history` returns `index.html`.

3. Session handling (`authSession.js`)

This is the most subtle frontend module. Its goals: keep the access token out of persistent storage, never run two refreshes at once (the refresh token is **single-use** — a second concurrent use would look like a replay and revoke the session), and keep multiple tabs consistent.

```
let session=null, pendingRefresh=null, generation=0, signingOut=false;
const listeners=new Set(), knownTokens=new Set();
const channel=new BroadcastChannel('ridez-auth');            // cross-tab sign-out
const locked=action=>navigator.locks
  ? navigator.locks.request('ridez-refresh-cookie',action)  // cross-tab mutual exclusion
  : action();
```

Mechanism	Purpose
`pendingRefresh`	Single-flight within a tab: concurrent callers share one promise
`navigator.locks` (`ridez-refresh-cookie`)	Serializes refresh/login/logout **across tabs**, so two tabs never present the same refresh cookie simultaneously
`generation` counter	Incremented on sign-in, sign-out and forced sign-out; a refresh that started in an older generation discards its result
`knownTokens`	All tokens issued during this session; `resolveToken(token)` maps any of them to the current token and rejects tokens from another session
`BroadcastChannel('ridez-auth')`	`signOut` posts `signed-out`; other tabs publish `null`
`signingOut` flag	Blocks refreshes during logout

Every auth request sets `X-Session-Request: 1` and `credentials: 'same-origin'`.

Refresh triggers:

1. Boot (`AuthProvider` effect).
2. Timer 60 s before `expires_at` (rescheduled whenever `access_token` changes); failure retries in 15 s.
3. Any API call that returns 401 (`api.js`).
4. `signOut(true)` refreshes first to obtain a guaranteed-valid Bearer token for `logout-all`.

4. HTTP client (`api.js`)

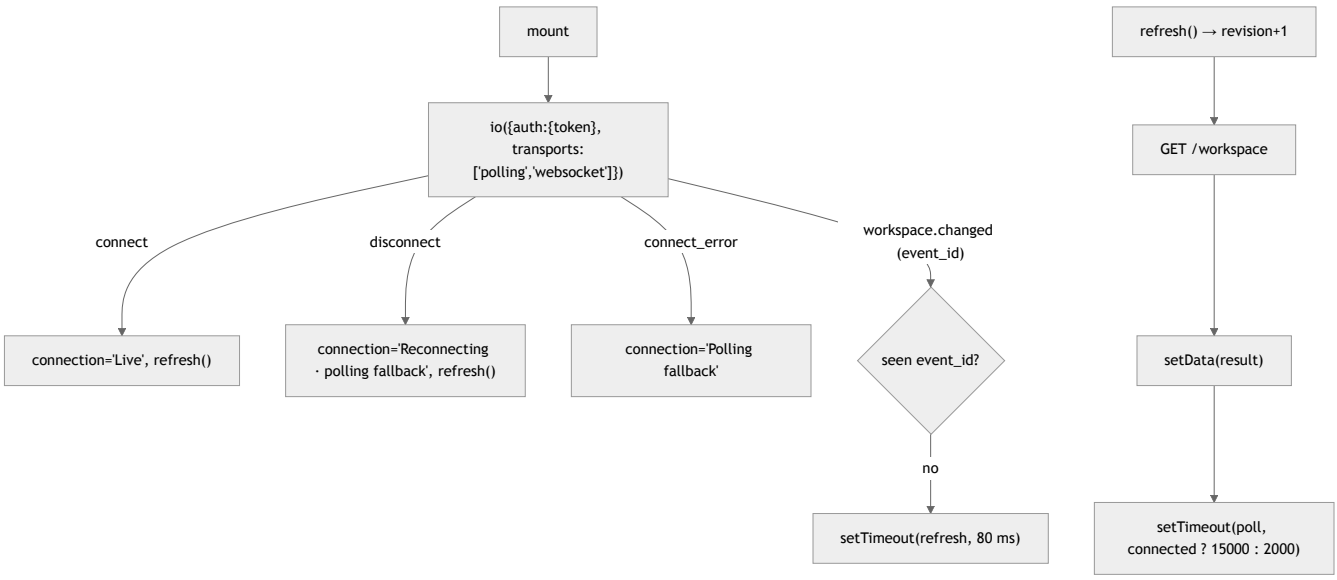
```
export async function api(path, { token, ...options } = {}) {
  const activeToken = resolveToken(token);                      // current token for this session
  const send = t => fetch(`/api/v1${path}`, { ...options, credentials:'same-origin',
      headers: { 'Content-Type':'application/json', ...(t ? {Authorization:`Bearer ${t}`} : {}),
...options.headers } });
  let response = await send(activeToken);
  if (response.status === 401 && activeToken && !options.signal?.aborted) {
    const renewed = currentSession()?.access_token !== activeToken ? currentSession() : await
refreshSession();
    if (renewed && resolveToken(token)) response = await send(renewed.access_token);  // retry once
  }
  let data; try { data = await response.json(); } catch { throw new Error('Unable to read the
server response. Please retry.'); }
  if (!response.ok) throw new Error(data.error || 'Something went wrong. Please try again.');
  return data;
}
```

Consequences:

- Components pass the token they received as a prop; even if it is 14 minutes old, the request uses the latest token.
- The backend's `{"error": "..."}` message is surfaced verbatim in the UI.
- Supports `AbortController` signals (used by polling hooks to cancel on unmount).

5. The workspace: data model and refresh loop

`useWorkspace(token)`



- The socket effect depends on `token`; every token rotation (≈ every 14 min) reconnects with the new token. This matters because the server disconnects sockets whose JWT `exp` has passed.
- The polling effect depends on `revision`; `refresh()` bumps it, which aborts the in-flight poll and starts a new one immediately.

- `updateAvailability(result)` patches `available` / `resume_available` optimistically after the availability POST, before the next poll.

`perform(path, body)` — the mutation helper

```
async function perform(path, body = {}) {
  if (busy) return;                     // one action at a time
  setBusy(true); setActionError('');
  try {
    const response = await api(path, {token, method:'POST', body: JSON.stringify(body)});
    if (typeof response.available === 'boolean' && typeof response.resume_available === 'boolean')
updateAvailability(response);
    if (response.ride?.status === 'COMPLETED') setCompleted(response.ride);   // driver "trip
completed" notice
    refresh();
    return response;
  } catch (e) { setActionError(e.message); return null; }
  finally { setBusy(false); }
}
```

It is passed down to `BookingPanel`, `RideCard`, `CancelRideControl`, `DriverCollectCard`. All buttons use `disabled = busy || !!error`, so a broken connection pauses all actions.

What each role sees (`RideWorkspace` render tree)

Rider – overview

1. Recent cancellation notice (if `data.recent_cancellation` and not dismissed).
2. If `data.trip_otp` and no driver yet: "Your ride OTP" card.
3. If `active_ride` → `ActiveRidePanel`; else → `BookingPanel` + "Most recent journey" card.
4. `PaymentPrompt` modal when `usePaymentPrompt` selects a ride.

Driver – overview

1. Approval/availability banner with **Go available / Go offline** (only for active drivers without an active ride; disabled when bookings are disabled).
2. Suspension reason (from `data.suspension`).
3. `PaymentConfirmedFlash`, then one `DriverCollectCard` per completed ride with pending payment.
4. Own rating badge, location freshness line.
5. If active ride → `ActiveRidePanel` with the **"Stay available after this trip"** checkbox (calls `changeDriverAvailability(checked)`); else "Incoming requests" invitation cards with Countdown, quote and Accept/Reject (disabled when expired).

Driver – earnings: two totals (`confirmed_earnings`, `pending_earnings`) and one card per ledger entry with a paid/awaiting badge.

Admin – overview: metrics grid (10 tiles), `DriverManagement`, recent rides list. **Admin – drivers**: `DriverManagement`. **Admin – rides**: rides list. Ride cards for admins include settlement and both ratings with comments.

History (rider/driver): all rides via `RideCard`.

`RideCard` decision logic

Condition	Rendered
`status === 'CANCELLED'`	`CancellationNotice`
Driver assigned and status in ASSIGNED/ARRIVING/WAITING/IN_PROGRESS and viewer is rider/driver	`AssignedRideDetails` + collapsible fare breakdown
otherwise	`QuoteDetails` + driver line + rider name (for drivers)
`SEARCHING`	`Countdown` to `expires_at`, round number and radius
driver & `WAITING` & not cancelling	`OtpInput`
driver & next action exists	Primary button; `start` disabled until 6 OTP digits; `complete` asks `window.confirm`
rider in SEARCHING/ASSIGNED/ARRIVING/WAITING, or assigned driver in ASSIGNED/ARRIVING/WAITING	`CancelRideControl` (opening it hides the action buttons)
driver/admin & `settlement`	"Allocation recorded · not a payment" block
always	`PaymentStatusLine` (only renders for completed rides with a payment)
COMPLETED & participant & (already rated or completed < 7 days ago)	`RatingForm compact`
admin & ratings	Both ratings with comments
always	`TestPaymentCard` (self-hides unless relevant)

`usePaymentPrompt(role, data)`

```
const previous = lastActive.current;               // undefined on first render
lastActive.current = data.active_ride?.id || null;
if (previous && !data.active_ride) candidate = data.rides.find(r => r.id === previous);   // active
ride just ended
else if (previous === undefined && !data.active_ride) {                                    //
first load
  const latest = data.rides[0];
  if (latest?.completed_at && Date.now() - Date.parse(latest.completed_at) < 30*60000) candidate =
latest;
}
if (candidate?.status === 'COMPLETED' && candidate.payment?.status === 'PENDING' &&
!paymentDismissed(candidate.id)) setRideId(candidate.id);
```

"Pay later" stores `ridez.payment-dismissed.{rideId}` in `sessionStorage` (per tab).

6. Booking flow components

`BookingPanel`

- `provider === 'google'` → `AddressSearch` fields + map picker; otherwise free-text labels with coordinates visible.
- Form state keeps coordinates as **strings** (inputs) and converts with `Number()` on submit.
- Editing an address label in Google mode clears its coordinates (`change`), forcing a fresh selection — prevents a label/coordinate mismatch.
- "Use my location" reuses `getPosition()` from `useDriverLocation.js` to set the pickup.

- After a quote: shows `QuoteDetails`, expiry time, `fare_rule_version`, and **Confirm ride** → `perform('/rides', {quote_id})`; on success the quote is cleared and the workspace refresh shows the active ride.

`AddressSearch` — race-safe autocomplete

Three refs make async results safe:

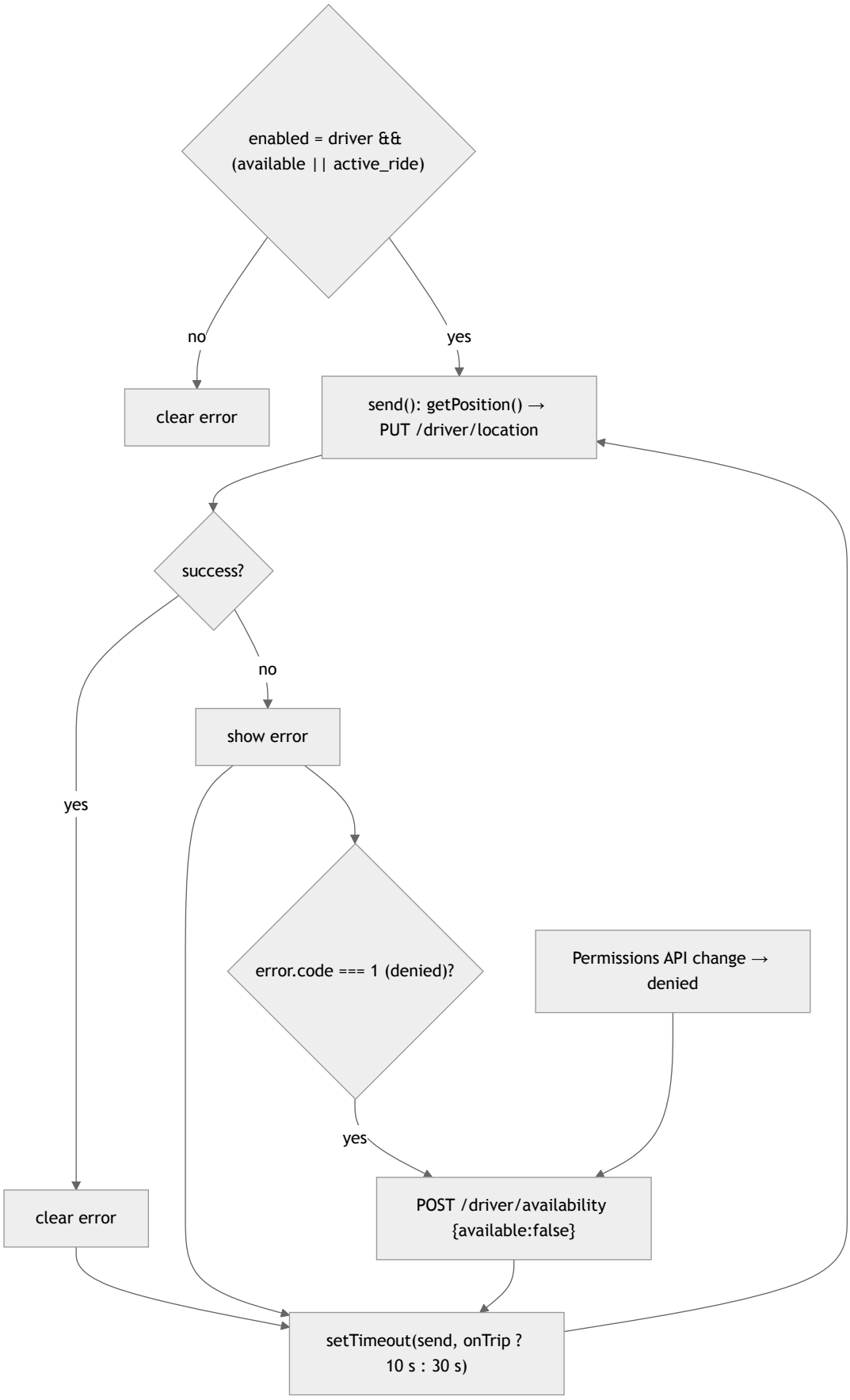
- `sequence` — incremented on every keystroke/blur/Escape; a suggestions response is applied only if its sequence is still current **and** the input value hasn't changed (`latest.current === query`).
- `selection` — same idea for the place-details fetch after clicking a suggestion.
- `alive` — prevents state updates after unmount.

It creates one `AutocompleteSessionToken` per typing session and clears it after a selection or blur, which is how Google groups autocomplete + details into one billing session **(Google documentation; not verifiable from code)**.

`GoogleRouteMap`

Two effects: (1) initialize the map once per `attempt` (retry increments it); (2) redraw markers/polyline whenever pickup, destination or polyline changes, then `fitBounds` with 55 px padding. A third effect moves a single driver marker without redrawing everything. Map clicks call `onSelect(mode, {lat, lng})` where `mode` is `pickup` or `destination`.

7. Driver GPS (`useDriverLocation.js`)



- Requires `window.isSecureContext` (HTTPS or localhost).

- High-accuracy attempt (15 s) then low-accuracy retry (20 s) — desktop browsers often time out on high accuracy; the server still enforces ≤ 200 m accuracy.
- The effect restarts when `available` or `onTrip` changes, which also changes the interval.
- Background tabs are throttled by browsers; the UI tells drivers to keep the tab open.

8. Trip-in-progress UI

`ActiveRidePanel` calls `useTripRoute(ride, token)` once and passes its state to both `TripStatsCard` and `TripRouteMap`, so only one poll runs. The poll key is `ride.id:phase`, so it restarts immediately on phase change (e.g. ARRIVING → WAITING). The server dictates the poll interval via `refresh_after_seconds` (10 s).

`TripStatsCard` only shows live numbers when `data.origin_source === 'driver'` and `data.target` matches the phase (pickup during approach, destination during the trip). Otherwise it shows the booked estimate with an explanatory note. Arrival time = `generated_at + duration_seconds`.

9. Payments and ratings UI

- `PaymentPrompt` (rider modal) views: `choose` → `cash` (waiting for driver) or `online` (QR) → `failed` (retry/cash) → confirmed (`PaymentSuccess` + `RatingForm`). It closes on Escape or backdrop click. Choosing ONLINE immediately generates the QR.
- `TestPaymentCard` appears on completed ride cards when test payments are enabled (or a mock record exists) and the ride is not already paid by cash.
- `DriverCollectCard` shows the rider's chosen method and a "Cash received" button for every completed, unpaid ride.
- `usePaymentConfirmedFlash` compares the previous set of pending ride IDs with the new data to detect a confirmation and shows a 6-second success banner.
- `RatingForm` filters tags by sentiment: 4–5 stars show only positive tags, 1–2 only negative (`NEGATIVE` set), 3 shows all; tags reset when the star value changes; max 5 tags. The server is the source of the tag list (`workspace.rating_tags`, via `RatingTags` context).

10. Admin pages

All admin pages manage their own `data/error/busy/message` state and talk to the API directly.

Page	Load	Save	Concurrency handling
`FareSettings`	`GET /admin/fare-settings`	`PUT` with `version`, `reason`, full `policy`	409 "Fare settings changed. Reload before saving" → "Reload settings" button
`RevenueSettings`	`GET /admin/revenue-share`	`PUT` with `version`, pct strings, `effective_from` ISO or null	Same; client blocks save unless total = 100
`DriverManagement`	`GET /admin/drivers?page&status&q` (refetches on `refreshKey` = `metrics.as_of`)	`POST /admin/drivers/{id}/{action}` with `version: status_revision`	Stale version → error + "Reload drivers"
`VehicleEditor`	from driver row	`PUT /admin/drivers/{id}/vehicle` with `version`	Stale → error
`AdminAudit`	`GET /admin/audit?page&category&q` ; detail on demand	read-only	—

11. Forms and validation

Client-side validation is **convenience only**; the server re-validates everything.

Form	Client checks	Server checks (authoritative)
Register	`required`, `maxLength=100`, `minLength=10` password, `type=tel`	name 1–100, password 10–128, E.164 mobile, role
Login	required	normalized, generic 401
Booking	coordinates present	1–160 char labels, coordinate bounds, distinct pickup/destination
OTP	6 digits before enabling Start	6 ASCII digits + HMAC match + rate limit
Cancel	reason chosen; text required for "Other"	1–300 chars
Rating	stars > 0	stars int 1–5, ≤ 5 known tags, comment ≤ 300
Fare settings	`type=number` min/max, `required`	strict decimals, ranges, zone schema
Revenue	total = 100	2-dp percentages, sum = 100, date window
Vehicle	`required`, `maxLength`	regex for registration, lengths

12. Accessibility notes (observed in code)

- Error messages use `role="alert"`, status lines `role="status"` / `aria-live`.
- `AddressSearch` implements the ARIA combobox pattern (`aria-expanded`, `aria-activedescendant`, listbox/option).
- `OtpInput` uses a `fieldset`/`legend`, per-digit `aria-label`, `inputMode="numeric"`.
- Star rating uses `role="radiogroup"` / `role="radio"`.
- `PaymentPrompt` is `role="dialog" aria-modal="true"`, focuses its first button, closes on Escape (no full focus trap).

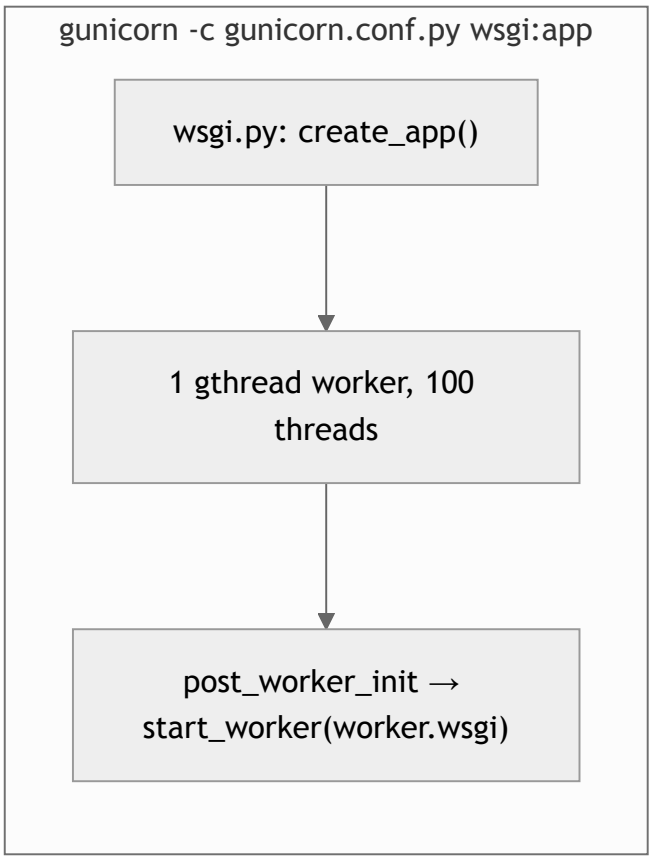
13. Known frontend limitations

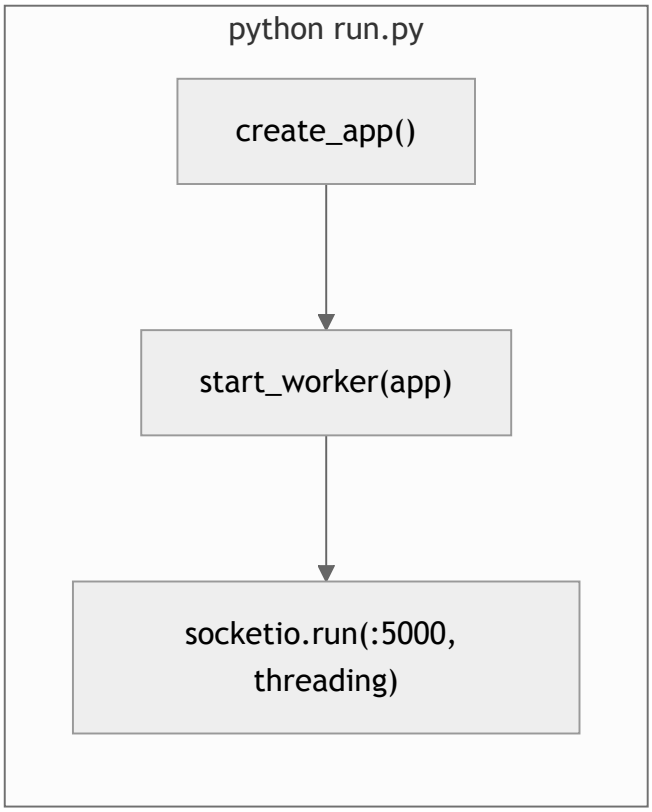
- Workspace header always shows "Development" (hard-coded) even in production.
- No global error boundary; a render exception blanks the page.
- `Account` page: profile editing and account recovery explicitly not implemented.
- No unit tests; the only browser test is not in CI.
- Admin ride list is limited to the 100 most recent rides returned by `/workspace` (no pagination).

05 — Backend documentation

File-level summaries and full function tables are in 02. This document explains the **logic** of the important backend functions in logical blocks, with code excerpts from the repository.

1. Process startup





`flask run` would **not** start the worker; the README explicitly says to use `python run.py`. `create_app` details: 02.

2. Request plumbing (`routes.py`)

```
def authenticated(role=None):
    def decorate(fn):
        @wraps(fn)
        def wrapped(*args, **kwargs):
            header = request.headers.get('Authorization', '')
            from .session_service import authenticate
            if not header.startswith('Bearer '):
                raise ApiError('Please sign in again', 401)
            g.user, g.auth_claims = authenticate(header[7:])   # JWT + DB session check
            if role and g.user.role != role:
                raise ApiError('You do not have access to this workspace', 403)
            return fn(*args, **kwargs)
        return wrapped
    return decorate
```

Every protected view gets a fresh `User` row in `g.user`, so a status change (e.g. suspension) is visible on the very next request.

3. Accounts (`services.py`)

Permanent account code

```
digest = base64.b32encode(hmac.new(USER_CODE_SECRET, f'{role}:{mobile}'.encode(),
sha256).digest()).decode().rstrip('=')
prefix = {'rider': 'RID', 'driver': 'DRV', 'admin': 'ADM'}[role]
for length in range(12, 33, 4):          # 12, 16, 20, 24, 28, 32 chars
    candidate = f'{prefix}-{digest[:length]}'
    if not users.code_exists(candidate):
        code = candidate; break
```

- Deterministic for a given role + mobile + secret, but **unguessable** without the secret.
- Collision handling lengthens the code instead of failing.
- Changing `USER_CODE_SECRET` would not change existing stored codes, but would change future derivations and **all trip OTPs** (README warns to keep it stable).

Login timing defence

`login()` always runs `check_password_hash`, against a cached dummy hash when the mobile is unknown, so response time doesn't reveal which mobiles are registered. All failures (bad format, unknown, wrong password, disabled) return the same 401 message.

4. Sessions (`session_service.py`)

`refresh()` — rotation with replay detection

```
value = request.cookies.get(COOKIE, '')
record = db.session.get(RefreshToken, digest(value)) if value and len(value) <= 128 else None
session = valid_session(record.session_id) if record else None
user = db.session.get(User, session.user_id) if session else None
if not session or not user or user.status == 'disabled':
    if session: revoke(session); db.session.commit()
    raise ApiError('Please sign in again', 401)
changed = db.session.execute(update(RefreshToken)
    .where(RefreshToken.token_hash == record.token_hash, RefreshToken.used_at.is_(None))
    .values(used_at=now())).rowcount
if changed != 1:                       # token already used → stolen or replayed
    revoke(session); db.session.commit()
    raise ApiError('Session credential was already used. Please sign in again', 401)
token = new_refresh(session); db.session.commit()
return payload(user, session), token
```

Block by block:

1. Only the SHA-256 hash is stored, so a DB leak does not leak usable refresh tokens.
2. Session must be un-revoked, unexpired (absolute 30-day lifetime — refreshing does **not** extend `expires_at`), and the user not disabled.
3. The conditional `UPDATE … WHERE used_at IS NULL` makes the token single-use even across threads; a second use revokes the whole session (OAuth-style refresh-token reuse detection).
4. A new refresh token is chained to the same session.

`authenticate(token)`

Decodes HS256 with required claims `sub, sid, iat, exp`, then checks the user and the **session row** on every request. That makes logout and suspension-of-sessions immediate even though JWTs are normally stateless.

`stop_requests(user_id)`

Used by logout, refresh-replay revocation and admin status changes. For drivers: lock the user row, set `available=False, resume_available=False`, close PENDING invitations, notify. An active trip is **not** affected.

5. Ride service (`ride_service.py`)

`quote(user, data)`

1. `simulator()` guard.
2. `pickup`, `destination` labels 1–160 chars; `pickup_lat/lng` required numbers; must differ (case-insensitive).
3. Destination coordinates optional (required by Google provider).
4. `expire()` to catch up matching (affects demand surge counts).
5. `route_provider.compute(...)` → distance/duration/polyline.
6. `fare_service.calculate(route, pickup)` → itemized fare dict.

7. Insert `FareQuote` with `expires_at = now + 5 min`, `distance_km = round(distance_m/1000, 2)`, `duration_min = ceil(seconds/60)`.
8. Return `quote.public()` + `polyline` (the polyline is **not stored**).

`book(user, data)`

```
quoted = repo.get(FareQuote, quote_id)
if not quoted or quoted.rider_id != user.id: raise ApiError('Quote not found', 404)
existing = repo.ride_for_quote(quote_id)
if existing: return public(existing, user)                 # idempotent retry
if quoted.expires_at <= now(): raise ApiError('Quote expired. Request a new estimate', 409)
if quoted.provider != current_app.config['ROUTE_PROVIDER']: raise ApiError('Route mode changed…',
409)
if quoted.pickup_lat is None: raise ApiError('Get a new quote with pickup coordinates', 409)
if repo.get(ActiveRide, user.id): raise ApiError('You already have an active ride', 409)
ride = repo.add(Ride(..., status='SEARCHING', round_no=1, radius_m=500,
                     requested_at=now(), expires_at=now()+timedelta(seconds=MATCH_ROUND_SECONDS)))
try:
    db.session.flush()
    repo.add(ActiveRide(user_id=user.id, ride_id=ride.id))  # PK user_id: one active ride
    matching.invite(ride, ride.requested_at)
    repo.notify_ride(ride)
    repo.commit()
except IntegrityError:                                      # concurrent double-submit
    repo.rollback()
    existing = repo.ride_for_quote(quote_id)
    if existing: return public(existing, user)
    raise ApiError('You already have an active ride', 409)
```

Two uniqueness constraints carry the correctness: `rides.quote_id` (same quote twice) and `active_rides.user_id` (two different quotes at once).

`availability(user, data)`

```
locked = repo.lock_active_driver(user.id) if available else repo.lock_driver(user.id)
if not locked: raise ApiError('Your driver status does not allow new requests', 403)
db.session.refresh(user)
active = db.session.scalar(select(ActiveRide).where(ActiveRide.user_id == user.id))
if available:
    location_service.update(user, data, initializing=not bool(active))
online = available and not bool(active)                 # busy drivers stay unavailable
repo.set_available(user.id, online, resume_available=available)
if not available:
    UPDATE ride_invitations SET status='OFFLINE' WHERE driver_id AND status='PENDING'
repo.notify(user.id); repo.commit()
return dict(available=online, resume_available=available, active_trip=bool(active))
```

- Going online requires an **active** status and a valid GPS fix in the same request.
- While on a trip, the same call only stores the "stay available after this trip" preference (`resume_available`).

- Going offline always works (even when suspended) and withdraws pending invitations.

`accept(user, ride_id)`

Checks (in order): invitation exists (404) → idempotent if already this driver's ride → driver has no active claim (409) → active and available (403) → fresh location (409) → invitation PENDING, ride SEARCHING, round not expired (409). Then:

```
if not repo.lock_active_driver(user.id): ... 403
if not repo.assign(ride.id, user.id, now(), repo.get(DriverVehicle, user.id)): ... 409 'Another
driver has already accepted this ride'
repo.add(ActiveRide(user_id=user.id, ride_id=ride.id))
repo.close_invitations(ride.id, 'CLOSED'); invitation.status = 'ACCEPTED'
repo.set_available(user.id, False)
matching.close_round(ride, 'ASSIGNED')
repo.notify_ride(ride); repo.commit()                    # IntegrityError → 409
```

`ride_repository.assign` re-validates **everything inside the UPDATE's WHERE clause** so a check-then-act race cannot slip through:

```
valid_invitation = select(Invitation.ride_id).where(Invitation.driver_id == driver_id,
    Invitation.status == 'PENDING', Invitation.expires_at > now)
eligible = select(DriverPresence.driver_id).join(User…).join(DriverLocation…).where(
    DriverPresence.driver_id == driver_id, DriverPresence.available.is_(True), User.status ==
'active',
    DriverLocation.fresh.is_(True), DriverLocation.recorded_at > cutoff, DriverLocation.updated_at
> cutoff).exists()
update(Ride).where(Ride.id == ride_id, Ride.status == 'SEARCHING', Ride.expires_at > now,
                   Ride.id.in_(valid_invitation), eligible)
            .values(status='ASSIGNED', driver_id=driver_id, vehicle_registration=…,
vehicle_model=…, vehicle_color=…)
```

`move(user, ride_id, action, data)`

Cancel branch

Step	Rule
Authorization	rider of this ride, or the assigned driver (admins cannot cancel)
Idempotency	already CANCELLED → return current record
State	only SEARCHING, ASSIGNED, ARRIVING, WAITING
Reason	required (1–300) except SEARCHING without a `reason` key → `"Rider cancelled the search"` (legacy clients)
Write	`transition(old → CANCELLED, cancelled_at, cancelled_by, cancellation_reason)`
Side effects	close invitations, driver offline with `resume_available=False`, release both claims, close round if searching

Driver transitions

```
transitions = {'depart': ('ASSIGNED','ARRIVING'), 'arrive': ('ARRIVING','WAITING'),
               'start': ('WAITING','IN_PROGRESS'), 'complete': ('IN_PROGRESS','COMPLETED')}
old, new = transitions[action]
if action == 'start' and ride.status in (old, new): verify_trip_otp(ride, user.id, data.get('otp'))
if ride.status == new: return public(ride, user)          # idempotent retry
if ride.status != old: raise ApiError(f'Cannot {action} a trip in {ride.status.lower()} state',
409)
```

Note: the OTP is checked even on a `start` retry (status already `IN_PROGRESS`), so a retry must resend the code.

Completion side effects

```
repo.lock_driver(user.id); db.session.refresh(user)        # serialize with suspend/offline
transition(IN_PROGRESS → COMPLETED, completed_at)
revenue_service.allocate(ride, quoted, user.id, completed_at)
payment_service.record_completion(ride, quoted, completed_at)
repo.release(ride.id)
eligible = (user.status == 'active' and presence.resume_available
            and location_service.is_fresh(driver_location, completed_at)
            and (ROUTE_PROVIDER == 'google' or SIMULATOR_ENABLED))
repo.set_available(user.id, eligible)
```

All in one transaction with the outbox events, then commit.

`snapshot(user)` — the `/workspace` payload

Common keys: `rides` (≤ 100, role-filtered, each via `public()`), `active_ride`, `simulator`, `route_provider`, `booking_enabled`, `user`.

Role	Extra keys
rider, driver	`recent_cancellation` (latest ride cancelled within 10 min and no active ride), `my_rating_summary`, `rating_tags`
rider	`trip_otp` while the active ride is SEARCHING/ASSIGNED/ARRIVING/WAITING
driver	`location` (freshness only), `suspension` (reason from matching audit row), `resume_available`, `available`, `invitations` (only if active, available, fresh and not busy), `earnings` (ledger + payment), `allocated_earnings`, `confirmed_earnings`, `pending_earnings`
admin	`metrics`, `drivers` (≤ 100 with location freshness)

6. Matching (`matching.py`)

`invite(ride, started_at)`

```
repo.add(MatchRound(ride_id=ride.id, round_no=ride.round_no, radius_m=ride.radius_m,
                    started_at=started_at, expires_at=ride.expires_at, outcome='SEARCHING'))
if ride.expires_at <= utcnow() or quoted.pickup_lat is None: return       # catch-up: don't invite
into a dead round
previous = set(driver_ids already invited for this ride)
for presence in repo.eligible_drivers(utcnow()):
    location = repo.get(DriverLocation, presence.driver_id)
    if presence.driver_id not in previous and distance_m(pickup, location) <= ride.radius_m:
        if not repo.lock_active_driver(presence.driver_id, require_available=True): continue
        repo.add(Invitation(ride_id=ride.id, driver_id=presence.driver_id, status='PENDING',
                            round_no=ride.round_no, expires_at=ride.expires_at))
```

Eligibility is evaluated in Python with haversine over **all** eligible drivers (no spatial index). Each round invites only drivers newly in range.

`advance()`

```
at = utcnow()
for _ in range(MATCH_MAX_RADIUS_M // 500 + 1):            # bounded catch-up (11 passes at 5 km)
    due = repo.expired_searches(at)
    if not due: break
    for ride in due:
        old_no, deadline = ride.round_no, ride.expires_at
        if not repo.expire_round(ride, at): repo.rollback(); continue   # someone else claimed it
        db.session.refresh(ride)
        old_round.outcome = 'EXPIRED'; repo.close_invitations(ride.id)  # PENDING → EXPIRED
        if ride.radius_m >= MATCH_MAX_RADIUS_M or quote has no pickup:
            ride.round_no = old_no; ride.status = 'NO_DRIVER_FOUND'; repo.release(ride.id)
        else:
            ride.round_no = old_no + 1; ride.radius_m += 500
            ride.expires_at = deadline + timedelta(seconds=MATCH_ROUND_SECONDS)   # schedule from
old deadline, not "now"
            invite(ride, deadline)
        repo.notify_ride(ride); repo.commit()
    db.session.expire_all()
# stale GPS sweep
for location in fresh locations with recorded_at or updated_at ≤ at − 90 s:
    location.fresh = False; repo.notify(location.driver_id)
```

Why the new deadline is `deadline + 30 s` rather than `now + 30 s`: after downtime the loop replays missed rounds on the original schedule; rounds whose deadline already passed are recorded but send no invitations (`invite` returns early), and the loop continues until it reaches a round that is still open or the maximum radius.

With defaults, a search lasts at most 10 rounds × 30 s = 5 minutes (radius 500 → 5000 m).

7. Locations (`location_service.update`)

```
if user.status != 'active' and not (user.status == 'suspended' and active and not initializing):
    raise 403 'Driver approval is required'          # suspended drivers may keep tracking an
active trip
if not initializing and not presence.available and not active:
    raise 409 'Location updates are only allowed while available or on a trip'
lat, lng = coordinate(...); accuracy = coordinate(accuracy_m, 'Accuracy', 200)   # ≤ 200 m
captured = parse ISO with timezone → UTC
if captured > now + 15 s or captured <= now − 90 s: raise 'Location time is stale or in the future'
if location exists:
    if captured < location.recorded_at: raise 409 'older than the latest accepted update'
    if is_fresh(location):                                      # only when continuously tracked
        movement = distance_m(old, new)
        seconds = captured − old.recorded_at
        if movement > seconds × 55.56 m/s (200 km/h) + old.accuracy + new.accuracy + 100:
            raise 409 'Location movement is implausible'
    if captured == location.recorded_at: return public(location)   # replay: do not bump updated_at
else: create
set lat/lng/accuracy, recorded_at=captured, updated_at=now, fresh=True; notify(driver)
```

Freshness uses **two clocks**: `recorded_at` (device time) and `updated_at` (server receipt). Both must be within 90 s.

8. Routing (`route_provider.compute`)

Simulator

```
seed = int(sha256('|'.join(label.casefold() for label in labels)).hexdigest()[:8], 16)
distance = (200 + seed % 1800) * 10              # 2 000 … 19 990 m
duration_seconds = max(5, int(distance_km * 3)) * 60   # 3 min per km, ≥ 5 min
```

Deterministic per label pair, so repeated quotes are stable. Example: "City centre" → "Riverside" = 17 340 m, 3 120 s (52 min).

Google

```
requests.post('https://routes.googleapis.com/directions/v2:computeRoutes',
  json={'origin': waypoint(pickup), 'destination': waypoint(destination), 'travelMode': 'DRIVE',
        'routingPreference': 'TRAFFIC_AWARE', 'computeAlternativeRoutes': False, 'polylineQuality':
'OVERVIEW'},
  headers={'X-Goog-Api-Key': key,
           'X-Goog-FieldMask':
'routes.distanceMeters,routes.duration,routes.staticDuration,routes.polyline.encodedPolyline'},
  timeout=(3, 10), allow_redirects=False)
```

`duration` (traffic-aware) is the billed/displayed duration; `staticDuration` (no traffic) is stored as `baseline_seconds` and used for the traffic-surge ratio. Error mapping: see 11 — Integrations.

9. Fares (`fare_service.calculate`)

```
base = policy.base
distance_fare = money(distance_m / 1000 × per_km)
time_fare     = money(duration_seconds / 60 × per_minute)        # prorated per second
components = base + distance_fare + time_fare
subtotal = max(components, minimum)                               # minimum_adjustment = subtotal - components
multipliers = [(1, 'No surge')]
traffic_ratio = duration / baseline  (if baseline available)
if surge_enabled:
    for zone in zones: if enabled and scheduled(zone, local_time) and haversine(pickup, centre) ≤ radius:
        multipliers.append((zone.multiplier, 'Zone: ' + name))
    if traffic_enabled and traffic_ratio ≥ traffic_threshold: append((traffic_multiplier, 'Traffic threshold'))
    if demand_enabled:
        requests, ratio = demand_signal(pickup, at)               # within 1 km
        if requests ≥ demand_min_requests and ratio ≥ demand_threshold: append((demand_multiplier, 'High demand within 1 km'))
highest = max(multipliers); multiplier = min(highest, max_multiplier)
reason = '; '.join(reasons with value == highest and > 1) or 'No surge'; add ' (capped)' if capped
total = money(subtotal × multiplier)
```

Multipliers are **not** compounded — only the single highest applies. Worked example with defaults (simulator, City centre → Riverside):

Component	Value
base	₹40.00
distance 17.34 km × ₹12	₹208.08
time 52 min × ₹2	₹104.00
total (no surge, above minimum)	**₹352.08**
driver allocation 85 %	₹299.27
company allocation	₹52.81

`scheduled(zone, local)`: if `start == end` the zone is active all day on selected days; if `start < end` it's a same-day window; otherwise it's an overnight window that **belongs to its start day** (e.g. Fri 22:00–02:00 is active Fri 22:00–24:00 and Sat 00:00–02:00).

10. Revenue (`revenue_service`)

- `active(at)` picks the latest setting with `effective_from ≤ at`; ties go to the highest version.
- `allocate` rounds the driver share half-up and gives the company the **remainder**, so `driver + company == total` exactly.
- `save` stores `previous_driver_pct/previous_company_pct` = split active at save time, for audit display.

11. Payments (`payment_service`) and mock QR

- `record_completion` creates `ride_payments` PENDING at completion with `amount = quote.total`.

- `choose` only changes `method` while PENDING (conditional update). Choosing the same method again is a no-op.
- `confirm` is the single conditional write that finalizes payment. Both the driver's cash confirmation and the mock QR success call it; whichever commits first wins.
- Mock QR tokens are `itsdangerous` signatures of the ride ID with salt `mock-qr-payment-v1` and `max_age=900`. They are bearer capabilities (no login).

12. Ratings (`rating_service.submit`)

Participants only (404), COMPLETED only (409), `stars` int 1–5 (bool rejected), ≤ 5 tags from `TAGS[role]`, comment ≤ 300 (trimmed, empty → null). Tags stored `|`-joined with duplicates removed. Insert; on `IntegrityError`, same stars → return existing (idempotent), different → 409 "You have already rated this trip". There is **no server-side time window** for rating; the 7-day window is UI-only.

`summary(user_id)` = `AVG(stars)` formatted to 1 decimal and `COUNT(*)` over ratings received.

13. Admin (`admin_service`)

`change_driver_status`

```
old, new, event = DRIVER_TRANSITIONS[action]
legacy = action == 'approve' and version is None         # old UI sent no version
if legacy: if driver.status == new: return driver.public(); version = driver.status_revision
if driver.status == new and driver.status_revision == version + 1:
    if matching audit row (same admin, reason, revision, event): return driver.public()   # exact
retry
if driver.status_revision != version: 409 'Driver status changed…'
if driver.status != old: 409 f'Cannot {action} a {driver.status} driver'
UPDATE users SET status=new, status_revision=version+1
  WHERE id AND role='driver' AND status=old AND status_revision=version     # rowcount must be 1
stop_requests(driver_id)
INSERT admin_audit(before_status=old, after_status=new, status_revision=version+1, reason)
```

`audit(params)` **and** `audit_detail`

`events()` builds one `UNION ALL` of three tables into a common shape, joined to `users` for `actor_name`. Search uses `ILIKE` with `\`, `%`, `_` escaped so user input is literal. Details produce friendly before/after rows; fare policy diffs compare version N with N−1 (or `DEFAULT_POLICY` for version 1) and list only changed keys.

`metrics()`

Counts rides by status with one `GROUP BY`; "available" drivers = available + active + not on a trip; "fresh" additionally requires a fresh location; stale = available − fresh; allocation sums over the whole ledger.

14. Trip map (`trip_map_service`)

```
phase  = 'in_progress' if IN_PROGRESS else 'approach' if ASSIGNED/ARRIVING else 'overview'
driver = fresh driver position or None
source = 'driver' if phase in ('in_progress','approach') and driver else 'pickup'
origin = driver if source == 'driver' else pickup
target = pickup if (phase == 'approach' and source == 'driver') else destination
value  = cached_route((ride.id, phase, source, towards, at_destination), compute)
db.session.expire_all(); ride = accessible(...)                 # re-check after the slow Google
call
if ride no longer active or phase changed: 409; if source changed: 409
return {..., driver_position, origin_source, target, polyline, distance_m, duration_seconds,
        refresh_after_seconds: 10, generated_at}
```

`cached_route` keeps a dict of `{key: {'future': Future, 'expires': t}}` under a `threading.Lock`. The first caller for a key computes; concurrent callers wait on the same `Future` (≤ 15 s). Results live 60 s (errors 10 s), capacity 256 entries. Because the cache key does **not** include the driver's coordinates, the polyline from the driver's position is refreshed at most once a minute while the `driver_position` marker is updated on every call.

15. Real-time (`realtime.py`)

- `connect(auth)`: requires `auth = {token: <JWT>}`; uses the same `authenticate()` as HTTP; returns `False` (reject) on failure. Records `(user_id, exp, sid)` per socket id and joins `user:{id}`; admins also join `admins`.
- There are no client→server events other than connect/disconnect. Server→client event: `workspace.changed {event_id, server_time}`.
- `dispatch()` runs in the worker thread; see 13.

16. CLI commands

Command (from `backend/`)	Effect
`flask --app run db upgrade`	Apply migrations (production uses `--app wsgi`)
`flask --app run init-db`	`create_all()` (development only)
`flask --app run create-admin`	Interactive admin creation
`flask --app wsgi bootstrap-admin`	Env-driven admin creation
`flask --app run check-routing [--probe]`	Routing diagnostics without printing the key

06 — API documentation

Every endpoint below was read from `backend/app/routes.py`, `backend/app/ride_routes.py` and `backend/app/test_payment_routes.py`. Base URL: `/api/v1`. There are **39 route handlers**; `POST /rides/{ride_id}/{action}` serves five actions, giving 43 logical operations. Two non-API routes in `web.py` serve the SPA.

Conventions

Topic	Rule (source)
Content type	Requests with a body must be a JSON **object**; otherwise 400 `A JSON object is required` (`routes.body`)
Auth header	`Authorization: Bearer <access_token>` for protected endpoints (`routes.authenticated`)
Error body	`{"error": "<message>"}` (`routes.api_error`)
Caching	Every API response carries `Cache-Control: no-store`
Money	Decimal strings with 2 dp, currency `INR`
Timestamps	ISO-8601 UTC with `Z` (e.g. `2026-11-02T09:15:00.123456Z`); `expires_at` in auth payloads is a Unix integer
Not found vs forbidden	Resources you don't participate in return **404**, not 403

Common auth errors for every protected endpoint: **401** `Please sign in again` (missing/invalid/expired token, revoked session, disabled user); **403** `You do not have access to this workspace` (wrong role).

Shared response objects

User (`User.public()`):

```
{"id": 7, "name": "Asha", "mobile": "+919876543210", "user_code": "DRV-ABCDEFGHIJKL",
 "role": "driver", "status": "active", "status_revision": 1,
 "vehicle": {"registration": "MH 12 AB 1234", "model": "Maruti Suzuki Dzire", "color": "White",
"version": 1}}
```

`vehicle` appears only for drivers (`null` if none recorded).

Quote (`FareQuote.public()`), example from the simulator with default rates:

```
{"id": "3f2c…-uuid", "pickup": "City centre", "destination": "Riverside",
 "pickup_lat": 19.076, "pickup_lng": 72.8777, "destination_lat": null, "destination_lng": null,
 "distance_km": "17.34", "distance_m": 17340, "duration_min": 52, "duration_seconds": 3120,
 "traffic_available": false, "base": "40.00", "distance_fare": "208.08", "time_fare": "104.00",
 "minimum_adjustment": "0.00", "subtotal": "352.08", "surge_multiplier": "1.00", "surge_amount":
"0.00",
 "surge_reason": "No surge", "fare_rule_version": 0, "total": "352.08", "currency": "INR",
 "provider": "simulator", "expires_at": "2026-11-02T09:20:00.000000Z"}
```

Ride (`ride_service.public(ride, user)`):

```
{"id": "9b1e…-uuid", "status": "ASSIGNED", "round_no": 1, "radius_m": 500,
 "quote": { …Quote… },
 "requested_at": "…Z", "expires_at": "…Z", "completed_at": null,
 "driver": {"name": "Ravi", "user_code": "DRV-…", "vehicle": {"registration": "MH 12 AB 1234",
"model": "Dzire", "color": "White"},
            "rating": {"average": "4.8", "count": 12}},
 "rider": {"name": "Asha", "rating": {"average": null, "count": 0}},
 "test_payment": null, "payment": null, "test_payments_enabled": true, "cancellation": null}
```


Role-dependent keys:

- `my_rating` — rider/driver viewers, COMPLETED rides only (`null` if not rated yet).
- `ratings` — admins only: list of both ratings with comments.
- `settlement` — drivers and admins only (never riders, never in invitation cards): `{eligible_fare, driver_pct, company_pct, driver_amount, company_amount, currency, status:"ALLOCATED", ride_id, share_version, created_at}`.
- `cancellation` — for CANCELLED rides: `{by: "rider"|"driver"|null, by_you, reason, cancelled_at}`.
- `payment` — `{method, status, amount, currency, confirmed_at}` after completion.
- `test_payment` — `{status, amount, currency, test_only: true}` if a mock record exists.

Payment (`RidePayment.public()`): `{"method": "CASH", "status": "CONFIRMED", "amount": "352.08", "currency": "INR", "confirmed_at": "…Z"}`

---

1. System endpoints

`GET /health`

- **File/handler:** `routes.py::health`. **Auth:** none.
- **Response 200:**


```
{"status": "ok", "stage": "location-matching", "routing_diagnostics_version": 1, "route_provider":
"simulator",
 "matching_worker": true, "simulator": true, "storage": "sqlite-development"}
```


`matching_worker` is true if the background loop ticked within 10 s. `storage` is `sqlite-development` for any SQLite URL (including production SQLite) and `sql-server` otherwise.

- **Frontend usage:** none (operational).

`GET /ready`

- **Handler:** `routes.py::readiness`. **Auth:** none.
- **Logic:** compute Alembic head from `migrations/` (cached in `extensions['migration_head']`), read `alembic_version`, require exact match and a worker tick within 10 s. Any exception → rollback, not ready.
- **200:** `{"status":"ready","database":true,"matching_worker":true}` · **503:** `{"status":"not_ready","database":false,"matching_worker":true}`
- **Used by:** Docker `HEALTHCHECK`, Render `healthCheckPath`, `deploy/smoke.py`.

`GET /openapi.json` • `GET /docs`

- `spec.SPEC` as JSON; Swagger UI HTML loading `swagger-ui-dist@5` from jsDelivr. **Auth:** none. Partial coverage (see 02).

---

2. Authentication endpoints

`POST /auth/register`

Item	Value
Handler	`routes.py::registration` → `services.register`
Middleware	`limit_auth` (10 POSTs/min/IP to login+register combined per IP key)
Auth	none
Body	`name` (1–100), `mobile` (with country code, ≤ 40 chars), `password` (10–128), `role` (`rider` default, or `driver`)
DB	`SELECT users WHERE mobile`; `SELECT users.id WHERE user_code` (loop); `INSERT users`
Success	**201** `{"user": User}`; drivers get `status: "pending"`
Errors	400 name/password/role/mobile validation messages; 409 `An account with this mobile already exists` / `Account already exists; please sign in`; 429 `Too many attempts. Try again in one minute`
Frontend	`main.jsx::AuthPage.submit`

Request: `{"name":"Asha","mobile":"+91 98765 43210","password":"correct-horse-battery","role":"rider"}` → mobile stored as `+919876543210`.

`POST /auth/login`

Item	Value
Handler	`routes.py::sign_in` → `session_service.check_origin(require_header=False)` → `services.login` → `session_service.issue` → `cookie()`
Headers	`Content-Type: application/json`; the SPA also sends `X-Session-Request: 1` (not required here); `Origin` must be same-host or trusted if present; `Sec-Fetch-Site: cross-site` rejected
Body	`mobile`, `password`
DB	`SELECT users`; `INSERT auth_sessions`, `INSERT refresh_tokens`
Success	**200** `{"access_token":"<JWT>","expires_in":900,"expires_at":1793603700,"user":User}` + `Set-Cookie: ridez_refresh=…; HttpOnly; Path=/api/v1/auth; SameSite=Lax; Max-Age=2592000[; Secure]`
Errors	401 `Mobile or password is incorrect` (unknown, wrong, malformed, disabled); 403 origin errors; 429 rate limit
Frontend	`authSession.signIn`

Pending and suspended drivers **can** log in.

`POST /auth/refresh`

Item	Value
Handler	`routes.py::refresh_session` → `check_origin()` → `session_service.refresh`
Headers	`X-Session-Request: 1` **required**; cookie `ridez_refresh`
Body	ignored (SPA sends `{}`)
DB	`refresh_tokens` lookup; conditional `UPDATE used_at`; `INSERT refresh_tokens`; on failure `UPDATE auth_sessions.revoked_at`, presence/invitation updates for drivers
Success	200 same shape as login + new cookie
Errors	403 `Session request header is required` / `Request origin is not allowed` / `Cross-site session requests are not allowed`; 401 `Please sign in again` or `Session credential was already used. Please sign in again` (both **clear the cookie**)
Frontend	`authSession.refreshSession` (boot, timer, 401 retry)

`POST /auth/logout`

- **Handler:** `logout_session` → `check_origin()` → `session_service.logout()`. **Headers:** `X-Session-Request: 1`; cookie and/or `Authorization` (fallback if the cookie is missing).
- **Effect:** revoke the matching session, stop driver requests, disconnect revoked sockets. Active rides are preserved.
- **200:** `{"status":"signed_out"}` + cookie deleted. Never fails because of a missing session.

`POST /auth/logout-all`

- **Handler:** `logout_all_sessions` (`@authenticated()`), `check_origin()`, `logout(g.user, all_devices=True)`.
- **200:** `{"status":"signed_out_all_devices"}` + cookie deleted. **401** also clears the cookie.

`GET /me`

- `@authenticated()` → `{"user": User}`. Not used by the SPA (it uses the `user` in auth payloads and `/workspace`).

---

3. Workspace and rides

`GET /workspace`

Item	Value
Handler	`ride_routes.py::workspace` → `ride_service.snapshot(g.user)`
Auth	any role
Side effect	runs `matching.advance()` (may commit round expiries)
Success	200 role-specific snapshot (see 05 §5)
Frontend	`RideWorkspace.jsx::useWorkspace` (poll 2 s/15 s + on socket events)

Rider example (abridged):

```
{"rides": [ …Ride… ], "active_ride": { …Ride… }, "simulator": true, "route_provider": "simulator",
 "booking_enabled": true, "user": { …User… }, "recent_cancellation": null,
 "my_rating_summary": {"average": "4.9", "count": 8},
 "rating_tags": ["Great driving","Clean car","Friendly","On time","Smooth ride","Unsafe
driving","Late","Rude","Dirty car","Wrong route"],
 "trip_otp": "042917"}
```

Driver adds: `location {fresh, recorded_at, accuracy_m}`, `available`, `resume_available`, `invitations [Ride…]`, `earnings [ledger + payment]`, `allocated_earnings`, `confirmed_earnings`, `pending_earnings`, and `suspension {reason, created_at}` when suspended. Admin adds `metrics` and `drivers`.

`POST /fares/quotes`

Item	Value
Handler	`ride_routes.py::quote` → `ride_service.quote`
Auth	rider
Middleware	inline limiter: 10 requests / 60 s per rider (`extensions['quote_attempts']`)
Body	`pickup` (1–160), `destination` (1–160), `pickup_lat` (–90..90), `pickup_lng` (–180..180), optional `destination_lat`, `destination_lng` (**required in Google mode**)
External	Google Routes `computeRoutes` in Google mode
DB	reads `fare_policies`, (demand surge) `rides` / `fare_quotes` / `driver_presence` / `driver_locations`; `INSERT fare_quotes`
Success	**201** `{"quote": Quote + "polyline": "<encoded> or null"}`
Errors	400 validation (`Pickup must contain 1–160 characters`, `Pickup latitude must be a finite number between −90 and 90`, `Pickup and destination must be different`, `Select destination coordinates on the map or enter them manually`, `Pickup and destination coordinates must be different`); 422 `No driving route was found between these locations`, `Fare exceeds the supported maximum…`; 429 `Too many fare requests…`; 502 Google denied/invalid/failed; 503 simulator disabled, Google key missing, Google quota; 504 Google timeout
Frontend	`BookingPanel.submit` via `perform`

Request (Google mode):

```
{"pickup":"Bandra Kurla Complex, Mumbai","destination":"Chhatrapati Shivaji Maharaj International
Airport",
 "pickup_lat":19.0662,"pickup_lng":72.8679,"destination_lat":19.0896,"destination_lng":72.8656}
```

`POST /rides`

Item	Value
Handler	`ride_routes.py::book` → `ride_service.book`
Auth	rider
Body	`{"quote_id": "<uuid>"}` — the **idempotency key**
DB	`fare_quotes`, `rides` (insert SEARCHING), `active_rides` (insert rider), `ride_match_rounds`, `ride_invitations`, `users` (lock), `outbox_events`
Success	**201** `{"ride": Ride}`; a retry with the same quote returns the same ride (still 201)
Errors	400 `A quote_id is required`; 404 `Quote not found`; 409 `Quote expired. Request a new estimate`, `Route mode changed. Get a new quote before booking`, `Get a new quote with pickup coordinates`, `You already have an active ride`; 503 simulator disabled
Frontend	`BookingPanel` "Confirm ride"

`GET /rides/{ride_id}`

- `ride_routes.py::details`; any authenticated user; `expire()`, `accessible()` (participant or admin, else 404), returns `{"ride": Ride}`. Not used by the SPA.

`POST /rides/{ride_id}/accept`

Item	Value
Handler	`ride_routes.py::accept` → `ride_service.accept`
Auth	driver
Body	none
DB	lock `users`; conditional `UPDATE rides` (assign + vehicle snapshot); `INSERT active_rides`; `UPDATE ride_invitations`; `driver_presence`; `ride_match_rounds.outcome`; outbox
Success	200 `{"ride": Ride}` (status ASSIGNED); idempotent if already this driver's ride
Errors	404 `Invitation not found`; 409 `You already have an active trip`, `Refresh your location before accepting`, `This invitation is no longer available`, `Another driver has already accepted this ride`, `Driver or ride was assigned elsewhere. Refresh and retry`; 403 `You must be approved and available to accept`, `Your driver status changed. New rides are blocked`; 503 simulator disabled
Frontend	invitation card "Accept ride" in `RideWorkspace`

`POST /rides/{ride_id}/reject`

- Driver. `ride_service.reject`: invitation must exist (404). PENDING → REJECTED + notify. Returns `{"status": "REJECTED"}` (or the current status if not pending).

`POST /rides/{ride_id}/{action}` — `depart`, `arrive`, `start`, `complete`, `cancel`

Item	Value
Handler	`ride_routes.py::progress` → `ride_service.move`
Auth	any authenticated; service enforces: transitions = assigned **driver** only (403 `Only the assigned driver can progress this trip`); cancel = ride's rider or assigned driver (403 `Only the rider or assigned driver can cancel this ride`)
Body	`start`: `{"otp":"123456"}` (required JSON object); `cancel`: `{"reason":"Plans changed"}` (optional only while SEARCHING); others: ignored
Success	200 `{"ride": Ride}`; retries of an already-applied transition return the current ride
Errors	404 `Ride not found`, `Unknown trip action`; 409 `Cannot depart a trip in searching state` (etc.), `Ride changed. Refresh and retry`, `Only rides that have not started can be cancelled`, `Ride changed. Refresh and review before cancelling`; 400 `Enter the correct six-digit OTP provided by the rider`, `Reason must contain 1–300 characters`; 429 `Too many OTP attempts. Try again in one minute`
`complete` side effects	`revenue_ledger` insert, `ride_payments` PENDING insert, claims released, driver availability decided
Frontend	`RideCard` primary button, `CancelRideControl`

`GET /rides/{ride_id}/map`

Item	Value
Handler	`ride_routes.py::trip_map` → `trip_map_service.trip_map`
Auth	rider or driver participant (admins → 403 `Trip maps are available to the rider and assigned driver only`)
External	Google Routes (cached 60 s per ride/phase/source/target)
Success	200 `{"map": {...}}`
Errors	404 not a participant; 409 `Live trip maps are only available for active rides`, `Google road routes are unavailable for simulated rides`, `This ride does not have Google route coordinates`, `Ride state changed. Refresh the trip map`, `Driver location availability changed. Refresh the trip map`; 503 `Google trip routing is currently unavailable`; 429 `Route updates are busy…`; 502/503/504 Google errors
Frontend	`TripRouteMap.jsx::useTripRoute`

Response example:

```
{"map": {"provider":"google","ride_id":"…","phase":"approach","pickup":
{"lat":19.0662,"lng":72.8679},
 "destination":{"lat":19.0896,"lng":72.8656},
 "driver_position":{"lat":19.0601,"lng":72.8611,"recorded_at":"…Z","accuracy_m":12.0},
 "origin_source":"driver","target":"pickup","generated_at":"…Z","refresh_after_seconds":10,
 "polyline":"a~l~Fjk~uO…","distance_m":1200,"duration_seconds":240,"at_destination":false}}
```

---

4. Driver endpoints

`POST /driver/availability`

Item	Value
Handler	`ride_routes.py::availability` → `ride_service.availability`
Auth	driver
Body	`{"available": true, "latitude": 19.076, "longitude": 72.8777, "accuracy_m": 12.5, "recorded_at": "2026-11-02T09:15:00.000Z"}` or `{"available": false}`
Success	200 `{"available": true, "resume_available": true, "active_trip": false}`
Errors	400 `Available must be true or false` + location validation messages; 403 `Your driver status does not allow new requests`, `Driver approval is required`; 409 `Availability changed concurrently. Refresh and retry`, `Location is older than the latest accepted update`, `Location movement is implausible`; 503 simulator disabled (going online only)
Frontend	`RideWorkspace.changeDriverAvailability`, `useDriverLocation.pauseRequests`

`PUT /driver/location`

- **Handler:** `ride_routes.py::location` → `location_service.update` + commit. **Auth:** driver.
- **Body:** `latitude`, `longitude`, `accuracy_m` (0–200), `recorded_at` (ISO with timezone, ≤ 15 s in future, newer than 90 s ago).
- **200:** `{"location": {"fresh": true, "recorded_at": "…Z", "accuracy_m": 12.5}}` (never echoes coordinates).
- **Errors:** 400 validation / `Location time is stale or in the future`; 403 `Driver approval is required`; 409 `Location updates are only allowed while available or on a trip`, older/implausible.
- **Frontend:** `useDriverLocation.send` every 30 s (10 s on trip).

`GET /driver/overview`

- Driver. Returns `{approval, trips, allocated_earnings, availability_enabled, available}`. Not used by the SPA.

---

5. Payments and ratings

`POST /rides/{ride_id}/payment-method`

- **Handler:** `payment_method` → `payment_service.choose`. **Auth:** rider (owner, else 404).
- **Body:** `{"method": "CASH" | "ONLINE"}`.
- **200:** `{"payment": Payment}`.
- **Errors:** 400 `Choose CASH or ONLINE`; 404 `Ride not found`; 409 `Payment is only available for completed rides`, `Online payment is not available. Please pay by cash` (test payments disabled), `This ride is already paid`.
- **Frontend:** `PaymentPrompt.choose`.

`POST /rides/{ride_id}/confirm-cash`

- **Handler:** `confirm_cash` → `payment_service.confirm_cash`. **Auth:** driver (assigned, else 404). **Body:** none (the SPA sends `{}`).
- **200:** `{"payment": {"method":"CASH","status":"CONFIRMED",…}}` (idempotent).
- **Errors:** 409 `Payment is only available for completed rides`, `The rider has already paid online`.
- **Frontend:** `DriverCollectCard.received`.

`POST /rides/{ride_id}/test-payment`

- **Handler:** `test_payment_routes.py::create_test_payment`. **Auth:** rider (owner).
- **Preconditions:** `TEST_PAYMENTS_ENABLED` (else **404** `Test payments are disabled`), ride COMPLETED (409), not already paid by cash (409 `This ride is already paid`).
- **200:** `{"payment": {"status":"PENDING","amount":"352.08","currency":"INR","test_only":true}, "path": "/test-payment/<signed-token>", "expires_in": 900}`.
- **Frontend:** `TestPayment.jsx::useTestQr.generate`.

`GET /test-payments/{token}` • `POST /test-payments/{token}`

- **Handlers:** `view_test_payment`, `simulate_test_payment`. **Auth:** none — the signed token is the credential (15 min).
- **POST body:** `{"status": "SUCCESS" | "FAILED" | "PENDING"}`.
- **Logic:** `UPDATE mock_payments SET status WHERE ride_id AND status != 'SUCCESS' AND status != new`; if changed to SUCCESS → `payment_service.confirm(ride,'ONLINE')`; notify rider and driver.
- **200:** `{"payment": {...}}`. **Errors:** 404 disabled/not found; 410 `Test QR expired or invalid…`; 400 `Choose PENDING, FAILED or SUCCESS`; 409 `This ride is already paid in cash`.
- **Frontend:** `TestPaymentPage`.

`POST /rides/{ride_id}/rating`

- **Handler:** `rate` → `rating_service.submit`. **Auth:** rider or driver (admin → 403 `Only the rider and driver of a trip can rate it`).
- **Body:** `{"stars": 5, "tags": ["Great driving","On time"], "comment": "Smooth ride"}`.
- **200:** `{"rating": {"rater_role":"rider","stars":5,"tags":["Great driving","On time"],"created_at":"…Z","comment":"Smooth ride"}}`.
- **Errors:** 404 `Ride not found`; 409 `You can rate a trip once it is completed`, `You have already rated this trip`; 400 `Choose a rating from 1 to 5 stars`, `Choose up to five of the listed tags`, `Comment must be text of at most 300 characters`.
- **Frontend:** `Rating.jsx::RatingForm.submit`.

6. Admin endpoints (all require role `admin`)

`GET /admin/overview`

Full admin `snapshot()` (same as `/workspace` for admins). Not used by the SPA.

`GET /admin/drivers?page=1&status=&q=`

- **Handler:** `admin_drivers` → `admin_service.driver_list`.
- **Query:** `page` 1–100000, `status` ∈ `''|pending|active|suspended|disabled`, `q` ≤ 100 chars (literal ILIKE over name, user_code, mobile).
- **200:** `{"page":1,"page_size":20,"total":42,"items":[User + {"location":{fresh,recorded_at,accuracy_m},"active_ride":{"id","status"}|null,"rating":{average,count}}]}`.
- **Errors:** 400 `Page must be a positive integer`, `Choose a valid driver status`, `Search must contain at most 100 characters`.

`POST /admin/drivers/{driver_id}/approve` · `/suspend` · `/reinstate`

- **Handlers:** `approve` (via `ride_service.approve`), `suspend_driver`, `reinstate_driver` → `admin_service.change_driver_status`.
- **Body:** `{"version": <status_revision>, "reason": "Documents verified"}` (`version` optional for approve — legacy clients).
- **Transitions:** approve pending→active, suspend active→suspended, reinstate suspended→active. Each also stops new requests (driver left offline).
- **200:** `{"user": User}`; exact retries are idempotent.
- **Errors:** 404 `Driver not found`; 400 `Reason must contain 1–300 characters`, `A valid driver status version is required`; 409 `Driver status changed. Reload drivers and review again`, `Cannot suspend a pending driver`.
- **Frontend:** `DriverManagement.save`.

`PUT /admin/drivers/{driver_id}/vehicle`

- **Handler:** `update_driver_vehicle` → `admin_service.save_driver_vehicle`.
- **Body:** `{"version":0,"registration":"mh 12 ab 1234","model":"Dzire","color":"White","reason":"RC verified"}` (registration upper-cased and whitespace-collapsed).
- **200:** `{"user": User}` with updated `vehicle`.
- **Errors:** 404; 400 `A valid vehicle version is required`, `Car number must contain 4–20 letters, numbers, spaces or hyphens`, field length messages; 409 `Disabled driver accounts require a separate review`, `Vehicle details cannot change during an active trip`, `Vehicle details changed. Reload drivers and review again`.
- **Frontend:** `VehicleEditor.save`.

`GET /admin/fare-settings` · `PUT /admin/fare-settings`

- **Handlers:** `fare_settings`, `update_fare_settings` → `fare_service.settings` / `save`.
- **GET 200:** `{"version": 3, "policy": {...}, "history": [{"version":3,"reason":"…","updated_by":1,"created_at":"…Z"}, …]}`.
- **PUT body:**

```
{"version": 3, "reason": "Weekend airport surge",
 "policy":
{"base":"40.00","per_km":"12.00","per_minute":"2.00","minimum":"40.00","timezone":"Asia/Kolkata",

"surge_enabled":true,"max_multiplier":"2.00","traffic_enabled":false,"traffic_threshold":"1.25","tr
affic_multiplier":"1.25",

"demand_enabled":false,"demand_threshold":"2.00","demand_multiplier":"1.25","demand_min_requests":5
,
  "zones":
[{"id":"airport","name":"Airport","latitude":19.0896,"longitude":72.8656,"radius_m":2000,"multiplie
r":"1.50",
            "days":[4,5,6],"start":"18:00","end":"02:00","enabled":true}]}}
```


- **PUT 200:** same as GET. **Errors:** 409 `Fare settings changed. Reload before saving` (stale version or concurrent save); 400 specific validation messages (e.g. `base must be between 0 and 100000, with at most two decimals`, `Enter a valid IANA timezone…`, `Use at most 20 surge zones`, `Zone times must use HH:MM (24-hour format)`).

`GET /admin/revenue-share` · `PUT /admin/revenue-share`

- **GET 200:** `{"version":2,"active":{…split…},"upcoming":[…],"history":[…],"server_time":"…Z"}`; default active `{"version":0,"driver_pct":"85.00","company_pct":"15.00","effective_from":null}`.
- **PUT body:** `{"version":2,"driver_pct":"80.00","company_pct":"20.00","effective_from":"2026-12-01T00:00:00Z","reason":"New contract"}` (`effective_from: null` = immediately).
- **Errors:** 409 `Revenue settings changed. Reload before saving`; 400 `Percentages must be between 0 and 100 with at most two decimals`, `Driver and company percentages must total exactly 100`, `Effective date must include a timezone, or use null for immediately`, `Scheduled changes must be in the future and within one year`, reason length.

`GET /admin/audit?page=1&category=&q=`

- **Handler:** `admin_audit` → `admin_service.audit`. `category` ∈ `''|driver|fares|revenue`.
- **200:** `{"page":1,"page_size":20,"total":5,"items":[{"id":"…","category":"driver","action":"driver.suspended","actor_id":1,"actor_name":"Admin","created_at":"…Z","reason":"…","target":"7"}]}`.

`GET /admin/audit/{category}/{identifier}`

- `identifier` is an `admin_audit` UUID for `driver`, or a version integer for `fares` / `revenue`. Invalid → 404 `Audit entry not found`.
- **200:** `{"title":"Driver suspension","subject":"Ravi · DRV-…","changes":[{"field":"Account status","before":"Active","after":"Suspended"}]}`; revenue adds `effective_from`.

---

7. Non-API routes (`web.py`)

Route	Behaviour
`GET /`	`index.html` from `FRONTEND_DIST` (or JSON 404 if no build)
`GET /<path>`	Static file; SPA fallback for `login`, `register`, `rider/*`, `driver/*`, `admin/*`, `test-payment/*`; JSON 404 for `api/*`, `socket.io/*`, dot-files, other unknown paths

8. Socket.IO contract (`realtime.py`)

Item	Value
Path	`/socket.io` (default) — proxied by Vite in development
Transports	client requests `['polling','websocket']`
Connect auth	`io({auth: {token: "<access_token>"}})` ; invalid/missing → connection refused
Rooms	`user:{id}` ; admins also `admins`
Server → client	`workspace.changed` with `{"event_id": "<uuid>", "server_time": "…Z"}`
Client → server	none (besides connect/disconnect)
Server disconnects	when the JWT `exp` passes, the session is revoked, or the user is disabled (checked every tick and on logout)
Delivery semantics	at-least-once; clients deduplicate by `event_id` and re-fetch `/workspace`

9. Endpoints planned in `docs/PRODUCT_BLUEPRINT.md` but not implemented

`POST /driver/status`, `GET /driver/invitations`, `GET /driver/earnings`, `POST /maps/routes`, `GET /rides` (list), `GET /admin/rides`, fare-rule/surge-zone CRUD as separate resources, and socket events `invitation.created`, `ride.assigned`, `driver.location_changed`. The implementation replaced these with `/workspace` snapshots and a single `workspace.changed` invalidation event.

07 — Database documentation

Source of truth: `backend/app/models.py` and `backend/migrations/versions/*.py`. ERD: diagrams/erd.md.

1. Database engines

Context	URL	Notes
Local dev	`sqlite:///ridez.db` (default when `APP_ENV=development` and no `DATABASE_URL`)	Relative path resolved by Flask-SQLAlchemy into the instance folder **(inferred)**
Tests	`sqlite://` (memory) or `sqlite:///<tmp>/…`	`db.create_all()` (not migrations) in fixtures
Render	`sqlite:////var/data/ridez.db`	Persistent disk; single instance
Intended	`mssql+pyodbc://…?driver=ODBC+Driver+18+for+SQL+Server`	**Not validated**; driver not in image

Engine option: `pool_pre_ping=True`. All migrations use `batch_alter_table`, which lets Alembic rebuild tables on SQLite (SQLite cannot ALTER constraints).

2. Conventions

- **IDs**: `users.id` is an auto-increment integer. Rides, quotes, sessions, outbox events and admin audit rows use `String(36)` UUID4 strings generated in Python (`str(uuid4())`).
- **Timestamps**: naive UTC `DateTime` (except `users.created_at`, `DateTime(timezone=True)`).
- **Money**: `Numeric(10,2)` (mock payments `Numeric(12,2)`), percentages `Numeric(5,2)`.
- **Enums** are strings guarded by CHECK constraints where defined.
- **No ORM relationships** (`relationship()`) are declared; joins are explicit in queries.
- **No ON DELETE cascades**; the application never deletes users, rides or quotes. The only deletes are `active_rides` (claim release) and old delivered `outbox_events`.

3. Tables

Legend: **PK** primary key, **FK** foreign key, **UQ** unique, **IX** index, **NN** not null.

3.1 `users` — accounts (`User`)

Column	Type	NN	Default	Key	Notes
`id`	Integer	✓	autoincrement	PK	
`name`	String(100)	✓			trimmed, 1–100
`mobile`	String(20)	✓		UQ	E.164 (`+919876543210`)
`password_hash`	String(512)	✓			Werkzeug hash string
`user_code`	String(40)	✓		UQ	`RID-` / `DRV-` / `ADM-` + base32 HMAC prefix

Column	Type	NN	Default	Key	Notes
`role`	String(16)	✓			CHECK `valid_user_role`: rider, driver, admin
`status`	String(16)	✓	`'active'`		CHECK `valid_user_status`: active, pending, disabled, suspended; suspended only if role = driver
`status_revision`	Integer	✓	0 (server default `'0'`)		Optimistic version for admin reviews; also target of no-op lock UPDATEs
`created_at`	DateTime(tz)	✓	now UTC		

Create: `services.register` (API/CLI/bootstrap). **Read:** login, authenticate (every request), ride JSON, admin lists. **Update:** `status` / `status_revision` by `admin_service.change_driver_status`; no-op `status_revision = status_revision` locks by `ride_repository.lock_*` and vehicle save. **Delete:** never.

3.2 `driver_presence` — online flag (`DriverPresence`)

Column	Type	NN	Default	Key
`driver_id`	Integer	✓		PK, FK users.id
`available`	Boolean	✓	False	
`resume_available`	Boolean	✓	True (server `true`)	

`available` = currently eligible to receive invitations. `resume_available` = driver's choice to stay available after the current trip (added by `e83a01cf742d`). Written by `ride_repository.set_available`, `session_service.stop_requests`.

3.3 `driver_locations` — latest GPS fix (`DriverLocation`)

Column	Type	NN	Key	Notes
`driver_id`	Integer	✓	PK, FK	one row per driver (no history)
`latitude`, `longitude`	Float	✓		
`accuracy_m`	Float	✓		0–200
`recorded_at`	DateTime	✓		device capture time
`updated_at`	DateTime	✓	IX	server receipt time
`fresh`	Boolean	✓		default True; set False by `matching.advance` after 90 s

3.4 `driver_vehicles` — admin-reviewed car (`DriverVehicle`)

Column	Type	NN	Key
`driver_id`	Integer	✓	PK, FK
`registration`	String(20)	✓	
`model`	String(80)	✓	
`color`	String(40)	✓	
`version`	Integer	✓	optimistic version (1 on first save)

3.5 `fare_quotes` — immutable price snapshots (`FareQuote`)

Column	Type	NN	Default	Notes
`id`	String(36)	✓		PK UUID
`rider_id`	Integer	✓		FK users
`pickup`, `destination`	String(160)	✓		labels
`distance_km`	Numeric(10,2)	✓		
`duration_min`	Integer	✓		ceil(seconds/60)
`base`, `distance_fare`, `time_fare`, `total`	Numeric(10,2)	✓		
`expires_at`	DateTime	✓		created + 5 min
`pickup_lat`, `pickup_lng`	Float			required by current code; nullable for legacy rows
`destination_lat`, `destination_lng`	Float			null in simulator mode unless supplied
`provider`	String(16)	✓	`'simulator'`	`google` or `simulator`
`distance_m`, `duration_seconds`, `baseline_seconds`	Integer			baseline = Google `staticDuration`
`subtotal`, `minimum_adjustment`	Numeric(10,2)			
`surge_multiplier`	Numeric(5,2)	✓	1	
`surge_amount`	Numeric(10,2)	✓	0	
`surge_reason`	String(500)	✓	`'No surge'`	
`fare_rule_version`	Integer	✓	0	`fare_policies.version` used (0 = defaults)
`rate_snapshot`	Text			JSON: rates, cap, timezone, applied `zone_ids`, `traffic_ratio`, `demand_ratio`

Never updated after insert. Polyline is **not** stored.

3.6 `fare_policies` — versioned fare rules (`FarePolicy`)

Column	Type	NN	Key
`version`	Integer	✓	PK (manual, no autoincrement)
`settings`	Text	✓	JSON policy (see `fare_service.DEFAULT_POLICY` for keys)
`updated_by`	Integer	✓	FK users
`reason`	String(300)	✓	
`created_at`	DateTime	✓	

Append-only. Latest version = current policy. No rows → `DEFAULT_POLICY` with version 0.

3.7 `rides` — trips (`Ride`)

Column	Type	NN	Default	Key / notes
`id`	String(36)	✓		PK UUID
`rider_id`	Integer	✓		FK users
`driver_id`	Integer			FK users; set on assignment
`quote_id`	String(36)	✓		FK fare_quotes, **UQ** (booking idempotency)
`status`	String(24)	✓		CHECK `valid_ride_state` (8 values)
`requested_at`	DateTime	✓		
`expires_at`	DateTime	✓		**current matching round deadline** (meaningless after assignment)
`completed_at`	DateTime			
`cancelled_at`	DateTime			
`cancelled_by`	Integer			FK users (`fk_rides_cancelled_by_users`)
`cancellation_reason`	String(300)			
`vehicle_registration`	String(20)			snapshot at acceptance
`vehicle_model`	String(80)			snapshot
`vehicle_color`	String(40)			snapshot
`round_no`	Integer	✓	1	
`radius_m`	Integer	✓	500	

No index on `rider_id`, `driver_id` or `status` beyond the PK/UQ (see 18).

3.8 `active_rides` — participation claims (`ActiveRide`)

Column	Type	NN	Key
`user_id`	Integer	✓	**PK**, FK users
`ride_id`	String(36)	✓	FK rides

Rows exist for the rider from booking until completion/cancellation/no-driver, and for the driver from acceptance until completion/cancellation. **PK on `user_id` = a user can be in at most one active ride.** Deleted by `ride_repository.release(ride_id)`.

3.9 `ride_invitations` — offers (`Invitation`)

Column	Type	NN	Default	Key
`ride_id`	String(36)	✓		PK, FK
`driver_id`	Integer	✓		PK, FK
`status`	String(16)	✓	`'PENDING'`	PENDING, ACCEPTED, REJECTED, EXPIRED, CLOSED, OFFLINE (no CHECK)
`round_no`	Integer	✓	1	round in which the invite was sent
`expires_at`	DateTime			= round deadline

Composite PK prevents re-inviting the same driver for the same ride.

3.10 `ride_match_rounds` — round history (`MatchRound`)

Column	Type	NN	Key
`ride_id`	String(36)	✓	PK, FK
`round_no`	Integer	✓	PK
`radius_m`	Integer	✓	
`started_at`	DateTime	✓	
`expires_at`	DateTime	✓	
`outcome`	String(24)	✓	SEARCHING (default), EXPIRED, ASSIGNED, CANCELLED

Audit/history only; matching logic reads `rides.round_no/expires_at`, not this table. (The final round of a NO_DRIVER_FOUND ride is marked EXPIRED.)

3.11 `outbox_events` — notification queue (`OutboxEvent`)

Column	Type	NN	Key	Notes
`id`	String(36)	✓	PK	event_id sent to clients
`user_id`	Integer		FK	NULL → `admins` room
`created_at`	DateTime	✓		ordering
`delivered_at`	DateTime		IX	NULL = pending

Written in the same transaction as the business change (transactional outbox). Delivered rows are deleted after 7 days.

3.12 `revenue_ledger` — allocations (`RevenueLedger`)

Column	Type	NN	Default	Key
`ride_id`	String(36)	✓		**PK**, FK rides (one allocation per ride)
`driver_id`	Integer	✓		FK users
`eligible_fare`	Numeric(10,2)	✓		= quote total
`driver_pct`, `company_pct`	Numeric(5,2)	✓		
`driver_amount`, `company_amount`	Numeric(10,2)	✓		sum = eligible_fare
`share_version`	Integer	✓	0	revenue setting version used
`created_at`	DateTime			= completed_at

3.13 `revenue_share_settings` — split versions (`RevenueShareSetting`)

Column	Type	NN	Key / constraint
`version`	Integer	✓	PK (manual)
`driver_pct`, `company_pct`	Numeric(5, 2)	✓	CHECK `valid_revenue_split`: each 0–100 and sum = 100
`previous_driver_pct`, `previous_company_pct`	Numeric(5, 2)	✓	split active when saved
`effective_from`	DateTime	✓	IX
`created_at`	DateTime	✓	
`updated_by`	Integer	✓	FK users

Column	Type	NN	Key / constraint
`reason`	String(300)	✓	

3.14 `admin_audit` — driver review log (`AdminAudit`)

Column	Type	NN	Notes
`id`	String(36)	✓	PK UUID
`admin_id`	Integer	✓	FK users
`driver_id`	Integer	✓	FK users
`action`	String(40)	✓	`driver.approved`, `driver.suspended`, `driver.reinstated`, `driver.vehicle.updated`
`reason`	String(300)	✓	shown to driver for suspensions
`created_at`	DateTime	✓	
`before_status`, `after_status`	String(16)		status changes
`status_revision`	Integer		revision after change (used to find current suspension reason and detect retries)
`details`	Text		JSON `{before, after}` for vehicle edits

3.15 `auth_sessions` — server sessions (`AuthSession`)

Column	Type	NN	Key
`id`	String(36)	✓	PK (JWT `sid`)
`user_id`	Integer	✓	FK, IX
`created_at`, `expires_at`	DateTime	✓	30-day absolute lifetime
`revoked_at`	DateTime		

3.16 `refresh_tokens` — rotating credentials (`RefreshToken`)

Column	Type	NN	Key
`token_hash`	String(64)	✓	PK (SHA-256 hex)
`session_id`	String(36)	✓	FK auth_sessions, IX
`created_at`	DateTime	✓	
`used_at`	DateTime		set on rotation (single use)

Old rows are never deleted (see 18).

3.17 `ride_payments` — real payment state (`RidePayment`)

Column	Type	NN	Default	Constraint
`ride_id`	String(36)	✓		PK, FK
`method`	String(16)			CHECK `valid_ride_payment_method`: NULL or CASH/ONLINE/UNRECORDED

Column	Type	NN	Default	Constraint
`status`	String(16)	✓	`'PENDING'`	CHECK `valid_ride_payment_status` : PENDING/CONFIRMED
`amount`	Numeric(10,2)	✓		= quote total
`updated_at`	DateTime	✓		
`confirmed_at`	DateTime			
`confirmed_by`	Integer			FK users (driver for cash; NULL for online)
				CHECK `confirmed_ride_payment_complete` : status = PENDING OR (method and confirmed_at not null)

3.18 `mock_payments` — test QR state (`MockPayment`)

Column	Type	NN	Default	Constraint
`ride_id`	String(36)	✓		PK, FK
`amount`	Numeric(12,2)	✓		
`status`	String(16)	✓	`'PENDING'`	CHECK PENDING/FAILED/SUCCESS

3.19 `ride_ratings` — two-way ratings (`RideRating`)

Column	Type	NN	Key / constraint
`ride_id`	String(36)	✓	PK, FK
`rater_role`	String(8)	✓	PK; CHECK rider/driver
`rater_id`	Integer	✓	FK users
`ratee_id`	Integer	✓	FK users, IX `ix_ride_ratings_ratee_id` (for averages)
`stars`	Integer	✓	CHECK 1–5
`tags`	String(200)		pipe-joined (`|`)
`comment`	String(300)		admin-only visibility
`created_at`	DateTime	✓	

Plus Alembic's own `alembic_version` table (read by `/ready`).

4. Relationships summary

From	To	Cardinality	Enforced by
users (rider)	fare_quotes	1 : N	FK
fare_quotes	rides	1 : 0..1	FK + UQ `rides.quote_id`
users	rides (rider / driver / cancelled_by)	1 : N	FKs
rides	active_rides	1 : 0..2	FK; PK on user
rides	ride_invitations	1 : N	composite PK
rides	ride_match_rounds	1 : N	composite PK
rides	revenue_ledger, ride_payments, mock_payments	1 : 0..1	PK = ride_id

From	To	Cardinality	Enforced by
rides	ride_ratings	1 : 0..2	PK (ride_id, rater_role)
users (driver)	driver_presence, driver_locations, driver_vehicles	1 : 0..1	PK = driver_id
users	auth_sessions → refresh_tokens	1 : N : N	FKs
users (admin)	fare_policies, revenue_share_settings, admin_audit	1 : N	FKs

5. Important queries

Purpose	Query (simplified)	File
Eligible drivers	`SELECT driver_presence JOIN users JOIN driver_locations WHERE available AND users.status='active' AND fresh AND recorded_at>cutoff AND updated_at>cutoff AND driver_id NOT IN (SELECT user_id FROM active_rides)`	`ride_repository.eligible_drivers`
Atomic assignment	`UPDATE rides SET status='ASSIGNED', driver_id, vehicle_* WHERE id=? AND status='SEARCHING' AND expires_at>now AND id IN (valid invitation) AND EXISTS(eligible driver)`	`ride_repository.assign`
State transition	`UPDATE rides SET status=:new … WHERE id=? AND status=:old`	`ride_repository.transition`
Round claim	`UPDATE rides SET round_no=round_no+1 WHERE id AND status='SEARCHING' AND round_no=? AND expires_at=? AND expires_at<=now`	`ride_repository.expire_round`
Driver row lock	`UPDATE users SET status_revision=status_revision WHERE id=? AND role='driver' [AND status='active'] [AND EXISTS available]`	`ride_repository.lock_active_driver/lock_driver`
Refresh single-use	`UPDATE refresh_tokens SET used_at=now WHERE token_hash=? AND used_at IS NULL`	`session_service.refresh`
Payment first-wins	`UPDATE ride_payments SET status='CONFIRMED', method … WHERE ride_id=? AND status='PENDING'`	`payment_service.confirm`
Mock no-downgrade	`UPDATE mock_payments SET status=? WHERE ride_id=? AND status!='SUCCESS' AND status!=?`	`test_payment_routes.simulate_test_payment`
Active revenue split	`SELECT … WHERE effective_from<=at ORDER BY effective_from DESC, version DESC LIMIT 1`	`revenue_service.active`
Rating summary	`SELECT AVG(stars), COUNT(*) FROM ride_ratings WHERE ratee_id=?`	`rating_service.summary`
Audit feed	`UNION ALL` of three tables → JOIN users → ILIKE → ORDER BY created_at DESC, category, id DESC LIMIT 20 OFFSET …	`admin_service.audit`
Metrics	`SELECT status, COUNT(*) FROM rides GROUP BY status`, counts and sums	`admin_service.metrics`

6. Transactions and consistency

- One SQLAlchemy session per request/app context (Flask-SQLAlchemy scoped session). Services commit explicitly; uncommitted work is rolled back at teardown.

- Multi-table operations (booking, acceptance, completion, cancellation, review) run in **one transaction** including their outbox rows.
- Race safety comes from conditional UPDATEs, uniqueness constraints and catching `IntegrityError`, not from `SELECT … FOR UPDATE`.
- Status/business invariants enforced in the database: ride status values, user role/status combination, revenue split sum, payment status/method completeness, rating stars range and role.
- Invariants enforced only in code: invitation status values, match round outcomes, the ride state machine order (enforced through `transition(old,new)`), vehicle registration format.

7. Migrations

Linear chain (oldest → newest). Apply with `flask --app run db upgrade` (dev) or `flask --app wsgi db upgrade` (prod, done automatically by `deploy/start.sh`).

#	Revision	Adds
1	`15e7a3910568`	`users` with role/status CHECKs, UQ mobile and user_code
2	`4b12d0134b2f`	`admin_audit`, `driver_presence`, `fare_quotes`, `rides` (state CHECK, UQ quote_id), `active_rides`, `revenue_ledger`, `ride_invitations`
3	`61c1900fe70b`	`driver_locations` (+IX), `outbox_events` (+IX), `ride_match_rounds`; `fare_quotes.pickup_lat/lng`; `ride_invitations.round_no/expires_at`; `rides.round_no/radius_m`
4	`11ca7c6daf35`	`fare_policies`; quote destination coords, provider, route metrics, subtotal, minimum adjustment, surge columns, `fare_rule_version`, `rate_snapshot`
5	`5854899d25f7`	`revenue_share_settings` (+IX effective_from); `revenue_ledger.share_version/created_at`
6	`9f22dbe09a11`	`auth_sessions`, `refresh_tokens` (+IXs)
7	`b3c8d1f09e22`	`users.status_revision`, new status CHECK incl. `suspended`; `admin_audit.before_status/after_status/status_revision`. **Downgrade refuses** if any driver is suspended
8	`c4a76be0139d`	`driver_vehicles`; `rides.vehicle_*`; `admin_audit.details`
9	`d72f9e014ac3`	`rides.cancelled_at/cancelled_by/cancellation_reason` + FK
10	`e83a01cf742d`	`driver_presence.resume_available` (server default true)
11	`f94b12de853e`	`mock_payments`
12	`7ba01cddd40c`	`ride_payments` + **data back-fill**: every already-COMPLETED ride gets a CONFIRMED payment with method ONLINE (if mock SUCCESS) or UNRECORDED, so historic earnings stay counted
13	`c51e8a2f90d4` (**head**)	`ride_ratings` + IX ratee_id

There are **no seed scripts**. No demo users or fare policies are inserted; defaults live in code (`DEFAULT_POLICY`, 85/15 split). Admins are created by CLI/bootstrap.

8. Backups

`deploy/backup.py <destination>` uses SQLite's online backup API from a read-only source connection, creates the destination exclusively (no overwrite, mode 0600) and runs `PRAGMA integrity_check`. Only works for file-backed SQLite.

08 — Authentication, authorization and security

1. Summary

Concern	Implementation	Status
Identity	Mobile number (E.164) + password	Implemented; **mobile ownership is not verified** (no SMS)
Password storage	Werkzeug `generate_password_hash` (default algorithm of Werkzeug 3.x, which is scrypt **(inferred from library version)**)	Implemented
Access token	JWT HS256, 15 min, claims `sub, sid, iat, exp`, held in browser memory	Implemented
Refresh	Opaque 48-byte random token in HttpOnly cookie, SHA-256 hashed in DB, single-use rotation, replay → revoke session, 30-day absolute session	Implemented
Revocation	Every request validates the DB session row	Implemented
RBAC	`@authenticated(role)` + per-resource ownership checks	Implemented
CSRF	Bearer tokens for business APIs; custom header + Origin + Sec-Fetch-Site checks on cookie endpoints	Implemented
CORS	None configured — same-origin deployment	By design
Rate limiting	In-memory, per-process: auth 10/min/IP, quotes 10/min/rider, OTP 5 failures/min/driver	**Partial** (not shared, resets on restart)
Input validation	Hand-written per field in services	Implemented
Security headers	nosniff, X-Frame-Options DENY, Referrer-Policy, Permissions-Policy, HSTS on HTTPS	Implemented; **no CSP**
Secrets	Env vars; prod requires ≥ 32 chars and distinct; dev auto-generates to `instance/`	Implemented
Account recovery / password reset / MFA	—	**Not implemented**
Account disabling	`disabled` status honoured everywhere	Honoured, but **no endpoint sets it**

2. Registration

`POST /api/v1/auth/register` → `routes.registration` → `services.register(data)`:

1. `limit_auth` before_request allows 10 POSTs per minute per client IP across login/register.
2. `body()` requires a JSON object.
3. Validates name (1–100 after trim), password (10–128 chars; no complexity rules), role (`rider` / `driver`; `admin` only when `allow_admin=True`, i.e. CLI/bootstrap).
4. `normalize(mobile)`: `phonenumbers.parse(value, None)` requires an international prefix, `is_valid_number`, returns E.164.
5. Duplicate check → 409; unique constraint is the final guard (IntegrityError → 409).
6. Permanent `user_code` from HMAC (see 05 §3). Codes are identifiers, **not** credentials.
7. Drivers start `pending`; riders `active`.

3. Login

`POST /api/v1/auth/login` → `check_origin(require_header=False)` → `services.login` → `session_service.issue`:

1. Malformed mobile, non-string or > 128-char password, unknown user, wrong password, disabled user → identical `401 Mobile or password is incorrect`.
2. For unknown users the password is still checked against a cached dummy hash (`DUMMY_HASH`), equalizing timing.
3. `issue(user)` creates `auth_sessions` (expires in `REFRESH_SESSION_DAYS`, 30) and a refresh token; commits.
4. Response: `{access_token, expires_in, expires_at, user}` + `Set-Cookie ridez_refresh`.

Login doesn't require `X-Session-Request` (so a plain form could post) but does reject a foreign `Origin` and `Sec-Fetch-Site: cross-site` — this blocks **login CSRF** (an attacker logging the victim into the attacker's account).

4. Tokens

Access token (JWT)

```
expires = min(issued + timedelta(minutes=15), session.expires_at)
jwt.encode({'sub': str(user.id), 'sid': session.id, 'iat': issued, 'exp': expires}, JWT_SECRET,
algorithm='HS256')
```

- Verified by `session_service.authenticate` with `algorithms=['HS256']` (prevents algorithm confusion) and `options={'require': ['sub','sid','iat','exp']}`.
- Because `sid` must reference a valid, unrevoked, unexpired session owned by `sub`, a stolen JWT stops working immediately after logout.
- Stored only in a JS module variable; never in `localStorage` / `sessionStorage` — not readable after a reload, which is why the SPA refreshes on boot.

Refresh token (cookie)

Attribute	Value	Reason
Name	`ridez_refresh`	
Value	`secrets.token_urlsafe(48)`	~384 bits of entropy
Storage	`refresh_tokens.token_hash = sha256(value)`	DB leak doesn't expose usable tokens
`HttpOnly`	yes	XSS cannot read it
`Secure`	`AUTH_COOKIE_SECURE` (= not development)	HTTPS only in prod
`SameSite`	`Lax`	Not sent on cross-site POSTs
`Path`	`/api/v1/auth`	Not sent to any other endpoint
`Max-Age`	30 days	

Rotation: each refresh marks the presented token used (conditional UPDATE) and issues a new one in the same session. **Replay detection:** presenting an already-used token revokes the whole session (both attacker and victim are signed out). **Expiry:** the session's absolute expiry is not extended; after 30 days the user must log in again.

5. How a request is authenticated and authorized (exact path)

1. Browser `api()` sends `Authorization: Bearer <token>`.
2. `routes.authenticated(role)` rejects missing/malformed headers → 401.
3. `session_service.authenticate(token)`:
   - `jwt.decode` (signature, `exp`, required claims) → `jwt.InvalidTokenError` → 401.
   - `User` by `sub`; missing or `disabled` → 401.
   - `valid_session(sid, user.id)` → None → 401.
4. Role gate: `role` argument vs `g.user.role` → 403.
5. Service-level authorization, e.g.:
   - `ride_service.accessible`: participant or admin, else 404.
   - `move`: transitions only by `ride.driver_id`; cancel by rider or assigned driver.
   - `payment_service.choose`: `ride.rider_id == user.id`; `confirm_cash`: `ride.driver_id == user.id`.
   - `rating_service.submit`: participant of the ride.
   - `trip_map`: rider/driver participant only (admins excluded to protect live location).
   - `location_service.update`: only the authenticated driver's own row (`user.id`).
6. Data-level privacy in serializers: riders never get `settlement`; invitations never include settlement; `location_service.public` omits coordinates; `ratings` with comments only for admins; `trip_otp` only in the rider's own snapshot.

Role matrix

Capability	Rider	Driver	Admin
Quote / book	✓		
Cancel	own ride (pre-start)	assigned ride (ASSIGNED..WAITING)	✗
Availability / location / accept / reject / progress		✓ (active)	
Choose payment method / test QR	✓		
Confirm cash		✓	
Rate	✓	✓	✗
View ride	participant	participant	any
Trip map	participant	participant	✗
Admin endpoints			✓
`/workspace`	own	own	all rides (100), metrics, drivers

6. Logout

- `POST /auth/logout`: session found via cookie, else Bearer `sid`; `revoke()` sets `revoked_at` and calls `stop_requests`; cookie deleted; `realtime.disconnect_invalid()` closes the session's sockets.
- `POST /auth/logout-all`: requires a valid Bearer; revokes **all** sessions of the user.
- Frontend broadcasts `signed-out` to other tabs (`BroadcastChannel`).
- Logout **does not** cancel an active trip (tested in `test_sessions.py::test_driver_logout_stops_matching_and_preserves_active_ride`).

7. CSRF and origin defence (`session_service.check_origin`)

```
if require_header and request.headers.get('X-Session-Request') != '1': 403
origin = request.headers.get('Origin')
allowed = {request.host_url.rstrip('/'), *AUTH_TRUSTED_ORIGINS}
if origin and origin not in allowed: 403
if request.headers.get('Sec-Fetch-Site') == 'cross-site': 403
```

- A custom header cannot be added by a cross-site HTML form, and a cross-site `fetch` with a custom header would need a CORS preflight that the server never approves.
- `TRUST_PROXY_HEADERS=1` + `ProxyFix(x_proto=1)` makes `request.host_url` `https://…` behind Render so the Origin comparison works.
- Business endpoints use Bearer tokens (not cookies), so they are not CSRF-able.

8. Rate limiting

Limiter	Key	Limit	Storage	File
Auth	`request.remote_addr`	10 POSTs to `/auth/login` + `/auth/register` per 60 s	`app.extensions['auth_attempts']`	`routes.limit_auth`
Quotes	rider id	10 per 60 s	`extensions['quote_attempts']`	`ride_routes.quote`
Trip OTP	driver id	5 **failed** attempts per 60 s	`extensions['trip_otp_attempts']`	`services.verify_trip_otp`

All are sliding windows using `collections.deque` and `time.monotonic()`. They are **process-local** (comment in code: "Development limiter only; deploy a shared Redis limiter before multiple workers") and reset on restart. Behind a proxy without `TRUST_PROXY_HEADERS=1`, every client shares the proxy's IP and therefore one auth bucket. There is no per-account lockout.

9. Trip OTP

`trip_otp(ride) = HMAC_SHA256(USER_CODE_SECRET, "trip-otp:v2:" + ride.id) mod 10^6`, zero-padded.

- Different for every ride; stable across restarts (no DB column needed).
- Shown only to the rider (`snapshot.trip_otp`) while the ride is SEARCHING/ASSIGNED/ARRIVING/WAITING.
- Verified with `hmac.compare_digest`; 5 failures/min/driver.
- It is a reusable pickup PIN, **not** an SMS one-time code. Rotating `USER_CODE_SECRET` changes OTPs for unstarted rides.

10. Mock payment capability URLs

`/test-payment/<token>` uses `itsdangerous.URLSafeTimedSerializer(JWT_SECRET, salt='mock-qr-payment-v1')`, `max_age=900`. Anyone holding the URL can change that ride's **mock** status for 15 minutes without logging in; it exposes only amount, currency and status. `Referrer-Policy: no-referrer` on that page prevents the token leaking through the Referer header. The whole feature is disabled unless `TEST_PAYMENTS_ENABLED=1` (default off outside development).

11. Input validation and sanitization

- No schema library; each service validates types explicitly (e.g. rejects `bool` where numbers are expected, because `True` is an `int` in Python).
- Strings are trimmed and length-bounded; numbers are bounded and must be finite.
- SQL injection: all queries use SQLAlchemy parameter binding. User search terms are additionally escaped for `ILIKE` wildcards (`\`, `%`, `_`).
- Output encoding: React escapes text by default; there is no `dangerouslySetInnerHTML` in the codebase. The only raw HTML is the static Swagger page string in `routes.docs`.
- Google error responses are filtered (`denial_reason`) so vendor text is never echoed to users.
- Logging: Gunicorn access log is disabled (URLs contain Socket.IO session IDs); the worker logs a generic message without exception payloads (to avoid leaking coordinates).

12. Security headers (`web.py::security_headers`, app-wide)

Header	Value
`X-Content-Type-Options`	`nosniff`
`Referrer-Policy`	`no-referrer` on `/test-payment/*`, otherwise `strict-origin-when-cross-origin`
`Permissions-Policy`	`geolocation=(self)`
`X-Frame-Options`	`DENY` (clickjacking)
`Strict-Transport-Security`	`max-age=31536000` when the request is HTTPS
`Cache-Control`	`no-store` on all API responses

Missing: `Content-Security-Policy`.

13. Security-related environment variables (names only)

`APP_ENV`, `JWT_SECRET`, `USER_CODE_SECRET`, `AUTH_TRUSTED_ORIGINS`, `TRUST_PROXY_HEADERS`, `TEST_PAYMENTS_ENABLED`, `GOOGLE_MAPS_SERVER_KEY`, `VITE_GOOGLE_MAPS_BROWSER_KEY` (public by design, must be referrer-restricted), `BOOTSTRAP_ADMIN`, `BOOTSTRAP_ADMIN_MOBILE`, `BOOTSTRAP_ADMIN_NAME`, `BOOTSTRAP_ADMIN_PASSWORD` (unset before Gunicorn starts), `DATABASE_URL`.

Production boot fails if `JWT_SECRET` / `USER_CODE_SECRET` are missing, shorter than 32 characters, or equal.

14. Implemented vs missing security measures

Implemented	Missing / partial
Hashed passwords, generic login errors, timing equalization	Password reset, account recovery, MFA
Server-validated sessions, rotation, replay detection	Session/device list UI; refresh token table cleanup
Role + ownership checks, 404 for foreign resources	Admin cannot disable accounts via API
CSRF/origin checks on cookie endpoints	Content-Security-Policy
Per-process rate limits	Shared/distributed rate limiting; per-account lockout
Coordinates hidden from admins/unrelated users	Location data retention policy (only latest kept)

Implemented	Missing / partial
Secrets validation at boot, unset bootstrap creds	Secret rotation procedure (rotating `USER_CODE_SECRET` changes OTPs; `JWT_SECRET` also signs QR tokens)
Non-root container user, `.dockerignore` excludes secrets	`pytest` and dev tooling shipped in prod image
Public `/health`, `/openapi.json`, `/docs`	These disclose configuration (provider, simulator flag) — low risk, but unauthenticated

See 18 — Problems and improvements for recommendations.

09 — Feature-by-feature explanation

Each feature lists: what/why, user interaction, frontend, backend, APIs, DB, external services, execution flow, important files, errors, edge cases and dependencies. Sequence diagrams are linked rather than repeated.

F1. Registration and permanent account codes

What	Creates a rider or (pending) driver account identified by mobile number; issues a permanent code like `RID-K3J…`
Why	One account per real phone number; a human-friendly identifier for support and admin search that never changes
User interaction	`/register` (or `/register?role=driver`), pick "I want to ride/drive", enter name, mobile with country code, password ≥ 10
Frontend	`main.jsx::AuthPage.submit` → `api('/auth/register')` → success card shows `user_code` and "pending approval" note for drivers
Backend	`routes.registration` → `services.register` → `repositories.save`
API	`POST /api/v1/auth/register`
DB	`users` INSERT
Flow	validate → normalize E.164 → duplicate check → HMAC code with collision lengthening → hash password → insert → 201
Errors	400 validation; 409 duplicate; 429 rate limit
Edge cases	`+91 98765 43210` and `+919876543210` are the same account; concurrent duplicate registration → IntegrityError → 409; admins cannot self-register (`Choose rider or driver`)
Depends on	`USER_CODE_SECRET`

Sequence: diagrams/sequences-auth.md#registration.

F2. Login, session restore, refresh and logout

What	Password login producing a 15-min access token and a 30-day rotating refresh cookie; silent restore on reload; logout (this device / all devices)
Why	Keep tokens out of JS-readable storage while letting users stay signed in; allow immediate revocation
Interaction	`/login`; reloading any page restores the session; sidebar "Sign out"; Account → "Sign out all devices"
Frontend	`authSession.js` (signIn, refreshSession, signOut, locks, broadcast), `main.jsx::AuthProvider` (boot restore + timer), `api.js` (401 → refresh → retry)
Backend	`routes.sign_in/refresh_session/logout_session/logout_all_sessions`, `services.login`, `session_service.*`, `realtime.disconnect_invalid`
APIs	`POST /auth/login`, `/auth/refresh`, `/auth/logout`, `/auth/logout-all`
DB	`auth_sessions`, `refresh_tokens`, `users`, `driver_presence`, `ride_invitations`, `outbox_events`
Errors	401 generic credential error; 401 on refresh failure/replay clears cookie; 403 origin/header failures
Edge cases	Two tabs refreshing simultaneously are serialized by Web Locks; a reused refresh token revokes the session; a driver logging out stops receiving invitations but keeps an active trip; logout without cookie falls back to the Bearer `sid`

Depends on	F1; `JWT_SECRET`; HTTPS in production (`Secure` cookie)

Details: 08. Sequences: diagrams/sequences-auth.md.

F3. Admin provisioning (CLI and bootstrap)

What	Creates admin accounts without any public API
How	`flask --app run create-admin` (interactive, hidden password) or `BOOTSTRAP_ADMIN=1` + `BOOTSTRAP_ADMIN_MOBILE/PASSWORD[/NAME]` at container start (`deploy/start.sh` → `flask --app wsgi bootstrap-admin`)
Files	`app/__init__.py` (CLI), `app/bootstrap.py`, `deploy/start.sh`, `deploy/render-start.sh`
Edge cases	Bootstrap never promotes an existing rider/driver, never reactivates a non-active admin, never resets a password, and is idempotent across restarts; bootstrap env vars are `unset` before Gunicorn starts (verified by `deploy/smoke.py`)

F4. Driver approval, suspension and reinstatement

What	Admin review workflow: pending → active (approve), active → suspended (suspend), suspended → active (reinstate), each with a mandatory reason
Why	Drivers must be vetted; operations need to pause a driver without destroying their trip or earnings
Interaction	Admin → Drivers (or Overview) → filter/search → "Approve/Suspend/Reinstate driver" → reason → confirm
Frontend	`DriverManagement.jsx` (`choose`, `save`) sends `{version: status_revision, reason}`; driver sees suspension banner + reason in `RideWorkspace`
Backend	`admin_service.change_driver_status`, `session_service.stop_requests`; approve also routed via `ride_service.approve`
APIs	`POST /admin/drivers/{id}/approve`, `/suspend`, `/reinstate`; `GET /admin/drivers`
DB	`users` (status, status_revision), `admin_audit` INSERT, `driver_presence`, `ride_invitations`, `outbox_events`
Flow	validate → idempotent retry detection → version and status check → conditional UPDATE → stop requests (offline, close invitations) → audit row → commit
Errors	404 not a driver; 400 reason/version; 409 stale version or wrong status
Edge cases	Suspended driver keeps sign-in, active trip, GPS tracking for that trip, OTP start and completion, and earnings; after completion they are not auto-available. Reinstated drivers stay offline until they go available with fresh GPS. Two admins acting on the same version → one wins, the other gets 409. `disabled` drivers have no actions
Depends on	F2 (stop_requests), F6 (availability)

F5. Vehicle details and ride snapshots

What	Admin records the reviewed car (registration, model, colour); each accepted ride copies these onto the `rides` row
Why	Riders must verify the car before boarding; later edits must not rewrite trip history/receipts
Frontend	`DriverManagement` "Vehicle details" (disabled while the driver has an active ride) → `VehicleEditor`; riders see it in `AssignedRideDetails`

Backend	`admin_service.save_driver_vehicle`; snapshot in `ride_repository.assign`
API	`PUT /admin/drivers/{id}/vehicle`
DB	`driver_vehicles` upsert (version+1), `admin_audit` (`driver.vehicle.updated`, details JSON), `rides.vehicle_*` at acceptance
Edge cases	Edit during an active trip → 409; stale version → 409; registration normalized to upper case with single spaces; drivers without a vehicle are still matchable and the UI shows "Car number not provided"

F6. Driver availability and GPS tracking

What	Approved drivers go online with a fresh GPS fix and keep sending location; stale drivers drop out of matching automatically
Why	Matching must only offer rides to drivers who are actually nearby right now
Interaction	Driver overview → "Go available" (browser asks for location permission) → keep tab open; "Go offline"; during a trip, "Stay available after this trip" checkbox
Frontend	`RideWorkspace.changeDriverAvailability` (gets a fix first), `useDriverLocation` (30 s idle / 10 s on trip; permission revoked → go offline)
Backend	`ride_service.availability`, `location_service.update`, `matching.advance` (stale sweep), `ride_repository.lock_active_driver`
APIs	`POST /driver/availability`, `PUT /driver/location`
DB	`driver_presence`, `driver_locations`, `ride_invitations` (→ OFFLINE), `users` lock, `outbox_events`
Validation	accuracy ≤ 200 m; recorded_at within (now−90 s, now+15 s]; not older than last accepted; movement ≤ 200 km/h + accuracies + 100 m while tracking is fresh; replay of same timestamp is a no-op
Errors	403 not active; 409 not available and not on trip; 409 older/implausible; 400 validation; 503 simulator disabled when going online
Edge cases	Location older than 90 s → `fresh=False` within ≤ 1 s by the worker → excluded from matching and acceptance; a driver re-appearing after a gap is accepted anywhere (no jump check when not fresh); browsers throttle background tabs
Depends on	F4 (status active), browser geolocation (HTTPS/localhost)

Sequence: diagrams/sequences-rides.md#driver-goes-available-and-streams-gps.

F7. Address search and map selection

What	Google Places (New) autocomplete for pickup/destination, map click selection, "Use my location", manual coordinates
Why	Google-mode quotes need exact coordinates for both ends
Frontend	`AddressSearch.jsx`, `GoogleRouteMap.jsx` (picker mode), `BookingPanel.jsx`, `googleMaps.js`, `useDriverLocation.getPosition`
Backend	none (browser → Google directly)
External	Maps JavaScript API, Places API (New) with browser key `VITE_GOOGLE_MAPS_BROWSER_KEY`
Edge cases	< 3 characters → no search; stale responses ignored via sequence counters; editing a selected address clears its coordinates and quote; Google errors show "Address search unavailable…" and the map still allows clicking; in simulator mode only free-text labels + pickup coordinates are needed

F8. Fare quotes and surge pricing

What	Server computes route distance/duration and an itemized INR fare; the quote is locked for 5 minutes
Why	Upfront, auditable pricing that later rate changes cannot alter
Interaction	Booking form → "Get fare estimate" → "Your fare, upfront" card with breakdown, expiry time, rate version
Frontend	`BookingPanel.submit` → `perform('/fares/quotes')` → `QuoteDetails`; map draws the returned polyline
Backend	`ride_routes.quote` (rate limit) → `ride_service.quote` → `route_provider.compute` → `fare_service.calculate` (+ `demand_signal`, `scheduled`)
API	`POST /fares/quotes`
DB	read `fare_policies`; demand reads `rides` / `fare_quotes` / `driver_presence` / `driver_locations`; INSERT `fare_quotes`
External	Google Routes (`computeRoutes`) in Google mode
Pricing	`subtotal = max(base + km×per_km + min×per_minute, minimum)`; `total = subtotal × min(highest eligible multiplier, max_multiplier)`; sources: zone (circle around pickup on scheduled days/times in policy timezone), traffic (duration/static ≥ threshold), demand (open requests within 1 km ÷ fresh drivers within 1 km ≥ threshold with min requests)
Errors	Google: 502 denied/invalid/failed, 503 quota/key missing, 504 timeout, 422 no route; **no fallback to simulated data**
Edge cases	Missing `staticDuration` → no traffic surge and `traffic_available:false`; overnight zone windows belong to their start day; total > 99 999 999.99 → 422; 10 quotes/min/rider
Depends on	F17 (fare settings)

F9. Booking and expanding-radius matching

What	Turning a quote into a SEARCHING ride and inviting nearby drivers in rounds: 500 m, then +500 m every 30 s up to `MATCH_MAX_RADIUS_M` (5 km)
Why	Offer the ride to the closest drivers first, widening only when needed, and never spam the same driver twice
Interaction	"Confirm ride" → rider sees "Finding your driver", countdown, round and radius; rider OTP card appears
Frontend	`BookingPanel` → `perform('/rides')`; `RideCard` SEARCHING view with `Countdown`
Backend	`ride_service.book`, `matching.invite`, `matching.advance` (worker + per-request), `ride_repository.eligible_drivers/expire_round`
API	`POST /rides`
DB	`rides`, `active_rides`, `ride_match_rounds`, `ride_invitations`, `users` (lock), `outbox_events`
Errors	404 quote; 409 expired quote, provider changed, missing pickup coords, already active ride
Edge cases	Double-click / retry → same ride (quote id unique); rider can't hold two active rides; after the server was down, overdue rounds are replayed on the original schedule without sending invitations for expired rounds; final timeout → NO_DRIVER_FOUND and rider claim released

Depends on	F6, F8, F14

Sequences: booking, round expiry.

F10. Accepting / rejecting invitations

What	Driver accepts (first valid acceptance wins) or rejects an invitation
Interaction	Driver overview → "Incoming requests" card (rider name + rating, countdown, fare) → Accept/Reject
Frontend	`RideWorkspace` invitation cards → `perform('/rides/{id}/accept')` / `perform('/rides/{id}/reject')`
Backend	`ride_service.accept` / `reject`, `ride_repository.assign`, `matching.close_round`
DB	`rides` (ASSIGNED + driver + vehicle snapshot), `active_rides` (driver), `ride_invitations` (ACCEPTED/CLOSED/REJECTED), `driver_presence.available=false`, `ride_match_rounds.outcome`
Errors	409 lost race / expired / stale location / already on trip; 403 not active/available
Edge cases	Two drivers accept simultaneously → exactly one 200 (tested); one driver accepting two rides simultaneously → `active_rides` PK ensures only one; retrying accept after winning returns the ride

F11. Trip lifecycle with per-ride OTP

What	Driver-driven state machine ASSIGNED → ARRIVING → WAITING → IN_PROGRESS → COMPLETED; start requires the rider's OTP
Why	Clear progress for both sides; OTP proves the right passenger is in the car
Interaction	Driver buttons "Head to pickup", "I've arrived", enter 6-digit OTP + "Start trip", "Complete trip" (confirm dialog). Rider sees progress text, car details, OTP
Frontend	`RideCard` (`actions` map), `OtpInput`, `AssignedRideDetails`, `TripStatsCard`
Backend	`ride_service.move`, `services.trip_otp/verify_trip_otp`, `ride_repository.transition`
API	`POST /rides/{id}/depart`, `/arrive`, `/start`, `/complete`
Errors	403 not the assigned driver; 409 wrong state / changed; 400 wrong OTP; 429 too many OTP failures
Edge cases	Retry of an applied transition returns current state; OTP required on start retries; a code from a previous ride never works; suspended drivers can still progress their current trip

F12. Pre-trip cancellation

What	Rider (SEARCHING..WAITING) or assigned driver (ASSIGNED..WAITING) cancels with a reason
Interaction	"Cancel request"/"Cancel ride" → preset reasons (rider: plans changed, pickup changed, wait too long, details don't match; driver: vehicle problem, can't reach pickup, rider not at pickup; or Other) + notes → confirm
Frontend	`CancelRideControl`, `CancellationNotice` (in ride card and as a 10-min "recent cancellation" banner)
Backend	`ride_service.move(action='cancel')`, `ride_service.cancellation`

DB	`rides` (CANCELLED + who/when/why), `ride_invitations` (CLOSED), `driver_presence` (driver offline, resume false), `active_rides` DELETE, `ride_match_rounds.outcome`
Edge cases	IN_PROGRESS cannot be cancelled (409); repeated cancel returns the original record; legacy clients may cancel a search without a reason; cancellation vs acceptance/expiry races resolve to exactly one outcome (tested); no fees/refunds

F13. Completion, revenue allocation and driver auto-availability

What	On completion the system records the driver/company split once, opens a pending payment, releases claims, and decides whether the driver is immediately available again
Backend	`ride_service.move('complete')`, `revenue_service.allocate/active`, `payment_service.record_completion`
DB	`rides`, `revenue_ledger`, `ride_payments`, `active_rides`, `driver_presence`
Rules	Split = setting effective at server completion time (default 85/15); driver amount rounded half-up, company gets the remainder; auto-available only if driver active + chose "stay available" + location fresh at completion + bookings enabled
Edge cases	Completion retry never re-enables availability; offline choice during the trip is respected; a split edited later never changes recorded allocations

F14. Real-time updates

What	Near-instant UI updates for all parties via Socket.IO invalidation events, with polling fallback
Frontend	`useWorkspace` (socket + polling), connection badge "Live" / "Polling fallback"
Backend	`ride_repository.notify/notify_ride` (outbox), `realtime.dispatch`, `realtime.init_realtime`
DB	`outbox_events`
Edge cases	Events are at-least-once and carry no data (clients always refetch), so lost/duplicate events are harmless; expired/revoked sockets are disconnected each second

Details: 13.

F15. Live trip map and ETA card

What	For Google-mode rides, an embedded road route: driver → pickup while approaching, pickup → destination while waiting, driver → destination during the trip; ETA/distance card; "Navigate in Google Maps" link for drivers
Frontend	`ActiveRidePanel`, `useTripRoute`, `TripRouteMap`, `TripStatsCard`, `GoogleRouteMap`
Backend	`trip_map_service.trip_map`, `cached_route`, `route_provider.compute`
API	`GET /rides/{id}/map` (polled every 10 s)
Edge cases	Stale/missing driver GPS → pickup-origin route and no driver marker; driver exactly at target → no Google call; ride completes during the Google call → 409, no coordinates leaked; admins excluded; simulator rides get 409 and the map is hidden; Google cost bounded to ≤ 1 call/min per ride/phase/source

F16. Payments: cash, online (mock QR)

What	After completion the rider chooses cash or online; the driver confirms cash; online is a **simulated** QR flow; earnings count as "received" only when confirmed
Interaction	Rider modal "You've arrived!" → Pay online / Pay by cash / Pay later; driver "Collect payment" card → "Cash received"; QR opened on a phone → choose success/failed/pending
Frontend	`TestPayment.jsx` (`PaymentPrompt`, `TestPaymentCard`, `TestPaymentPage`), `PaymentStatus.jsx` (`DriverCollectCard`, flash), `RideWorkspace.usePaymentPrompt`
Backend	`payment_service.*`, `test_payment_routes.*`
APIs	`POST /rides/{id}/payment-method`, `/confirm-cash`, `/test-payment`; `GET/POST /test-payments/{token}`
DB	`ride_payments`, `mock_payments`, `outbox_events`
Edge cases	First confirmation wins (cash vs online race); mock SUCCESS is sticky; cash-confirmed rides refuse later test QR; online choice blocked if test payments are disabled; QR links expire after 15 min; prompt re-opens on reload within 30 min of completion unless dismissed in that tab
Not implemented	Real payment gateway, receipts, refunds, tips, taxes

F17. Admin fare settings

What	Versioned fare policy editor (rates, minimum, timezone, global surge switch and cap, traffic/demand surge, up to 20 scheduled circular zones) with reason and history
Frontend	`FareSettings.jsx`
Backend	`fare_service.settings/save/validate`
APIs	`GET` / `PUT /admin/fare-settings`
DB	`fare_policies` (append-only)
Edge cases	Concurrent saves → one version wins, other 409; existing quotes keep their saved fare; every change visible in audit history with a field diff

F18. Admin revenue share

What	Immediate or scheduled (≤ 1 year ahead) driver/company split with reason; upcoming schedule and history
Frontend	`RevenueSettings.jsx` (browser local time → UTC ISO)
Backend	`revenue_service.settings/save/active`
DB	`revenue_share_settings`
Edge cases	Same effective timestamp → newest version wins; future schedules remain when a new immediate split is saved; percentages must sum exactly 100 (DB CHECK too)

F19. Admin monitoring and audit history

What	Overview metrics (active/searching/completed/cancelled rides, fresh vs stale available drivers, pending/suspended drivers, allocation totals), driver list with location freshness, recent rides with ratings, and a searchable audit log with before/after details
Front end	`RideWorkspace` (admin), `DriverManagement`, `AdminAudit`
Back end	`admin_service.metrics/driver_list/audit/audit_detail`, `ride_service.snapshot`
APIs	`GET /workspace`, `/admin/drivers`, `/admin/audit`, `/admin/audit/{category}/{id}`
Privacy	Admins see location freshness/accuracy but never coordinates; they see rating comments

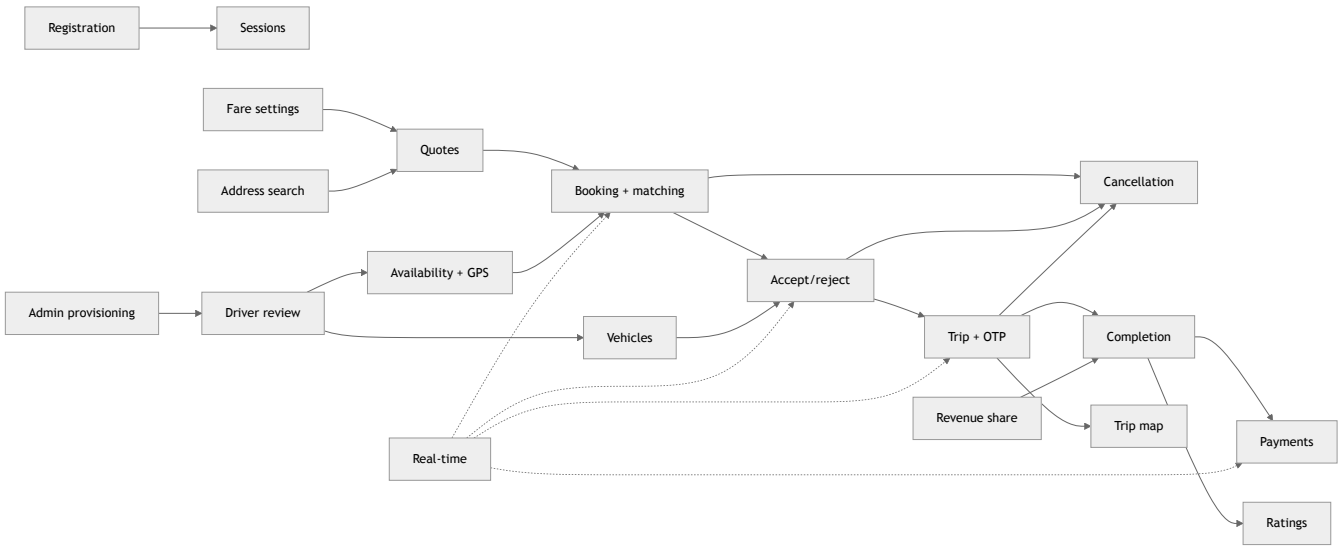
F20. Ratings

What	After completion, rider rates driver and driver rates rider (1–5 stars, ≤ 5 tags, optional comment), once each
Interaction	Rating form after payment success and on completed ride cards for 7 days; averages shown as badges (driver on assigned card; rider on invitations; own on driver overview; drivers in admin list)
Frontend	`Rating.jsx`
Backend	`rating_service.submit/summary/given/for_admin`
API	`POST /rides/{id}/rating`
DB	`ride_ratings`
Edge cases	Same-star retry is idempotent; different stars → 409; ratees never see comments about themselves; admins cannot rate

F21. Health, readiness and API docs

`GET /health` (liveness + config summary), `GET /ready` (schema at head + worker alive; used by Docker/Render), `GET /openapi.json`, `GET /docs`. See 06 §1.

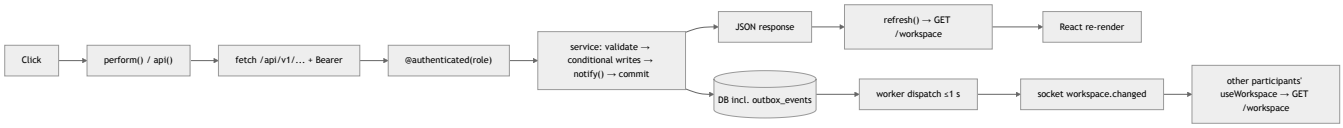
Feature dependency map



10 — Complete data flow

Each flow is traced through the same 14 steps: (1) user action, (2) component, (3) handler, (4) data collection, (5) client validation, (6) endpoint, (7) controller, (8) service logic, (9) database, (10) external services, (11) response, (12) frontend receipt, (13) UI update, (14) errors. Diagrams for each flow are in diagrams/.

The universal pattern



The initiating client updates from its own refresh; **other** users update via the outbox → socket → refetch path (or their 2 s/15 s poll).

Flow 1 — Login

Step	Detail
1	User submits the `/login` form
2	`main.jsx::AuthPage`
3	`submit(event)` → `signIn(fields)` (`authSession.js`)
4	`Object.fromEntries(new FormData(form))` → `{mobile, password}`
5	HTML `required`, `maxLength=128`
6	`POST /api/v1/auth/login` with `X-Session-Request: 1`
7	`routes.sign_in` (+ `limit_auth` before_request, `check_origin(require_header=False)`)
8	`services.login`: normalize mobile, constant-work password check, disabled check → `session_service.issue`
9	SELECT `users`; INSERT `auth_sessions`, `refresh_tokens`; COMMIT
10	none
11	200 `{access_token, expires_in, expires_at, user}` + `Set-Cookie ridez_refresh`
12	`request()` parses JSON; `signIn` bumps `generation`, clears `knownTokens`, `publish(data)`
13	`AuthProvider` listener sets `session`; `AuthPage` → `navigate('/{role}')` → `Protected` → `Workspace`
14	401 → `setError('Mobile or password is incorrect')`; 429 → rate-limit message; 403 → origin message

Flow 2 — Getting a fare quote

Step	Detail
1	Rider fills pickup/destination and clicks "Get fare estimate"
2	`BookingPanel`
3	`submit(event)`
4	`form` state: labels from `AddressSearch.onChoose`/map `select()`/typing; coordinates as strings
5	Both coordinate pairs required in Google mode (pickup only in simulator) → else inline error

Step	Detail
6	`perform('/fares/quotes', {pickup, destination, pickup_lat, pickup_lng, destination_lat?, destination_lng?})` → `POST /api/v1/fares/quotes`
7	`ride_routes.quote` : `@authenticated('rider')` , 10/min limiter
8	`ride_service.quote` : `simulator()` guard, `text()` , `coordinate()` , pickup≠destination, `expire()` , `route_provider.compute` , `fare_service.calculate` (policy, zones, traffic, demand)
9	Reads `fare_policies` (+ demand tables); INSERT `fare_quotes` ; COMMIT
10	Google Routes `computeRoutes` (Google mode)
11	201 `{quote: {...itemized, expires_at, polyline}}`
12	`perform` returns response; `refresh()` also called
13	`setQuote(response.quote)` → "Your fare, upfront." card; map draws polyline
14	`actionError` banner with backend message (e.g. "The route service timed out. Try again shortly"); no simulated fallback in Google mode

Flow 3 — Booking and matching

Step	Detail
1	Rider clicks "Confirm ride"
2	`BookingPanel` quote card
3	inline `onClick` → `perform('/rides', {quote_id})`
4	`quote.id` from Flow 2
5	Button disabled while busy
6	`POST /api/v1/rides`
7	`ride_routes.book` (`@authenticated('rider')`)
8	`ride_service.book` : ownership, idempotency, expiry, provider, active-claim check → create ride → `matching.invite` → `notify_ride`
9	INSERT `rides` , `active_rides` , `ride_match_rounds` , `ride_invitations` ; no-op UPDATE `users` (driver locks); INSERT `outbox_events` ; COMMIT
10	none
11	201 `{ride}` (SEARCHING)
12	`perform` → `setQuote(null)` , `refresh()`
13	`/workspace` now has `active_ride` → `ActiveRidePanel` with countdown; `trip_otp` card. **Drivers** receive `workspace.changed` within ~1 s → their `/workspace` includes `invitations` → "New ride request" card
14	409 messages (expired quote, already active, provider changed) in the banner; quote stays visible

Background continuation: every second `matching.advance()` expires the round; radius grows; new invitations; eventually NO_DRIVER_FOUND (rider claim released, rider sees "No driver found" and the booking form returns).

Flow 4 — Driver location update

Step	Detail
1	Timer (30 s idle / 10 s on trip) — no click
2	`RideWorkspace` → `useDriverLocation`
3	`send()`

Step	Detail
4	`getPosition()` → `navigator.geolocation.getCurrentPosition`
5	Secure context check; geolocation errors mapped to messages
6	`PUT /api/v1/driver/location` `{latitude, longitude, accuracy_m, recorded_at}`
7	`ride_routes.location` (`@authenticated('driver')`)
8	`location_service.update`: status/availability rules, bounds, time window, ordering, plausibility, replay
9	UPSERT `driver_locations`; INSERT `outbox_events`; COMMIT (in the route)
10	Browser geolocation only
11	200 `{location:{fresh, recorded_at, accuracy_m}}`
12	Error cleared
13	Next `/workspace` shows "Location fresh" + time + accuracy
14	Error banner (`trackingError`); permission denied → `POST /driver/availability {available:false}`

Flow 5 — Driver accepts a ride

Step	Detail
1	Driver clicks "Accept ride"
2	Invitation card in `RideWorkspace`
3	`perform('/rides/{id}/accept')`
4	ride id from `data.invitations`
5	Button disabled when `expires_at` passed
6	`POST /api/v1/rides/{id}/accept`
7	`ride_routes.accept` (`@authenticated('driver')`)
8	`ride_service.accept` guards → `lock_active_driver` → `ride_repository.assign` (conditional)
9	UPDATE `rides`; INSERT `active_rides`; UPDATE `ride_invitations`, `driver_presence`, `ride_match_rounds`; INSERT outbox; COMMIT
10	none
11	200 `{ride}` ASSIGNED with vehicle snapshot
12	`refresh()`
13	Driver: `ActiveRidePanel` with "Head to pickup". Rider (via socket): `AssignedRideDetails` with driver name, rating, car plate, fare, OTP. Other invited drivers: invitation disappears
14	409 "Another driver has already accepted this ride" etc.

Flow 6 — Starting the trip with OTP

Step	Detail
1	Rider reads OTP to driver; driver types 6 digits and clicks "Start trip"
2	`RideCard` + `OtpInput`
3	button `onClick` → `perform('/rides/{id}/start', {otp})`
4	`otp` state from `OtpInput.onChange`
5	Button disabled until `otp.length === 6`
6	`POST /api/v1/rides/{id}/start`

Step	Detail
7	`ride_routes.progress` → `body()` required for start
8	`ride_service.move` : assigned driver check → `verify_trip_otp` (rate limit, HMAC compare) → `transition(WAITING→IN_PROGRESS)`
9	UPDATE `rides` ; outbox; COMMIT
10	none
11	200 `{ride}`
12–13	Driver sees "Complete trip"; map switches to driver→destination; rider sees "You're on your way"; OTP no longer shown
14	400 wrong OTP; 429 after 5 failures/min

Flow 7 — Completion and cash payment

Step	Detail
1	Driver clicks "Complete trip" and confirms
2–3	`RideCard` → `perform('/rides/{id}/complete')`
6–8	`ride_service.move('complete')` → `revenue_service.allocate` + `payment_service.record_completion` + release + auto-availability
9	UPDATE `rides` ; INSERT `revenue_ledger` , `ride_payments` ; DELETE `active_rides` ; UPDATE `driver_presence` ; outbox; COMMIT
11–13	Driver: completion notice, `DriverCollectCard` . Rider: `usePaymentPrompt` detects the active ride turned COMPLETED/PENDING → `PaymentPrompt` modal
—	Rider clicks "Pay by cash" → `POST /rides/{id}/payment-method {method:'CASH'}` → UPDATE `ride_payments.method`
—	Driver clicks "Cash received" (confirm) → `POST /rides/{id}/confirm-cash` → conditional UPDATE to CONFIRMED
—	Rider modal (via socket refresh) shows `PaymentSuccess` + `RatingForm` ; driver sees `PaymentConfirmedFlash` ; earnings move from "Awaiting payment" to "Earnings received"
14	409 "The rider has already paid online" if the mock QR won first

Flow 8 — Mock online payment

Step	Detail
1	Rider chooses "Pay online" (or "Pay by QR (test)")
3	`PaymentPrompt.choose('ONLINE')` → `useTestQr.generate()`
6	`POST /rides/{id}/payment-method {ONLINE}` then `POST /rides/{id}/test-payment`
8–9	INSERT `mock_payments` PENDING; return signed path
12–13	`QRCode.toDataURL(absolute URL)` → QR image + "Open payment page"
—	Phone opens `/test-payment/<token>` → `TestPaymentPage` → `GET /test-payments/<token>` → shows amount
—	"Simulate success" → `POST /test-payments/<token> {status:'SUCCESS'}` → UPDATE `mock_payments` , `payment_service.confirm(ONLINE)` , notify
—	Rider modal flips to success via socket; driver earnings update
14	410 expired token; 404 feature disabled; 409 already paid in cash

Flow 9 — Admin suspends a driver

Step	Detail
1	Admin clicks "Suspend driver" on a row, types a reason, confirms
2–3	`DriverManagement.choose` → `save(event)`
4	`{version: selected.status_revision, reason}`
5	Reason required (button disabled when empty)
6	`POST /api/v1/admin/drivers/{id}/suspend`
7	`ride_routes.suspend_driver` (`@authenticated('admin')`)
8	`admin_service.change_driver_status` (version, retry detection, conditional UPDATE) → `session_service.stop_requests`
9	UPDATE `users`, `driver_presence`, `ride_invitations`; INSERT `admin_audit`, outbox; COMMIT
11	200 `{user}`
12–13	Success message; list reloads; driver's workspace (via socket) shows "Your driver account is suspended" + reason; invitations disappear
14	409 stale version → error + "Reload drivers"

Flow 10 — Rating

Step	Detail
1	User taps stars, optional tags/comment, "Submit rating"
2–3	`RatingForm.submit`
4	`{stars, tags: chosen, comment?}`
5	Submit only visible after a star is chosen; ≤ 5 tags enforced in `toggle`
6	`POST /api/v1/rides/{id}/rating`
7–8	`ride_routes.rate` → `rating_service.submit`
9	INSERT `ride_ratings`; COMMIT (no outbox event)
11–13	`setSaved(data.rating)` → "You rated … Thanks!"; averages update on next workspace load
14	409 already rated with different stars

Flow 11 — Admin edits fare policy

`FareSettings.save` → `PUT /admin/fare-settings {version, reason, policy}` → `fare_service.save` (version check, `validate`, INSERT `fare_policies` version+1, notify admins) → returns new settings → form reloads with new version; next quote uses it, existing quotes unchanged; audit list shows "Fare policy" entry with diff.

Data visibility summary

Data	Rider	Driver	Admin
Fare breakdown	✓	✓	✓
Revenue split (`settlement`)	✗	own rides	✓
Trip OTP	own active ride	✗	✗
Driver coordinates	via trip map (active ride)	own (implicitly)	✗ (freshness only)

Data	Rider	Driver	Admin
Vehicle snapshot	assigned ride	✓	✓
Rating average	of driver	of rider + own	of drivers
Rating comments	✗	✗	✓
Cancellation reason	✓	✓	✓
Mobile numbers	✗	✗	drivers' (admin list)

11 — Third-party integrations and external services

Only integrations actually present in the code are listed. There is **no** payment gateway, SMS, email, push notification, analytics, error tracking, object storage or cloud database integration.

Google Routes API (server-side)

Item	Detail
Why	Real road distance and traffic-aware duration for fares; road polylines for maps
Configured in	`GOOGLE_MAPS_SERVER_KEY`, `ROUTE_PROVIDER=google` (default outside development); validated in `create_app` (required in production when provider is Google)
Code	`backend/app/route_provider.py::compute`, called by `ride_service.quote`, `trip_map_service.trip_map`, CLI `check-routing --probe`
Request	`POST https://routes.googleapis.com/directions/v2:computeRoutes` JSON `{origin, destination (latLng), travelMode: DRIVE, routingPreference: TRAFFIC_AWARE, computeAlternativeRoutes: false, polylineQuality: OVERVIEW}`
Auth	Header `X-Goog-Api-Key: <server key>`; field mask `routes.distanceMeters,routes.duration,routes.staticDuration,routes.polyline.encodedPolyline` (limits billing tier/response size)
Sent	Pickup and destination coordinates only (labels are not sent)
Received	Distance (m), duration and static duration (`"123s"` strings), encoded polyline
Timeouts	3 s connect, 10 s read; redirects disabled
Caching	Quotes: none (each quote calls Google). Trip maps: in-memory 60 s per `(ride, phase, source, target, at_destination)`, errors 10 s, concurrent callers coalesced

Error handling (`route_provider.compute`):

Condition	API response
Key missing	503 `Google routing is not configured. Set GOOGLE_MAPS_SERVER_KEY…`
No destination coordinates	400 `Select destination coordinates on the map or enter them manually`
`requests.Timeout`	504 `The route service timed out. Try again shortly`
Other `RequestException`	502 `The route service is unavailable. No fare was generated`
HTTP 429	503 `Google routing quota was reached…`
HTTP 401/403	502 `Google routing access was denied… Google reason: <ALLOW-LISTED CODE>`
Other non-200	502 `Google could not calculate this route…`
Empty `routes`	422 `No driving route was found between these locations`
Malformed fields	502 `The route provider returned an invalid estimate. Please retry`

If Google is unavailable: no new quotes can be created in Google mode (explicitly no fallback to simulated data — product rule in `docs/PRODUCT_BLUEPRINT.md`); existing rides continue (matching, OTP, completion, payments don't need Google); trip maps show an error with "Refresh route" while trip controls keep working.

Simulator alternative: `ROUTE_PROVIDER=simulator` + `SIMULATOR_ENABLED=1` produces deterministic sample routes (labelled "simulated route" in UI). Quotes record their `provider`; booking refuses a quote whose provider

differs from the current mode.

Google Maps JavaScript API (browser)

Item	Detail
Why	Interactive maps for picking points, showing quotes and live trips
Configured in	`frontend/.env.local` → `VITE_GOOGLE_MAPS_BROWSER_KEY` (and optional `VITE_GOOGLE_MAP_ID`, default `DEMO_MAP_ID`); Docker build args; `render.yaml` (`sync: false`)
Code	`frontend/src/googleMaps.js::loadMaps` (script loader), `GoogleRouteMap.jsx` (Map, `AdvancedMarkerElement`, `Polyline`, `geometry.encoding.decodePath`, `LatLngBounds`)
Auth	Browser key in the script URL — **public by design**; must be restricted by HTTP referrer and API in Google Cloud (documented in `docs/GOOGLE_MAPS_SETUP.md`)
Error handling	Missing key → explanatory error; script `onerror`; 15 s load timeout; `window.gm_authFailure` → `ridez.maps.error` event → "access was denied" message + Retry button
If unavailable	Booking still possible via manual coordinate fields; trip map hidden behind an error; no impact on backend

Google Places API (New) (browser)

Item	Detail
Why	Address autocomplete
Code	`frontend/src/AddressSearch.jsx`: `maps.importLibrary('places')`, `AutocompleteSessionToken`, `AutocompleteSuggestion.fetchAutocompleteSuggestions({input, sessionToken, locationBias: {center: pickup, radius: 50000}})`, `placePrediction.toPlace().fetchFields({fields: ['formattedAddress','location']})`
Sent	Typed text (≥ 3 chars, debounced 300 ms), optional bias centre
Received	Up to 5 predictions; selected place address and lat/lng
Attribution	"Powered by Google" image under suggestions
Errors	"Address search unavailable. Check Places API (New) and browser-key restrictions, or select on the map." / "Unable to load this address…"

Google Maps deep link (no API key)

`TripStatsCard.googleNavigationUrl` builds `https://www.google.com/maps/dir/?api=1&destination=<lat,lng>&travelmode=driving&dir_action=navigate`, opened in a new tab for drivers. No data is sent from the backend.

Browser platform APIs

API	Where	Purpose
Geolocation	`useDriverLocation.js::getPosition`	Driver location and rider "Use my location"
Permissions API	`useDriverLocation`	Detect revoked location permission

API	Where	Purpose
Web Locks (`navigator.locks`)	`authSession.js`	Cross-tab refresh serialization
BroadcastChannel	`authSession.js`	Cross-tab sign-out
sessionStorage	`RideWorkspace.jsx`	Remember dismissed payment prompts
`crypto.randomUUID`	`FareSettings.jsx`	New zone IDs

jsDelivr CDN

`GET /api/v1/docs` returns HTML that loads `swagger-ui-dist@5` CSS/JS from `cdn.jsdelivr.net`. If unavailable, only the Swagger UI page breaks; `/openapi.json` still works.

Render (hosting)

`render.yaml` defines one Docker web service (Starter plan, 1 instance, manual deploys, health check `/api/v1/ready`, 1 GB disk at `/var/data`), generates `JWT_SECRET` and `USER_CODE_SECRET`, and expects the Google keys to be entered in the dashboard. Render terminates TLS and forwards `X-Forwarded-For/Proto` (trusted through `TRUST_PROXY_HEADERS=1`). Details: 15.

GitHub Actions

`.github/workflows/checks.yml` runs on push to `main` and on pull requests: Python 3.12 + Node 22, `pip install -r backend/requirements.lock`, `pytest -q`, `npm ci && npm run build`, `docker build`, `python deploy/smoke.py --image ridez:ci`.

Dependency on external services — impact matrix

Service down	Effect
Google Routes	No new Google quotes; trip map errors; rides in progress unaffected
Google Maps JS / Places	No map/autocomplete; manual coordinates still work
jsDelivr	Swagger UI only
Render	Whole app down (single instance)
Browser geolocation denied	Driver cannot go available; active trip preserved; rider can still type/select pickup

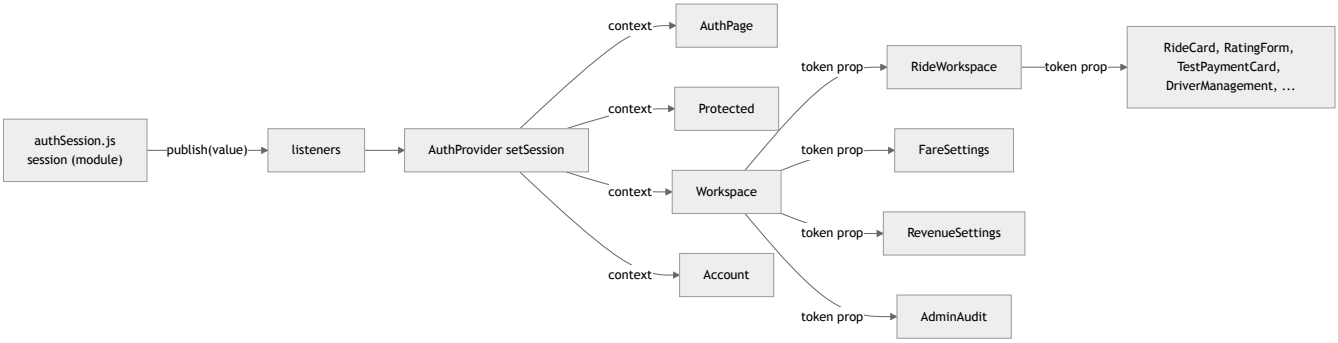
12 — State management and frontend flow

1. Where state lives

State	Location	Lifetime	Written by
Access token + user	`authSession.js` module variable `session`	Tab memory; lost on reload (restored via refresh cookie)	`signIn`, `refreshSession`, `signOut`
Refresh credential	HttpOnly cookie `ridez_refresh`	30 days max	Server only
Session for React	`Auth` context in `main.jsx::AuthProvider` (`session, ready, restoreError, signingOut, logoutError`)	App lifetime	`subscribeSession` listener
Server snapshot	`useWorkspace` → `data`	While `RideWorkspace` is mounted	Polling + socket-triggered refetch
Action status	`RideWorkspace` local `busy`, `actionError`, `completed`, `availabilityChoice`, `dismissedCancellation`, `closedPayment`	Component	`perform`, handlers
Form drafts	Local `useState` in each form component	Component	User input
Admin page data	Local `data/policy/...` in `FareSettings`, `RevenueSettings`, `AdminAudit`, `DriverManagement`	Component	Their own `api()` calls
Rating tag list	`RatingTags` React context, value = `data.rating_tags`	Workspace	Server
Payment prompt dismissal	`sessionStorage['ridez.payment-dismissed.<rideId>']`	Tab	`rememberPaymentDismissed`
Trip map data	`useTripRoute` local state keyed by `ride.id:phase`	Active ride panel	Polling `/rides/{id}/map`
Google Maps instance	`googleMaps.js` module `loading` promise + `window.google`	Page	`loadMaps`

There is **no global store** (Redux/Zustand/React Query). The design treats the server as the single source of truth and the client as a cache that is invalidated aggressively.

2. Global state: authentication



The token is passed as a prop, but `api()` always resolves it to the **current** token through `resolveToken`, so components holding an old token still work after rotation.

3. Server-state synchronization (the workspace)

- `useWorkspace(token)` keeps one `data` object = last `/workspace` response.
- Re-fetch triggers: initial mount; every 15 s (socket connected) or 2 s (disconnected); socket `connect` / `disconnect`; each new `workspace.changed` event (debounced 80 ms); any `perform()` success; the refresh button.
- No optimistic updates except `updateAvailability` (patches `available` / `resume_available` immediately after the availability POST).
- When `error` is set (last poll failed), all action buttons are disabled and the banner explains "Actions are paused until the connection recovers".

4. Local component state patterns

Pattern	Example
`busy` flag + disabled buttons	`perform`, `FareSettings.save`, `RatingForm.submit`, `VehicleEditor.save`
`error` string rendered in `role="alert"`	every form
`message` string for success	admin pages
AbortController in effects	`useWorkspace` poll, `useTripRoute`, `AdminAudit`, `DriverManagement`, `useDriverLocation`
Monotonic sequence refs to ignore stale async results	`AddressSearch` (`sequence`, `selection`)
"previous value" refs to detect transitions	`usePaymentPrompt` (`lastActive`), `usePaymentConfirmedFlash` (`pending`)
`revision` counter to force refetch	`useWorkspace`, `AdminAudit`, `DriverManagement`, `useTripRoute`
`key` prop to reset component state	`OtpInput key={ride.id}`, `VehicleEditor key={id:version}`, `PaymentPrompt key={ride.id}`, `AuthPage key="login"/"register"`

5. Passing data between components

- Top-down props only. `RideWorkspace` passes `ride`, `role`, `perform`, `disabled`, `token`, `tripOtp` into cards.
- Callbacks up: `onSelect` / `onChoose` (map/address → BookingPanel), `onChange` (OtpInput → RideCard), `onReview` (CancelRideControl hides action buttons), `onChanged` / `onSaved` (DriverManagement/VehicleEditor → workspace refresh), `onClose` (PaymentPrompt).
- Two contexts: `Auth` (session) and `RatingTags`.
- `ActiveRidePanel` lifts the trip-map poll (`useTripRoute`) so `TripStatsCard` and `TripRouteMap` share one request.

6. Loading states

Situation	UI
Session restoring	`SessionLoading` "Restoring your session…"
Workspace first load	"Loading your workspace…"

Situation	UI
Workspace failed before first data	"Unable to load your workspace. Use refresh to retry."
Button actions	Labels change: "Please wait…", "Saving…", "Submitting…", "Generating…", "Signing out…", "Getting your location…"
Maps	"Loading Google Maps…", "Loading Google road route…"
Address search	"Searching addresses…", "Loading selected address…"
Admin lists	"Loading drivers…", "Loading audit history…", "Loading fare settings…"

7. Error states

All backend errors arrive as `Error(message)` from `api()`. Each component shows the message near the action. Special cases:

- `Protected`: session restore failure (network) → message + "Retry restoring session" + "Sign in again".
- `useDriverLocation`: geolocation errors mapped to human messages (denied, unavailable, timeout).
- Google map errors: "Retry map" button.
- 409 conflicts on admin pages: "Reload settings" / "Reload drivers" buttons.

8. Forms

All forms are **uncontrolled-to-controlled hybrids** with `useState` and native HTML validation attributes; there is no form library. Submission always `event.preventDefault()`, sets `busy`, calls the API, then resets or shows the error. Details per form: 04 §11.

9. Navigation and routing

- `BrowserRouter` with role prefixes; `Navigate replace` for redirects (no history spam).
- After login: `navigate('/{role}', {replace:true})`.
- `NavLink end` highlights the active sidebar item; mobile menu toggled by `open` state.
- Unknown nested paths render `ComingSoon`; unknown top-level paths render the 404 page.
- Production server mirrors SPA paths (`web.py`), so deep links survive reloads.

10. Reusable component structure

Reusable	Used in
`GoogleRouteMap`	`BookingPanel` (picker) and `TripRouteMap` (read-only)
`QuoteDetails` (passed as prop to `BookingPanel`)	Booking, ride cards, invitations
`RideCard`	Overview, history, admin rides, recent trip
`RatingBadge`, `StarsShown`	Assigned details, invitations, driver list, admin ratings
`RatingForm`	Ride cards and payment success
`CancellationNotice`	Ride cards and recent banner
`Countdown`	Searching rides and invitations
`getPosition`	Driver availability and rider "Use my location"
`api`	All network calls

11. Important interactions traced UI → backend → UI

Interaction	UI → API → UI
Go available	button → `getPosition` → `POST /driver/availability` → `updateAvailability` + refresh → banner "You're available for requests"; GPS loop starts
Accept invitation	card button → `POST /rides/{id}/accept` → refresh → `ActiveRidePanel`
Rider waits for driver	socket event → refetch → `AssignedRideDetails` appears without clicking
Driver completes	button + confirm → `POST /complete` → rider socket refetch → `usePaymentPrompt` opens modal
Admin suspends	form → `POST /suspend` → `onChanged()` refresh metrics → driver socket refetch → suspension banner

13 — Background processes and real-time functionality

Inventory

Mechanism	Implemented?	Where
Background worker loop (1 s)	✓	`backend/app/realtime.py::start_worker/tick`
WebSockets (Socket.IO)	✓	`realtime.py`, `RideWorkspace.jsx::useWorkspace`
Transactional outbox	✓	`outbox_events`, `ride_repository.notify`, `realtime.dispatch`
Client polling	✓	`/workspace` every 2 s/15 s; `/rides/{id}/map` every 10 s
Location tracking	✓ (browser foreground only)	`useDriverLocation.js`
Deadline-based scheduling	✓ (persisted in DB)	`rides.expires_at`, `matching.advance`
Cron jobs / task queues / Celery / Redis	✗	—
Push notifications / SMS / email	✗	—
Retry mechanisms	✓ for outbox delivery, refresh scheduling, location loop, Google map polling; ✗ for Google quote calls (user retries)	

1. The worker loop

Start: `start_worker(app)` is called once per process — by `run.py` before `socketio.run`, or by Gunicorn's `post_worker_init` hook. It is guarded by `app.extensions['matching_worker']` so it never starts twice in the same app.

```
def start_worker(app):
    if app.extensions.get('matching_worker'): return
    def loop():
        while True:
            try: tick(app)
            except Exception:
                app.logger.error('Matching worker tick failed; retrying. Check database readiness
and migrations.')
            socketio.sleep(1)
    app.extensions['matching_worker'] = socketio.start_background_task(loop)
```

With `async_mode='threading'`, `start_background_task` starts a regular daemon thread **(inferred from Flask-SocketIO threading mode)**.

`tick(app)`:

```
with app.app_context():
    try:
        advance()     # matching + stale GPS
        dispatch()    # sockets + outbox
        app.extensions['matching_last_tick'] = time.monotonic()
    except Exception:
        db.session.rollback(); raise
```

`matching_last_tick` drives `/health.matching_worker` and `/ready` (must be < 10 s old).

What `advance()` does each second

1. Find SEARCHING rides whose `expires_at <= now`.
2. For each, claim the round with a conditional UPDATE (`expire_round`), so HTTP requests running `advance()` concurrently cannot double-process it.
3. Expire PENDING invitations, mark the round `EXPIRED`.
4. If max radius reached → `NO_DRIVER_FOUND`, release the rider's claim. Else radius += 500 m, deadline += 30 s, invite newly eligible drivers.
5. Notify the rider, invitees and admins; commit per ride.
6. Repeat up to `MATCH_MAX_RADIUS_M/500 + 1` times to catch up after downtime.
7. Mark `driver_locations` older than 90 s as `fresh=False` and notify those drivers.

Because the same `advance()` runs at the start of most HTTP requests (`ride_service.expire()`), the system also progresses if the worker thread dies — but only while there is traffic.

2. Outbox → Socket.IO

Writing events. Any service that changes state visible to a user calls:

```
def notify(*user_ids):
    for user_id in set(user_ids) | {None}:          # always one extra event for admins
        add(OutboxEvent(id=str(uuid4()), user_id=user_id, created_at=utcnow()))
```

The rows are committed **in the same transaction** as the business change. If the transaction rolls back, no event is emitted; if it commits, the event is guaranteed to be delivered eventually.

Who notifies whom:

Operation	Recipients
Book, round expiry, accept, trip transitions, cancel	rider, all invited drivers, assigned driver, admins (`notify_ride`)
Availability change, location update, stale sweep, reject	that driver, admins
Driver status change, vehicle save	that driver, admins
Fare/revenue settings save	admins only (`notify()` with no ids)
Payment method / cash confirm / mock status	rider, driver, admins
Logout / session revocation (drivers)	that driver, admins
Rating	none

Delivering events. `dispatch()`:

```
disconnect_invalid()
events = SELECT * FROM outbox_events WHERE delivered_at IS NULL ORDER BY created_at LIMIT 200
for event in events:
    socketio.emit('workspace.changed', {'event_id': event.id, 'server_time': now},
                  to=f'user:{event.user_id}' if event.user_id is not None else 'admins')
    event.delivered_at = now
DELETE FROM outbox_events WHERE delivered_at < now - 7 days
COMMIT
```

- Delivery is **at-least-once**: if the process dies after emitting but before committing, the events are re-emitted.
- Events are emitted even if the user is offline (no connected sockets) — they are simply marked delivered; the client catches up by polling/refetching on reconnect.
- At most 200 events per second are delivered; a backlog drains over subsequent ticks.

Socket authentication (`connect` handler): validates `auth.token` exactly like HTTP. Accepted sockets join `user:{id}` (and `admins` for admins). `disconnect_invalid()` checks every tracked socket each tick for JWT expiry, disabled user, or revoked session, and force-disconnects it.

Client handling (`useWorkspace`): dedupe by `event_id` (keeps the last 100 IDs), debounce 80 ms, refetch `/workspace`. The token-dependent effect reconnects the socket after each token refresh.

3. Polling

Poll	Interval	Purpose
`/workspace`	15 s when socket live, 2 s otherwise	Reconciliation and fallback
`/rides/{id}/map`	server-suggested `refresh_after_seconds` (10 s); 15 s after an error	Live route + driver marker
Session renewal	`expires_at − 60 s`; retry 15 s	Keep access token valid
Countdown display	500 ms (`setInterval`)	UI only

4. Location tracking

- Starts when a driver is available or has an active ride (`useDriverLocation` `enabled`).
- Sends immediately, then every 30 s (idle) or 10 s (on trip).
- Server freshness window: 90 s; worker marks stale within ~1 s of crossing it.
- Permission revocation (Permissions API `change` event) or a denial error triggers `POST /driver/availability {available:false}` — new matching stops, the active trip is preserved.
- Limitation: browsers throttle timers in background tabs and mobile browsers may suspend pages, so a driver who switches apps can become stale. The README notes native background tracking would require a mobile app.

5. Time-based rules summary

Rule	Value	Enforced by
Quote validity	5 min	`quote()` sets `expires_at`; `book()` checks
Matching round	30 s (`MATCH_ROUND_SECONDS`)	`rides.expires_at`, `advance()`
Max search	`MATCH_MAX_RADIUS_M/500` rounds (default 10 → 5 min)	`advance()`

Rule	Value	Enforced by
Location freshness	90 s (`LOCATION_STALE_SECONDS`)	`is_fresh`, `eligible_drivers`, `assign`, stale sweep
Location future tolerance	15 s	`location_service.update`
Access token	15 min	`session_service.payload`
Session	30 days absolute	`issue()`
Mock QR link	15 min	`payment_for` `max_age=900`
Trip map cache	60 s success / 10 s error	`cached_route`
Recent cancellation banner	10 min	`snapshot()`
Rating form shown	7 days (UI only)	`RATING_WINDOW_MS`
Payment prompt on reload	30 min (UI only)	`usePaymentPrompt`
Outbox retention	7 days after delivery	`dispatch()`
Rate-limit windows	60 s	limiters

6. Why single-process matters

The following live only in process memory: Socket.IO connections and rooms, `ride_connections` map, rate-limit buckets, trip-map cache, `matching_last_tick`. Running two Gunicorn workers or two instances would mean: a user connected to worker A would miss emits from worker B's dispatcher (Socket.IO has no message queue configured), rate limits would be per-worker, and readiness would reflect only one worker. Both worker loops would still be **safe** for matching (conditional claims), but notifications would be lost. This is why `gunicorn.conf.py` sets `workers = 1` and `render.yaml` sets `numInstances: 1`.

14 — Error handling and edge cases

1. Mechanisms

Layer	Mechanism	File
Domain	`raise ApiError(message, status)`	`services.py` (class), all services
Blueprint	`@api.errorhandler(ApiError)` → `{"error": message}`, status, `no-store`; 401 on refresh/logout-all clears cookie	`routes.py::api_error`
App	JSON 404 `Resource not found`, 405 `Method not allowed`	`__init__.py`
Transactions	`repo.rollback()` / `db.session.rollback()` before raising after failed conditional writes; Flask-SQLAlchemy removes the session at teardown	services
Worker	rollback + generic log line, retry next second	`realtime.py`
Frontend	`api()` throws `Error(data.error)`; components show it	`api.js`

2. Status code usage

Code	Meaning in this app	Examples
400	Validation failure	missing/invalid fields, wrong OTP, non-object JSON
401	Not authenticated	bad/expired JWT, revoked session, wrong credentials
403	Authenticated but not allowed	wrong role, inactive driver, CSRF/origin check
404	Not found **or not yours**	ride not accessible, invitation missing, feature disabled (test payments)
409	State conflict / race lost / stale version	quote expired, ride changed, already accepted, stale admin version
410	Expired capability	mock QR token expired
422	Semantically impossible	no driving route, fare above maximum
429	Rate limit / busy	auth, quotes, OTP, trip-map cache full
502	Upstream failure	Google denied / invalid / unavailable
503	Feature/dependency unavailable	simulator disabled, Google key missing, quota, not ready
504	Upstream timeout	Google timeout, trip-map wait > 15 s

3. Scenario table

Scenario	Handling	Where
Invalid user input	Explicit type/range/length checks; `bool` rejected for numbers; messages name the field	every service
Missing body / not JSON object	400 `A JSON object is required`	`routes.body`
Missing required data	e.g. `A quote_id is required`, `Reason must contain 1–300 characters`	services
Unauthorized	401/403 via `authenticated`; 404 for foreign resources	`routes.py`, `ride_service.accessible`

Scenario	Handling	Where
Invalid / expired token	401 `Please sign in again`; frontend refreshes once and retries	`session_service.authenticate`, `api.js`
Refresh token reuse	Session revoked, 401, cookie cleared	`session_service.refresh`
Database integrity race	`IntegrityError` caught → idempotent return or 409	`book`, `accept`, `availability`, `register`, `rating_service.submit`, `fare_service.save`, `revenue_service.save`, `create_test_payment`
Lost conditional update	rollback + 409 `… changed. Refresh and retry`	`move`, `accept`, `change_driver_status`, `payment_service`
Duplicate requests	Idempotent handlers return the current state	booking (quote id), accept, reject, transitions, cancel, review, payment choice/confirm, mock status, same-star rating
Concurrent requests	Exactly one winner by conditional writes and PKs; tested with thread barriers on SQLite	see 03 §12
Empty responses	`rides: []`, `invitations: []` with empty-state UI; Google `routes: []` → 422	
Network failure (client)	`fetch` rejects → error banner; polling continues (2 s) and recovers; actions disabled while `error` set	`useWorkspace`
Unreadable server response	`Unable to read the server response. Please retry.`	`api.js`
External API failure	Mapped 502/503/504/422 with safe messages; no simulated fallback	`route_provider.compute`
Timeouts	Google 3 s/10 s; trip-map wait 15 s; Maps script 15 s; geolocation 15 s + 20 s	various
Database failure	Unhandled `SQLAlchemyError` → Flask 500 (not JSON); `/ready` reports `database:false`; worker logs and retries	—
Worker crash	Exceptions caught per tick; deadlines still advanced by request traffic	`realtime.py`
Server restart mid-search	Deadlines persisted; catch-up replays rounds without inviting into expired rounds	`matching.advance`
Trip completes during map fetch	Re-check after Google call → 409, no driver coordinates	`trip_map_service`
Driver loses GPS mid-trip	Map falls back to pickup route; completion still allowed; not auto-available after	`trip_map_service`, `move`
Suspended driver mid-trip	Can track, start with OTP, complete; earnings recorded	`location_service.update`, tests
Quote from other provider mode	409 `Route mode changed…`	`book`
Expired QR	410 with guidance	`payment_for`

4. Edge cases worth knowing

- **Start retry requires the OTP again** because the OTP check runs when status is `WAITING` *or* `IN_PROGRESS`.
- **Cancel without reason** is only accepted for SEARCHING rides and only if the `reason` key is absent (an empty string fails validation).
- **Invitation acceptance after round expiry**: rejected even if the invitation row still says PENDING, because `assign()` checks `Invitation.expires_at > now` and `Ride.expires_at > now`.
- **Driver goes offline with invitations open**: invitations become `OFFLINE`; ride keeps searching.
- `reject` **for a non-pending invitation** returns its current status instead of an error.
- **Admin approve without version** (legacy) is allowed and idempotent.
- **Rider rating window**: the backend accepts ratings at any time after completion; only the UI hides the form after 7 days.

5. Missing error handling / failure points

Gap	Impact	Recommendation
No JSON handler for 500 / unexpected exceptions	Clients receive Flask's HTML 500; `api()` then shows "Unable to read the server response"	Add `@app.errorhandler(Exception)` returning `{"error": "Unexpected server error"}` and logging with a request ID
No request IDs / structured logging	Hard to correlate errors	Add a request-ID middleware and JSON logs
SQLite `database is locked` under load	Surfaces as 500	Configure SQLite busy timeout / WAL, or move to a server DB
Google quote calls not retried	User must retry manually	Optional single retry with jitter for 5xx/timeouts
Outbox dispatch emits before commit	Duplicate emits after crash (harmless)	Acceptable; documented as at-least-once
Frontend has no error boundary	A render error blanks the app	Add a top-level React error boundary
`disconnect_invalid` and `dispatch` run DB queries per socket per second	Load grows with connections	Batch session validation
No timeout on SQLAlchemy queries	A stuck DB blocks a thread	Set statement timeouts on the server DB

15 — Application execution and deployment

Commands below are copied from `README.md`, `Dockerfile`, `deploy/*.sh`, `render.yaml` and `.github/workflows/checks.yml`. Items marked **(recommended)** are suggestions, not existing configuration.

1. Prerequisites

Software	Version	Source
Python	3.12 (`.python-version` 3.12.12); 3.14 unsupported by pinned `pyodbc`	README
Node.js	20.19+ (22.12+ recommended); `.node-version` 22	README
npm	bundled with Node	
Docker	for image builds and `deploy/smoke.py`	CI
Google Cloud project	Routes API (server key); Maps JavaScript API + Places API (New) (browser key)	`docs/GOOGLE_MAPS_SETUP.md`
Microsoft ODBC Driver 18	only if using SQL Server (not in the image)	README

2. Environment variables

Backend (`backend/.env` locally, platform env in production)

Variable	Default	Purpose
`APP_ENV`	`development`	Anything else = production rules
`DATABASE_URL`	dev: `sqlite:///ridez.db`; prod: **required**	SQLAlchemy URL
`JWT_SECRET`	dev: auto-generated `instance/jwt.key`; prod: **required ≥ 32 chars**	JWT + QR token signing
`USER_CODE_SECRET`	dev: auto `instance/codes.key`; prod: **required ≥ 32, ≠ JWT_SECRET**	Account codes and trip OTPs
`ROUTE_PROVIDER`	dev `simulator`, prod `google`	`google` or `simulator`
`GOOGLE_MAPS_SERVER_KEY`	empty; required in prod with Google	Routes API
`SIMULATOR_ENABLED`	dev `1`, prod `0`	Allow simulated quotes/bookings
`TEST_PAYMENTS_ENABLED`	dev `1`, prod `0`	Mock QR payments and ONLINE method
`MATCH_MAX_RADIUS_M`	`5000`	Multiple of 500 in 500–10000
`AUTH_TRUSTED_ORIGINS`	empty	Extra allowed Origins (comma-separated)
`TRUST_PROXY_HEADERS`	`0`	`1` behind exactly one trusted TLS proxy (Render)

Variable	Default	Purpose
`PORT`	`5000` (`run.py`), `10000` (Gunicorn/Docker)	Listen port
`BOOTSTRAP_ADMIN`, `BOOTSTRAP_ADMIN_MOBILE`, `BOOTSTRAP_ADMIN_NAME`, `BOOTSTRAP_ADMIN_PASSWORD`	unset	Startup admin creation

Not configurable by env (hard-coded config defaults): `REFRESH_SESSION_DAYS=30`, `LOCATION_STALE_SECONDS=90`, `MATCH_ROUND_SECONDS=30`, `AUTH_COOKIE_SECURE` (= production), `FRONTEND_DIST`.

Frontend (`frontend/.env.local`, or Docker build args)

Variable	Purpose
`VITE_GOOGLE_MAPS_BROWSER_KEY`	Browser Maps/Places key (public; restrict by referrer)
`VITE_GOOGLE_MAP_ID`	Optional Map ID (default `DEMO_MAP_ID`)
`API_PROXY_TARGET`	Vite dev proxy target (default `http://127.0.0.1:5000`)

3. Local development (verified commands from README)

Backend (Linux/macOS):

```
python3 -m venv backend/.venv
backend/.venv/bin/pip install -r backend/requirements.lock
cd backend
.venv/bin/flask --app run db upgrade
.venv/bin/python run.py            # API + Socket.IO + matching worker on :5000
```

Frontend (second terminal):

```
cd frontend
npm ci
npm run dev                        # Vite on :5173, proxies /api and /socket.io to :5000
```

Open `http://localhost:5173`. Port 5000's root returns JSON 404 locally (no SPA build there).

Create an admin: `cd backend && .venv/bin/flask --app run create-admin`.

Windows: the README provides PowerShell equivalents (`py -3.12 -m venv backend\.venv312`, `npm.cmd ci`, …).

Gotchas:

- Use `python run.py`, not `flask run` (the latter does not start the worker).
- Geolocation needs `localhost` or HTTPS.
- If port 5000 is occupied, set `PORT` for `run.py` and `API_PROXY_TARGET` for Vite **(recommended)**.

4. Tests and build

```
cd backend && .venv/bin/python -m pytest -q
cd ../frontend && npm run build    # outputs frontend/dist
```

Tests use SQLite only; they do not establish SQL Server behaviour (README). Live Google calls are mocked.

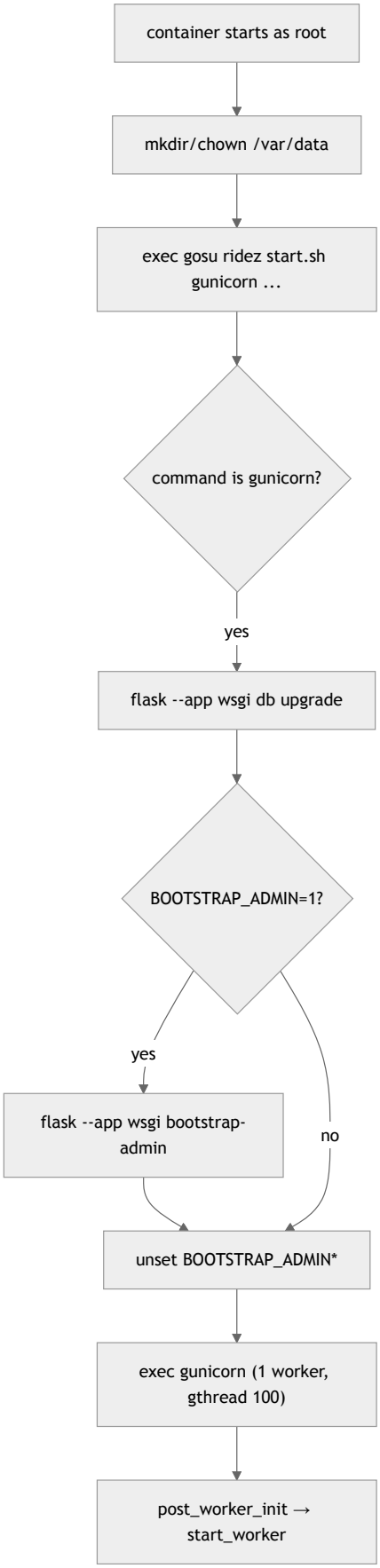
5. Production image (Docker)

```
docker build -t ridez:local .
python deploy/smoke.py --image ridez:local
```

Build stages (`Dockerfile`):

1. `node:22-bookworm-slim`: `npm ci --strict-ssl=true` (optional `proxy_ca` build secret), copy source, `ARG VITE_GOOGLE_MAPS_BROWSER_KEY`, `ARG VITE_GOOGLE_MAP_ID`, `npm run build`.
2. `python:3.12-slim-bookworm`: `APP_ENV=production`, `PORT=10000`; install `libodbc2 ca-certificates gosu`; user `ridez` (uid/gid 10001); `pip install -r requirements.lock`; copy backend app, migrations, `wsgi.py`, `gunicorn.conf.py`, `start.sh`, `backup.py`, SPA → `/app/frontend_dist`; create `/var/data`, `/app/instance` owned by `ridez`.
3. `HEALTHCHECK` every 30 s → `GET /api/v1/ready` (start period 45 s).
4. `ENTRYPOINT /app/start.sh`, `CMD gunicorn --config gunicorn.conf.py wsgi:app`.

Startup sequence (`deploy/start.sh`):



Migrations run at **runtime** (not build) because Render attaches the persistent disk only at runtime.

6. Render deployment (verified configuration)

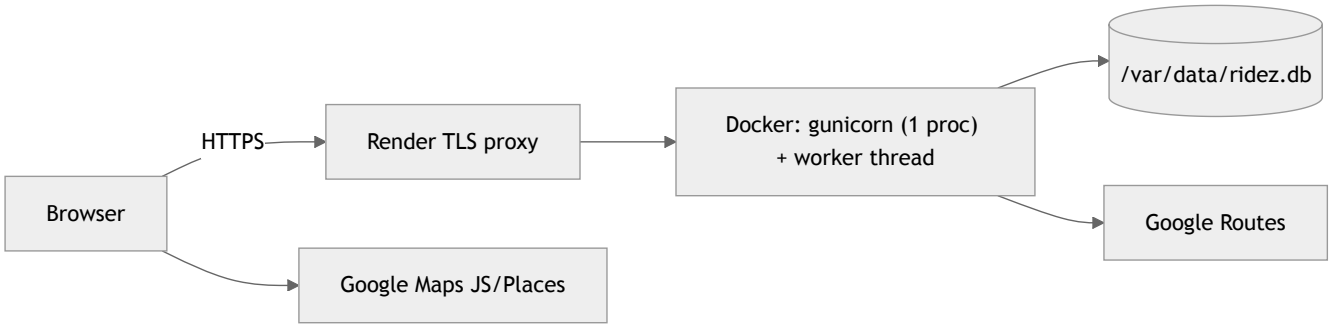
Blueprint (`render.yaml`): Docker runtime, plan `starter`, `numInstances: 1`, `autoDeploy: false`, `healthCheckPath: /api/v1/ready`, disk `ridez-data` 1 GB at `/var/data`, env: `APP_ENV=production`, `DATABASE_URL=sqlite:////var/data/ridez.db`, `JWT_SECRET`/`USER_CODE_SECRET` generated, `TRUST_PROXY_HEADERS=1`, `ROUTE_PROVIDER=google`, `SIMULATOR_ENABLED=0`, `GOOGLE_MAPS_SERVER_KEY` and `VITE_GOOGLE_MAPS_BROWSER_KEY` set manually.

Native Python alternative (from `docs/RENDER_DEPLOYMENT.md`): Build `sh deploy/render-build.sh`, Start `sh deploy/render-start.sh`, health `/api/v1/ready`, `PYTHON_VERSION=3.12.12`, `NODE_VERSION=22`.

First admin: Render Shell `gosu ridez python -m flask --app wsgi create-admin`, or env bootstrap (then set `BOOTSTRAP_ADMIN=0` and remove credentials).

Post-deploy checks (from the deployment guide): `/` and deep links load; `/api/v1/ready` = 200 with `database:true, matching_worker:true`; full ride flow; restart persistence; Socket.IO reaches WebSocket transport.

7. Deployment architecture



One origin serves SPA + API + WebSocket; no CDN, no separate static site, no CORS.

8. CI pipeline (`.github/workflows/checks.yml`)

1. checkout → setup Python 3.12 and Node 22 (npm cache)
2. `python -m pip install -r backend/requirements.lock`
3. `python -m pytest -q` (in `backend`)
4. `npm ci && npm run build` (in `frontend`)
5. `docker build -t ridez:ci .`
6. `python deploy/smoke.py --image ridez:ci` — runs the container with simulator mode, bootstrap admin, checks readiness, admin login, bootstrap env removal, SPA deep links, register/login, Secure+HttpOnly cookie, quote → book → cancel, OTP persistence across `docker restart`, refresh, logout revocation.

9. Logging and monitoring

Aspect	Current state
Access logs	Disabled (`accesslog = None`) to avoid logging Socket.IO session IDs / URLs
Error logs	Gunicorn `errorlog='-'` (stderr), `capture_output=True`
App logs	Only the worker failure message; no structured logging
Health	`/api/v1/health` (liveness + config), `/api/v1/ready` (schema + worker)

Aspect	Current state
Metrics/APM/alerts	None **(recommended: add error tracking and uptime alerts)**

10. Backups and recovery

`gosu ridez python backup.py /var/data/backups/<new-name>.db` in Render Shell. Backups on the same disk are not disaster recovery; download copies off-host. Restore the DB **together with** its `JWT_SECRET` / `USER_CODE_SECRET`. Image rollbacks don't roll back the schema; the suspension migration refuses downgrade while drivers are suspended.

11. Hosting requirements and limits

- Exactly one process/instance (in-process sockets, limiters, caches).
- Persistent disk for SQLite; ephemeral free instances lose data.
- HTTPS (Secure cookies, geolocation).
- Outbound HTTPS to `routes.googleapis.com`.
- Scaling beyond one instance requires a shared Socket.IO message queue (e.g. Redis), shared rate limiting, a server database and worker ownership — **not implemented**.

16 — Dependencies and package explanation

1. Frontend (`frontend/package.json`)

Versions are semver ranges; the exact resolved versions are in `package-lock.json`.

Package	Range	Kind	Purpose	Used in	Essential?
`react`	^19.0.0	runtime	Component model, hooks, context	every `.jsx`	Yes
`react-dom`	^19.0.0	runtime	`createRoot` rendering	`main.jsx`	Yes
`react-router-dom`	^7.0.0	runtime	`BrowserRouter`, `Routes`, `Route`, `Navigate`, `Link`, `NavLink`, `useNavigate`, `useParams`	`main.jsx`, `TestPayment.jsx`	Yes
`socket.io-client`	^4.8.1	runtime	Real-time invalidation events	`RideWorkspace.jsx`	Yes (polling fallback exists)
`lucide-react`	^0.468.0	runtime	SVG icons	most components	No (cosmetic)
`qrcode`	^1.5.4	runtime	QR data URLs for mock payments	`TestPayment.jsx`	Only for test payments
`vite`	^6.0.0	dev	Dev server, proxy, production bundler	`vite.config.js`, scripts	Yes (build)
`@vitejs/plugin-react`	^4.3.0	dev	JSX/Fast Refresh	`vite.config.js`	Yes (build)

Google Maps JS is **not** an npm dependency; it is loaded at runtime from `maps.googleapis.com` by `googleMaps.js`.

npm scripts

Script	Command	Use
`dev`	`vite --host 0.0.0.0`	Local dev server on :5173 (reachable from LAN devices)
`build`	`vite build`	Production bundle in `frontend/dist` (used by Docker, CI, `render-build.sh`)
`preview`	`vite preview --host 0.0.0.0`	Serve the built bundle locally

2. Backend direct dependencies (`backend/requirements.txt`)

Package	Pinned	Purpose	Where used	Essential?
Flask	3.1.0	Web framework	everywhere	Yes
Flask-SQLAlchemy	3.1.1	ORM integration, `db`	`__init__.py`, all data access	Yes
Flask-Migrate	4.1.0	Alembic integration, `flask db` CLI	`__init__.py`, `migrations/env.py`	Yes

Package	Pinned	Purpose	Where used	Essential?
PyJWT	2.10.1	Access tokens	`session_service.py`; tests	Yes
phonenumbers	8.13.55	Mobile normalization	`services.normalize`	Yes
pytest	8.3.4	Tests	`backend/tests`	Dev only (but installed in prod image)
pyodbc	5.2.0	SQL Server DBAPI	only when `DATABASE_URL` is `mssql+pyodbc`	No for SQLite deployments
Flask-SocketIO	5.5.1	WebSocket server	`realtime.py`, `run.py`	Yes
requests	2.32.5	Google Routes HTTP	`route_provider.py`	Yes in Google mode
python-dotenv	1.2.1	`.env` loading, `check-routing`	`__init__.py`	Convenience
tzdata	2025.2	IANA tz database for `zoneinfo`	`fare_service`	Yes for surge schedules on slim/Windows
gunicorn	26.2.0	Production server	`gunicorn.conf.py`, Docker CMD	Yes in production

3. Transitive dependencies (`backend/requirements.lock`)

Package	Version	Pulled in by	Role
SQLAlchemy	2.1.6	Flask-SQLAlchemy	ORM/Core; conditional UPDATE statements
alembic	1.20.1	Flask-Migrate	Migrations; `/ready` reads the script head
Mako	1.4.3	alembic	Revision templates
Werkzeug	3.1.9	Flask	Routing, password hashing, `ProxyFix`
Jinja2 / MarkupSafe	3.1.6 / 3.0.4	Flask	Templating (unused by app code)
itsdangerous	2.2.0	Flask	Mock QR token signing (used directly)
click	8.5.0	Flask	CLI commands
blinker	1.9.0	Flask	Signals
python-socketio / python-engineio	5.17.0 / 4.15.1	Flask-SocketIO	Socket.IO protocol
bidict	0.24.1	python-socketio	Session maps
simple-websocket / wsproto / h11	1.1.0 / 1.3.2 / 0.16.0	python-engineio	WebSocket transport in threading mode
certifi, charset-normalizer, idna, urllib3	2026.10.23, 3.5.2, 3.20, 2.8.0	requests	HTTPS/TLS
typing_extensions	4.16.0	SQLAlchemy/alembic	Typing backports
packaging, pluggy, iniconfig	26.3, 1.7.0, 2.3.1	pytest/gunicorn	

The lock file is what Docker, CI, the README and root `requirements.txt` install, so builds are reproducible.

4. Unused or questionable dependencies

Item	Observation
`pytest` in the runtime lock	Installed into the production image; harmless but unnecessary
`pyodbc` + `libodbc2`	Present for a SQL Server target that the deployment doesn't use; the ODBC Driver 18 itself is not installed, so SQL Server cannot work from the image as-is
`Jinja2`	Not used by app code (only via Flask internals)
`repositories.by_id`	Unused function (code, not a package)

No unused **npm** packages were found: every listed package is imported by `frontend/src`.

5. What depends on what (critical paths)

If this package broke…	…this would fail
SQLAlchemy / Flask-SQLAlchemy	Everything
PyJWT	All authenticated endpoints and sockets
Flask-SocketIO	App boot (imported in `create_app`), worker loop, real-time updates
requests	Google quotes and trip maps
phonenumbers	Register, login, bootstrap
tzdata	Surge zone schedules (`ZoneInfo('Asia/Kolkata')`) on systems without tz data
socket.io-client	Live updates (polling still works at 2 s)
qrcode	QR image only

17 — Technical decisions and project design

Each decision is tagged **Documented** (stated in code comments, README or `docs/`) or **Inferred** (my reading of why the code is shaped this way).

1. Architecture decisions

Decision	Rationale	Tag
Single modular monolith (Flask app serving SPA + API + Socket.IO)	One origin removes CORS and simplifies cookie security; one deployable	Documented (`web.py` docstring "Serve the Vite build from the same origin as HTTP and Socket.IO APIs"; Render guide)
Exactly one process	Matching loop, socket dispatcher, limiters and caches are in-process	Documented (`gunicorn.conf.py` comment, README)
Flask-SocketIO in **threading** mode, no Redis	Avoid a separate Redis install for local Windows development	Documented (README)
Persist matching deadlines and events in the DB	Restarts must recover overdue work; events must not be lost	Documented (README, blueprint)
Socket events are invalidations; clients fetch canonical state	Avoid divergent client state; reconnect is trivial	Documented (blueprint: "events are notifications, not the only source of truth")
Route-provider interface with deterministic simulator	Develop without Google; never silently substitute simulated data in production	Documented (blueprint)
SQL Server as target DB, SQLite in practice	Organizational target (blueprint) vs. single-instance pilot on Render disk	Documented
Conditional UPDATEs + uniqueness for concurrency	Portable across SQLite and SQL Server; no `SELECT FOR UPDATE` (unsupported by SQLite)	Inferred (comment on `ActiveRide`: "portable uniqueness guard")
No-op UPDATE as a lock	Serialize availability/acceptance/suspension on the same row	Documented (comment "Serialize availability/assignment with suspension using the same user row")
Access token in memory + HttpOnly rotating refresh cookie	XSS cannot steal long-lived credentials; reload restores session	Documented (`authSession.js` comment; blueprint)
Session row check on every request	Immediate revocation	Inferred
Immutable quotes and versioned policies/splits	Later changes must not reprice existing quotes/settlements	Documented (README)
Decimal money, strings in JSON	Avoid float accounting errors	Documented (blueprint)
Highest multiplier wins, capped (not compounded)	Predictable surge	Documented (UI text, README)
Revenue allocation ≠ payment	Accounting entry only; earnings "received" only when payment confirmed	Documented
Per-ride HMAC OTP, no DB column	Stable across restarts, unique per ride, no storage	Documented (README)
Vehicle snapshot on ride	History/receipts immutable	Documented
Trip map geometry cached 60 s, shared by participants	Bound Google billing	Documented (README, spec)

Decision	Rationale	Tag
Admin cannot self-register	Privilege escalation prevention	Documented
Riders never see revenue split	Financial privacy	Documented (`docs/ADMIN_MONITORING.md`, spec)

2. Design patterns used

Pattern	Where
Application factory	`create_app`
Blueprint + decorator-based auth	`routes.api`, `authenticated(role)`
Service layer / repository	`*_service.py`, `ride_repository.py`, `repositories.py`
Transactional outbox	`OutboxEvent` + `dispatch()`
Optimistic concurrency (version numbers)	fare policy, revenue split, driver status, vehicle
Compare-and-set via SQL	`transition`, `assign`, `expire_round`, `confirm`
Idempotency keys	`rides.quote_id`; exact-retry detection elsewhere
Strategy / adapter	`route_provider.compute` (simulator vs Google)
Snapshot / event-sourced history (light)	`fare_quotes.rate_snapshot`, `admin_audit`, `ride_match_rounds`
Single-flight / request coalescing	`authSession.refreshSession` (`pendingRefresh`), `trip_map_service.cached_route` (`Future`)
Capability URL	Mock QR token
Serializer methods on models	`public()`
Custom hooks	`useWorkspace`, `useDriverLocation`, `useTripRoute`, `usePaymentPrompt`, `usePaymentConfirmedFlash`, `useTestQr`

3. Code organization

- **Backend:** flat `app/` package, one module per domain area. Route files are thin. Some naming is historical: `services.py` is really "auth/account service", and there are two repository modules (`repositories.py` for users, `ride_repository.py` for everything else).
- **Frontend:** flat `src/` with one file per component/hook, contrary to the blueprint's suggested `features/` folders **(documented plan vs actual)**.
- **Docs:** topic notes per milestone in `docs/`.

4. Reusable logic

Logic	Reused by
`ride_service.expire()`	quote, book, accept, availability, move, snapshot, details
`ride_repository.notify/notify_ride`	every mutating service
`location_service.is_fresh/distance_m/coordinate`	matching, fare demand/zones, trip map, completion
`route_provider.compute`	quotes, trip map, CLI
`session_service.stop_requests`	logout, refresh revoke, admin reviews
`ride_service.public`	workspace, booking, accept, move, details
Frontend `api`, `perform`, `getPosition`, `GoogleRouteMap`	everywhere

5. Separation of concerns — strengths and leaks

Strengths: route handlers hold no business logic; services own validation and transactions; serializers decide visibility.

Leaks:

- Rate limiting is inline in `ride_routes.quote` and `services.verify_trip_otp` rather than in middleware.
- `test_payment_routes.py` contains business logic (status update, notifications) in the route module.
- `ride_service.public` lazily imports models and reads config, and performs many queries (presentation + data access mixed).
- `spec.py` is mutated incrementally instead of declared once.

6. Scalability limitations

Limitation	Cause
One process only	In-memory socket rooms, limiters, caches
SQLite write serialization	Every location update, outbox write and state change needs the single DB write lock; workspace polls also run `advance()`, which writes whenever rounds or locations are due
`/workspace` is query-heavy	Up to 100 rides × ~8–10 queries each (quote, users, two rating aggregates, payments, ledger) per poll; polls every 2–15 s per client
Haversine over all eligible drivers in Python	No spatial index; O(drivers) per matching round and per demand calculation
`demand_signal` scans all SEARCHING rides	O(open requests) per quote
`advance()` runs on most requests	Extra write attempts under load
Admin event per notification	Every `notify()` adds an admins event; with many drivers sending locations every 10–30 s, outbox volume grows quickly
`dispatch()` 200 events/s cap	Backlog under heavy load
`disconnect_invalid()` queries per socket each second	O(connections) DB queries per second

7. Performance considerations already present

- `pool_pre_ping`, indexes on `driver_locations.updated_at`, `outbox_events.delivered_at`, `revenue_share_settings.effective_from`, `auth_sessions.user_id`, `refresh_tokens.session_id`, `ride_ratings.ratee_id`.
- Long-term immutable caching of hashed `assets/*`.
- Trip-map route cache and coalescing.
- Polling slows to 15 s when sockets are live.
- Google field masks and `polylineQuality: OVERVIEW` to reduce payload/cost.
- Debounced address search with session tokens.

8. Technical compromises (acknowledged in code/docs)

Compromise	Source
In-process rate limiters	"Development limiter only; deploy a shared Redis limiter before multiple workers" (`routes.py`)

Compromise	Source
SQLite in production pilot	Render guide: "suitable for a single-instance pilot"
Browser geolocation instead of native tracking	README: "Production background tracking will require a native/mobile implementation"
Mock online payments	`TEST_PAYMENTS.md`, `MockPayment` docstring
SQL Server not validated	README
Legacy compatibility paths	approve without `version`; cancel search without `reason`

Documented plans vs actual implementation

`docs/PRODUCT_BLUEPRINT.md` says	Actual code
SQL Server via pyodbc	SQLite in all exercised environments
Redis-backed background worker	In-process thread, no Redis
`frontend/src/{app,components,features,lib,styles}`	Flat `src/`
`backend/app/{api,services,repositories,models,schemas,jobs}`	Flat modules, no schemas package
Socket events `invitation.created`, `ride.assigned`, `driver.location_changed` with versions	Single `workspace.changed {event_id, server_time}`
Routes like `GET /driver/invitations`, `/admin/rides`, `POST /maps/routes`	Replaced by `/workspace` snapshot
Driver state enum (OFFLINE, AVAILABLE, …)	Derived from `users.status` + `driver_presence` + `active_rides` + location freshness
Event envelopes with entity ID, state version, deadline	Only `event_id` and `server_time`

18 — Problems, limitations and future improvements

Every item below was found by reading the code. Severity: **High** (correctness/security risk in realistic use), **Medium** (operational or scaling risk), **Low** (polish/maintainability). No code was changed.

Incomplete or missing functionality

#	Problem	Where	Why it matters	Impact	Recommendation
M1	No real payment gateway; "online" is a mock QR	`test_payment_routes.py`, `payment_service.choose` (ONLINE requires `TEST_PAYMENTS_ENABLED`)	Production can only take cash	Business blocker	Integrate a gateway (e.g. UPI provider) behind `payment_service.confirm`, with webhooks and idempotency keys
M2	No password reset/account recovery, no profile editing	`main.jsx::Account` text	Locked-out users need manual DB work	Support burden	Add OTP/email-based recovery once mobile verification exists
M3	Mobile numbers are not verified	`services.register`	Anyone can register any number	Fraud/impersonation	SMS verification at registration
M4	`disabled` status has no API/UI	`models.User`, no writer	Admins cannot ban riders	Abuse handling gap	Add admin disable/enable endpoints with audit
M5	No in-progress incident/abort flow	`move` blocks cancel for IN_PROGRESS	A breakdown mid-trip can't be closed except by completing	Wrong charges	Add an incident state with admin resolution
M6	No cancellation fees/no-show policy, refunds, taxes, tips	README "remaining milestones"	Commercial readiness	—	Product decisions + ledger types
M7	Admin cannot list/search all rides with pagination	`/workspace` caps at 100	Ops visibility	—	`GET /admin/rides?page&status&q`
M8	Browser-only GPS (foreground)	`useDriverLocation`	Drivers go stale when switching apps	Missed matches	Native app or PWA background location
M9	OpenAPI incomplete	`spec.py`	Integrators can't rely on it	—	Generate the spec from code (e.g. apispec/pydantic) and cover all routes/response schemas

Potential bugs

#	Severity	Problem	Where	Failure scenario	Recommendation
B1	Medium	In-memory limiter dicts are mutated from up to 100 threads without a lock	`ride_routes.quote` (plain `dict`), `services.verify_trip_otp` (plain `dict`), `routes.limit_auth`	Thread A deletes an empty bucket while thread B is in the `for key in list(buckets): while buckets[key]` loop → `KeyError` → HTML 500 (rare, under concurrency)	Guard with a `threading.Lock`, or move to a shared limiter
B2	Low	Mock QR SUCCESS can coexist with a cash-confirmed payment	`test_payment_routes.simulate_test_payment`	Driver confirms cash between the `confirmed_by_other_method` check and the mock UPDATE → `mock_payments.status=SUCCESS` while `ride_payments.method=CASH`; UI shows both "Simulation successful" and "Paid in cash"	Perform the mock update conditionally on `ride_payments.status='PENDING'` in the same statement/transaction
B3	Low	Clearing a zone latitude/longitude input becomes `0`	`FareSettings.jsx` (`Number(e.target.value)`)	Admin clears a field and saves → zone centred at 0°,0° silently	Keep raw strings and validate before submit
B4	Low	Simulator mode: entering only one destination coordinate sends `0` for the other	`BookingPanel.submit`	Destination saved at lat or lng 0	Require both or neither
B5	Low	Without Web Locks support, two tabs refreshing at once can self-revoke the session	`authSession.locked` falls back to no lock	User signed out unexpectedly in older browsers	Add a localStorage-based mutex fallback
B6	Low	`signOut(true)` keeps the local session if the pre-refresh fails	`authSession.signOut`	User sees an error but still appears signed in with a dead session	On 401 during sign-out, clear local state anyway
B7	Low	`/health.storage` reports `sql-server` for any non-SQLite URL	`routes.health`	Misleading diagnostics	Report the dialect name
B8	Low	Workspace header always says "Development"	`main.jsx::Workspace`	Confusing in production	Drive from `/health` or a build flag
B9	Low	ETA on `TripStatsCard` uses `generated_at` of a cached route (≤ 60 s old)	`TripStatsCard.liveRoute`	Arrival time can be up to a minute stale	Return cache age or compute from request time

#	Severity	Problem	Where	Failure scenario	Recommendation
B10	Low	No generic JSON 500 handler	`__init__.py`	Frontend shows "Unable to read the server response" on any crash	Add an `Exception` handler

Security

#	Severity	Issue	Where	Recommendation
S1	Medium	Rate limits are per-process and IP-only; reset on restart; no per-account lockout	`routes.limit_auth` etc.	Shared limiter (Redis) keyed by IP **and** mobile; progressive delays
S2	Medium	No Content-Security-Policy	`web.py::security_headers`	Add CSP allowing self + Google Maps domains
S3	Low	Swagger UI loads scripts from jsDelivr without Subresource Integrity, and `/docs`, `/openapi.json`, `/health` are public	`routes.docs`, `routes.health`	Pin with SRI or self-host; restrict docs in production
S4	Low	`JWT_SECRET` also signs mock QR tokens (salted)	`test_payment_routes.signer`	Separate key per purpose
S5	Low	Password policy is length-only (10–128)	`services.register`	Add breached-password check
S6	Low	`pytest` and ODBC libs shipped in runtime image	`Dockerfile`, `requirements.lock`	Split runtime vs dev requirements
S7	Info	Trip OTP is a static per-ride PIN	`services.trip_otp`	Acceptable by design; document rotation impact of `USER_CODE_SECRET`
S8	Info	Mock QR link is a 15-min bearer capability	`test_payment_routes`	Keep feature disabled in production (default)

Performance bottlenecks

#	Issue	Where	Impact	Recommendation
P1	N+1 queries in ride serialization	`ride_service.public` (quote, driver, rider, 2× `rating_service.summary`, mock payment, payment, ledger per ride) × up to 100 rides	A single `/workspace` can issue hundreds of queries; polled every 2–15 s per client	Batch-load related rows; cache rating summaries; paginate history
P2	Admin snapshot computes `drivers` (≤ 100 with locations) and drivers compute `allocated_earnings` that the UI never reads	`ride_service.snapshot`	Wasted queries on every admin poll	Remove or move behind explicit endpoints
P3	Haversine filter over all eligible drivers in Python	`matching.invite`, `fare_service.demand_signal`	O(N) per round/quote	Bounding-box prefilter in SQL or a spatial index

#	Issue	Where	Impact	Recommendation
P4	Missing indexes: `rides(status, expires_at)`, `rides(rider_id)`, `rides(driver_id)`, `ride_invitations(driver_id, status)`, `revenue_ledger(driver_id)`, `outbox_events(user_id)`	`models.py`	Full scans as data grows (`expired_searches` runs every second)	Add indexes via migration
P5	Every `notify()` adds an admin event; location updates notify every 10–30 s per driver	`ride_repository.notify`, `location_service.update`	Outbox write amplification; admins refetch constantly	Don't notify on routine location updates (only on freshness changes); coalesce admin events
P6	`disconnect_invalid()` runs DB lookups for every socket every second	`realtime.py`	O(connections) queries/s	Check only on expiry time or on revocation events
P7	Unbounded growth: `refresh_tokens`, `auth_sessions`, `fare_quotes`, `ride_match_rounds`	—	DB size	Periodic cleanup job

Scalability concerns

- Single process/instance (sockets, limiters, caches in memory) — see 13 §6.
- SQLite single-writer; SQL Server path untested and driver not installed.
- Recommendation path: (1) move to PostgreSQL/SQL Server with proper indexes; (2) add Redis as the Socket.IO message queue and rate-limit store; (3) elect one worker instance for `advance()`/`dispatch()` or keep them safe for concurrency (matching already is) and make dispatch use `SELECT … FOR UPDATE SKIP LOCKED`; (4) scale Gunicorn horizontally.

Code organization and maintainability

#	Issue	Where	Recommendation
C1	Misleading module names: `services.py` = auth/OTP; two repository modules	`backend/app`	Rename to `auth_service.py`, `user_repository.py`
C2	Many function-local imports to avoid cycles	`ride_service`, `routes`, `ride_routes`	Restructure dependencies (e.g. move `accessible` to a shared module)
C3	`spec.py` mutated incrementally	`spec.py`	Declarative spec or generated
C4	Very long single-line JSX and Python lines	`RideWorkspace.jsx`, `main.jsx`, `models.py`	Format with Prettier/Black; split `RideWorkspace` into rider/driver/admin views
C5	Manual validation duplicated (text lengths, versions, reasons)	services	Shared validators or a schema library
C6	No frontend tests in CI; browser script has hard-coded paths	`frontend/tests`	Add Vitest + Playwright in CI
C7	Unused code: `repositories.by_id`, `GET /me`, `/driver/overview`, `/admin/overview`, `GET /rides/{id}` (not used by SPA)		Keep if intended for API clients; otherwise remove

#	Issue	Where	Recommendation
C8	Duplicate currency formatters across files	`RideWorkspace`, `AssignedRideDetails`, `PaymentStatus`, `TestPayment`	Shared `format.js`
C9	Naive vs aware datetimes mixed (`users.created_at` aware, others naive)	`models.py`	Standardize

Database optimization opportunities

1. Indexes listed in P4.
2. Store rating aggregates (average/count) on users or a materialized table updated on insert.
3. Partition/clean outbox and token tables.
4. Use `RETURNING` (where supported) to avoid re-reads after conditional updates.
5. Enable SQLite WAL mode + busy timeout while SQLite remains in use **(recommended; not currently configured)**.

Priority recommendation list

1. Real payments and mobile verification (business blockers).
2. Fix B1 (limiter locking) and add a JSON 500 handler.
3. Add indexes (P4) and reduce `/workspace` query count (P1, P2).
4. CSP + shared rate limiting (S1, S2).
5. Database migration off SQLite + Redis for multi-instance.

19 — Manager explanation guide

Speaking scripts you can deliver nearly verbatim. Numbers and names are from the code.

A. Project explanation (5–10 minutes)

1. What it is (30 s). ridez.io is a taxi-booking web application with three workspaces in one React app: riders book trips, drivers accept and run them, and admins operate the platform. The backend is a single Flask service that serves the API, real-time updates over Socket.IO, and the compiled website from the same origin.

2. The rider journey (1.5 min). A rider searches pickup and destination using Google Places autocomplete. The server calls Google's Routes API for real road distance and traffic-aware duration and calculates an itemized INR fare: base plus per-km plus per-minute, a minimum fare, and optional surge. That quote is stored, locked for five minutes, and records the fare-policy version, so later rate changes can't reprice it. When the rider confirms, the quote ID acts as an idempotency key, so double-clicks never create two rides.

3. Matching (1.5 min). Matching starts at 500 metres from pickup and expands by 500 metres every 30 seconds up to a configurable 5 km. Only approved, available drivers whose GPS is fresher than 90 seconds are invited, and a driver is never invited twice for the same ride. The round deadlines live in the database, and a one-second background loop advances them, so a server restart catches up instead of losing searches. When drivers race to accept, a single conditional SQL update decides the winner, and a unique "active ride" claim per user guarantees nobody is double-booked.

4. The trip (1 min). After acceptance the rider sees the driver's name, rating, and car number. The car details are snapshotted onto the ride. The driver moves the trip through head-to-pickup, arrived, start, and complete. Starting requires a six-digit OTP that is unique per ride and shown only to the rider. For Google-routed rides both sides see a live road map with ETA, refreshed every 10 seconds but cached server-side so we call Google at most once a minute per ride phase.

5. Money (1 min). On completion we record a revenue allocation: by default 85% driver, 15% company, using whatever split was effective at that moment; admins can schedule future splits. A payment record opens as pending. The rider chooses cash, and the driver confirms receipt, or "online", which today is a clearly-labelled test QR flow with no real money. The first confirmation wins, and driver earnings count as received only after confirmation.

6. Admin and safety (1 min). Admins approve, suspend, and reinstate drivers with mandatory reasons and version checks so two admins can't overwrite each other. Suspension stops new work but lets a driver finish their current trip. Admins edit versioned fare and surge rules and revenue splits, and every change appears in a searchable audit history with before and after values.

7. Security (45 s). Login is mobile plus password. The access token is a 15-minute JWT kept only in memory; a rotating, single-use, HttpOnly refresh cookie restores the session for up to 30 days, and reusing an old refresh token revokes the session. Every request also checks the server-side session, so logout is immediate.

8. Status and limits (1 min). It's deployed as one Docker container on Render with SQLite on a persistent disk, migrations at startup, and a readiness check. CI runs the test suite, builds the frontend and the image, and runs a container smoke test. The main limits are that it must run as a single process, payments are

> simulated, GPS depends on the driver keeping the browser tab open, and SQL Server is targeted but not validated.

B. Architecture explanation (simple but accurate)

> "It's a modular monolith. The React app talks to a Flask REST API under `/api/v1` with a bearer token. Inside Flask, thin route handlers call service modules like `ride_service`, `matching`, and `fare_service`, which use SQLAlchemy to read and write the database. Everything important, including matching deadlines and outgoing notifications, is stored in the database. A background thread runs every second: it advances matching rounds and turns 'outbox' rows into Socket.IO messages. The browser treats those messages as 'something changed' and re-fetches one endpoint, `/workspace`, which returns everything that user's screen needs. If the socket drops, the browser polls every two seconds instead. Google is called from the server for routes and from the browser for maps and address search."

Draw: Browser → (REST + WebSocket) → Flask [routes → services → repositories] → DB; background loop → DB → Socket.IO → Browser; server → Google Routes; browser → Google Maps/Places.

C. Feature speaking scripts

Authentication. "Mobile numbers are normalized to E.164 with Google's `phonenumbers` library, so formatting differences don't create duplicate accounts. Passwords are hashed with Werkzeug. Login failures always return the same message and do equal work for unknown users. We issue a 15-minute JWT plus a refresh cookie scoped to `/api/v1/auth`, HttpOnly and SameSite Lax. The refresh token is stored only as a SHA-256 hash and is single-use. The frontend serializes refreshes across tabs with the Web Locks API so tabs don't trip replay detection."

Driver availability. "A driver goes available by sending a fresh GPS fix with the request. The server checks accuracy (200 m or better), timestamp freshness, ordering, and physically plausible movement. The browser keeps sending location every 30 seconds while idle and every 10 seconds on a trip. The background loop marks any location older than 90 seconds as stale, which removes the driver from matching."

Quotes and surge. "Fares are Decimal-based and rounded half-up. Surge has three sources: scheduled circular zones around the pickup, traffic (Google's traffic duration divided by no-traffic duration), and demand (open requests divided by fresh drivers within 1 km). We take the single highest multiplier and cap it; we never multiply them together. The reason is stored on the quote and shown to the rider."

Matching and acceptance. "Booking creates the ride, the rider's active-ride claim, round 1, and invitations in one transaction, plus outbox events. Acceptance is one UPDATE whose WHERE clause re-checks everything: the ride is still searching, the round hasn't expired, this driver has a valid pending invitation, and they're still available with fresh GPS. If the row count is 1 they won; otherwise they get a 409."

Trip and OTP. "The OTP is an HMAC of the ride ID with a server secret, reduced to six digits. Nothing is stored, it survives restarts, and a code from one ride can never start another. Wrong attempts are rate-limited to five per minute per driver."

Cancellation. "Before the trip starts, the rider or assigned driver can cancel with a reason. It's recorded with who, when, and why, shown to both sides, and both active claims are released. The driver is left offline. In-progress trips can't be cancelled."

Payments. "Completion creates a pending payment for the quoted total. Cash is confirmed by the driver; online is currently a signed, 15-minute test QR link. Both go through one conditional update, so whichever confirmation commits first wins."

Ratings. "Each side rates the other once per completed trip with stars, tags, and an optional comment. Comments are visible only to admins. A retry with the same stars is idempotent; changing the rating is refused."

Admin. "Every admin write uses an optimistic version number. Driver status changes also call the same 'stop requests' routine as logout, so a suspended driver immediately stops receiving invitations. The audit page is a UNION over three history tables with escaped search and pagination."

Real-time. "We use a transactional outbox: events are rows written in the same transaction as the change, so you never notify about something that rolled back. The worker emits them within a second. Events carry no data, so duplicates or missed events are harmless; the client just refetches."

D. Demo script (if asked to show it)

1. Three browsers: admin, driver, rider (simulator mode works without Google).
2. Admin → Drivers → approve the driver with a reason; add vehicle details.
3. Driver → Go available (allow location).
4. Rider → enter pickup at the driver's location (or use DevTools Sensors), get the estimate, confirm.
5. Driver accepts → rider sees car and OTP → driver: head to pickup → arrived → enter OTP → start → complete.
6. Rider payment modal → pay by cash → driver "Cash received" → both rate.
7. Admin → overview metrics, audit history.

E. One-line answers to "what did you learn / what's next?"

- **Hardest part:** making every state change race-safe with conditional updates and unique claims, and making the system restart-safe by persisting deadlines and events.
- **Next steps:** real payment gateway and mobile verification; reduce `/workspace` query load and add indexes; move off SQLite and add Redis to scale beyond one process; add CSP and shared rate limiting; frontend tests in CI.

20 — Technical interview questions and answers

75 questions with answers grounded in this repository. File references let you jump to the code if challenged.

Architecture

1. Describe the overall architecture. A modular monolith: React 19 SPA → Flask REST API `/api/v1` + Flask-SocketIO in one Python process → SQLAlchemy → SQLite (Render) with SQL Server as an untested target. Layers: route modules (`routes.py`, `ride_routes.py`, `test_payment_routes.py`) → services (`ride_service`, `matching`, `fare_service`, …) → repositories (`ride_repository`, `repositories`) → `models.py`. A 1-second background thread (`realtime.py`) advances matching and dispatches outbox events.

2. Why a single process? The Socket.IO rooms, connection map, rate limiters and trip-map cache are in memory, and the outbox dispatcher emits through the local Socket.IO server. `gunicorn.conf.py` hard-codes `workers = 1` with 100 threads, and `render.yaml` sets one instance. It was a deliberate choice to avoid Redis for local Windows development (README).

3. How does the frontend get data? Mostly from one endpoint, `GET /api/v1/workspace`, which returns a role-specific snapshot built by `ride_service.snapshot`. `useWorkspace` polls it (15 s with a live socket, 2 s without) and refetches whenever a `workspace.changed` event arrives.

4. Why send "something changed" events instead of the data itself? Events are at-least-once and may be missed while disconnected; refetching canonical state makes duplicates and losses harmless and keeps client logic simple. The blueprint states "events are notifications, not the only source of truth".

5. What is the transactional outbox here? `ride_repository.notify()` inserts `outbox_events` rows in the same DB transaction as the business change. `realtime.dispatch()` reads undelivered rows, emits `workspace.changed` to `user:{id}` or `admins`, marks them delivered, and deletes delivered rows older than 7 days.

6. Where does the background work run and how does it start? `realtime.start_worker(app)` → `socketio.start_background_task(loop)`; `loop` calls `tick()` every second. Started by `run.py` (dev) or Gunicorn's `post_worker_init` (prod). `flask run` does not start it.

7. What happens if the worker thread dies? `tick` exceptions are caught and logged generically and the loop continues. If the thread itself stopped, `/ready` turns 503 after 10 s (Render health check), and matching still progresses on requests because `ride_service.expire()` calls `matching.advance()`; only socket pushes would stop (clients fall back to polling).

8. How is the code organized? Flat `backend/app/` with one module per domain; flat `frontend/src/` with one file per component/hook. Tests in `backend/tests/`, migrations in `backend/migrations/versions/`, deployment in `deploy/`.

Backend

9. Walk me through `create_app`. Loads `backend/.env` without overriding env, decides development vs production, auto-generates dev secrets into `instance/`, sets DB URL/secrets/flags, validates required settings (≥ 32-char distinct secrets in prod, Google key with Google provider), optionally wraps `ProxyFix`, initializes SQLAlchemy/Migrate, imports route modules, registers the blueprint at `/api/v1`, initializes Socket.IO and SPA serving, adds JSON 404/405 and CLI commands.

10. How does a route enforce roles? `@authenticated('driver')` in `routes.py` requires a Bearer token, calls `session_service.authenticate`, stores `g.user`, and returns 403 if the role differs. Ownership is checked again inside services.

11. Which file controls booking? `ride_service.book` (`backend/app/ride_service.py`), exposed by `POST /rides` in `ride_routes.py`.

12. Which file controls trip state transitions? `ride_service.move` with the `transitions` map; writes via `ride_repository.transition`. Route: `POST /rides/<id>/<action>` (`ride_routes.progress`).

13. Why are some imports inside functions? To break circular imports (`ride_service` ↔ `admin_service` / `trip_map_service`, `routes` ↔ `session_service`) and delay heavy imports (Alembic in `/ready`).

14. How are errors returned? Services raise `ApiError(message, status)`; `routes.api_error` returns `{"error": message}` with that status and `no-store`. There's no JSON handler for unexpected exceptions (a known gap).

15. How is money handled? `Decimal` with `ROUND_HALF_UP` to 0.01 (`money()` helpers), stored as `Numeric(10,2)`, sent as strings. Revenue allocation rounds the driver amount and gives the company the remainder so the sum is exact.

16. How is the fare calculated? `fare_service.calculate`: `base + distance_m/1000 × per_km + duration_seconds/60 × per_minute`, then `max(…, minimum)`, then × the highest eligible surge multiplier capped at `max_multiplier`. Defaults: ₹40 + ₹12/km + ₹2/min, minimum ₹40, surge off.

17. How does surge work? Only if `surge_enabled`. Candidates: zones (enabled, scheduled in policy timezone, pickup within radius), traffic (`duration/staticDuration ≥ traffic_threshold`), demand (`requests ≥ demand_min_requests` and `requests/max(1,drivers) ≥ demand_threshold` within 1 km). Highest wins; reasons stored in `surge_reason`, `(capped)` appended if capped.

18. How do overnight surge zones work? `fare_service.scheduled`: if `start > end`, the window belongs to its start day — active on a selected day after `start`, or on the following day before `end`. `start == end` means all day.

19. What does the simulator do? `route_provider.compute` derives a stable distance (2–20 km) from a SHA-256 of the lowercase labels and 3 min/km duration (≥ 5 min). Enabled by `ROUTE_PROVIDER=simulator` and `SIMULATOR_ENABLED=1` (dev defaults).

20. What is `expire()` and why is it called everywhere? `ride_service.expire()` runs `matching.advance()` then `db.session.expire_all()`, so every request first catches up overdue matching deadlines and sees fresh data, even if the worker is lagging.

APIs

21. List the main ride endpoints. `POST /fares/quotes`, `POST /rides`, `POST /rides/{id}/accept|reject`, `POST /rides/{id}/depart|arrive|start|complete|cancel`, `GET /rides/{id}`, `GET /rides/{id}/map`, `POST /rides/{id}/payment-method|confirm-cash|rating|test-payment`, `GET /workspace`.

22. How is booking made idempotent? `rides.quote_id` is unique. `book()` returns the existing ride if one exists for the quote; on `IntegrityError` from a concurrent duplicate it rolls back and returns the existing ride.

23. What does `POST /driver/availability` expect? `{"available": true, latitude, longitude, accuracy_m, recorded_at}` to go online (fresh fix required) or `{"available": false}`. While on a trip, `available:true` only saves `resume_available`.

24. What's the difference between `/health` and `/ready`? `/health` always 200 with config and worker status. `/ready` returns 200 only if `alembic_version` equals the migration head and the worker ticked in the last 10 s; else 503. Docker and Render use `/ready`.

25. Why do some endpoints return 404 instead of 403? `ride_service.accessible` and payment/rating services return 404 for rides you don't participate in, so the API doesn't reveal that a ride ID exists.

26. What status codes does the API use for conflicts? 409 for state conflicts/stale versions/lost races; 410 for expired QR links; 422 for no route / fare too large; 429 for rate limits; 502/503/504 for Google failures.

27. Is there API documentation? `/api/v1/openapi.json` and `/api/v1/docs` (Swagger UI from jsDelivr), built in `spec.py`. It's incomplete (no response schemas, missing test-payment routes); this knowledge base is more complete.

Database

28. What are the core tables? `users`, `fare_quotes`, `rides`, `active_rides`, `ride_invitations`, `ride_match_rounds`, `driver_presence`, `driver_locations`, `driver_vehicles`, `revenue_ledger`, `revenue_share_settings`, `fare_policies`, `admin_audit`, `auth_sessions`, `refresh_tokens`, `outbox_events`, `ride_payments`, `mock_payments`, `ride_ratings` (19).

29. How do you guarantee a user has only one active ride? `active_rides` has primary key `user_id`. Booking inserts the rider's row, acceptance inserts the driver's; a second insert raises `IntegrityError`. Rows are deleted by `ride_repository.release` on completion, cancellation or NO_DRIVER_FOUND.

30. How do you prevent inviting the same driver twice? `ride_invitations` primary key `(ride_id, driver_id)`, plus `invite()` skips drivers already invited for that ride.

31. Why is the ride's `expires_at` reused for rounds? It is the current round deadline. `advance()` claims a round by matching both `round_no` and `expires_at`, which makes processing idempotent between the worker and requests.

32. How are migrations handled? Alembic via Flask-Migrate, 13 linear revisions (head `c51e8a2f90d4`), run at container start (`deploy/start.sh`). `batch_alter_table` is used for SQLite compatibility. Migration `7ba01cddd40c` back-fills payments for already-completed rides.

33. Are there seeds? No. Defaults are in code (`DEFAULT_POLICY`, 85/15 split); admins are created via CLI or bootstrap env vars.

34. Which DB constraints enforce business rules? CHECKs for ride status, user role/status (suspended only for drivers), revenue split sum = 100, payment status/method/completeness, rating stars 1–5 and role. Unique: mobile, user_code, quote_id; PK-based uniqueness for claims, ledgers, payments, ratings, versions.

35. What indexes exist and what's missing? Present: `driver_locations.updated_at`, `outbox_events.delivered_at`, `revenue_share_settings.effective_from`, `auth_sessions.user_id`, `refresh_tokens.session_id`, `ride_ratings.ratee_id`. Missing: `rides(status, expires_at)`, `rides(rider_id)`, `rides(driver_id)`, `ride_invitations(driver_id)`, `revenue_ledger(driver_id)`.

Authentication and security

36. How does authentication work internally? Login creates an `auth_sessions` row and a refresh token (hash stored). The response has a 15-min HS256 JWT `{sub, sid, iat, exp}` and sets the `ridez_refresh` HttpOnly cookie. Each request: decode JWT with required claims → load user (not disabled) → check the session row is valid and belongs to the user → role check.

37. Where is the token stored on the client? In a module variable in `authSession.js` (memory only). On reload, `AuthProvider` calls `/auth/refresh` with the cookie.

38. How does refresh rotation and replay detection work? `session_service.refresh` marks the presented token used with `UPDATE … WHERE used_at IS NULL`; if the row count isn't 1, the token was reused and the session is revoked. Otherwise a new token is issued.

39. How do you protect cookie endpoints from CSRF? `check_origin()` requires `X-Session-Request: 1`, a same-host or trusted `Origin`, and rejects `Sec-Fetch-Site: cross-site`. Cookie is `SameSite=Lax` and path-scoped. Business endpoints use Bearer tokens, which browsers don't attach automatically.

40. How are passwords stored? `werkzeug.security.generate_password_hash` (scrypt by default in Werkzeug 3.x). Login uses `check_password_hash` and a dummy hash for unknown users.

41. What rate limiting exists? In-memory sliding windows: 10 login/register attempts per minute per IP, 10 quotes per minute per rider, 5 failed OTPs per minute per driver. Per-process only.

42. How is admin creation protected? Public registration rejects `admin`. Admins come from `flask create-admin` or `flask bootstrap-admin` reading `BOOTSTRAP_ADMIN_*` env vars at startup; the bootstrap never promotes existing users and the env vars are unset before Gunicorn runs.

43. What privacy rules are enforced? Riders never receive `settlement`; driver coordinates are never in admin views (`location_service.public` omits them); trip maps are participant-only; OTP only in the rider's snapshot; rating comments only for admins.

44. What happens on logout to an active driver? `stop_requests`: `available=false`, `resume_available=false`, pending invitations closed, notify. The active trip continues and the driver can still complete it after signing back in.

Data flow

45. How does data move from frontend to database when booking? `BookingPanel` → `perform('/rides', {quote_id})` → `api()` with Bearer → `ride_routes.book` → `ride_service.book` → inserts `rides`, `active_rides`, `ride_match_rounds`, `ride_invitations`, `outbox_events` → commit → 201 ride JSON → `refresh()` → `/workspace` → `ActiveRidePanel`. Drivers get the socket event and see the invitation.

46. How does the rider learn a driver accepted? Acceptance writes outbox events for the rider; the worker emits `workspace.changed` within ~1 s; `useWorkspace` refetches; `AssignedRideDetails` renders. Without sockets, the 2-second poll catches it.

47. How does the driver's location reach the rider's map? `useDriverLocation` → `PUT /driver/location` → `driver_locations`. The rider's `useTripRoute` polls `GET /rides/{id}/map` every 10 s; `trip_map_service.position` returns fresh coordinates; `GoogleRouteMap` moves the taxi marker.

48. What happens at completion? In one transaction: ride → COMPLETED, `revenue_ledger` insert, `ride_payments` PENDING insert, claims deleted, driver availability decided, outbox events. The rider's `usePaymentPrompt` opens the payment modal.

Third-party integrations

49. What Google APIs are used and where? Routes API from the server (`route_provider.compute`, server key); Maps JavaScript API and Places API (New) from the browser (`googleMaps.js`, `AddressSearch.jsx`, `GoogleRouteMap.jsx`, browser key).

50. What happens when Google fails during a quote? Specific errors: timeout 504, connection 502, quota 503, denied 502 with an allow-listed reason, no route 422, malformed 502. No simulated fallback in Google mode.

51. How do you limit Google costs? Field masks, `polylineQuality: OVERVIEW`, quote rate limit, trip-map cache (60 s per ride/phase/source/target, shared by both participants, coalesced across threads), and Places session tokens.

52. Why two Google keys? The browser key is public and must be referrer/API-restricted; the server key stays secret on the backend. `.env.example` warns never to put the server key in a `VITE_` variable.

Error handling and edge cases

53. What if two drivers accept the same ride simultaneously? Both run `ride_repository.assign`, a single conditional UPDATE `WHERE status='SEARCHING' …`. One gets row count 1; the other gets 0 → rollback → 409 "Another driver has already accepted this ride". Tested in `test_rides.py::test_concurrent_acceptance_has_one_winner`.

54. What if one driver accepts two rides simultaneously? Both might pass `assign`, but inserting the second `active_rides` row with the same `user_id` fails → `IntegrityError` → rollback → 409.

55. What if a rider cancels while a driver accepts? Both are conditional updates on `rides.status`; whichever commits first wins. The loser's `transition` / `assign` returns 0 → 409. Tested in `test_cancellation.py::test_cancel_race_has_one_valid_outcome`.

56. What if cash is confirmed while the QR payment succeeds? Both call `payment_service.confirm`, `UPDATE … WHERE status='PENDING'`. First wins; cash confirmation then returns 409 "already paid online", or the mock success just doesn't confirm. (A minor inconsistency can leave the mock row SUCCESS; see 18 B2.)

57. What if the server restarts during a search? Deadlines are in `rides.expires_at`. On restart, `advance()` claims overdue rounds and schedules each next deadline from the previous one; expired rounds send no invitations; it stops at the first still-open round or ends in NO_DRIVER_FOUND.

58. What if the driver's GPS goes stale during a trip? The trip continues. The map shows the pickup→destination route without a marker; ETA falls back to the booked estimate; at completion the driver won't be auto-available.

59. What if a location update arrives out of order? Older than the stored `recorded_at` → 409; equal → treated as a replay (no change). Jumps faster than 200 km/h plus accuracies plus 100 m are rejected while tracking is fresh.

60. What if an admin edits fares while a rider holds a quote? The quote keeps its stored totals and `fare_rule_version`; booking uses the quote. New quotes use the new version.

Deployment

61. How is it deployed? Render Blueprint: one Docker web service, persistent 1 GB disk, SQLite at `/var/data/ridez.db`. `start.sh` fixes disk ownership, drops to user `ridez`, runs `flask db upgrade`, optional bootstrap admin, then Gunicorn. Health check `/api/v1/ready`. Manual deploys.

62. What does CI do? pytest, `npm ci && npm run build`, `docker build`, then `deploy/smoke.py` runs the real image: readiness, bootstrap admin, SPA deep links, register/login, Secure cookie, quote/book/cancel, OTP persistence across restart, refresh, logout revocation.

63. How do you run it locally? Backend: venv, `pip install -r backend/requirements.lock`, `flask --app run db upgrade`, `python run.py` (port 5000). Frontend: `npm ci`, `npm run dev` (5173, proxies `/api` and `/socket.io`).

64. How are backups done? `deploy/backup.py` with SQLite's online backup API and an integrity check; must be copied off the disk for disaster recovery.

Performance and scalability

65. What are the performance bottlenecks? `/workspace` N+1 queries (up to 100 rides × multiple lookups including two rating aggregates each), polling every 2–15 s; Python haversine over all drivers; missing indexes; outbox write amplification (an admin event for every notify, location updates every 10–30 s); SQLite single writer.

66. How would you scale this application? Move to PostgreSQL/SQL Server with indexes; add Redis as Socket.IO message queue and rate-limit store; run multiple Gunicorn workers/instances; make dispatch use `SKIP LOCKED` or leader election; batch-load in `snapshot`; paginate history; spatial index for drivers; reduce notifications for routine location updates.

67. Is matching safe if two workers ran it? Yes for correctness: `expire_round` and `assign` are compare-and-set. The unsafe parts are notifications (local Socket.IO) and limiters/caches.

Difficult "why" questions

68. Why Flask and not FastAPI/Django? The blueprint specified a Flask REST API with SQLAlchemy and Alembic. Flask-SocketIO's threading mode lets sockets, SQLAlchemy and `requests` share ordinary blocking code without an async rewrite **(inferred)**.

69. Why is `lock_active_driver` an UPDATE that sets a column to itself? It's a portable row lock: acquiring a write lock on the driver's row (or the whole DB in SQLite) serializes availability, acceptance, suspension and completion for that driver, and the WHERE clause doubles as a status check.

70. Why is the OTP derived instead of stored? No migration needed, stable across restarts, unique per ride, and it can't be read from the DB. The trade-off is that rotating `USER_CODE_SECRET` changes OTPs for unstarted rides.

71. Why does matching schedule the next deadline from the old deadline, not now? So catch-up after downtime replays rounds on the original timeline instead of restarting the clock, and stale rounds are recorded without inviting drivers.

72. Why keep the access token only in memory? XSS can't read it from storage, and the long-lived credential is an HttpOnly cookie. The cost is a refresh call on every page load.

73. What are the limitations of the current architecture? Single process; SQLite; foreground-only browser GPS; mock online payments; no recovery or verification; in-memory limits; heavy polling payload; SQL Server untested.

74. What would you improve with more time? Real payments, mobile verification, password recovery; JSON 500 handler and request IDs; fix limiter thread safety; indexes and snapshot batching; CSP; Redis + server DB for scaling; frontend tests in CI; split `RideWorkspace` into role-specific components.

75. How do you know it works? 18 pytest modules cover auth, sessions, rides, matching, cancellation, availability, reviews, vehicles, Google fares (mocked), trip maps, revenue, payments, ratings, OTP, privacy, bootstrap and deployment, including thread-race tests on file-backed SQLite. CI also runs a container smoke

test. Not covered: live Google, SQL Server, frontend unit tests. (The suite was not re-run while writing this documentation.)

21 — Quick revision sheet

Technologies and responsibilities

Tech	Job
React 19 + React Router 7 + Vite 6	SPA, routing, dev proxy, build
socket.io-client / Flask-SocketIO (threading)	`workspace.changed` push
Flask 3.1 + Blueprint `api` at `/api/v1`	REST API
SQLAlchemy 2.1 + Flask-Migrate/Alembic	ORM + 13 migrations (head `c51e8a2f90d4`)
SQLite (dev + Render) / SQL Server (untested target)	DB
PyJWT HS256	15-min access tokens
Werkzeug	password hashing, ProxyFix
phonenumbers	E.164 mobiles
requests	Google Routes
itsdangerous	mock QR tokens
Gunicorn 1 worker gthread 100 threads	production server
Docker + Render + GitHub Actions	deploy + CI

Key file paths

Path	One-liner
`backend/app/__init__.py`	`create_app`, config validation, CLI
`backend/app/models.py`	19 tables
`backend/app/routes.py`	blueprint, `authenticated`, auth/health/ready
`backend/app/ride_routes.py`	rides, driver, admin routes
`backend/app/test_payment_routes.py`	mock QR
`backend/app/ride_service.py`	quote, book, availability, accept, move, snapshot
`backend/app/matching.py`	invite, advance
`backend/app/ride_repository.py`	assign, transition, expire_round, locks, notify
`backend/app/session_service.py`	JWT, refresh rotation, logout, check_origin
`backend/app/services.py`	ApiError, register, login, trip OTP
`backend/app/fare_service.py`	policy, calculate, surge
`backend/app/route_provider.py`	Google/simulator routes
`backend/app/location_service.py`	GPS validation, freshness, haversine
`backend/app/payment_service.py`	payment choose/confirm
`backend/app/revenue_service.py`	split versions, allocate
`backend/app/rating_service.py`	ratings
`backend/app/admin_service.py`	metrics, audit, driver status, vehicles

Path	One-liner
`backend/app/trip_map_service.py`	live map + cache
`backend/app/realtime.py`	sockets, dispatch, worker
`backend/app/web.py`	SPA serving + security headers
`backend/run.py` / `wsgi.py` / `gunicorn.conf.py`	entry points
`frontend/src/main.jsx`	routes, AuthProvider, Workspace
`frontend/src/authSession.js` / `api.js`	session store / fetch wrapper
`frontend/src/RideWorkspace.jsx`	role dashboards, `useWorkspace`, `perform`
`frontend/src/BookingPanel.jsx`, `AddressSearch.jsx`	booking
`frontend/src/useDriverLocation.js`	GPS loop
`frontend/src/TripRouteMap.jsx`, `TripStatsCard.jsx`, `GoogleRouteMap.jsx`	maps
`frontend/src/TestPayment.jsx`, `PaymentStatus.jsx`, `Rating.jsx`	payments, ratings
`frontend/src/DriverManagement.jsx`, `FareSettings.jsx`, `RevenueSettings.jsx`, `AdminAudit.jsx`	admin
`deploy/start.sh`, `Dockerfile`, `render.yaml`	deployment

Main functions

`create_app` · `authenticated(role)` · `session_service.authenticate/issue/refresh/logout/stop_requests/check_origin` · `services.register/login/normalize/trip_otp/verify_trip_otp` · `ride_service.quote/book/availability/accept/reject/move/snapshot/public/expire/accessible` · `matching.invite/advance/close_round` · `ride_repository.assign/transition/expire_round/eligible_drivers/lock_active_driver/notify/notify_ride/release` · `location_service.update/is_fresh/distance_m` · `route_provider.compute` · `fare_service.calculate/validate/save/scheduled/demand_signal` · `revenue_service.active/save/allocate` · `payment_service.choose/confirm/confirm_cash` · `rating_service.submit/summary` · `admin_service.change_driver_status/save_driver_vehicle/driver_list/audit/audit_detail/metrics` · `trip_map_service.trip_map/cached_route` · `realtime.tick/dispatch/start_worker` · Frontend: `api`, `refreshSession`, `signIn`, `signOut`, `useWorkspace`, `perform`, `useDriverLocation`, `getPosition`, `useTripRoute`, `usePaymentPrompt`.

API endpoints (`/api/v1`)

Auth	Endpoints
public	`GET /health`, `GET /ready`, `GET /openapi.json`, `GET /docs`, `POST /auth/register`, `POST /auth/login`, `POST /auth/refresh`, `POST /auth/logout`, `GET/POST /test-payments/{token}`
any signed in	`GET /me`, `GET /workspace`, `GET /rides/{id}`, `POST /rides/{id}/{depart|arrive|start|complete|cancel}`, `POST /auth/logout-all*`
rider	`POST /fares/quotes`, `POST /rides`, `POST /rides/{id}/payment-method`, `POST /rides/{id}/test-payment`
driver	`POST /driver/availability`, `PUT /driver/location`, `GET /driver/overview`, `POST /rides/{id}/accept|reject|confirm-cash`

Auth	Endpoints
rider or driver	`POST /rides/{id}/rating`, `GET /rides/{id}/map`
admin	`GET /admin/overview`, `GET /admin/drivers`, `POST /admin/drivers/{id}/approve|suspend|reinstate`, `PUT /admin/drivers/{id}/vehicle`, `GET/PUT /admin/fare-settings`, `GET/PUT /admin/revenue-share`, `GET /admin/audit`, `GET /admin/audit/{category}/{id}`

* cookie endpoints need `X-Session-Request: 1`. Socket.IO: connect with `auth.token`; event `workspace.changed {event_id, server_time}`.

Database models

`User` (users) · `DriverPresence` · `DriverLocation` · `DriverVehicle` · `FareQuote` · `FarePolicy` · `Ride` · `ActiveRide` · `Invitation` (ride_invitations) · `MatchRound` (ride_match_rounds) · `OutboxEvent` · `RevenueLedger` · `RevenueShareSetting` · `AdminAudit` · `AuthSession` · `RefreshToken` · `RidePayment` · `MockPayment` · `RideRating`

Ride states: `SEARCHING → ASSIGNED → ARRIVING → WAITING → IN_PROGRESS → COMPLETED`; terminal `CANCELLED`, `NO_DRIVER_FOUND`.

Environment variables (names only)

Backend: `APP_ENV`, `DATABASE_URL`, `JWT_SECRET`, `USER_CODE_SECRET`, `ROUTE_PROVIDER`, `GOOGLE_MAPS_SERVER_KEY`, `SIMULATOR_ENABLED`, `TEST_PAYMENTS_ENABLED`, `MATCH_MAX_RADIUS_M`, `AUTH_TRUSTED_ORIGINS`, `TRUST_PROXY_HEADERS`, `PORT`, `BOOTSTRAP_ADMIN`, `BOOTSTRAP_ADMIN_MOBILE`, `BOOTSTRAP_ADMIN_NAME`, `BOOTSTRAP_ADMIN_PASSWORD`. Frontend/build: `VITE_GOOGLE_MAPS_BROWSER_KEY`, `VITE_GOOGLE_MAP_ID`, `API_PROXY_TARGET`. Render native: `PYTHON_VERSION`, `NODE_VERSION`. Browser test: `RIDEZ_TEST_URL`, `PLAYWRIGHT_MODULE`, `CHROMIUM_PATH`.

Numbers to remember

Value	Meaning
15 min / 30 days	access token / session lifetime
5 min	quote validity
500 m, +500 m / 30 s, 5 km	matching radius start/step/max (10 rounds)
90 s	location freshness
200 m	max GPS accuracy
200 km/h	implausible-movement threshold (+ accuracies + 100 m)
30 s / 10 s	location send interval idle / on trip
2 s / 15 s	workspace poll without / with socket
10 s	trip map refresh; worker health window
60 s	trip map Google cache
1 s	worker tick
200	outbox events per tick
7 days	outbox retention; rating form window (UI)
10/min	login+register per IP; quotes per rider

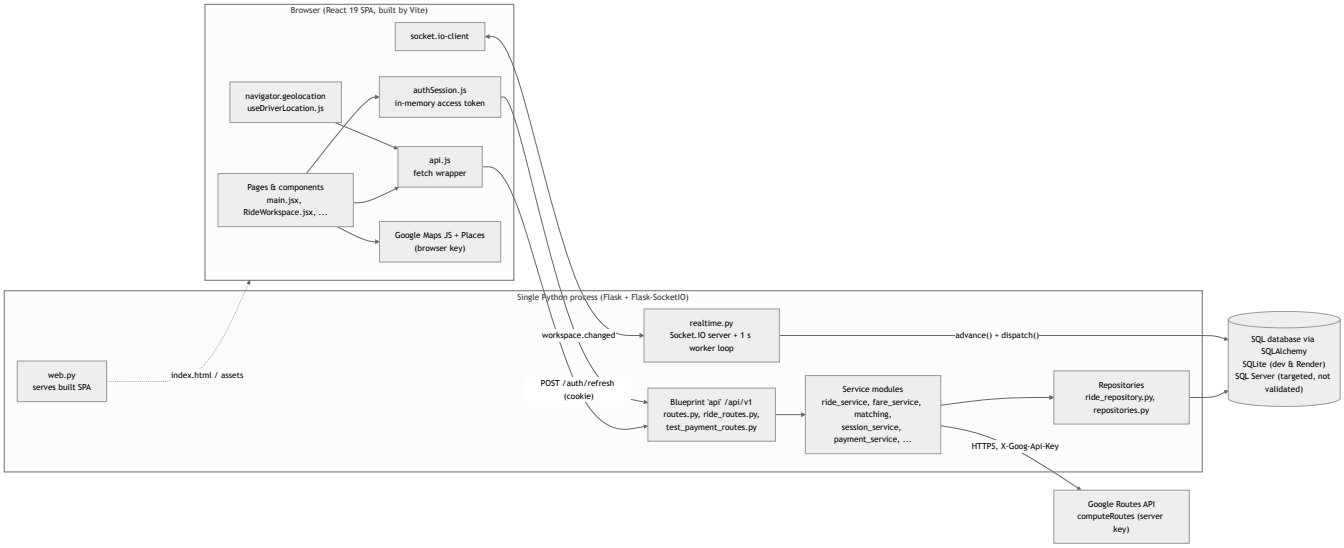
Value	Meaning
5/min	failed OTPs per driver
15 min	mock QR link
₹40 + ₹12/km + ₹2/min, min ₹40	default fare
85 / 15	default driver/company split
20	max surge zones; page size
100	rides/drivers in snapshot

Important workflows (one line each)

1. **Register** → E.164 → HMAC code → hash → users (driver pending).
2. **Login** → session + refresh cookie + 15-min JWT; reload → `/auth/refresh`.
3. **Driver online** → GPS fix → `/driver/availability` → presence + location → stale after 90 s.
4. **Quote** → Google/simulator route → fare (+surge) → `fare_quotes` (5 min).
5. **Book** → ride SEARCHING + rider claim + round 1 invitations + outbox.
6. **Match** → every 30 s radius +500 m; max → NO_DRIVER_FOUND.
7. **Accept** → conditional assign + driver claim + vehicle snapshot.
8. **Trip** → depart → arrive → start (OTP) → complete.
9. **Complete** → ledger split + payment PENDING + release + auto-availability.
10. **Pay** → cash (driver confirms) or mock QR → first confirmation wins.
11. **Rate** → once per side; comments admin-only.
12. **Admin** → approve/suspend/reinstate with version + reason; fares; revenue; audit.
13. **Real-time** → outbox → worker emits → client refetches `/workspace`.

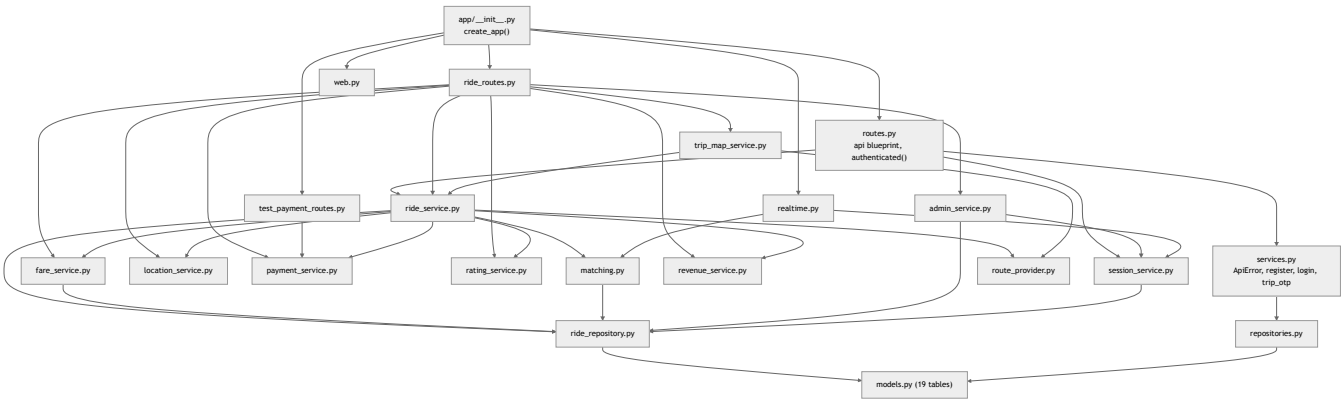
Architecture diagrams

1. High-level system

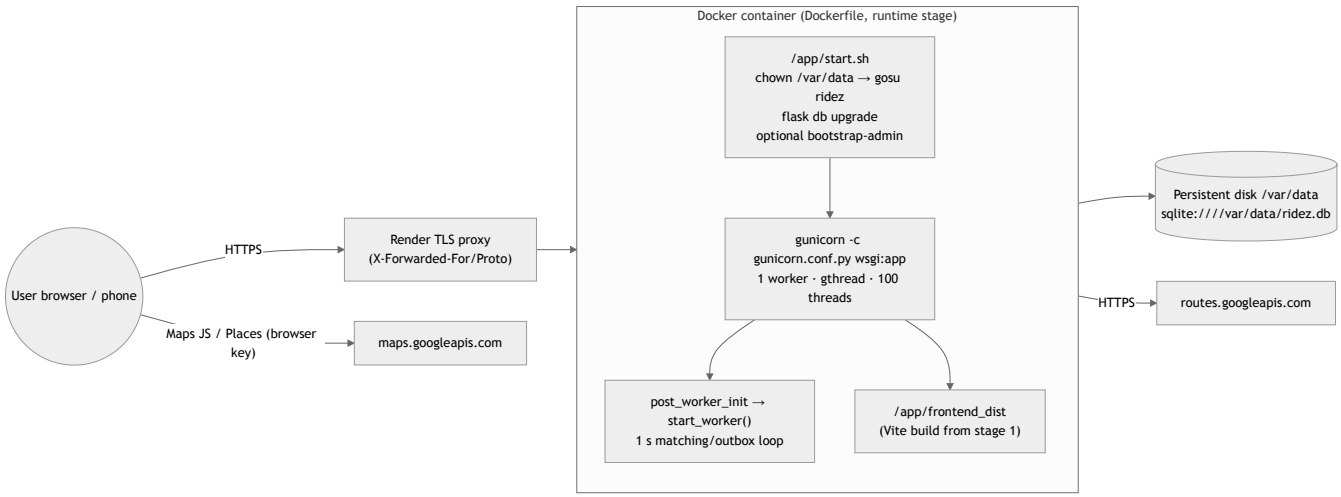


2. Backend module dependency graph

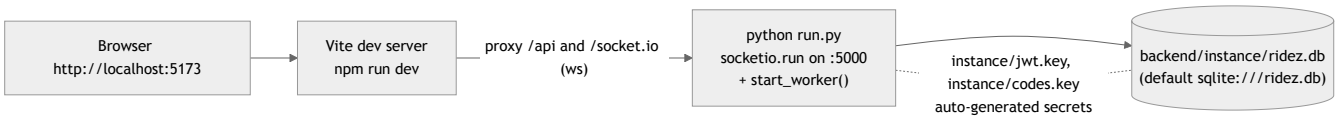
Arrows mean "imports / calls".



3. Deployment topology (Render Blueprint, `render.yaml`)



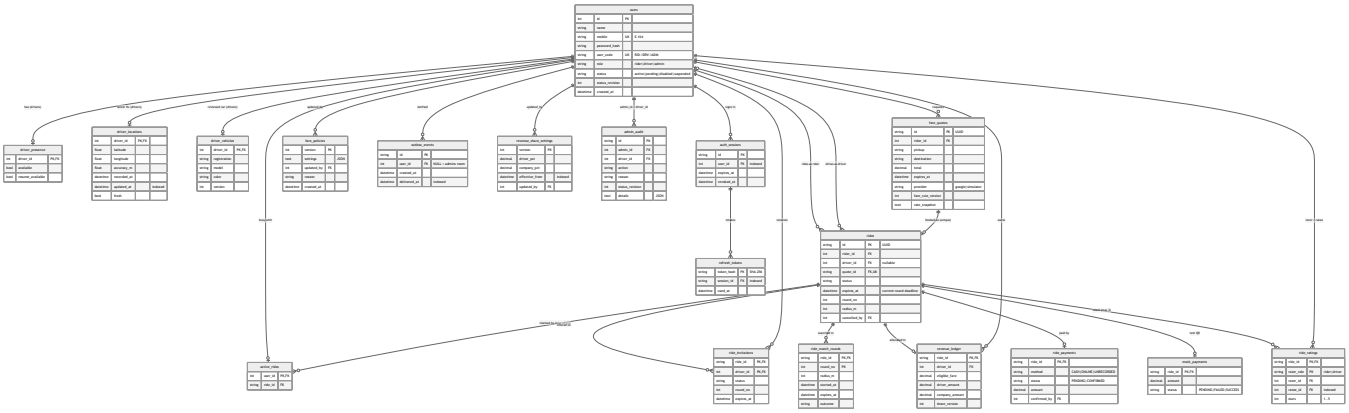
4. Local development topology



Note: Flask-SQLAlchemy resolves the relative SQLite URL `sqlite:///ridez.db` inside the Flask instance folder (`backend/instance/`). **(inferred from Flask-SQLAlchemy 3.x behaviour; the `instance/` folder in this checkout contains the generated key files.)**

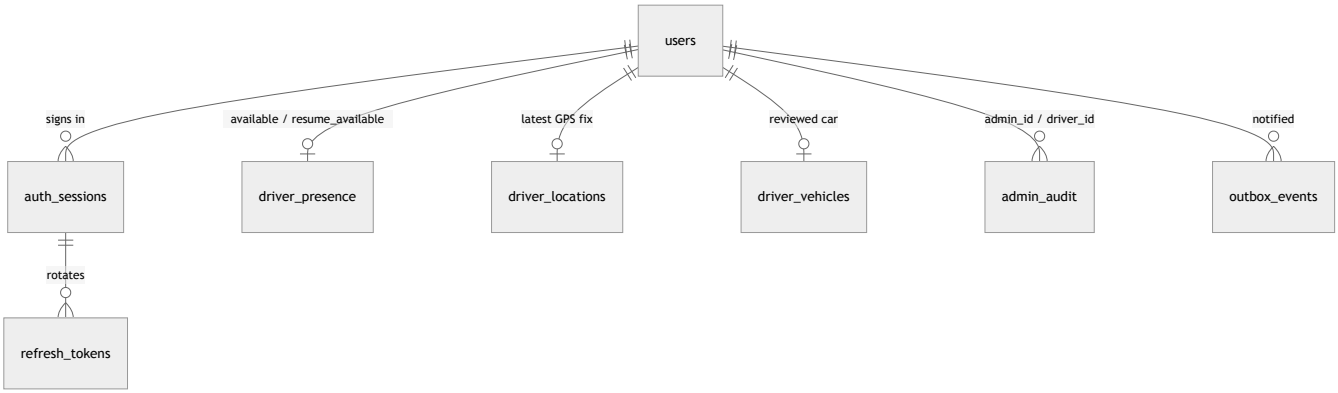
Entity-relationship diagram

All tables are defined in `backend/app/models.py` and created by the Alembic migrations in `backend/migrations/versions/`.

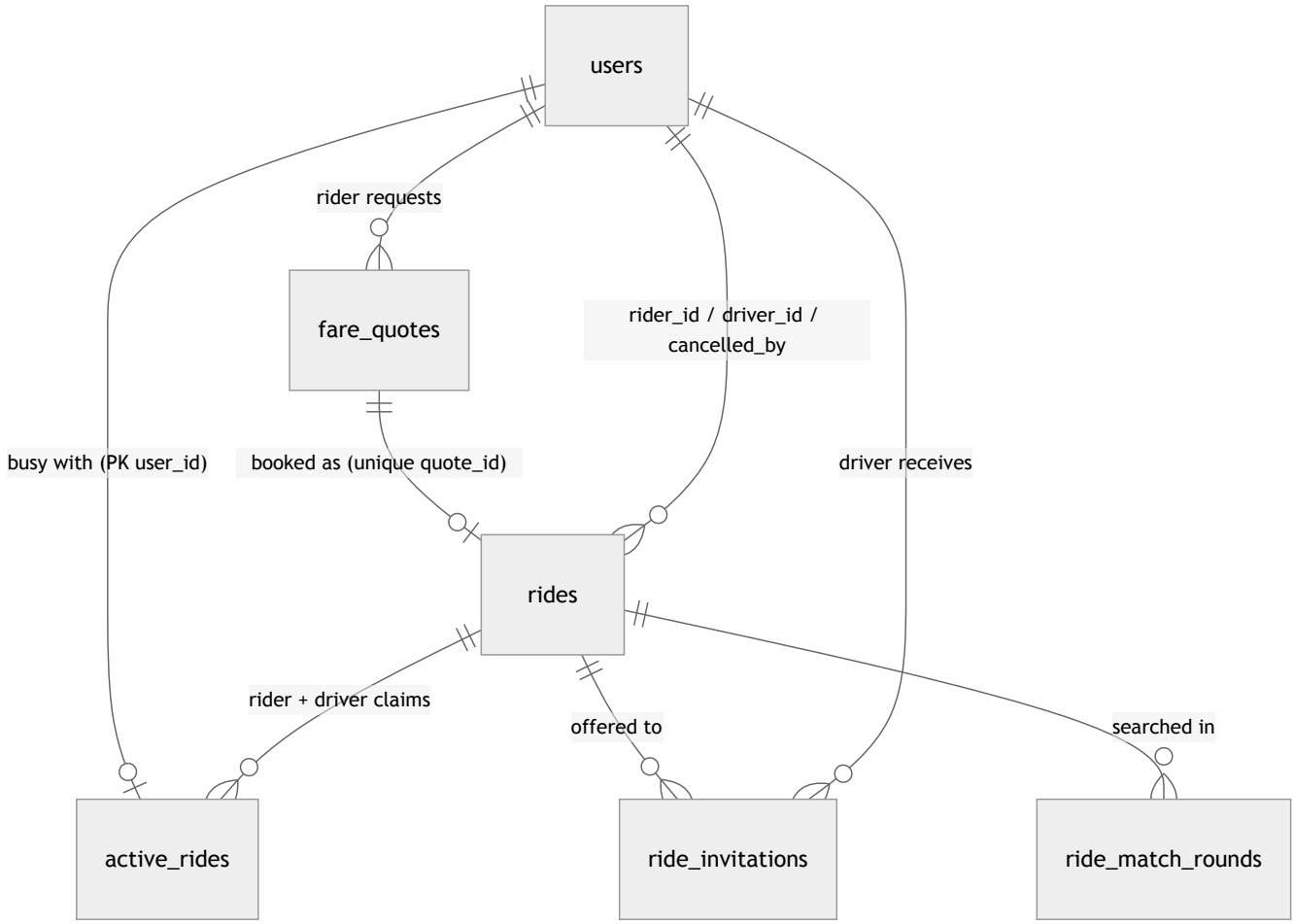


Domain views (same tables, split for readability)

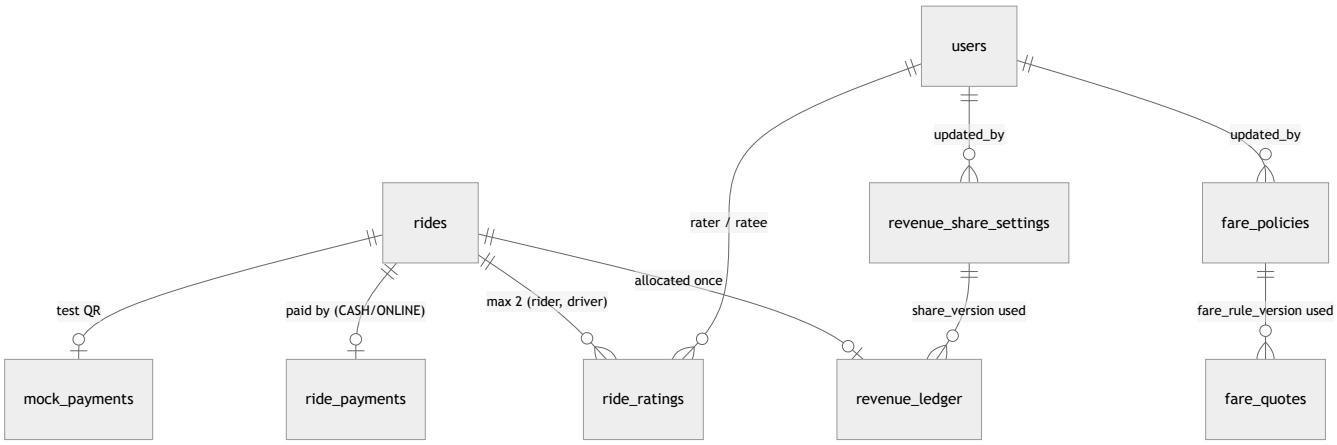
Accounts, sessions and driver state



Quotes, rides and matching



Money, settings and feedback



`share_version` and `fare_rule_version` are recorded version numbers, not database foreign keys (version 0 means "built-in defaults", which have no row).

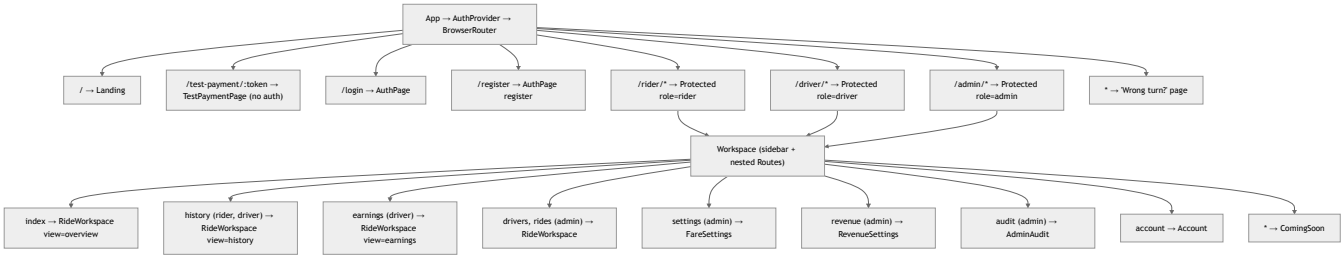
Reading notes

- `active_rides` has **at most two rows per ride** (rider + assigned driver) and **at most one row per user** (primary key `user_id`). This is the database-level guarantee that nobody has two active rides.
- `rides.quote_id` is **unique**, which makes the quote ID the booking idempotency key.

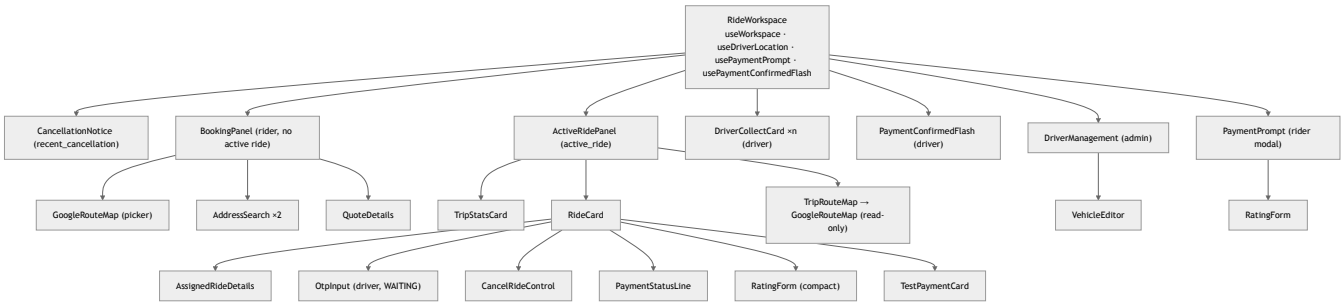
- `ride_ratings` primary key `(ride_id, rater_role)` allows exactly one rating from the rider and one from the driver per ride.
- `fare_policies.version` and `revenue_share_settings.version` are **manually assigned** (`autoincrement=False`); the primary key collision on concurrent saves is the optimistic-concurrency guard.
- `outbox_events.user_id = NULL` means "send to the `admins` Socket.IO room".

Frontend diagrams

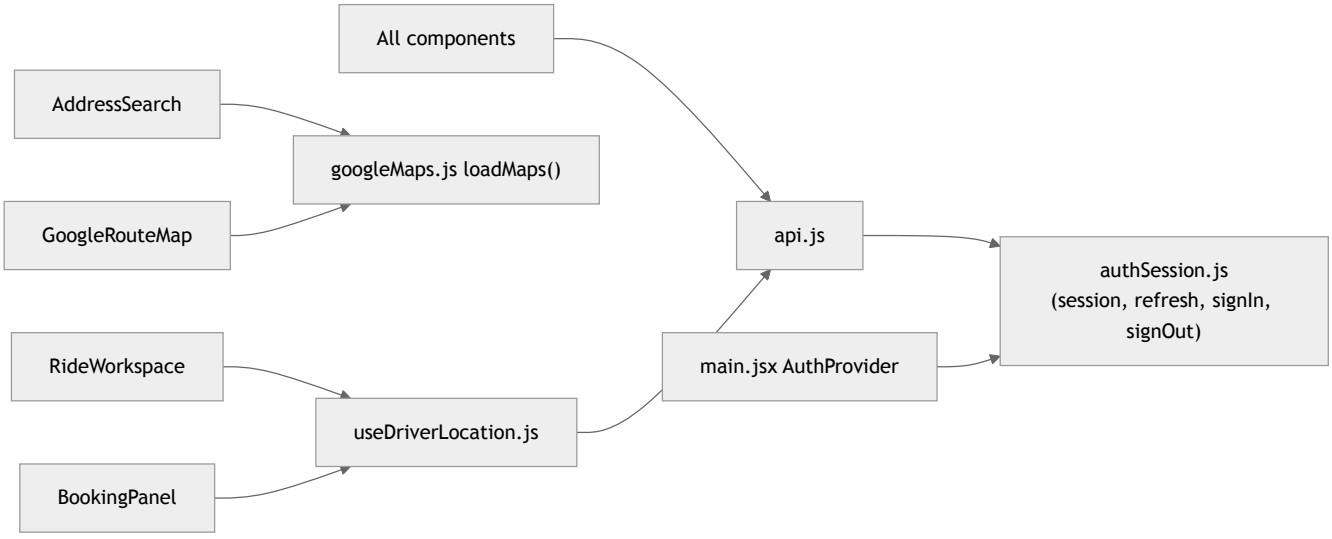
Route tree (`frontend/src/main.jsx::App`)



Component tree of `RideWorkspace`

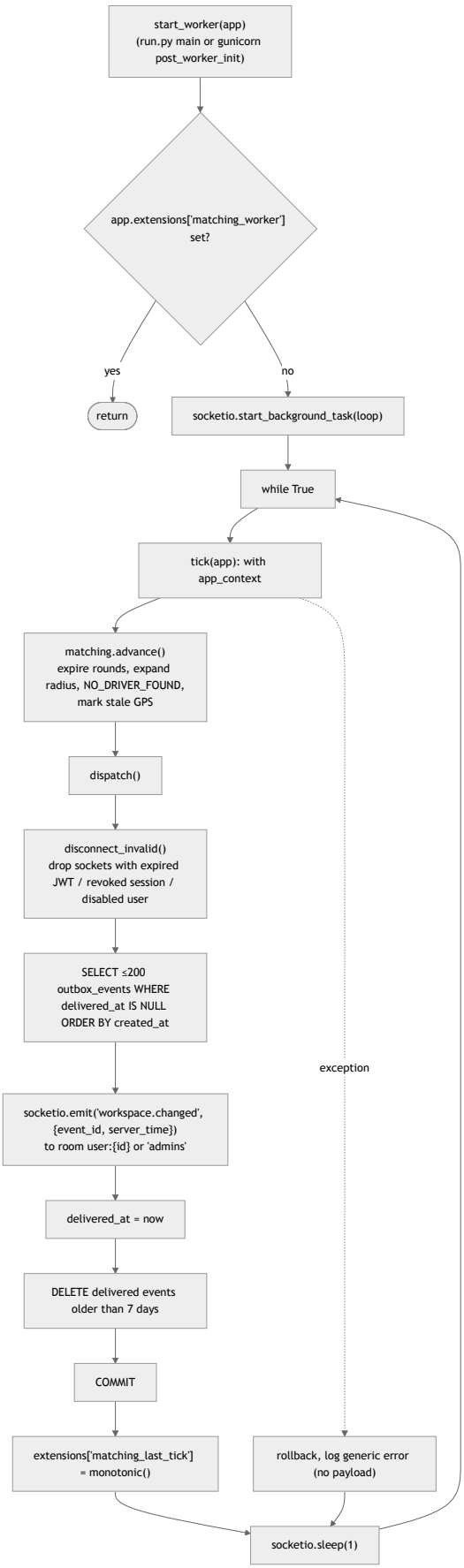


Shared modules

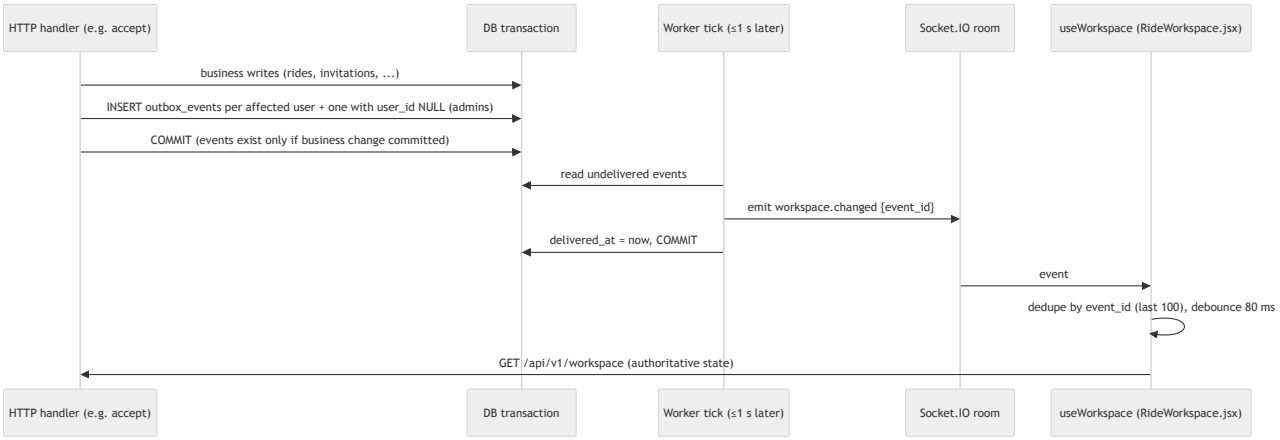


Real-time and background processing diagrams

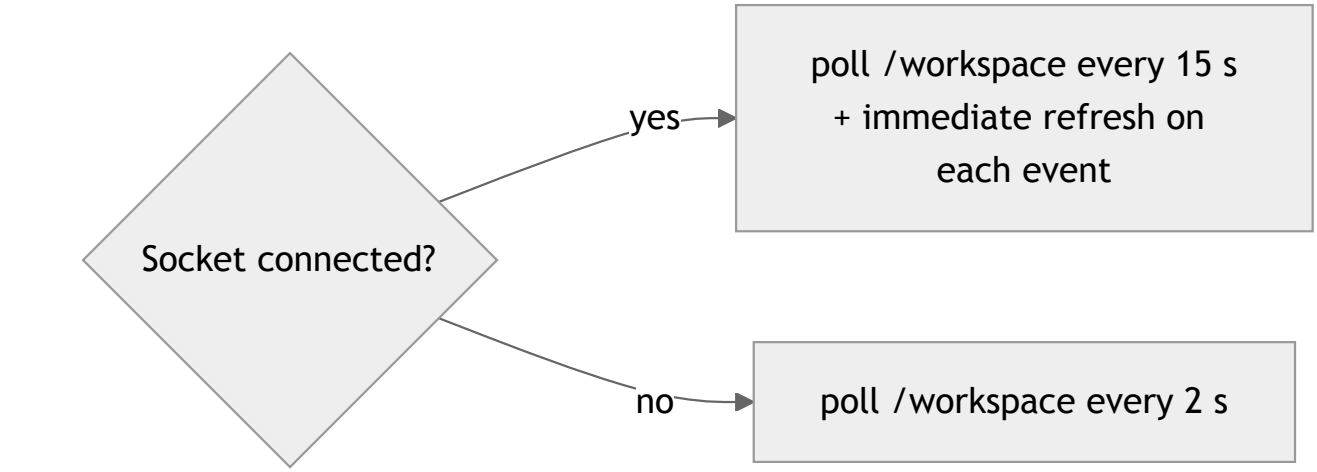
Worker loop (`backend/app/realtime.py`)



Transactional outbox pattern



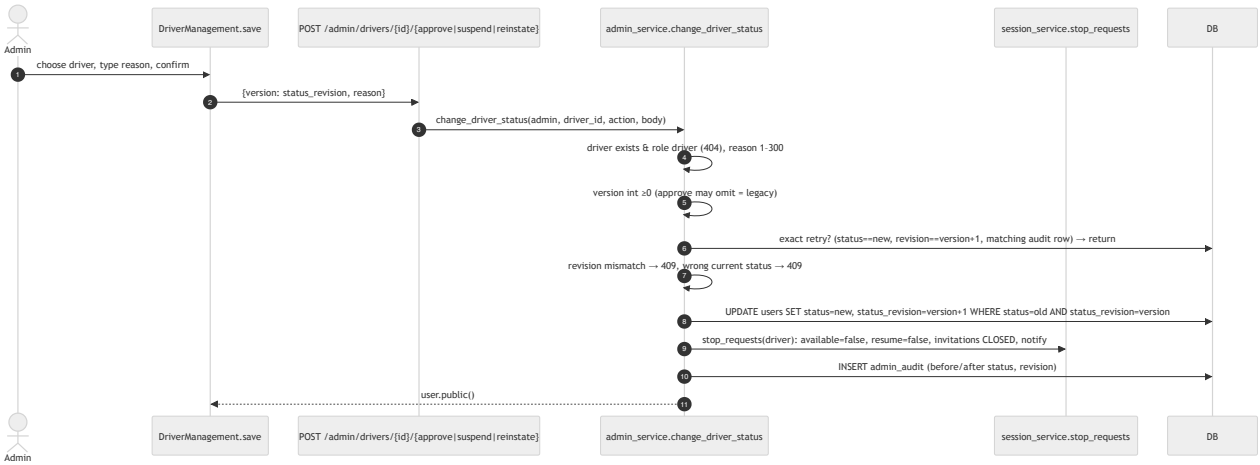
Client freshness strategy



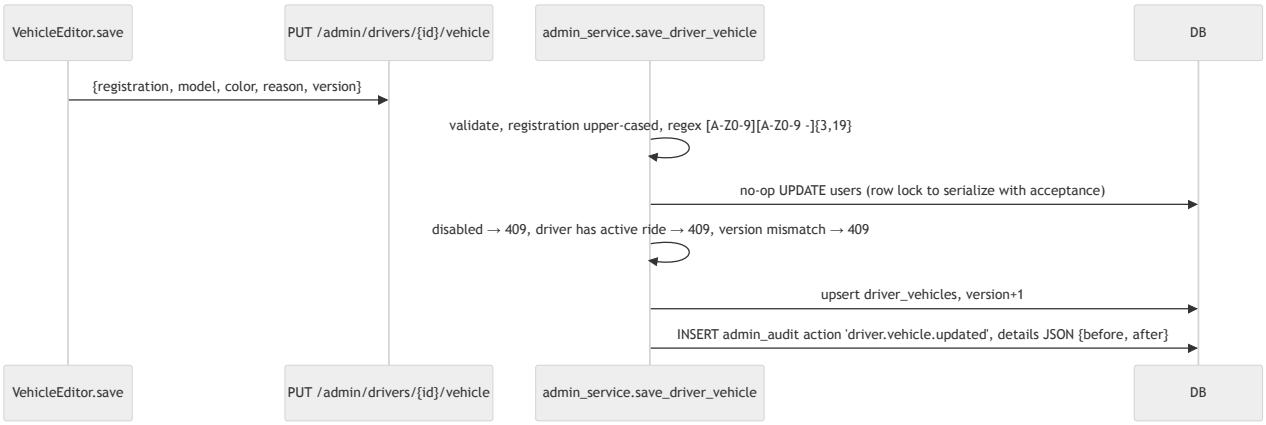


Admin sequence diagrams

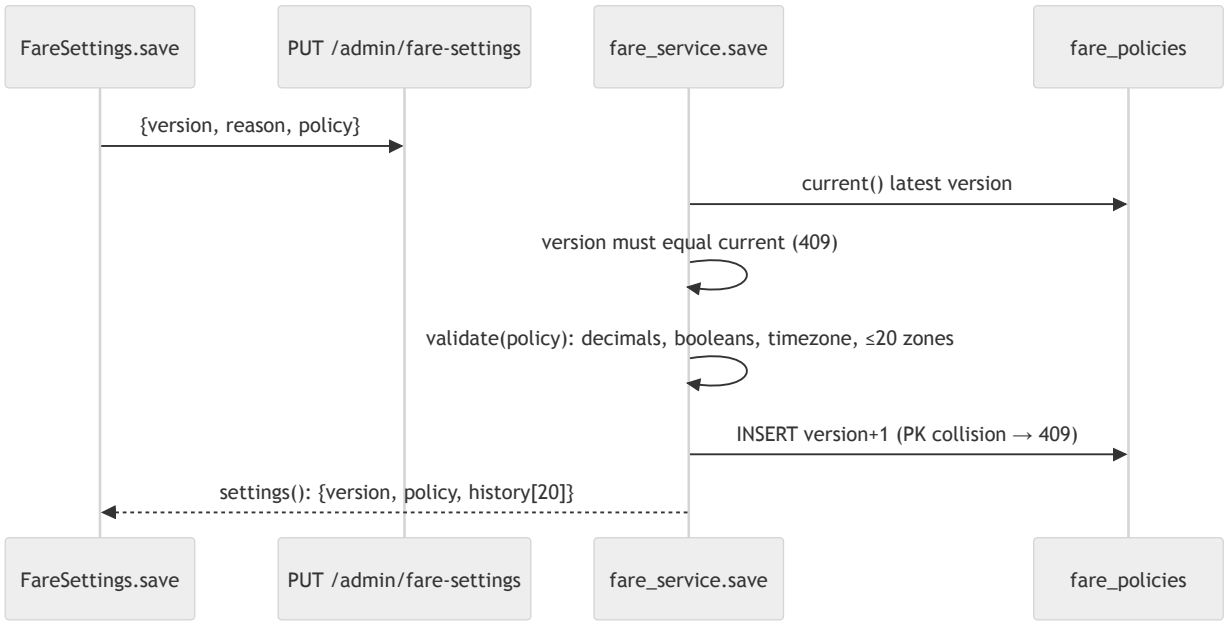
Approve / suspend / reinstate a driver



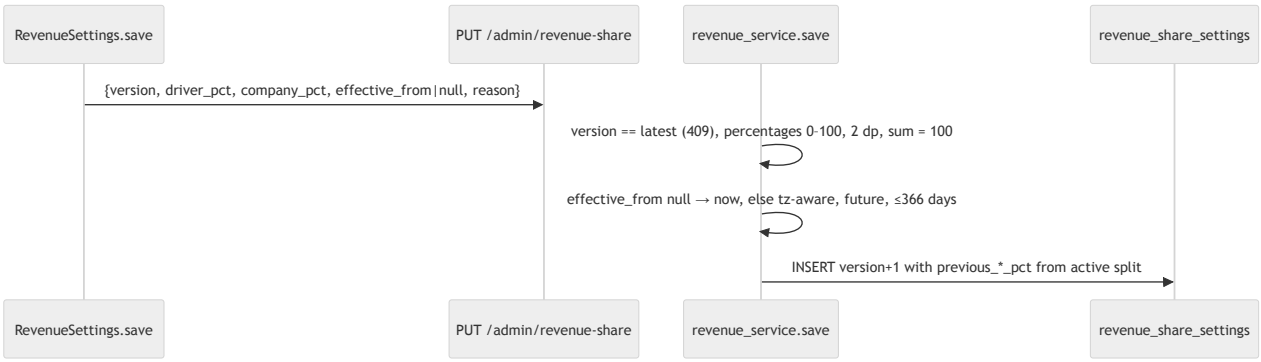
Save vehicle details



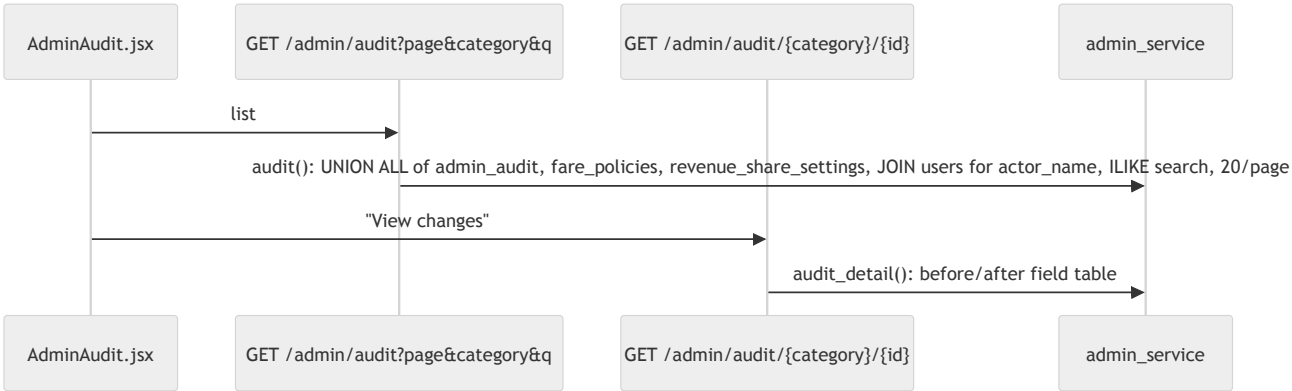
Fare policy save (optimistic concurrency)



Revenue share save

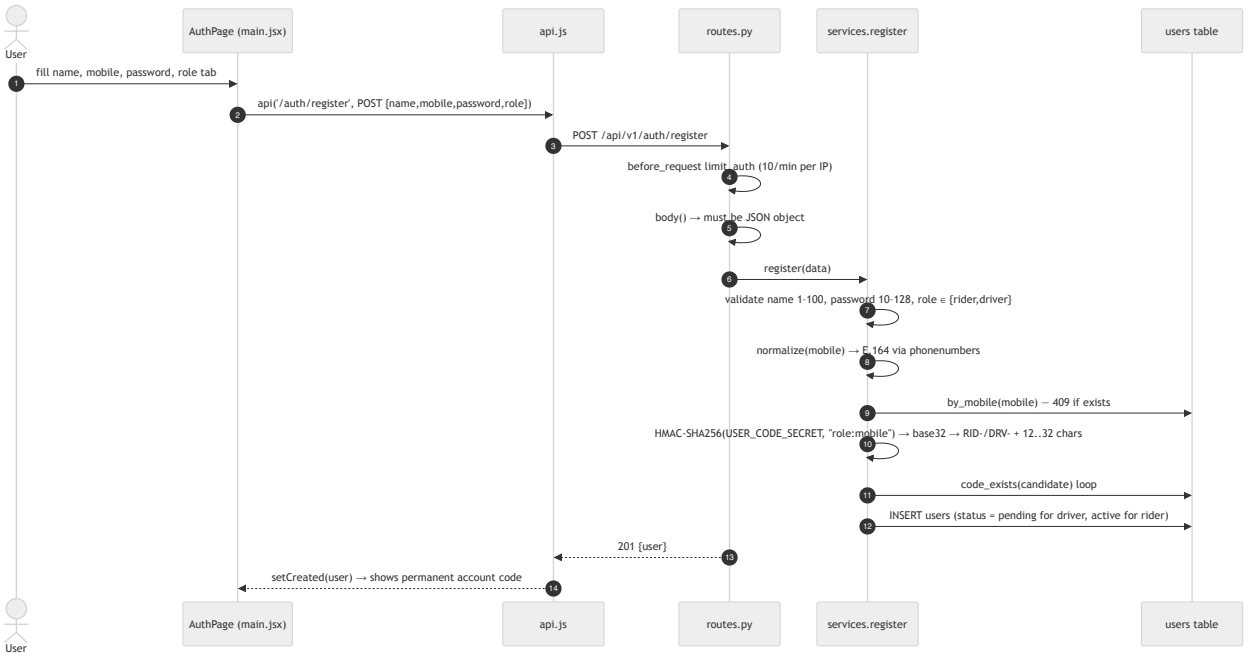


Audit history

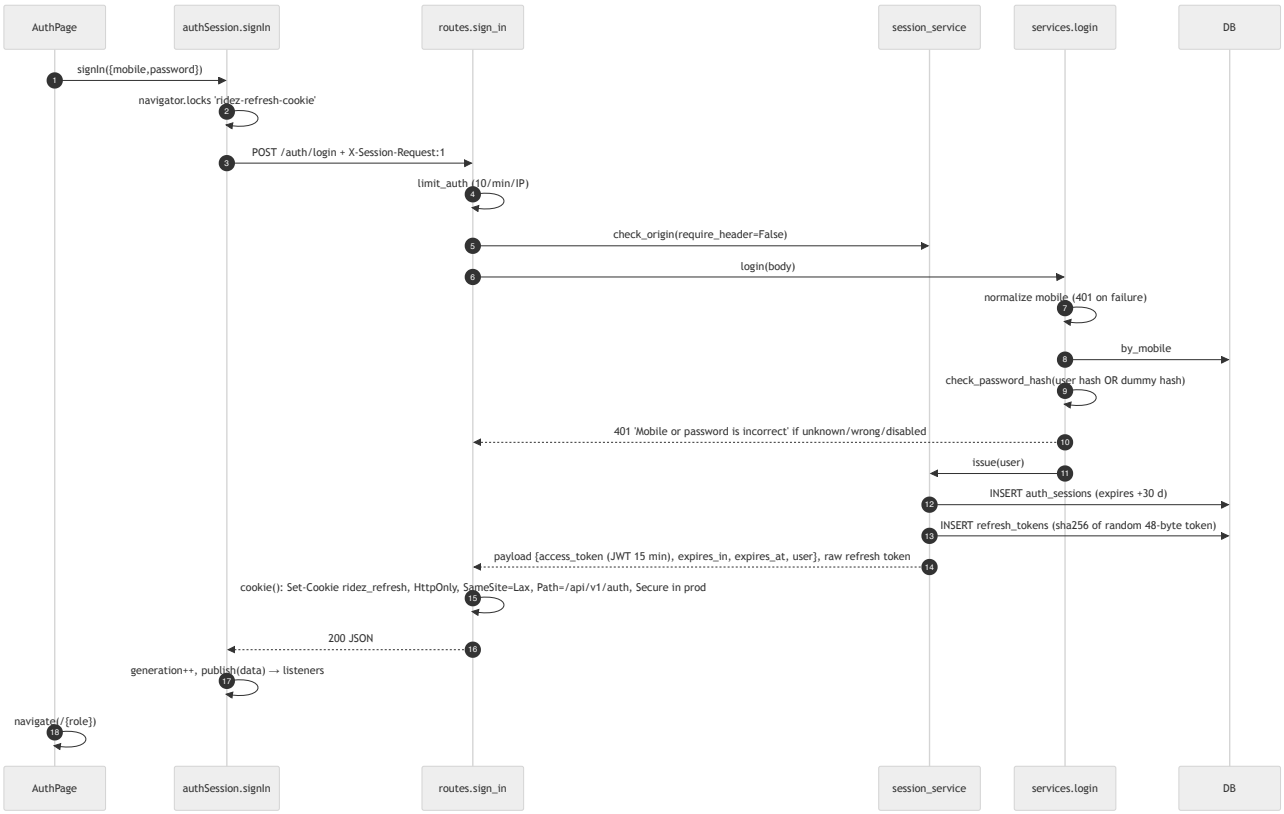


Authentication sequence diagrams

Registration



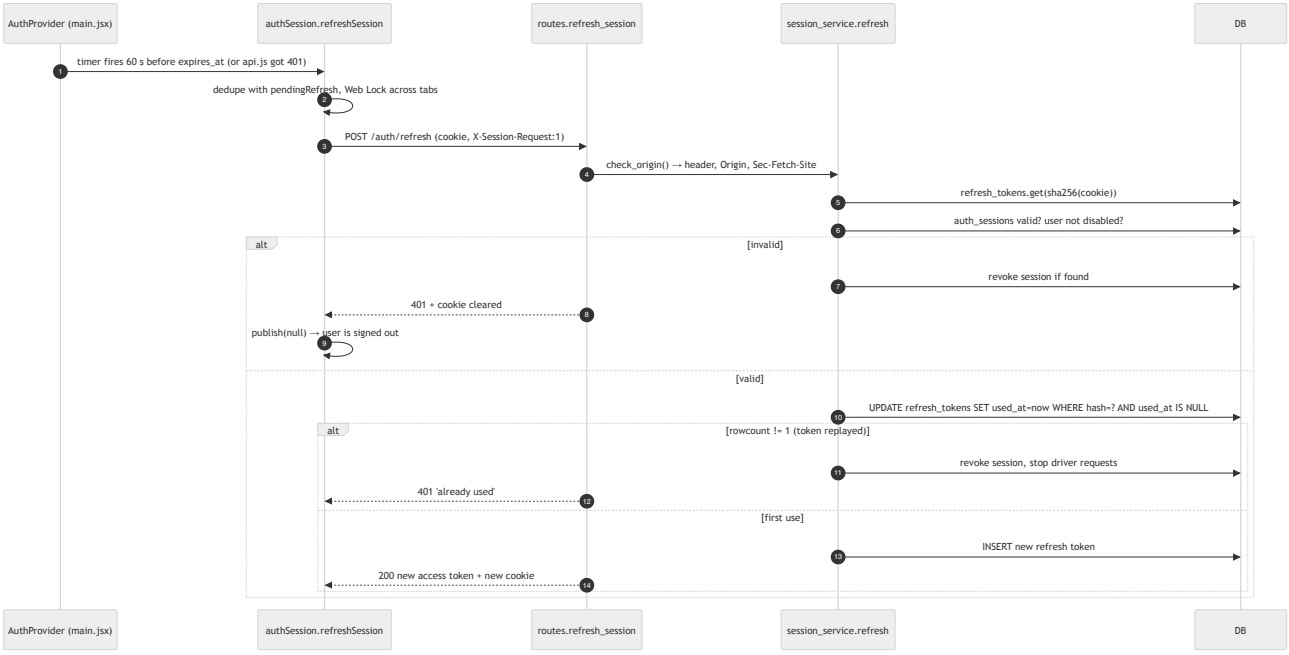
Login



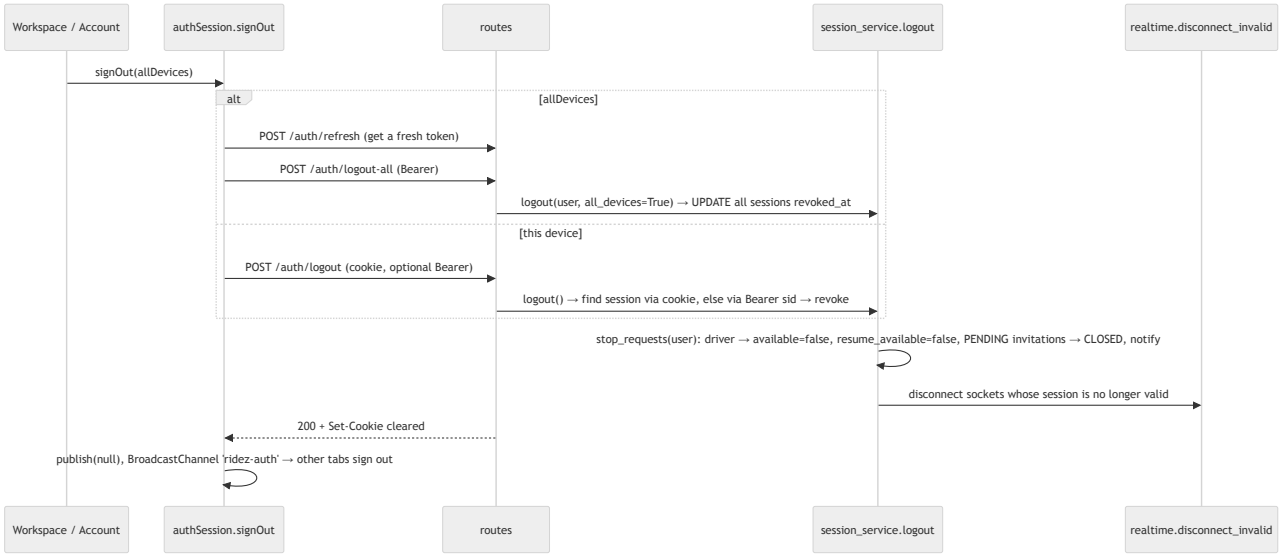
Authenticated request (`@authenticated(role)`)



Silent refresh and single-use rotation

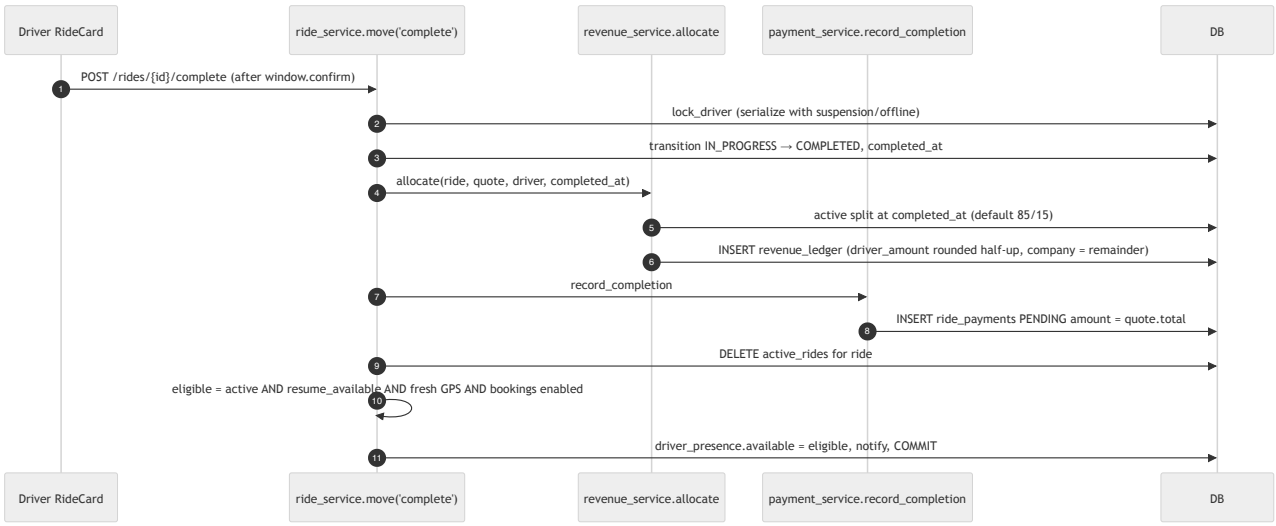


Logout and logout-all



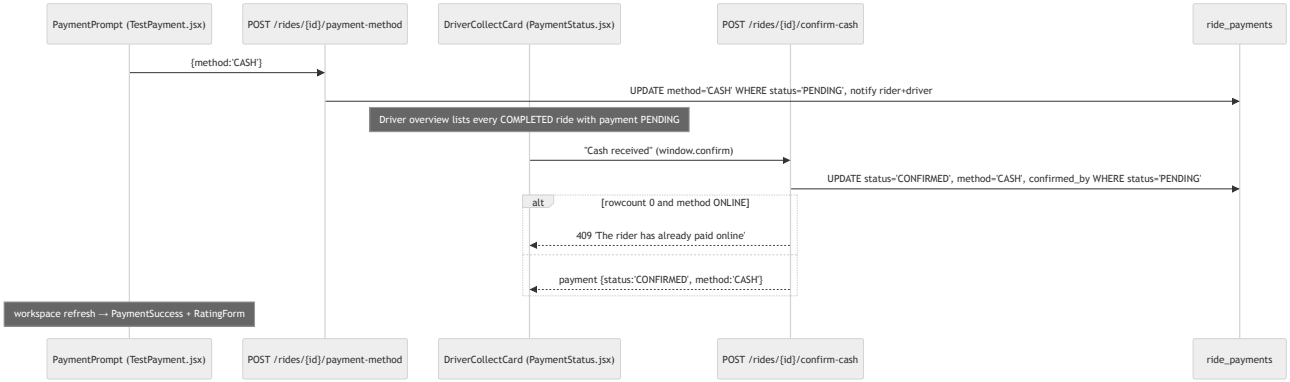
Completion, payment and rating sequence diagrams

Trip completion

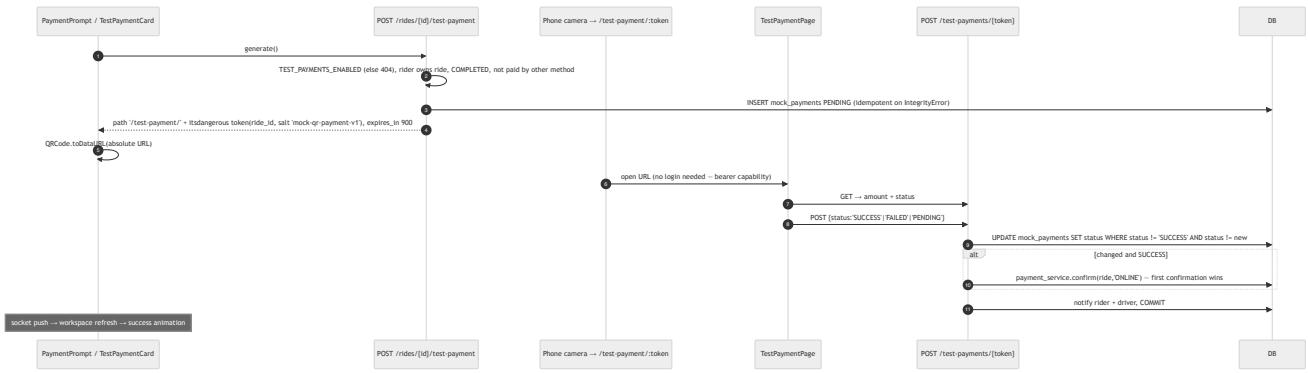


A retried `complete` returns the existing ride: `ride.status == new` short-circuits before any writes. The `revenue_ledger` and `ride_payments` primary keys (`ride_id`) are a second line of defence against double allocation.

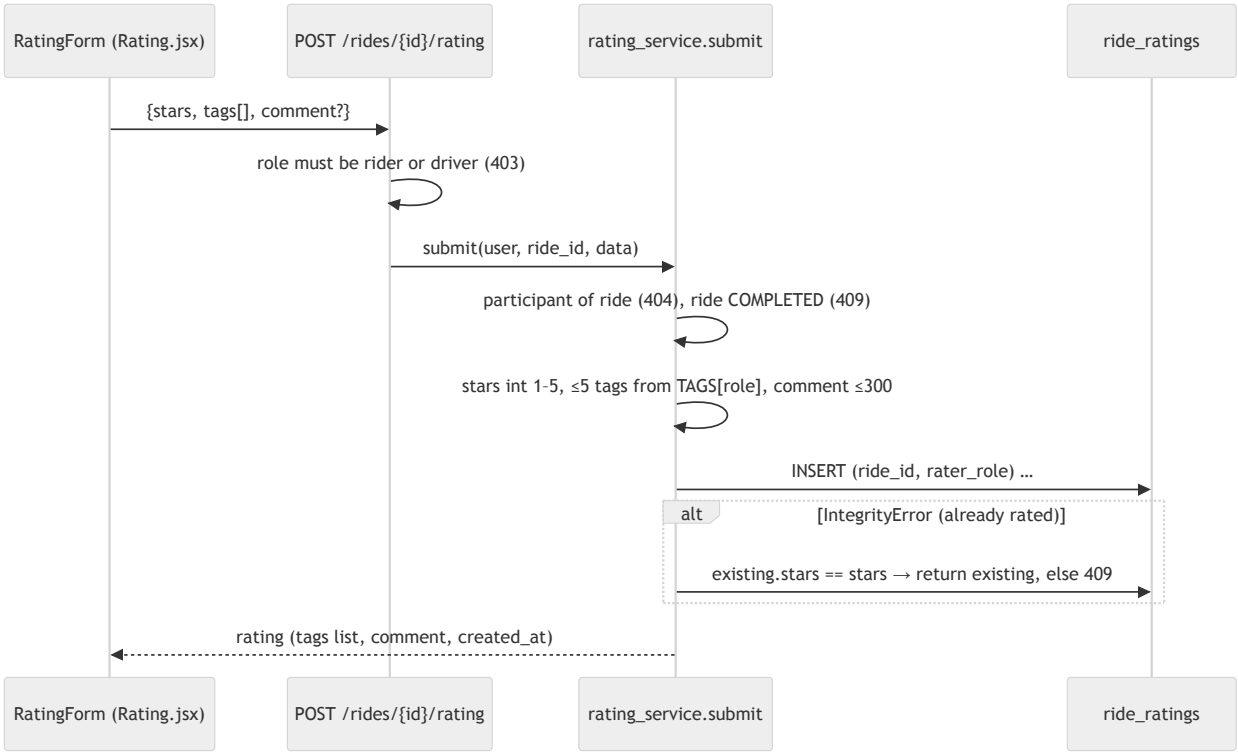
Rider pays in cash



Rider pays "online" (mock QR, test only)

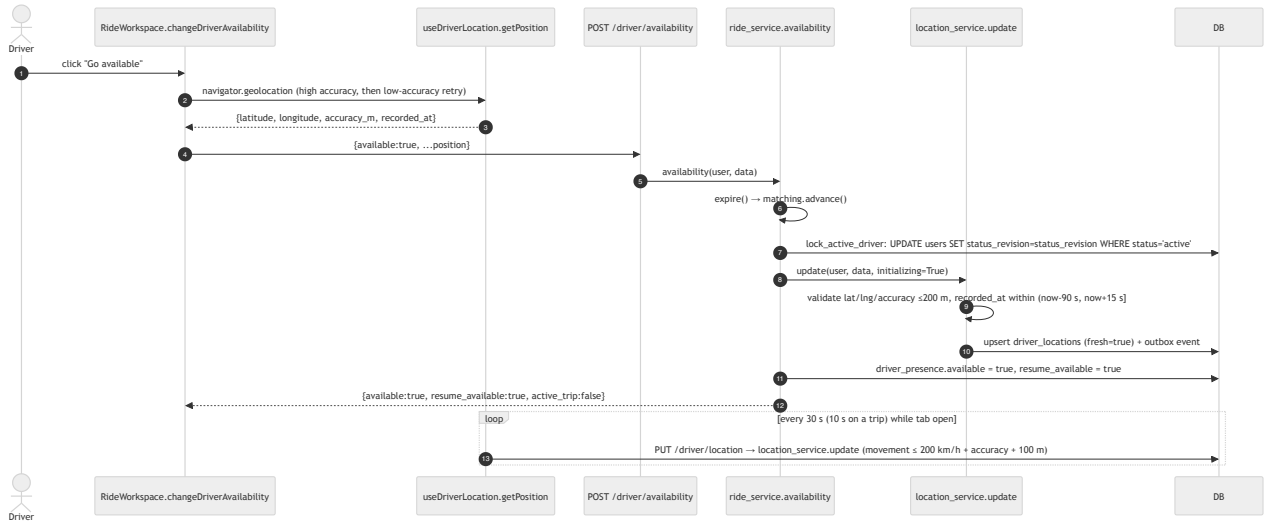


Two-way rating

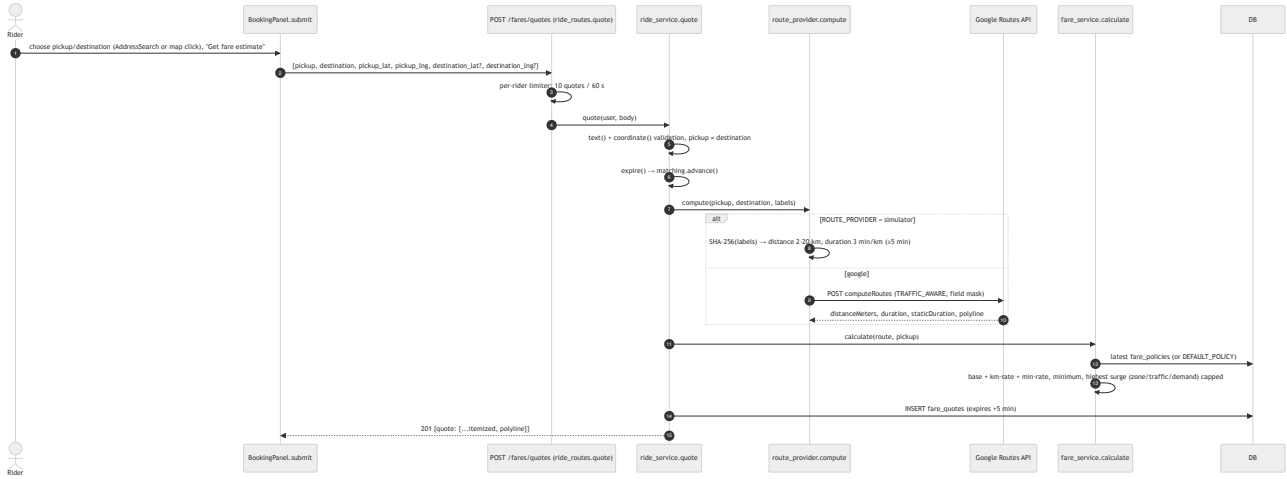


Ride sequence diagrams

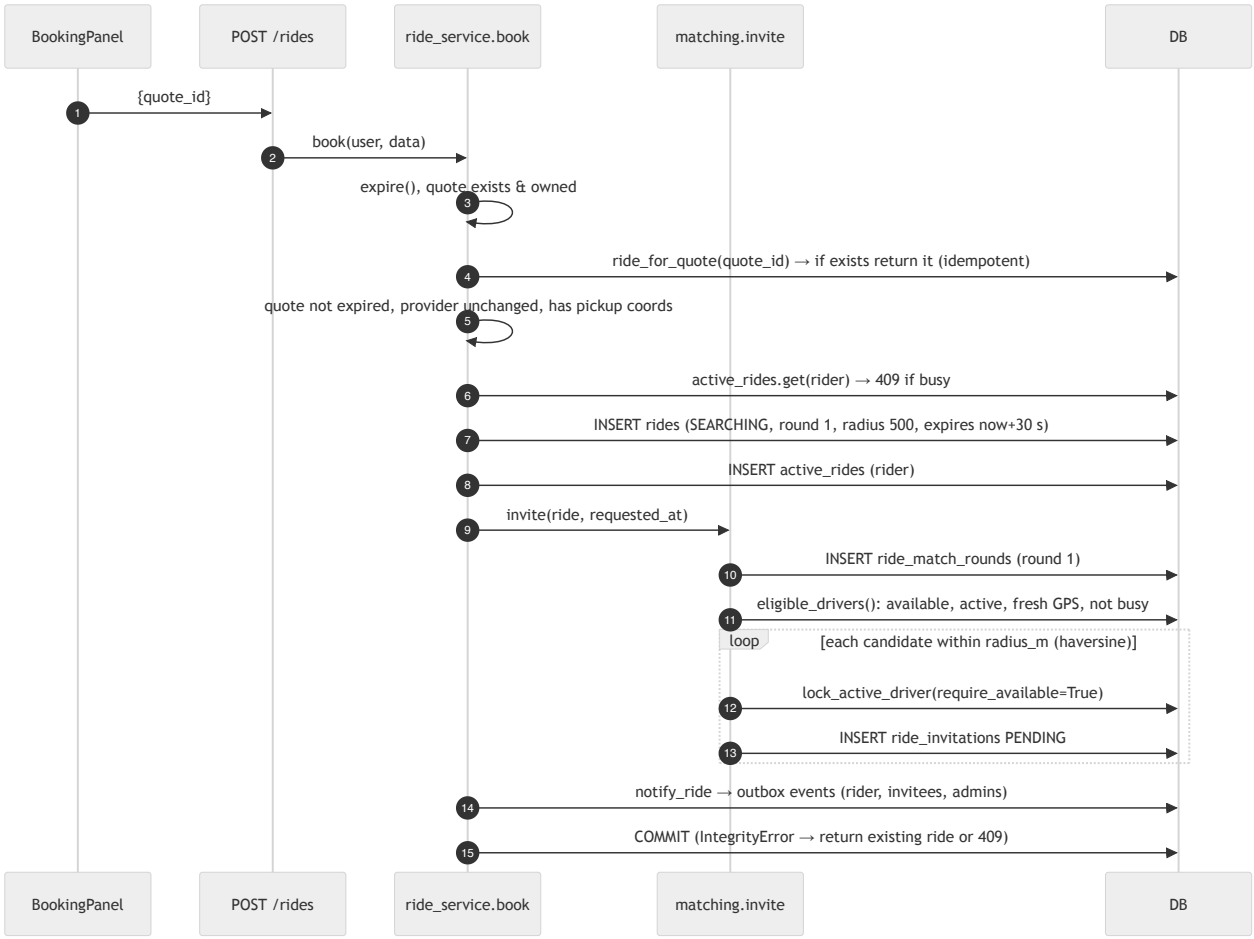
Driver goes available and streams GPS



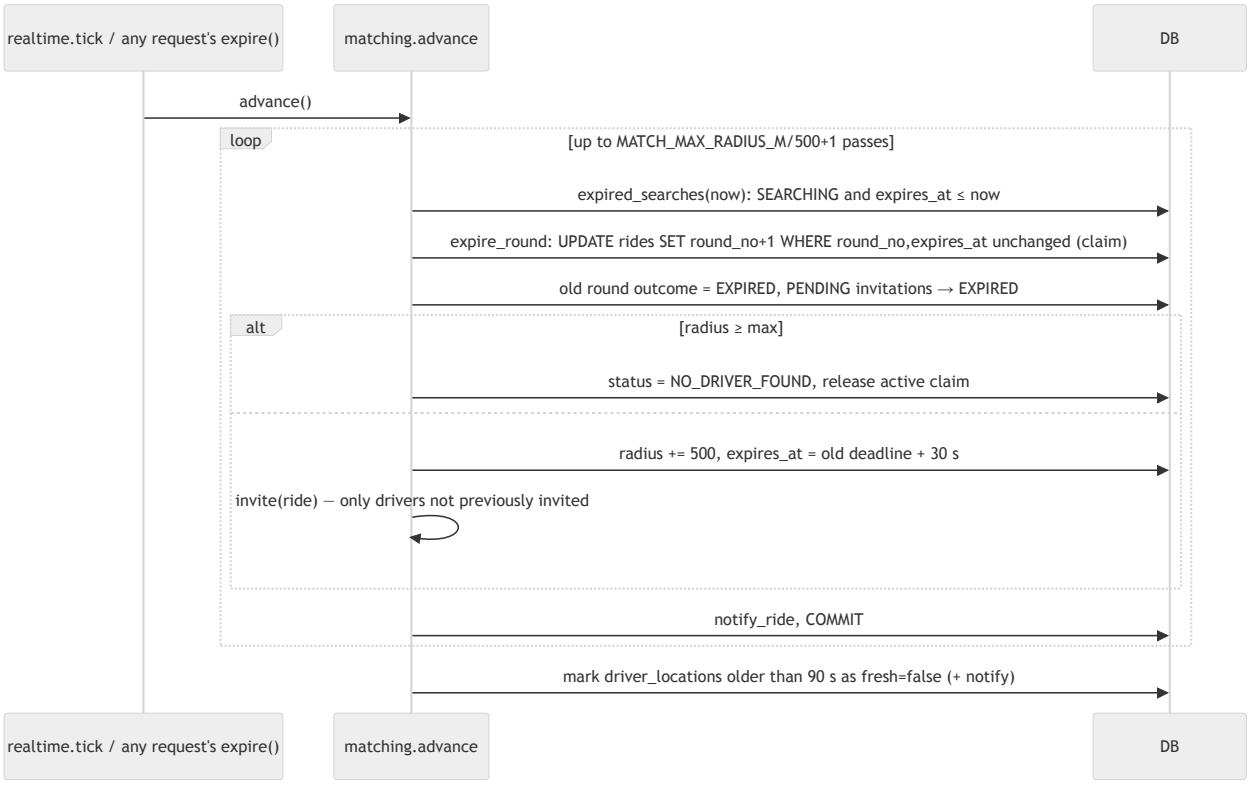
Fare quote



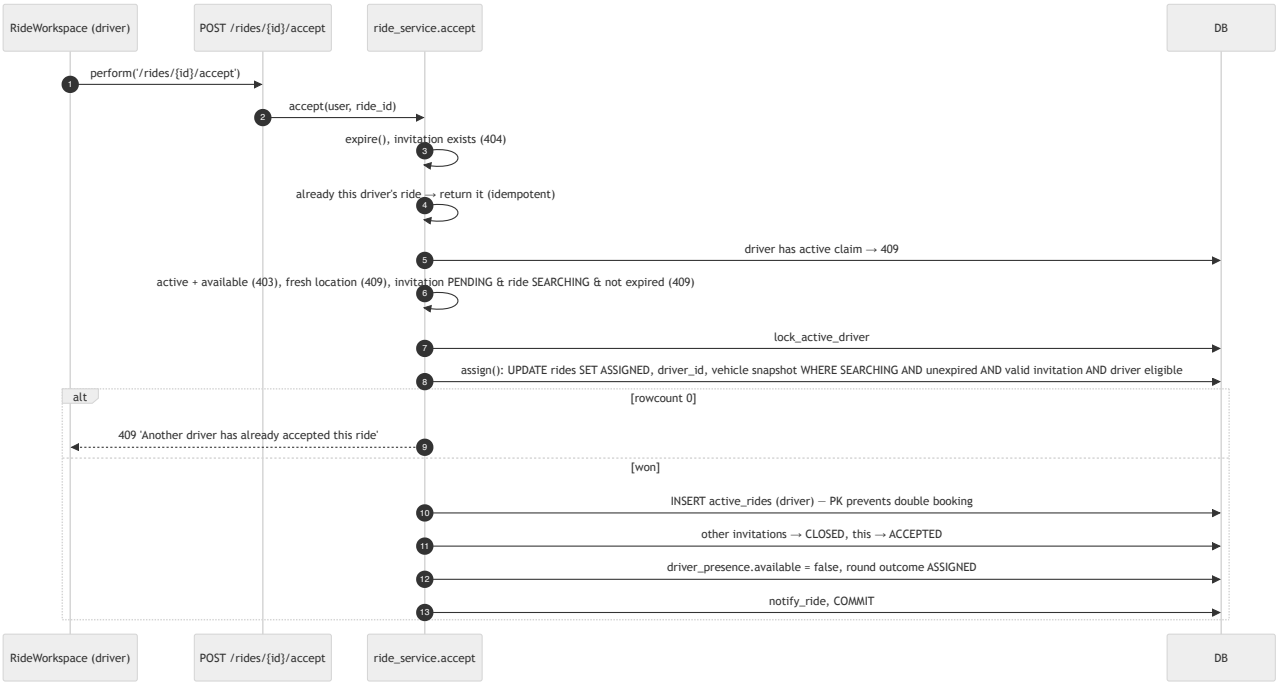
Booking and matching round 1



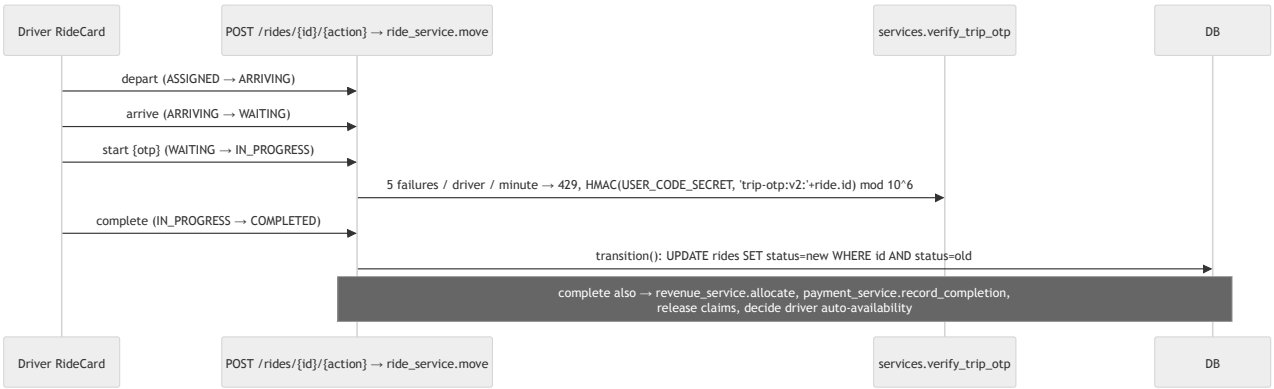
Round expiry (background worker or any request)



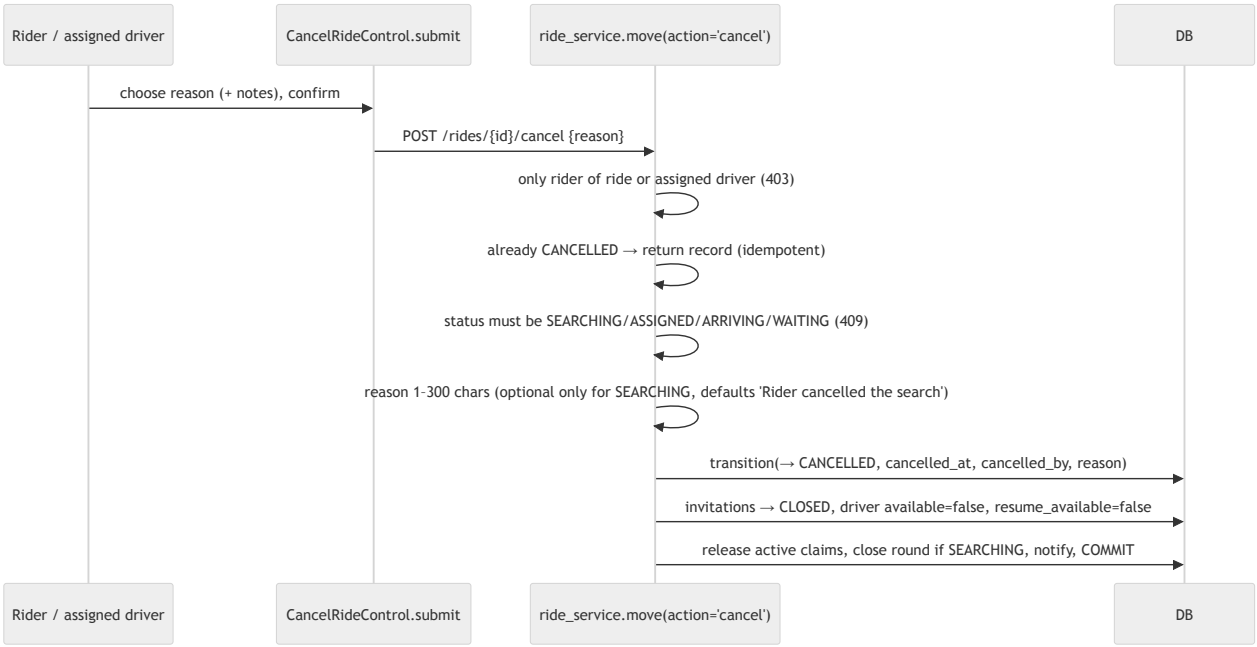
Driver accepts



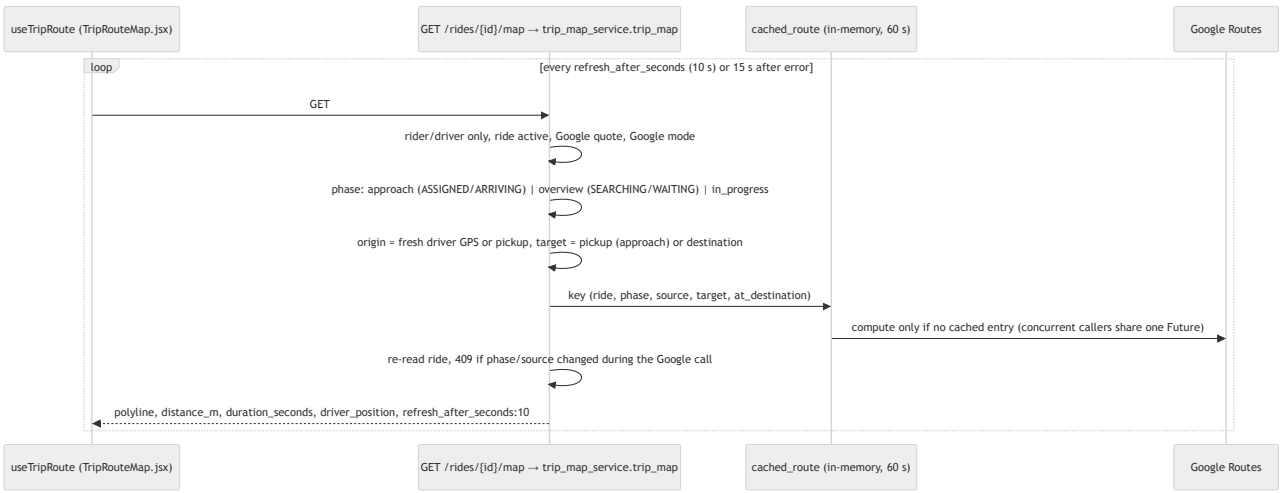
Trip progression and OTP



Cancellation



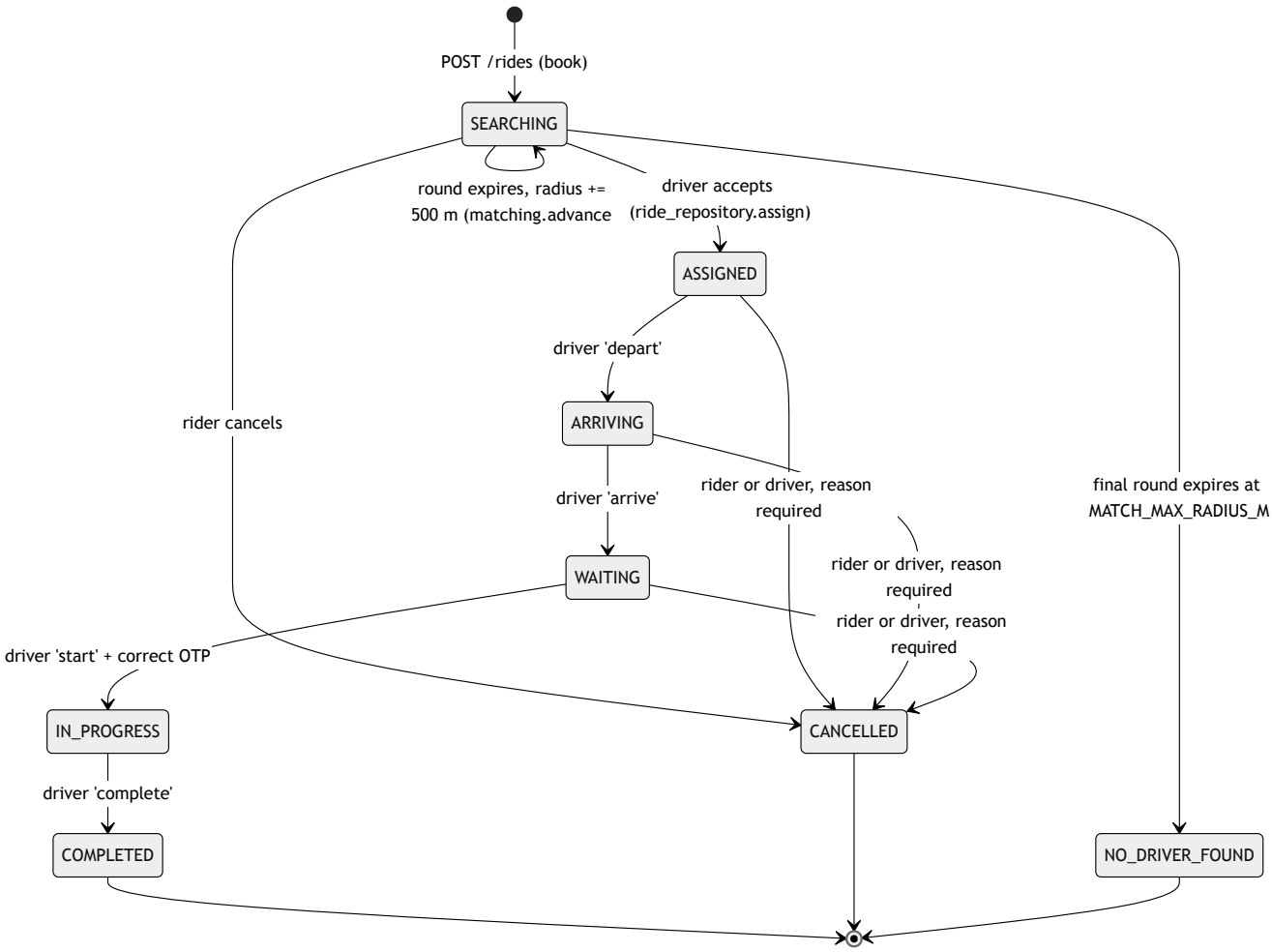
Live trip map



State machines

Ride status (`rides.status`)

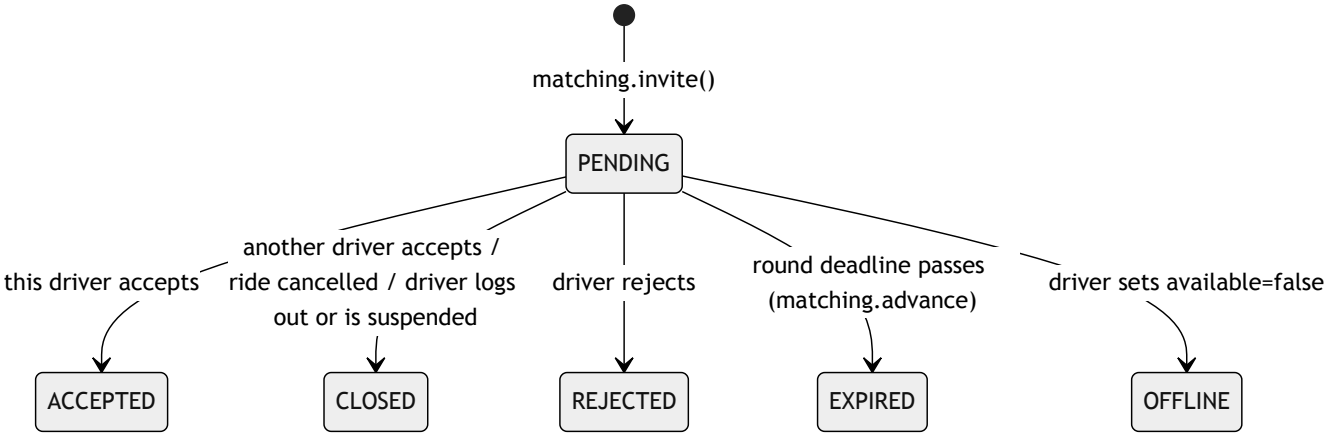
Allowed values are enforced by the `valid_ride_state` CHECK constraint. Transitions are implemented in `backend/app/ride_service.py` (`book`, `accept`, `move`) and `backend/app/matching.py` (`advance`). Every transition except the `NO_DRIVER_FOUND` timeout is a **conditional UPDATE** (`ride_repository.transition` / `ride_repository.assign`) so that only one concurrent request can win.



`IN_PROGRESS` rides **cannot be cancelled** (HTTP 409). There is no incident/abort workflow for an in-progress trip.

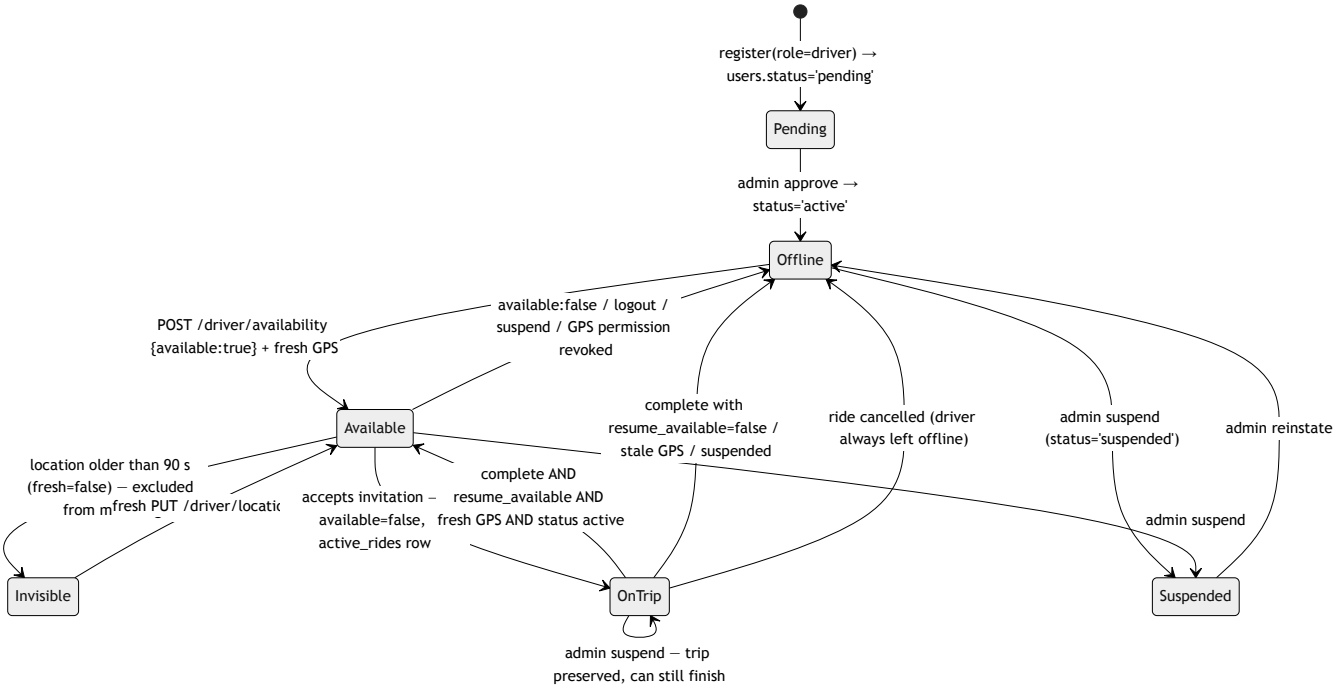
Invitation status (`ride_invitations.status`)

No CHECK constraint; values are set in code.

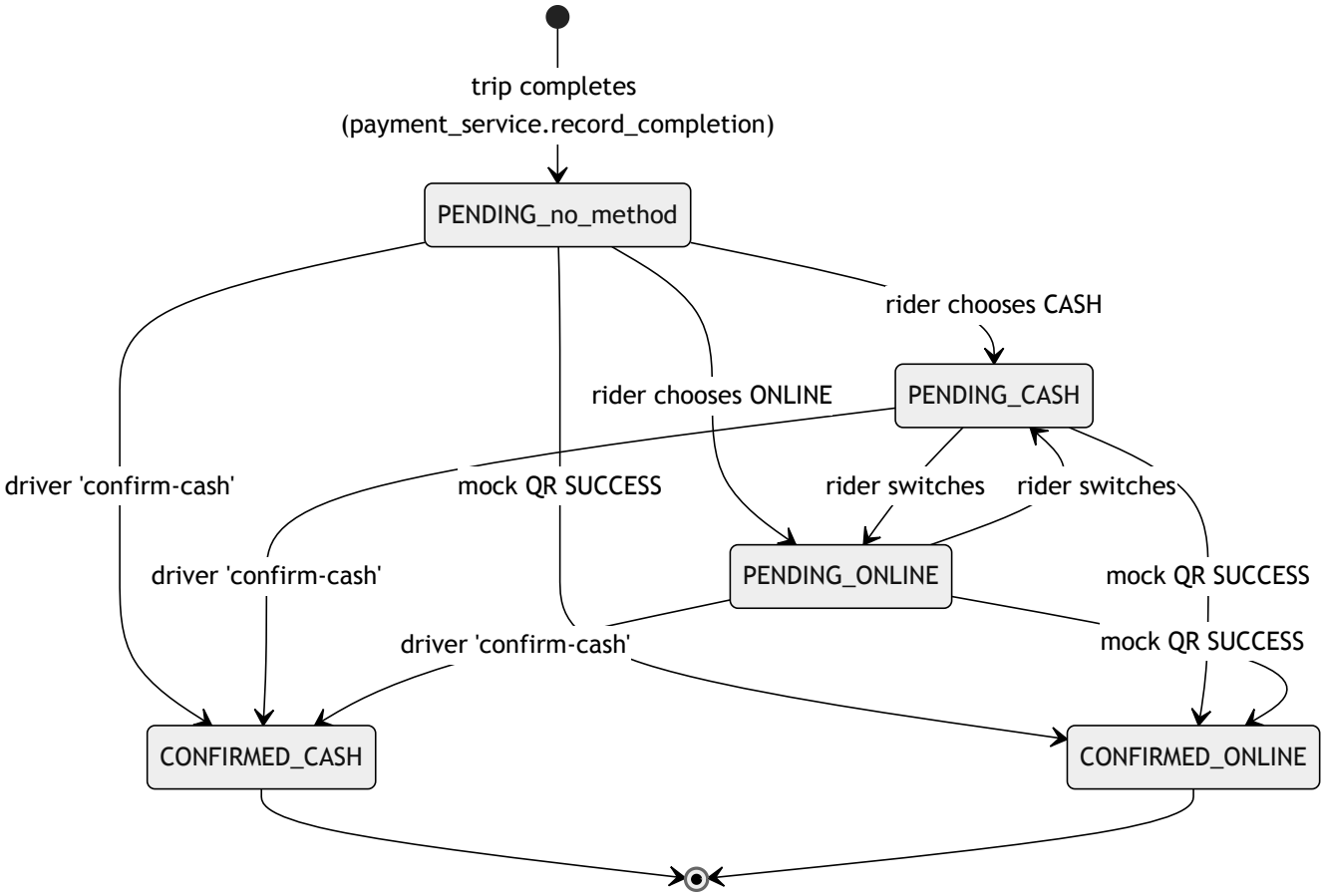


Driver operational state (derived, not a single column)

There is **no driver-state enum** in the database. A driver's operational state is derived from four sources: `users.status`, `driver_presence.available`, `driver_presence.resume_available`, `active_rides`, and the freshness of `driver_locations`.

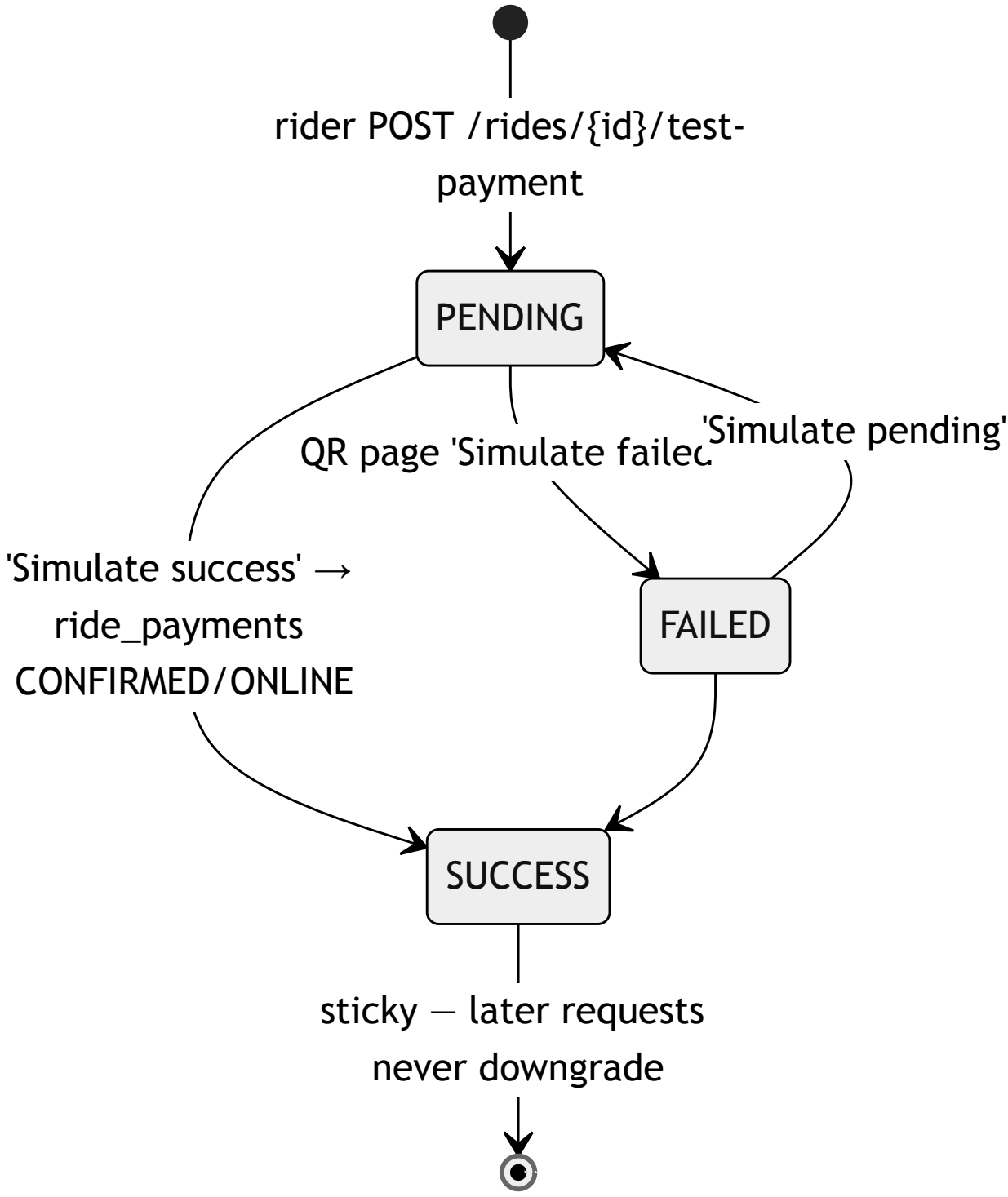


Ride payment (`ride_payments`)



`CONFIRMED` is final. The first confirmation wins because `payment_service.confirm` updates only `WHERE status='PENDING'`. A third method, `UNRECORDED`, only exists for rides completed **before** payment tracking was added (back-filled by migration `7ba01cddd40c`).

Mock QR payment (`mock_payments`)



Auth session (`auth_sessions` + `refresh_tokens`)

